Programming with Python

Thomas Weise (汤卫思)

November 1, 2024

Abstract

The goal of this book is to teach practical programming with the Python language to high school, undergraduate, and graduate students alike. Hopefully, readers without prior knowledge can follow the text. Therefore, all concepts are introduced using examples and discussed comprehensively. All examples are available online in the GitHub repository associated with this book, so that readers can play with them easily. Actually, the goal of the book is not just to teach programming, but to teach programming as a part of the software development process. This means that from the very beginning, we will attempt to push the reader towards writing clean code with comments and documentation as well as to use various tools for finding potential issues. While this book is work in progress, we hope that it will eventually teach all the elements of Python software creation. We hope that it can enable readers without prior programming experience to develop beautiful and maintainable software.

Contents

Contents i

1 Introduction 1
- 1.1 Why Programming *and* Software Engineering Tools? . . . 1
- 1.2 Why Python? . . . 2

I Basics 4

2 Getting Started 6
- 2.1 Installing Python . . . 6
- 2.2 Installing PyCharm . . . 9
- 2.3 Our First Program . . . 13
- 2.4 Python in the Terminal . . . 16
- 2.5 Summary . . . 21

3 Simple Datatypes and Operations 22
- 3.1 Introduction . . . 22
- 3.2 Integers . . . 22
- 3.3 Floating Point Numbers . . . 27
- 3.4 Boolean Values . . . 36
- 3.5 Text Strings . . . 38
- 3.6 None . . . 49
- 3.7 Summary . . . 50

4 Variables 51
- 4.1 Defining and Assigning Variables . . . 51
- 4.2 Multiple Assignments and Value Swapping . . . 59
- 4.3 Variable Types and Type Hints . . . 60
- 4.4 Object Equality and Identity . . . 65
- 4.5 Summary . . . 67

5 Collections 69
- 5.1 Lists . . . 69
- 5.2 Tuples . . . 77
- 5.3 Sets . . . 80
- 5.4 Dictionaries . . . 84
- 5.5 Summary . . . 85

II Control Flow Statements 87

6 Conditional Statements 89
- 6.1 The `if` Statement . . . 89
- 6.2 The `if...else` Statement . . . 90
- 6.3 The `if...elif...else` Statement . . . 92
- 6.4 The Inline `if...else` Statement . . . 95
- 6.5 Summary . . . 97

7 Loops **98**
7.1 The `for` Loop Statement . . . 98
7.2 The `continue` and `break` Statements . . . 100
7.3 Nesting Loops . . . 101
7.4 Loops over Sequences . . . 102
7.5 Enumerating over Sequences . . . 105
7.6 The `while` Loop . . . 107
7.7 The `else` Statement at the Bottom of Loops . . . 109
7.8 Summary . . . 111

8 Functions **112**
8.1 Defining and Calling Functions . . . 112
8.2 Functions in Modules . . . 116
8.3 Unit Testing . . . 118
8.4 Function Arguments: Default Values, Passing them by Name, and Constructing them . 124
8.5 Functions as Parameters or Variables, `Callable`, and `lambdas` . . . 127
8.6 Summary . . . 131

9 Exceptions **132**
9.1 Introduction . . . 132
9.2 Raising Exceptions . . . 133
9.3 Handling Exceptions . . . 136
9.4 Testing Exceptions . . . 145
9.5 Built-in Exceptions . . . 148

Backmatter **149**

Glossary **154**

Acronyms **158**

Symbols **159**

Python Commands **160**

Bibliography **163**

Scripts **174**

Preface

This book tries to teach undergraduate and graduate students as well as high school students how to program with the Python programming language. It aims to strike a good balance between theory and practice, leaning more to the practice side. In particular, we try to teach programming together with some software engineering concepts. It is the firm opinion of the author that these two cannot be separated. Teaching programming alone without introducing tools such as static code analysis, unit tests, and enforcing principles such as code style and proper commenting will create bad programmers. So we discuss these aspects while working our way through the principles of programming.

This book is intended to be read on an electronic device. Please do not print it. Help preserving the environment.

This book is work in progress. It will take years to be completed and I plan to keep improving and extending it for quite some time.

This book is freely available. You can download its newest version from https://thomasweise.github.io/programmingWithPython/. This version may change since this book is, well, work in progress. The book is released under the *Attribution-NonCommercial-ShareAlike 4.0 International license* (CC BY-NC-SA 4.0). You can freely share it. You can also copy text or figures, as long as you cite the book as the original source, e.g., by using the following BibTEX:

```
@book{programmingWithPython,
 author = {Thomas Weise},
 title = {Programming with Python},
 year = {2024},
 publisher = {Institute of Applied Optimization,
              School of Artificial Intelligence and Big Data,
              Hefei University},
 address = {Hefei, Anhui, China},
 url = {https://thomasweise.github.io/programmingWithPython}
}
```

This book contains a lot of examples. You can find all of them in the repository https://github.com/thomasWeise/programmingWithPythonCode. You can clone this repository and play with these example codes.

The text of the book itself is also available in the repository https://github.com/thomasWeise/programmingWithPython. There, you can also submit issues, such as change requests, suggestions, errors, typos, or you can inform me that something is unclear, so that I can improve the book. Such feedback is most welcome. The book is written using LATEX and this repository contains all the scripts, styles, graphics, and source files of the book (except the source files of the example programs).

Copyright © 2024

Prof. Dr. Thomas Weise (汤卫思教授)
at the Institute of Applied Optimization (应用优化研究所, IAO)
of the School of Artificial Intelligence and Big Data (人工智能与大数据学院)
of Hefei University (合肥大学),
in Hefei, Anhui, China (中国安徽省合肥市)

Contact me via email to tweise@hfuu.edu.cn with CC to tweise@ustc.edu.cn.

https://thomasweise.github.io/programmingWithPython/programmingWithPython.pdf

This book was built using the following software:

```
Alpine Linux 3.20.3
Linux 6.5.0-1025-azure x86_64

python: 3.12.7
latexgit_py: 0.8.25
latexgit_tex: 0.8.5
pycommons: 0.8.57
pdflatex: pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2024)
biber: 2.20
makeglossaries: 4.54 (2024-04-03)
ghostscript: 10.04.0 (2024-09-18)

date: 2024-11-01 22:40:11
```

Chapter 1

Introduction

1.1 Why Programming *and* Software Engineering Tools?

What is programming? Programming means to delegate a task to a computer. And it works just like delegating tasks in real life.

Our economy and society as a whole works because of the division of labor. In a factory or company, different works carry out different duties. If you are a chef in a kitchen, you have to tell the junior trainee chef: "First you wash the potatoes, then peel the potato skin, then you wash the potatoes again, and then you cook them." If you are visiting the hairdresser to get your hair done, you would say something like: "Wash my hair, then cut it down to 1cm on the top, trim the sides, then color it green."

In either case, you provide the other person with a clear and unambiguous sequence of instructions in a language they can understand. In this book, you will learn to do the same — with computers.

Definition 1.1 (Computer Program) A *computer program* is an unambiguous sequence of computational instructions for a computer to achieve a specific goal.

Definition 1.2 (Programming) *Programming* is the activity or job of writing computer programs [109].

Now, in the vast majority of situations, we do not create a program to just use it one single time. Actually, this is similar to the real life situation of work delegation again: If you were a chef, you basically "input" the "program" *cook potatoes* into the junior trainee once. In the future, you would like to be able go to them and invoke this program again by saying: "Please cook 2kg of potatoes."

Indeed, your "programs" often even have implicit parameters, like the quantity of 2kg mentioned above. Or, maybe, you go to the hairdresser again and want to say: "Same as usual, but today color it blue." In our day-to-day interactions, creating reusable and parameterized programs happens very often and very implicitly. We usually do not think about this in any explicit terms.

But if we write programs for computers, we *do* need to think about it in explicit terms and right from the start. The activity of pure programming listed in Definition 1.2 is only one part of software development. Imagine the following scenario:

In this book we discuss how to translate some kind of specification into `Python` code. Let's say that later in your job, you want to develop a program that can be used to solve a specific task. So, well, you write the program. You learned how to do that with this book. So you now have the file with the program code. The problem is solved.

Is that easy? On one hand, you may wonder whether you made any mistake. We are people, we all make mistakes. The more complex the task we tackle, the more (program code) we write, the more likely it is that we make some small error somewhere. So probably you want to *test* your program, i.e., check if it really computes the things that it should compute it the way you intended it to compute these things.

And what if your program is not just a single-use, stand-alone kind of program? What if it is part of some sort of software ecosystem? What if other programs may later need to use/run it? Maybe it

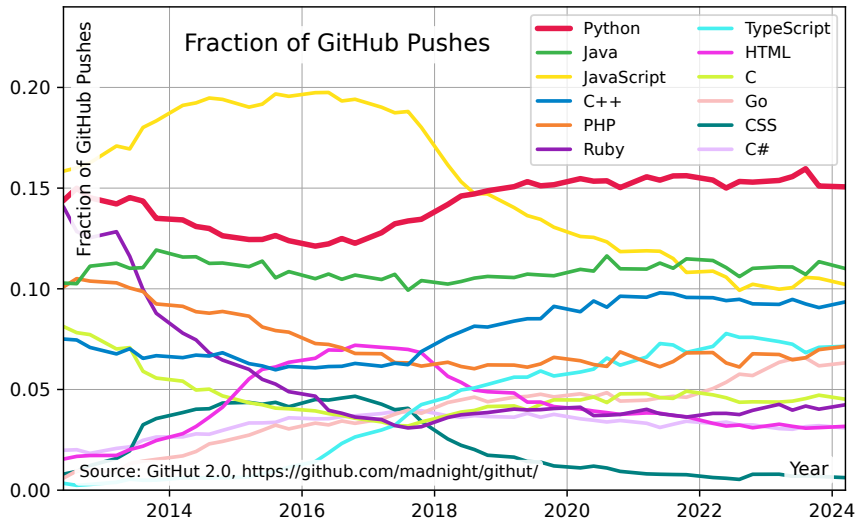


Figure 1.1: The twelve most popular programming languages chosen based on the GitHub pushes over the years. Source: [12].

can access some sort of sensor measuring something, or maybe it can convert data from one file format to another one. Then, other programmers may need to be able to at least understand how to correctly run the program and what kind of input and output data it will expect or produce, respectively.

What if this program is going to be needed for the next ten or so years? Maybe it could eventually become necessary to add new functionality? Maybe some of the software libraries you use get outdated and need to be replaced? There also are several situations in which a totally different person may need to work, read, modify, and improve *your* code.

Or maybe you are a researcher, using `Python` 3 to implement some algorithm and to run an experiment. In order to make your experiments and results replicable, you would probably want to publish your algorithm implementation together with the results. This only makes sense if your code is at least a bit readable.

All of these things need to be considered when we learn how to program. Because you do not just "program," you develop software.

In typical courses, such aspects are ignored. In a typical course, you learn how to write a small program that solves a certain task. The task is usually simple, so the teachers are able to read your code even if you have ugliest style imaginable. Also, nobody is ever going to look at or use the programs you write as your homework again. In such a pristine scenario, none of the above really matters. But in reality, things are not always simple.

One may hope that, as one of the very first things, surgery students in medical school are taught to wash their hands before performing surgery. And you, our dear students, can expect us to teach you how to develop software properly. And we will try to do that. We will teach you how to write clean, well-documented, and properly tested programs. Right from the start. Such that all of your code will be reusable, readable, clear, clean, and *beautiful*.

As a result, this will be a course which is very practice centered. We will learn programming the *right* way.[1] The downside of this approach is that we will sometimes go off on tangents in the text. While I am introducing variables in Chapter 4, for example, I will also explain how to use a static code analysis tool designed to find type errors in variable use. Also, the text will often have references to best practices that clarify common approaches and different code hygiene concepts. Our goal will be to learn how to do things right from the start and not put things off to later.

1.2 Why Python?

The center of this course is the `Python` programming language. Our goal is to get familiar with programming, with the programming language `Python`, and with the tools and ecosystem surrounding it. This makes sense for several reasons.

[1] Well, at least, what I consider as the *right* way.

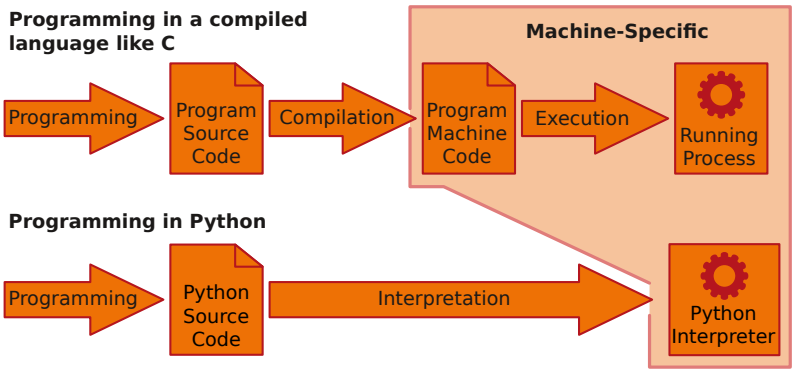


Figure 1.2: Python code is interpreted, which leads to an easier programming workflow compared to compiled programming languages like C.

First, Python is one of the most successful and widely used programming languages [21]. We plot the number of pushes to GitHub over time for the most popular programming and web development languages in Figure 1.1. We find that Python became the leading languages at some point in 2018. Python is intensely used [21] in the fields of Artificial Intelligence (AI) [116], Machine Learning (ML) [125], and Data Science (DS) [51] as well as optimization, which are among the most important areas of future technology. If you will do programming in any future employment or research position, chances are that Python knowledge will be useful.

Second, there exists a very large set of powerful libraries supporting both research and application development in these fields, including NumPy [34, 54, 64], Pandas [9, 84], Scikit-learn [103, 111], SciPy [64, 144], TensorFlow [2, 78], PyTorch [102, 111], Matplotlib [59, 64, 100], SimPy [150], and moptipy [145][2], just to name a few. This means that for many tasks, you can find suitable and efficient Python libraries that support your work.

Third, Python is very easy to learn [50, 141]. It has a simple and clean syntax and enforces a readable structure of programs. Programmers do not need to declare datatypes explicitly[3]. Python has expressive built-in types likes lists, tuples, and dictionaries. The fact that Python is an interpreted language makes it somewhat slower compared to compiled languages like C. However, this also leads to a much easier workflow when experimenting and programming, as sketched in Figure 1.2. It also is possible to interactively write programs in an interpreter window. This means that you can execute commands in a terminal instead of needing to compile and run programs. These features, in sum, make Python a good choice for learning how to write programs.

[2] Yes, I list moptipy here, next to very well-known and widely-used frameworks, because I am its developer.
[3] at least during the first steps of learning

Part I

Basics

In this part of the book, we will gain the following abilities:

- read, write, and execute simple `Python` programs
- use some tools to help us looking for errors

In particular, we will learn how to use the `Python` console and the editor `PyCharm`. We will learn about the most important simple datatypes in `Python`, such as integer numbers, floating point numbers, strings, and Boolean values. We will learn how to store values in variables. And we will learn about the most important collection datatypes offered by `Python`, namely lists, tuples, sets, and dictionaries.

Chapter 2

Getting Started

This course should be a practical course, so we should get started with practical things right away. In order to do practical things, we need to have all the necessary software on our computer. What software is necessary to do Python programming? Well, first of all, Python. If Python is not yet installed on your machine, then you can follow brief installation guide in Section 2.1.

Now with the programming language Python alone, you cannot really do much – in a convenient way, at least. You need a nice editor in which you can write the programs. Actually, you want an editor where you can not just write programs. You want an editor where you can also directly execute and test your programs. In software development, you often work with a Version Control Systems (VCS) like Git. You want to do that convenient from your editor. Such an editor, which integrates many of the common tasks that occur during programming, is called an Integrated Development Environment (IDE). In this book, we will use the PyCharm IDE [140, 148]. If you do not yet have PyCharm installed, then you can work through the setup instructions outlined in Section 2.2.

Before we get into these necessary installation and setup steps that we need to really learn programming, we face a small problem: Today, devices with many different Operating System (OS) are available. For each OS, the installation steps and software availability may be different, so I cannot possibly cover them all. Personally, I strongly recommend using Linux [7, 55, 135] for programming, work, and research. If you are a student of computer science or any related field, then it is my personal opinion that you should get familiar with this operating system. Maybe you could start with the very easy-to-use Ubuntu Linux [25]. Either way, in the following, I will try to provide examples and instructions for both Ubuntu and the commercial Microsoft Windows [14] OS.

Once the necessary software is installed, we will here also learn how to write and execute our very first small Python program in Section 2.3This first program will just print "Hello World!" and then exit. As final step to get our environment up and running, we discuss how Python programs can be executed in the terminal and the Python console in Section 2.4

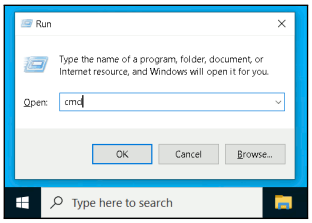
2.1 Installing Python

In order to learn and use Python, we first need to install it. There are two major versions of Python out there: Python 2 and Python 3. This book focuses entirely on Python 3. We assume that you have installed Python 3.12 or newer. We here provide some brief setup instructions. More help can be found at the following resources:

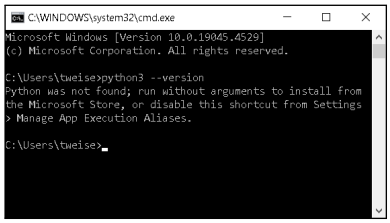
- the official Python setup and usage page [123],
- the Python Downloads at https://www.python.org/downloads/, and
- the Python 3 Installation & Setup Guide at https://realpython.com/installing-python/

2.1.1 Installing Python 3 under Microsoft Windows

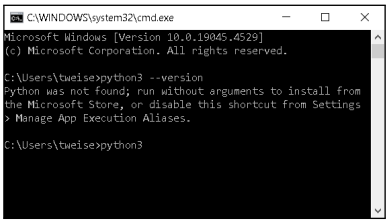
Example installation steps for Python on Windows (version 10) are sketched in Figure 2.1. First, you would open a terminal using and press [⊞] + [R], type in `cmd`, and hit [↵], as shown in Figure 2.1a. If Python is installed, then typing `python3 --version` in the terminal and hitting [↵] would print the version of the Python installation. If it is not installed, however, then Windows will print a message



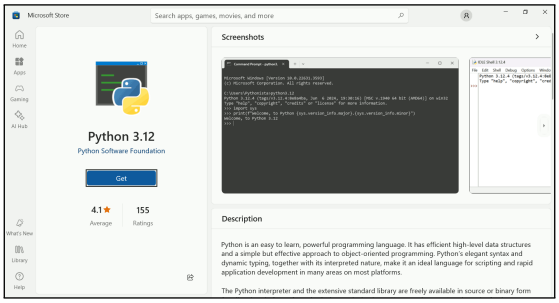
(a) Opening the terminal under `Windows`: press ⊞+R, type in `cmd`, and hit ↵.



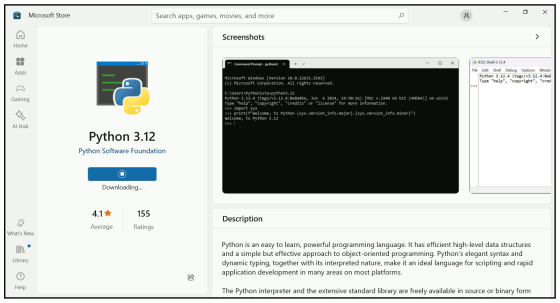
(b) Trying to get the `Python` version via `python3 --version`, but it is not installed.



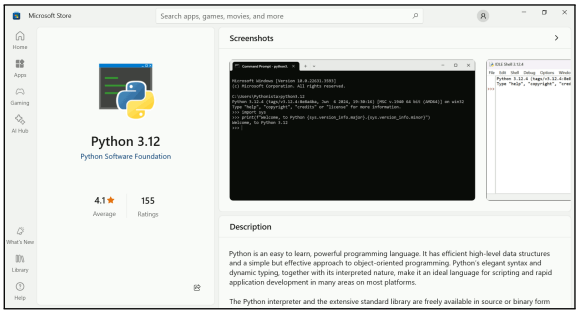
(c) Installing it by typing `python3` and hitting ↵.



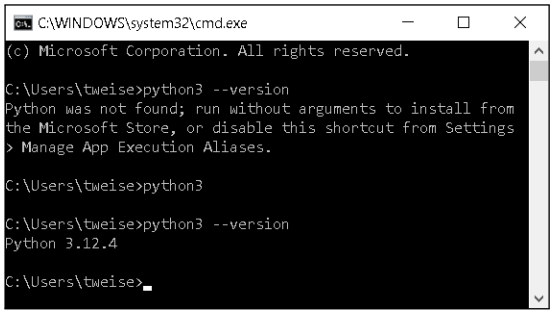
(d) The install screen, where we click Get.



(e) The install screen, downloading `Python`.



(f) The installation is finished.



(g) And the `python3 --version` command now works in the terminal.

Figure 2.1: Cropped screenshots of the installation steps for `Python` on `Windows`.

informing you that `Python` is not yet installed and that you can reach the web installer by just typing `python3` (and hitting ↵, of course).0 We do this in Figure 2.1c, which leads us to the installation screen (Figure 2.1d), where we need to press the Get button. This will then download (Figure 2.1e) and install `Python`. When this process is completed, the screen just shows nothing (Figure 2.1f). If we go back to the terminal and again try `python3 --version`, it will now work and print the version of our `Python` installation. In our case, this is version `3.12`.

Figure 2.2: Under my `Ubuntu Linux 22.04` system, typing `python3 --version` in a terminal and hitting return yields version `3.10.12`.

2.1.2 Python under Ubuntu Linux

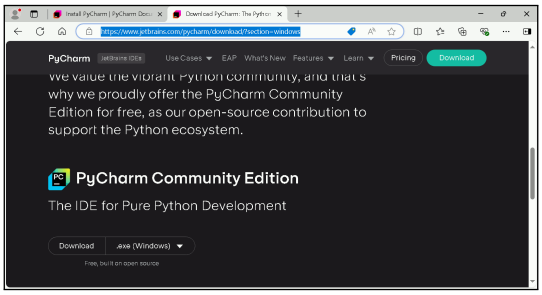
Under `Ubuntu Linux`, `Python` 3 is already pre-installed. You can open a terminal [7] by pressing Ctrl+ Alt+ T, then type in `python3 --version`, hit ↵, and get the result illustrated in Figure 2.2:

2.2 Installing PyCharm

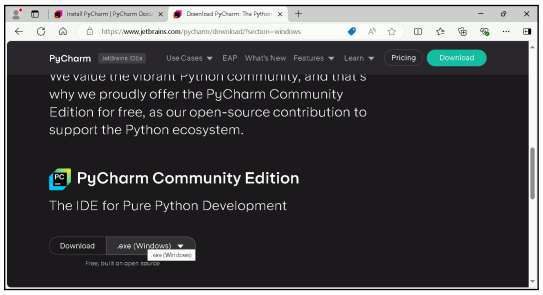
Just having a programming language and the corresponding interpreter on your system is not enough. Are you going to write programs in a simple text editor like a caveperson? No, of course not, you need an IDE, a program which allows you to do multiple of the necessary tasks involved in the software development process under one convenient user interface. For this book, I recommend using PyCharm, whose community edition is freely available. The installation guide for PyCharm can be found at https://www.jetbrains.com/help/pycharm/installation-guide.html.

2.2.1 Installing PyCharm under Microsoft Windows

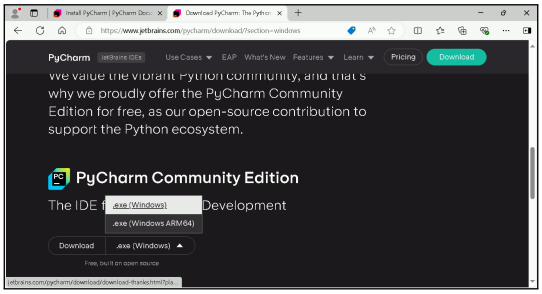
The process of installing PyCharm under Windows is illustrated in Figure 2.3. You first need to download the PyCharm Community Edition installation executable from https://www.jetbrains.com/pycharm/download. Make sure to download the Community Edition and nothing else, as shown in Figures 2.3a to 2.3c. Once the installer is downloaded, you start it and confirm that you wish to install PyCharm, as illustrated in Figures 2.3e to 2.3h. As Figures 2.3i to 2.3k show, the installation setup process is more or less automated, we just need to click Next here and there and finally click Install (Figure 2.3l). After the installation completes, we run PyCharm for the first time. Now we need to agree to the user agreement (Figure 2.3p) and should probably choose that we do not with to send any information about our PyCharm usage out (Figure 2.3q). Finally, as sketched in Figure 2.3r, we have a running and ready PyCharm IDE.



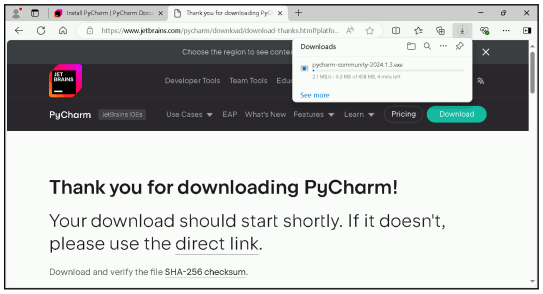
(a) The download website https://www.jetbrains.com/pycharm/download for PyCharm.



(b) Downloading the PyCharm Community Edition.

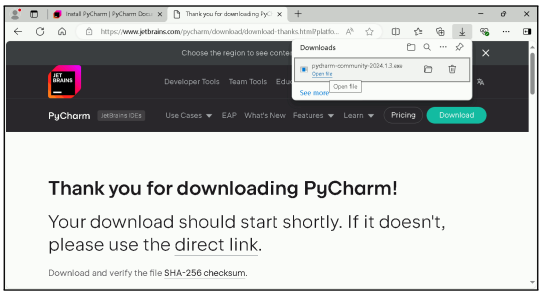


(c) Selecting the normal Windows installer for the PyCharm Community Edition.

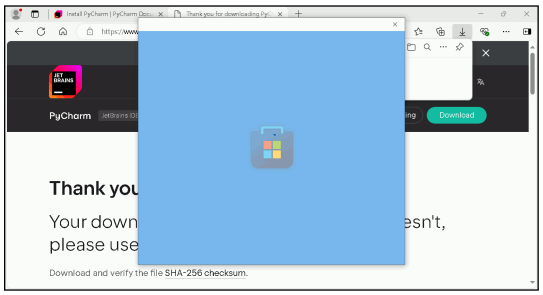


(d) The download is starting.

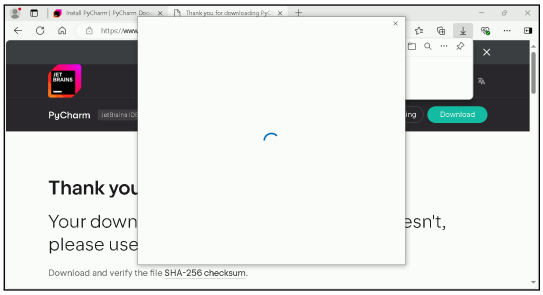
Figure 2.3: The installation steps of PyCharm under Windows.



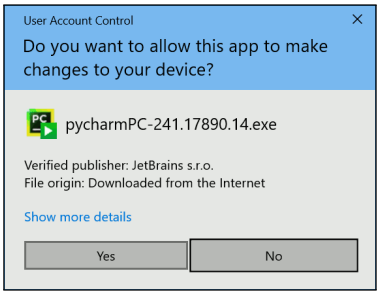
(e) The download is completed. We click Open file.



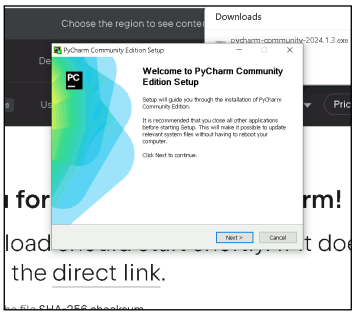
(f) The installer is starting.



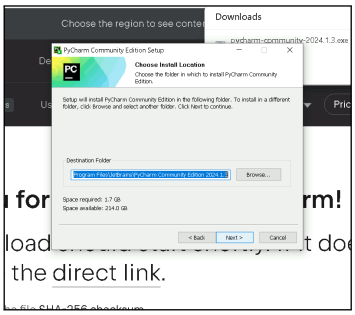
(g) The installer is starting.



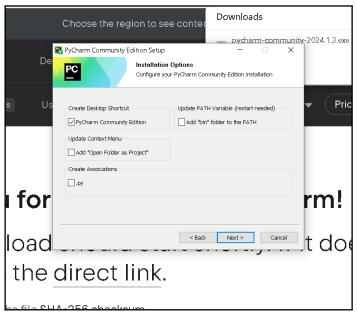
(h) We do want to install, so click Yes.



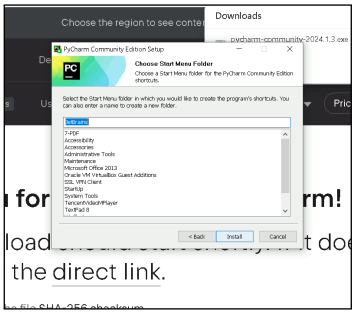
(i) The welcome screen of the installer. We click Next.



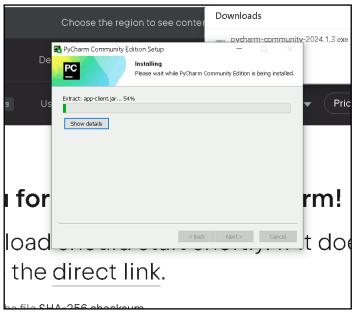
(j) The installation folder selection. We click Next.



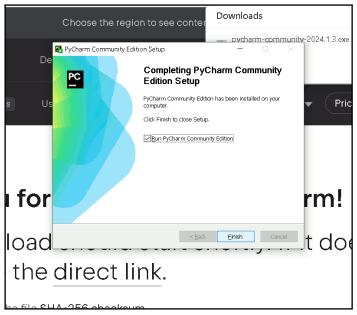
(k) The installation options. We can click Next.



(l) The start menu folder choose dialog. We click Install.

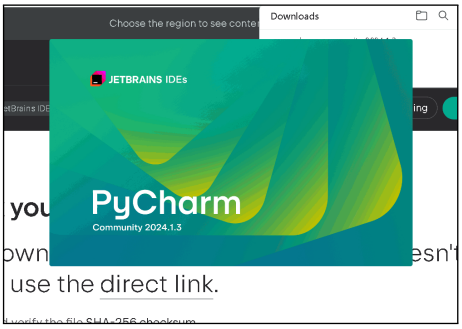


(m) The install process starts.

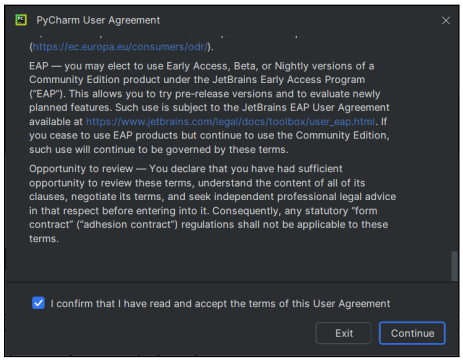


(n) The installation is finished. Select "Run PyCharm Community Edition" and click Finish.

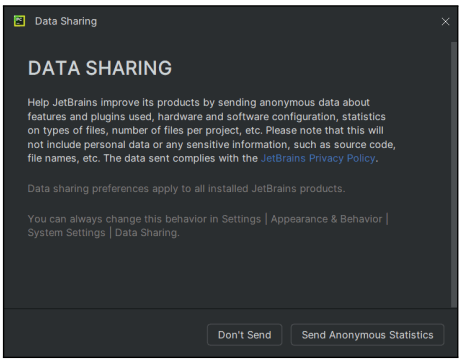
Figure 2.3: The installation steps of PyCharm under Windows (continued).



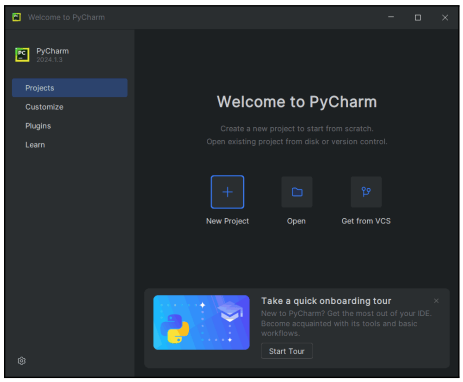
(o) The welcome screen of PyCharm.



(p) We read the user agreement, confirm that we read it, and click Continue.



(q) We do not want to send any data and click Don't Send.



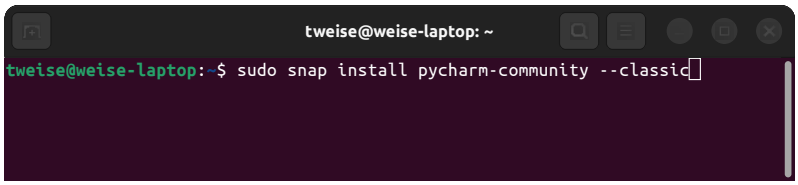
(r) Finally, PyCharm is ready to use.

Figure 2.3: The installation steps of PyCharm under Windows (continued).

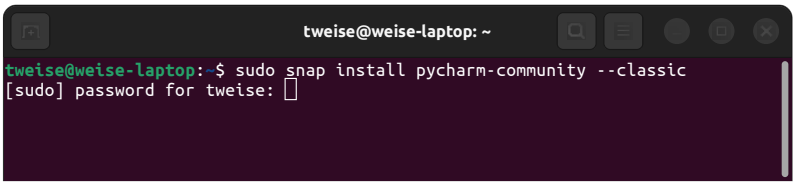
2.2.2 Installing PyCharm under Ubuntu Linux

Installing PyCharm under Ubuntu Linux is very easy, as illustrated in Figure 2.4. First, you open a terminal by pressing Ctrl + Alt + T. Then, enter the command `sudo snap install pycharm-community --classic` and hit return (Figure 2.4a). This installs the PyCharm software package and the necessary super user privileges are obtained via the pre-pended sudo, which will ask us to enter the root password, as sketched in Figure 2.4b. Then, the installation process basically runs automatically. Once it has completed (see Figure 2.4e), you can press the ⊞ key and type `pycharm` in the launcher window to find PyCharm (Figure 2.4f). A double-click will open PyCharm.

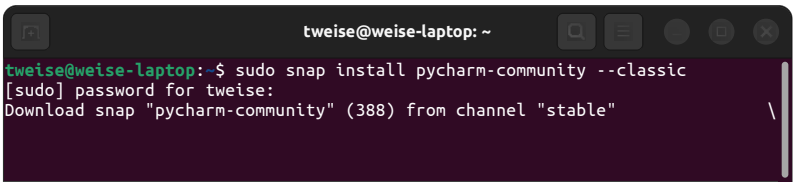
In the installation instructions for Windows, now a user agreement (Figure 2.3p) and data upload statement (Figure 2.3q) need to be performed. Since I already had PyCharm installed previously and probably already agreed/disagreed to them, respectively, these windows did not open in my current installation and I could not take screenshots of them. If they open, then you can probably treat them exactly as suggested in the Windows installation instructions Section 2.2.1. Either way, we finally get PyCharm up and running and can begin our coding (Figure 2.4h).



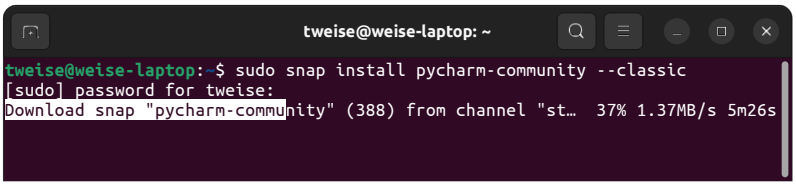
(a) Installing PyCharm using the snap install command in a terminal opened with Ctrl + Alt + T.



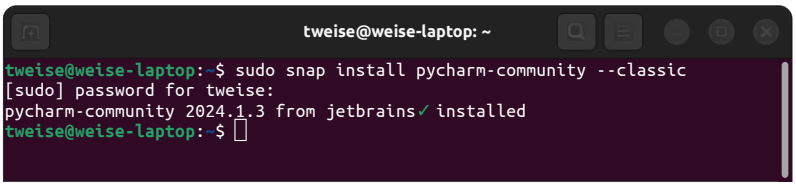
(b) This command requires the super user password.



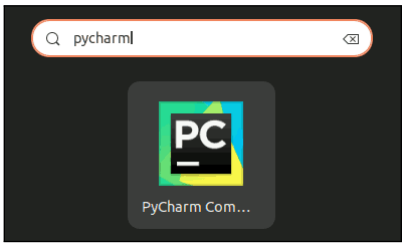
(c) The installation begins.



(d) The software package is downloaded.

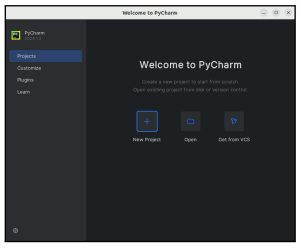


(e) The package is installed.



(f) Open the launcher by pressing ⊞ and type in pycharm to find the PyCharm executable, then double-click it.

(g) The PyCharm welcome screen appears.



(h) PyCharm has been started.

Figure 2.4: The installation steps of PyCharm under Ubuntu linux.

2.3 Our First Program

We now want to write and execute our very first Python program. This program should just print "Hello World!" to the text output and then exit. It therefore will consist of the single statement `print("Hello World!")`, as illustrated in Listing 2.1.

In PyCharm, we usually will not just create a single Python source code file. Instead, we will work in the context of *projects*, which can contain many Python files, settings, build scripts, and other resources. Inside such a project, we would create the Python file, write our source code from Listing 2.1 into it, and then run it. The steps for doing this are illustrated with screenshots in Figure 2.5, which were taken on my Ubuntu machine. On Windows, this will look very similar (maybe with the exception of "\" instead of "/" as directory separator characters in paths).

Since we started with a completely new PyCharm installation in Section 2.2, no project has yet been created. So when we open PyCharm, we will arrive at the project creation screen illustrated in Figure 2.5a. Here, we will New Project, which takes us to the second project creation screen shown in Figure 2.5b. In case that you already had PyCharm installed and already created some projects in the past, you can get to the same screen by clicking ≡ File New Project. Either way, when arriving at the screen, you will find several confusingly looking options. First, make sure that Pure Python is selected in the left pane. Then you can choose a name for the project in the Name: text box. For the sake of our example, let's call the new project `00_veryFirstProject`.

Below the name text box, you can select the destination directory in which the project folder should be created in the Location: box. You can ignore the text in the screenshot, as this is just the path that I am using on my Ubuntu machine. Instead, you will choose some suitable directory on your computer. (Notice that in Windows, the directory separator character is "\" and not "/" as on my Ubuntu Linux.) Either way, on my machine, I choose the rather elaborate path `home▸tweise▸local▸programming▸python▸programmingWithPythonCode`. PyCharm will create the folder `00_veryFirstProject` for our new project *inside* this directory. But, as said, you will choose some other suitable place on your computer.

The following options may not make any sense for you, which is totally OK for now. Please make sure to select Interpreter Type: Custom Environment as well as Environment: Select existing and Type: Python. Under Python Path:, you would choose the Python interpreter you have installed on your system (see Section 2.1). On my system that was "Python 3.10.12" at the time of this writing.[1] We finally click Create. We now have created our first and empty PyCharm Python project – as illustrated in Figure 2.5c.

We can now create the Python file to write our actual program code. To do this, we right-click on the folder `00_veryFirstProject` in our Project tree view pane on the left-hand side (see Figure 2.5d). In the menu that pops up, we select and click New Python File. This takes us the New Python file creation dialog illustrated in Figure 2.5e. Here, we enter the name for our first program. What could be more fitting than `very_first_program`? After hitting Enter, the file is created in our project folder and opened in the editor pane, as shown in Figure 2.5f.

We now enter the single line of code from Listing 2.1 into the editor, as illustrated in Figure 2.5g. We do not need to save the file, as PyCharm does this automatically for us.

Finally, we want to actually execute, i.e., run, our program. We can do this directly from the editor by pressing Ctrl + ⇧ + F10. We can also do it by right-clicking on the file in the project tree

[1] Now I am using Python 3.12, but this is not important.

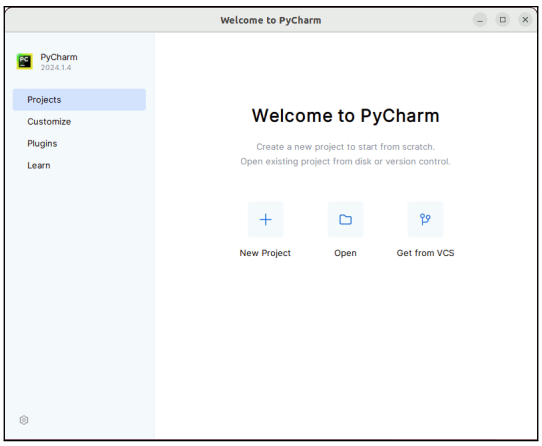
Listing 2.1: Our very first Python program, which just prints "Hello World!" (stored in file `very_first_program.py`; output in Listing 2.2)

```python
print("Hello World!")
```

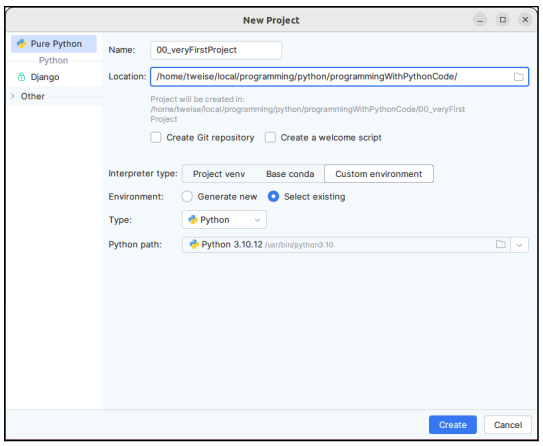
↓ `python3 very_first_program.py` ↓

Listing 2.2: The standard output stream (stdout) of the program `very_first_program.py` given in Listing 2.1.

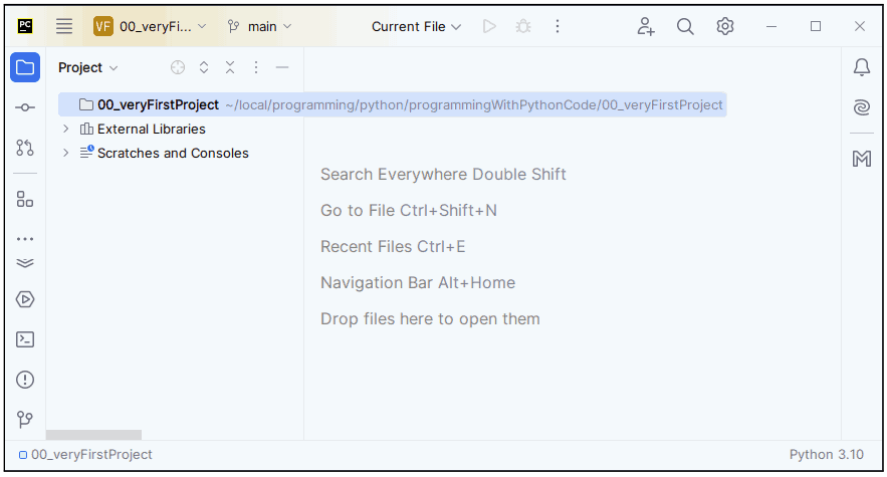
```
Hello World!
```



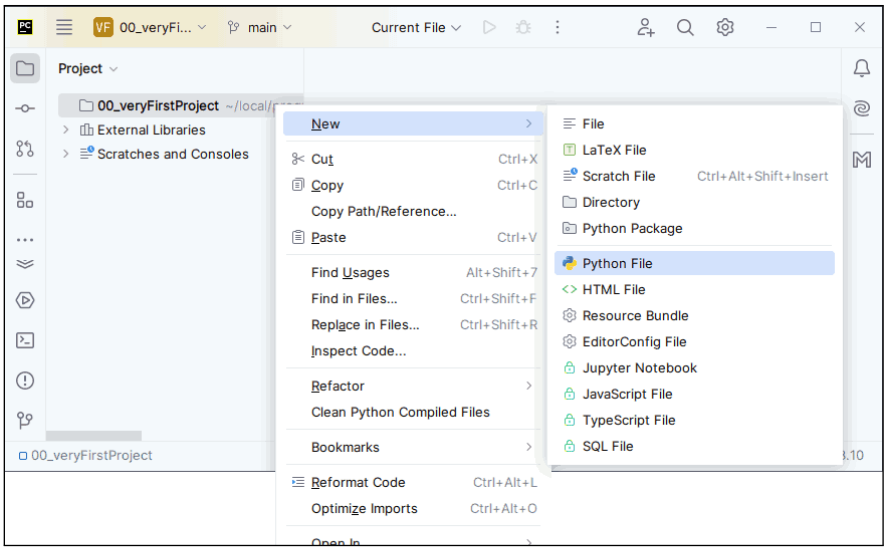
(a) Creating a new project in PyCharm, step 1: click New Project.



(b) Creating a new project in PyCharm, step 2: Make sure that Pure Python is selected in the left pane, then select a name for the project (here: 00_veryFirstProject), a directory location (some path on my Ubuntu machine, you will pick some other directory), and we select the current Python installation as Custom Environment. We finally click Create.

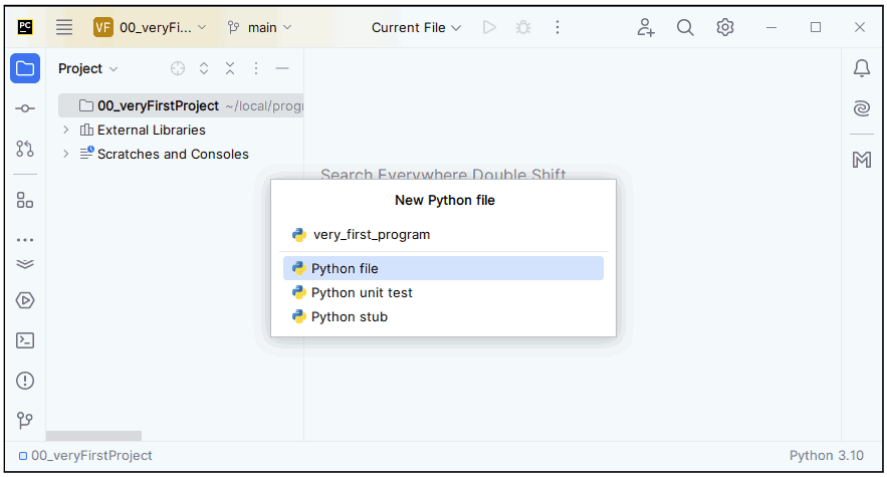


(c) The new and empty project has been created.

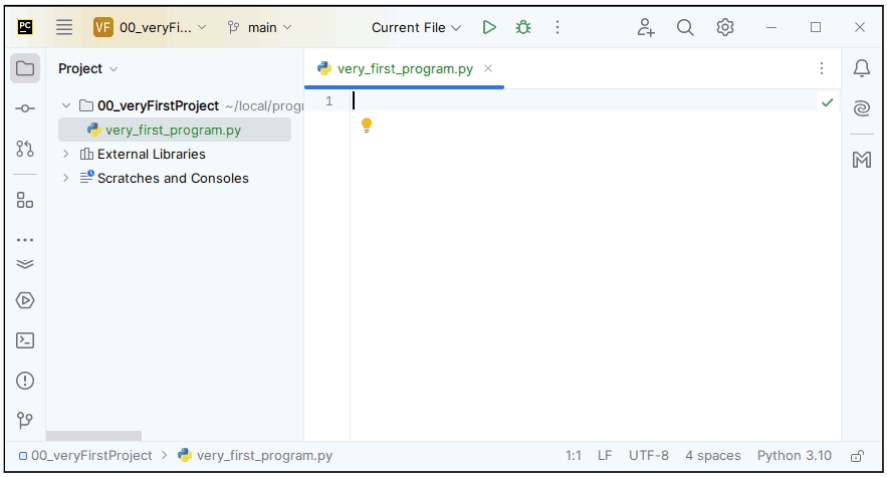


(d) We create a new Python file within this project by right-clicking on the project folder 00_veryFirstProject and selecting New ⟩ Python File.

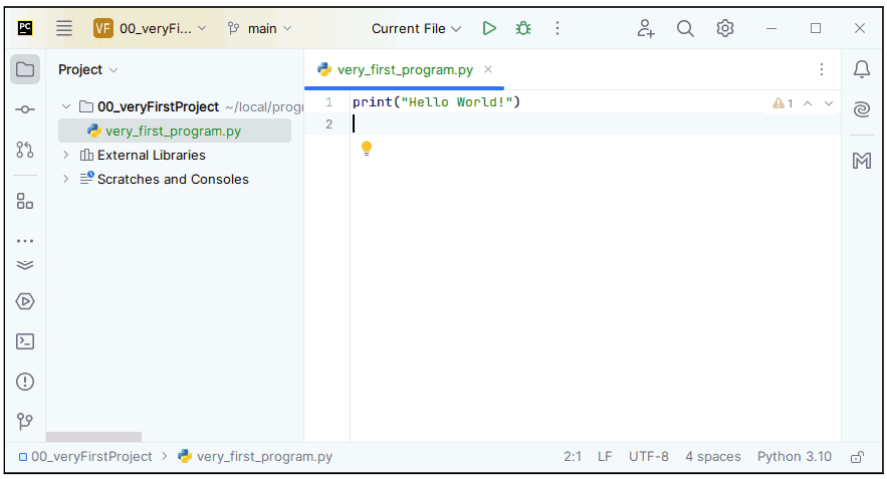
Figure 2.5: The steps to create a new Python file in a new PyCharm project and to then run it (Ubuntu).



(e) We enter a name for the new `Python` file (here: `very_first_program`) and hit Enter.

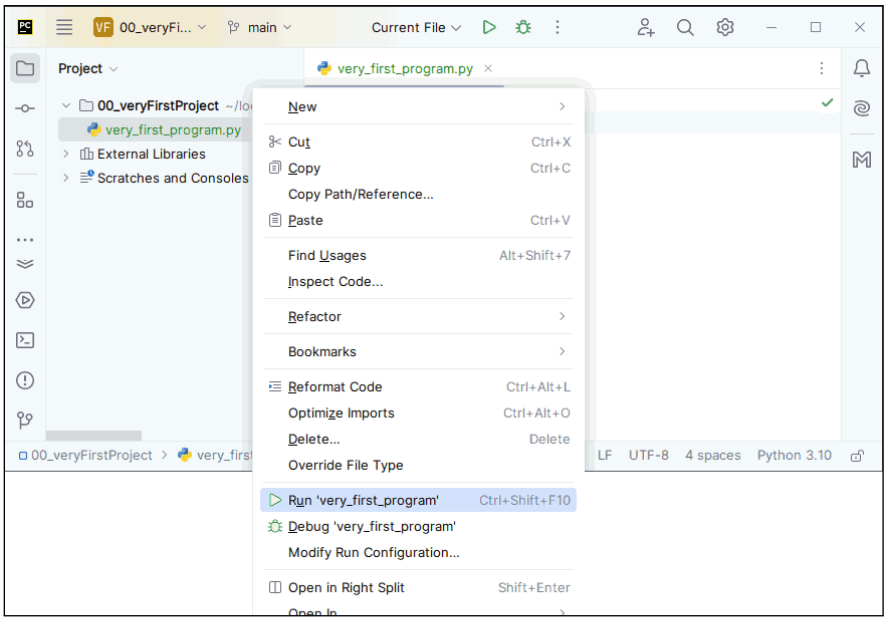


(f) The new and empty file `very_first_program.py` has been created in the project `00_very FirstProject`.

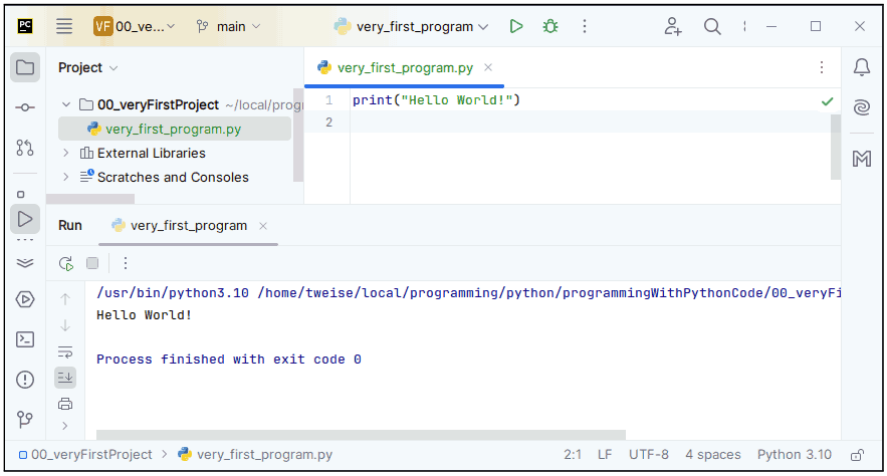


(g) We enter the text from Listing 2.1. `PyCharm` automatically saves it.

Figure 2.5: The steps to create a new `Python` file in a new `PyCharm` project and to then run it (`Ubuntu`, continued).



(h) In order to run this program, we right-click on the program file in the tree view and select Run 'very_first_program'. Alternatively, we could press Ctrl + ⇧ + F10.



(i) And indeed, in the console pane at the bottom of the PyCharm window, the text "Hello World!" appears.

Figure 2.5: The steps to create a new Python file in a new PyCharm project and to then run it (Ubuntu, continued).

view on the left-hand side and then clicking Run 'very_first_program' in the popup menu, as shown in Figure 2.5h.

Either way, a console with the title "very_first_program" opens at the bottom of our editor. And behold: Indeed, the text `Hello World!` appears.

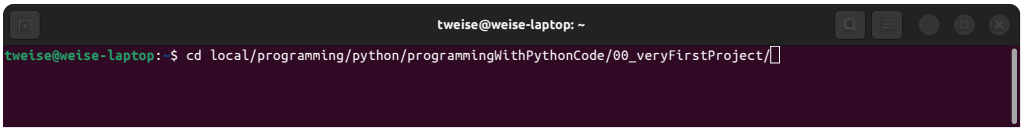
Well, before that text, we see the command line that was actually executed, namely the Python interpreter with our file's path as parameter. And after our program's output, we are notified that "Process finished with exit code 0," which means that the program has completed successfully and without error.

Congratulations. You now have written, saved, and executed your first ever Python program!

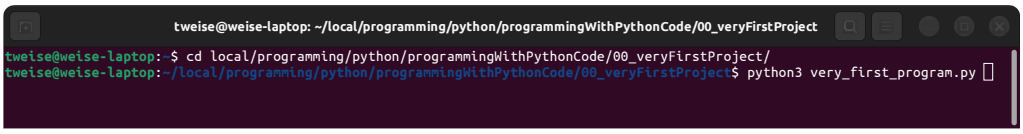
2.4 Python in the Terminal

In total, are four more ways in which we can execute a Python program:

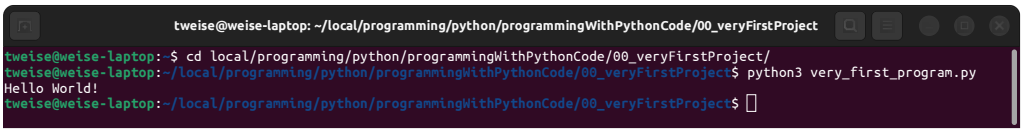
1. We can enter the program into a Python file in the PyCharm IDE and then run it from there. We



(a) Open a terminal (by pressing Ctrl + Alt + T under Ubuntu Linux; under Windows press ⊞ + R, type in cmd, and hit ↵). Change into the directory "directory" where your Python file is located, by typing cd directory and hit Enter.



(b) Execute a Python program "program.py" by typing python3 program.py and hit Enter. In our case, the program is "very_first_program.py".



(c) As expected, the text "Hello World!" appears in the terminal.

Figure 2.6: Example of executing a Python program in a terminal (on Ubuntu).

just did this in the previous section and illustrated it in Figure 2.5.

2. Actually, we can also write a Python program with a normal text editor. A Python program is just a normal text file, after all. We can execute such a text file by entering its directory and typing python3 programName (where programName is very_first_program.py, in our case) and hitting Enter. Then the program is executed directly in the terminal. This process is shown in Section 2.4.1 and Figure 2.6.

3. Alternatively, we could open the Python interpreter console in PyCharm and enter and execute our code line-by-line. This is sketched in Section 2.4.2 and Figure 2.7.

4. Besides using the Python console inside PyCharm, we can also open it inside a terminal. We can then enter separate Python instructions and run them there. This fourth option is outlined in Section 2.4.3 and Figure 2.8.

2.4.1 Executing a Python Program in a Terminal

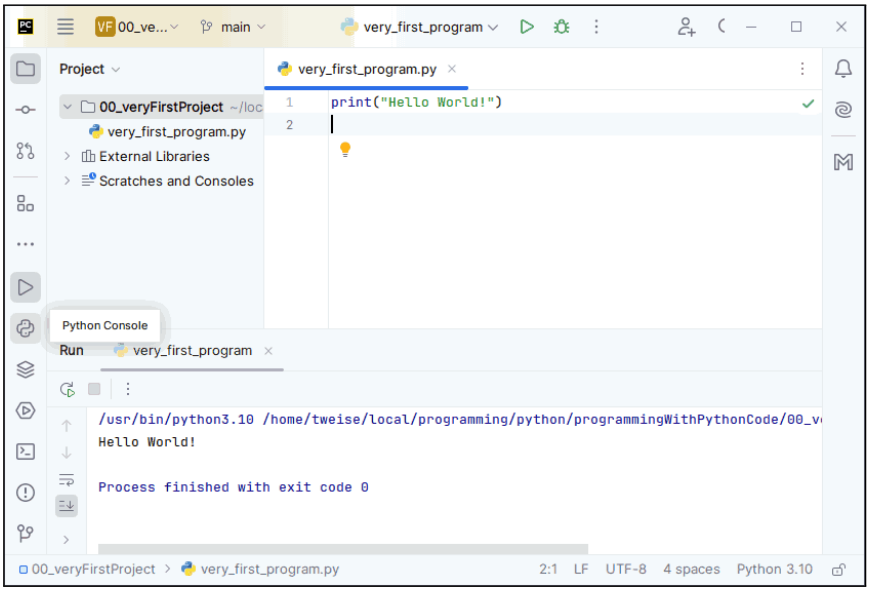
In order to directly execute a Python program in a terminal, we first need to open one. Under Ubuntu Linux, we simply press Ctrl + Alt + T. Under Windows, we have to press ⊞ + R, type in cmd, and hit ↵. Once the terminal is open, we need to change into the directory where the program is located. Under both Linux and Windows, this can be done by typing the command cd, followed by the path to the directory, and hitting Enter.[2] We provide a screenshot for that, taken under Ubuntu Linux, in Figure 2.6a. Now we simply call the Python interpreter by writing python3 followed by the file name of our program, which is very_first_program.py in our case. In Figure 2.6b we do this and hit Enter, which causes the Python interpreter to execute our program. The output "Hello World!" is then printed into the terminal in Figure 2.6c.

2.4.2 Entering Commands in the Python Console inside PyCharm

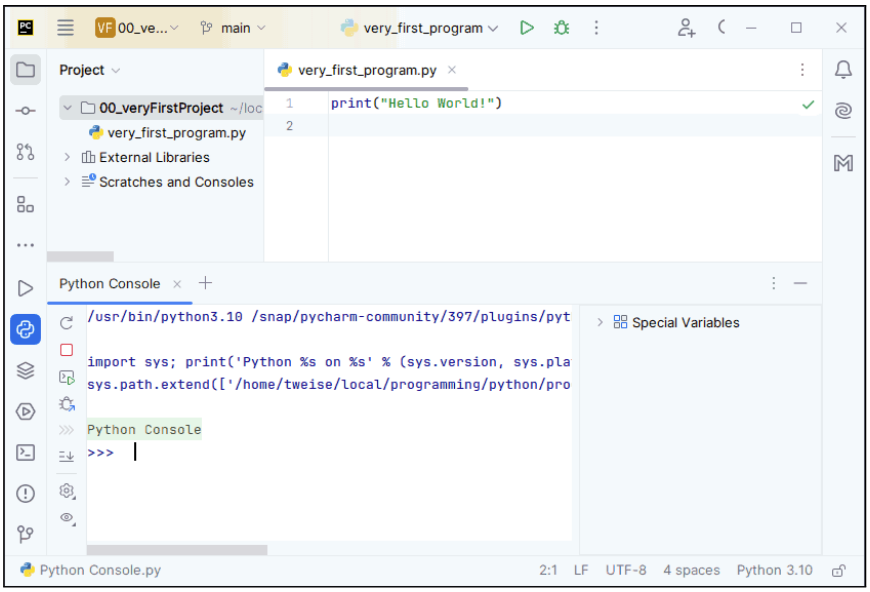
Besides writing programs in files and executing them, we can also directly enter them into the Python console and execute them step-by-step. This does not make sense if we want to reuse our programs later. But it does make a lot of sense when we just want to test some commands or functions or quickly test some idea. A Python console can be used directly in PyCharm (Figure 2.7) or opened in a terminal (Figure 2.8).

To explore entering Python code in the Python console inside PyCharm, we continue where we left of in Section 2.3. In PyCharm, we first click the on the vertical icon bar on the left side of the

[2] Under Windows, you may also need to change into the correct drive first.

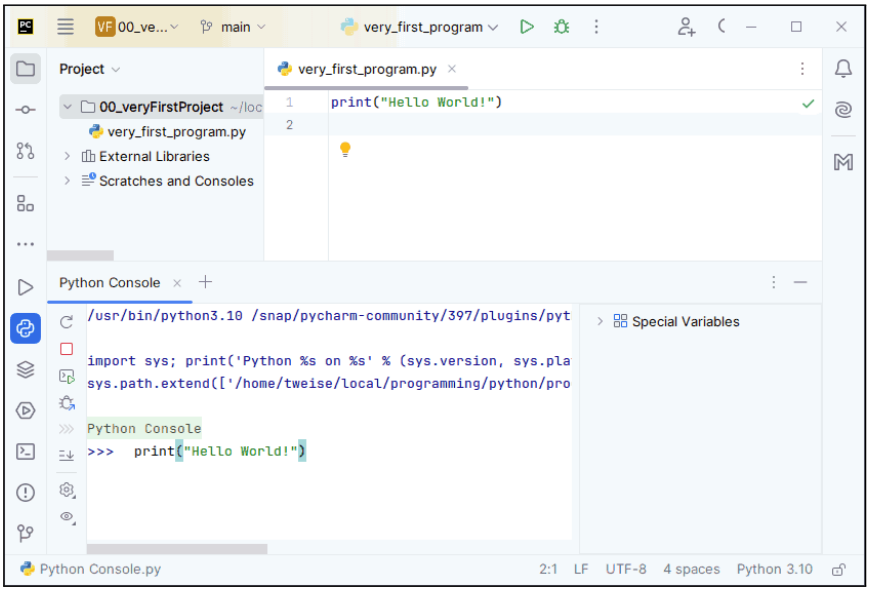


(a) Pressing the on the vertical icon bar on the left side of the PyCharm window.

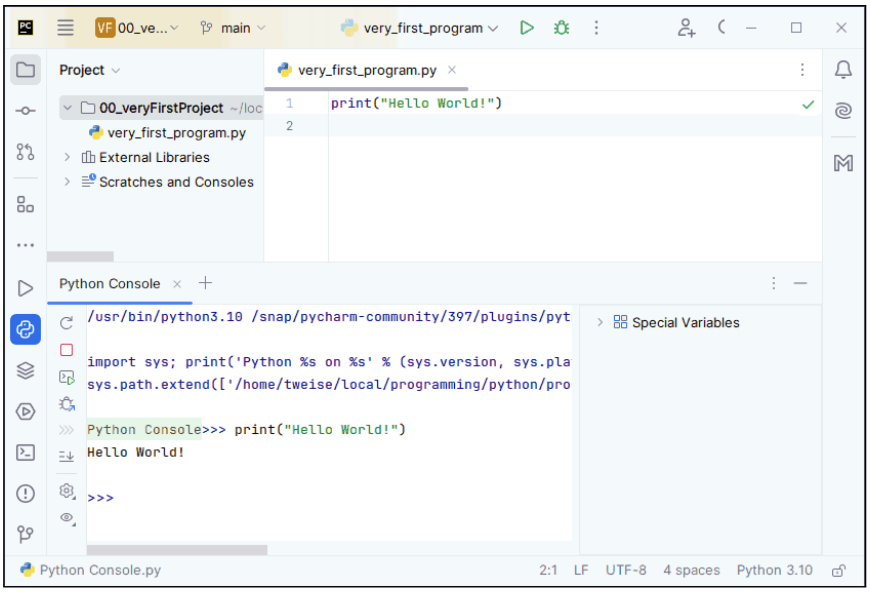


(b) The PyCharm Python console is open.

Figure 2.7: Entering the "Hello World!" program from Listing 2.1 directly into the Python console offered by PyCharm.



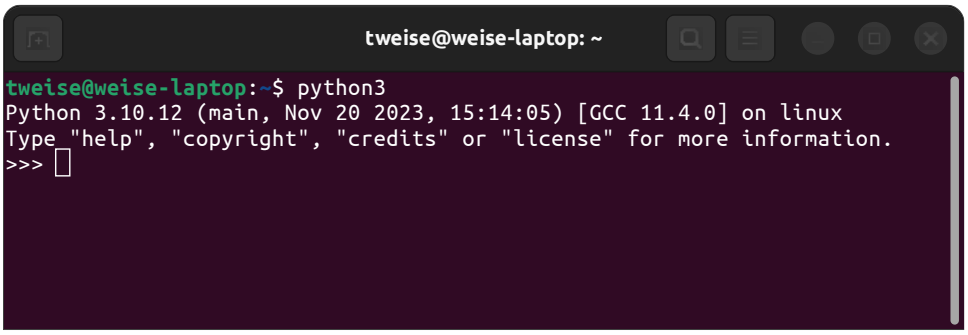
(c) We enter the "Hello World!" program from Listing 2.1 and press Enter.



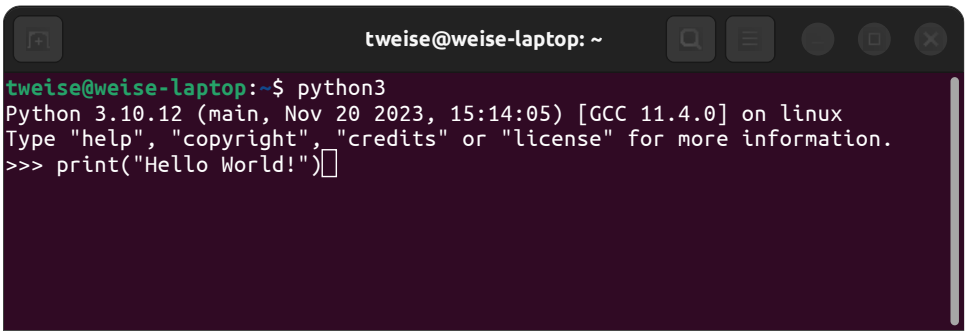
(d) And indeed, the output is "Hello World!".

Figure 2.7: Entering the "Hello World!" program from Listing 2.1 directly into the `Python` console offered by `PyCharm` (continued).

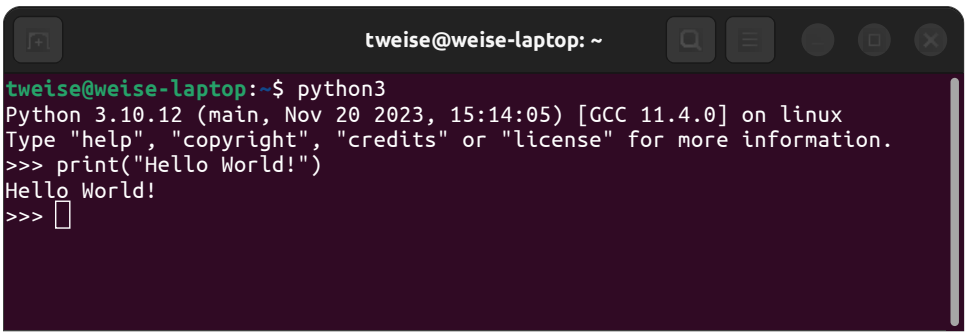
(a) Open a terminal (by pressing Ctrl + Alt + T under Ubuntu Linux; under Windows press ⊞ + R, type in cmd, and hit ↵), enter python3, then hit Enter.



(b) The Python console opens in the terminal.

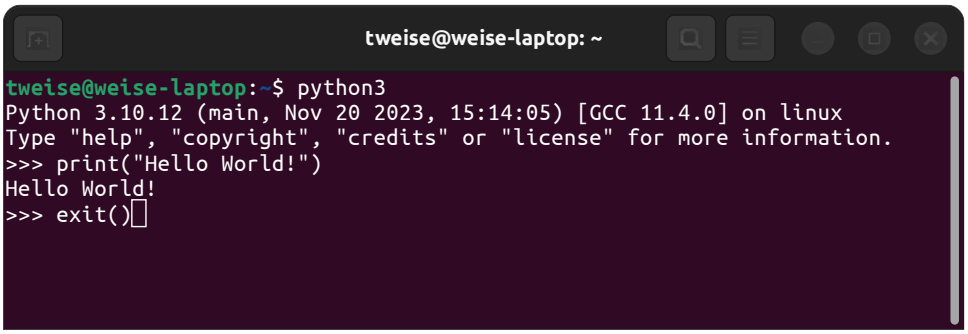


(c) We enter the "Hello World!" program from Listing 2.1 and press Enter.

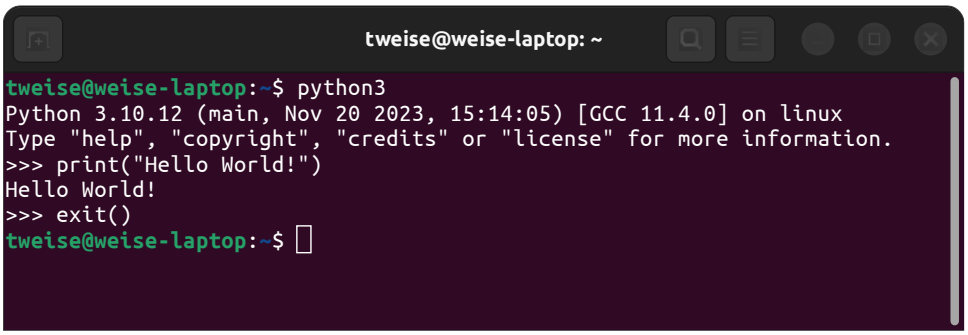


(d) And indeed, the output is "Hello World!".

Figure 2.8: Writing a program in the Python console in the terminal (Ubuntu).



(e) We exit the console by typing `exit()` and pressing Enter.



(f) We are back in the normal terminal.

Figure 2.8: Writing a program in the Python console in the terminal (Ubuntu, continued).

PyCharm window, as shown in Figure 2.7a. This directly brings us to the Python console (Figure 2.7b). We can enter the one-line-program from Listing 2.1, as illustrated in Figure 2.7c. Notice that the input prompt of the console is marked by the three greater characters `>>>` after which we enter our text. Pressing Enter after writing the code leads to the expected output shown in Figure 2.7d. This output directly appears in the console and is not preceded by any other text, in particular not by `>>>`, which makes it easy to visually distinguish what the input and output in a Python console are.

2.4.3 Entering Commands in the Python Console in a Terminal

Let us now open a Python console from the terminal instead of using the one in PyCharm. We therefore first need to open a normal terminal. Under Ubuntu Linux, we simply press Ctrl + Alt + T. Under Windows, we have to press ⊞ + R, type in `cmd`, and hit ↵. Either way, the terminal opens and we can enter `python3` and press Enter, as shown in Figure 2.8a. Now the Python interpreter starts right inside the terminal (Figure 2.8b). The prompt, i.e., the place where we can write our code, again is preceded by the `>>>` characters. As illustrated in Figure 2.8c, we copy the single line of code, `print("Hello World!")` from Listing 2.1 and press Enter. The output "Hello World!" is printed as expected in Figure 2.8d. However, we now are still in the Python interpreter. In order to leave it and to, maybe, enter other commands in the terminal, we have to use another new Python instruction: We type in `exit()` and press Enter, as shown in Figure 2.8e, which causes the Python interpreter to exit. We are now back in the basic terminal, as shown in Figure 2.8f. In these figures, I was using Ubuntu Linux. On Windows or other Linux variants, the process would have looked quite similar.

2.5 Summary

In this introductory section, we have performed the very first steps into the domain of Python programming. We have now a computer where both Python and the PyCharm IDE are installed. We can create program files and we can execute them in different ways. We are now ready to learn how to program.

Chapter 3

Simple Datatypes and Operations

3.1 Introduction

We now know how to create and run Python programs, both in the IDE and terminal. We have also already learned our first two Python commands:

- `print("Hello World!")` prints the text "Hello World!" to the output.
- `exit()` exits and terminates the Python interpreter.

Now, it would be very strange if the `print` function could print "Hello World!". That would not make much sense. `print` expects one parameter. This parameter cannot just be anything. It must be a text.

The command `exit`, on the other hand, can either have no parameter or one parameter. If it receives one parameter, this parameter will be the exit code of the program. Here, `0` indicates success. If no parameter is provided, this will be used as default value. We need to invoke `exit` if we use the Python console in the terminal explicitly. If we just run a program, then after the last instruction of the program was executed, then the interpreter will also terminate with exit code 0. Indeed, when we executed our first program in Section 2.3, we saw exactly that happen in Figure 2.5i. Different from the parameter of `print`, which must be some text, the parameter of `exit` needs to be a number.

We realize: Distinguishing different types of data makes sense. Sometimes we need to do something with text. Sometimes we want to do something with numbers. Somtimes, we want to just handle a decision which can be either "yes" or "no".

Of course, for these different situations, different possible operations may be useful. For example, when we use numbers, we may want to divide or multiply them. When we handle text, we may want to concatenate two portions of text, or maybe we want to convert lowercase characters to uppercase. We may want to do something if two decision variables are both "yes" or, maybe, if at least one of them is.

In this chapter, we will look into the simple datatypes of Python, namely:

- `int`: the integer datatype, which represents integers numbers $\mathbb{Z}$ (Section 3.2),
- `float`: the floating point numbers, i.e., a subset of the real numbers $\mathbb{R}$ (Section 3.3),
- `bool`: Boolean values, which can be either `True` or `False` (Section 3.4),
- `str`: strings, i.e., portions of text of arbitrary length (Section 3.5), and
- `None`: nothing, which is the result of any command that does not explicitly return a value (Section 3.6).

3.2 Integers

Integer arithmetic is the very first thing that you learn in mathematics in primary school. Integer arithmetic is also the very first thing you learn here. *Integer* is a Latin word that means "whole" or "intact." The integers include all whole numbers and negative numbers and zero, without fractions and decimals.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 4 + 3
7
>>> 7 * 5
35
>>> 4 + 3 * 5
19
>>> (4 + 3) * 5
35
>>> 4 - -12
16
>>> ((4 + 3) * (4 - -12) - 5) * 3
321
>>> 32 // 4
8
>>> 33 // 4
8
>>> 34 // 4
8
>>> 35 // 4
8
>>> 36 // 4
9
>>> 32 / 4
8.0
>>> 33 / 4
8.25
>>> 34 / 4
8.5
>>> 35 / 4
8.75
>>> 36 / 4
9.0
>>> 33 % 4
1
>>> 34 % 4
2
>>> 35 % 4
3
>>> 36 % 4
0
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.1: Examples for Python integer math in the console, part 1 (see Figure 3.2 for part 2).

In many programming languages, there are different integer datatypes with different ranges. In Java, a `byte` is an integer datatype with range $-2^7..2^7-1$, a `short` has range $-2^{15}..2^{17}-1$, an `int` has range $-2^{31}..2^{31}-1$, and `long` has range $-2^{63}..2^{63}-1$, for example. The draft for the C17 standard for the C programming language lists five signed and five unsigned integer types, plus several ways to extend them [108]. The different integer types of both languages have different ranges and sizes, and the programmer must carefully choose which she needs to use in which situation.

Python 3 only has one integer type, called `int`. This type has basically an unbounded range. The Python 3 interpreter will allocate as much memory as is needed to store the number you want.[1]

3.2.1 Integer Arithmetics

Now, what can we do with integer numbers? We can add, subtract, multiply, divide, modulo divide, and raise them to powers, for example.

In Figure 3.1, you can find some examples for this. Like back in Section 2.4.3, press Ctrl+Alt+T under Ubuntu Linux or press ⊞+R, type in `cmd`, and hit ↵ under Windows to open a terminal.

[1] Ok, the range is not actually *unbounded*, it is bounded by the amount of memory available on your computer... ...but for all intents and purposes within this book, we can assume that `int` $\equiv \mathbb{Z}$.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 ** 7
128
>>> 7 ** 11
1977326743
>>> 2 ** 63
9223372036854775808
>>> 2 ** 64
18446744073709551616
>>> 2 ** 1024
179769313486231590772930519078902473361797697894230657273430081157732675805500963132708477322407536021120113879871393357658789768814416622492847430639474124377767893424865485276302219601246094119453082952085005768838150682342462881473913110540827237163350510684586298239947245938479716304835356329624224137216
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.2: Examples for Python integer math in the console, part 2 (see Figure 3.1 for part 1).

After entering `python3` and hitting Enter, we can begin experimenting with integer maths. The lines with Python commands in the console begin with `>>>`, whereas the result is directly output below them without prefix string.

In the very first line of Figure 3.1, we enter `4 + 3` and hit Enter. The result is displayed on the next line and, as expected, is `7`. We then attempt to multiply the two integers `7` and `5` by typing `7 * 5` and hitting Enter. The result is `35`.

Python does not just support normal arithmetics as you have learned it in school, it also follows the operator precedence rules. If we type in `4 + 3 * 5`, it will compute $4 + (3 * 5) = 4 + 15 = 19$ and, hence, print `19`. We can also use parentheses and type in `(4 + 3)* 5`, which will be evaluated as, well $(4 + 3) * 5 = 7 * 5 = 35$, and we get `35`. Integers can be signed, so typing `4 - -12` yields Python16. Parentheses can be arbitrarily nested, so we can also compute `((4 + 3)* (4 - -12)- 5)* 3`, which evaluates to $((7 * 16) - 5) * 3 = (112 - 5) * 3 = 107 * 3 = 321$.

Division is a bit tricky in programming in general and in Python as well. There are *two kinds* of division in Python: Integer division, denoted by `//` and fractional division, denoted as `/`.

`32 // 4` yields `8`, because `4` fits `8` times into `32`. `33 // 4`, `34 // 4`, and `35 // 4` all still yield `8`, as `4` *completely* fits `8` times into these numbers (leaving some remainder left over). `36 // 4` then finally yields `9`. The results of the integer division operator `//` are always also `ints`.

Fractional division, however, returns `float` values, which we will explore in the next section (Section 3.3) in detail. For now, let's just say that they can represent fractional parts (to a limited precision), which is denoted by having a Python. in the text output of the numbers. Computing `32 / 4` thus yields `8.0`, `33 / 4` gives us `8.25`, `34 / 4` yields `8.5`, `35 / 4` results in `8.75`, and, finally, `36 / 4` returns `9.0`. Notice that the result of this division operator is always a floating point number, even if the number itself is an integer.

Best Practice 1 Always be careful with which division operator you use for `ints`. If you need an integer result, make sure to use `//`. Remember that `/` always returns a `float` (and see Best Practice 2), even if the result is a whole number.

Now above we have said that `33 // 4` yields the integer `8`. The remainder of this operation can be computed using the modulo division operator `%`, i.e., by typing `33 % 4`, which yields `1`. We also find that `34 % 4` yields `2`, `35 % 4` gives us `3`, and `36 % 4` is `0`.

As you will find in Figure 3.2, integers can also be raised to a power. For example, 2^7 is expressed as `2 ** 7` in Python (and yields `128`). `7 ** 11`, i.e., 7^{11} gives us 1 977 326 743 and shows as `1977326743` in the output.

In most programming languages such as Java and C, the largest integer type available off the shelf is 64 bits wide. If it is signed (can have negative values) like Java's `long`, it has range $-2^{63}..2^{63} - 1$. An unsinged 64 bit integer type, such as `unsigned long long` in C, would have range $0..2^{64} - 1$. In Python we can compute 2^{63} (`2 ** 63`), namely 9 223 372 036 854 775 808, and 2^{64} (`2 ** 64`), which

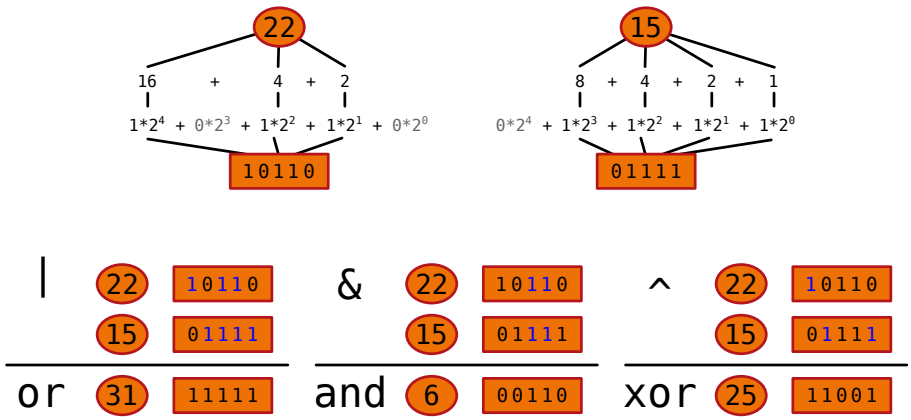


Figure 3.3: Examples for how integer numbers are represented as bit strings in the computer (upper part) and for the binary (bitwise) operations *and*, *or*, and *exclusive or* (often called *xor*).

is 18 446 744 073 709 551 616. These are very large numbers and the latter one would overflow the range of the standard integer types of `Java` and `C`. However, we can also keep going and compute `2 ** 1024`, which is such a huge number that it wraps four times in the output of our `Python` console in Figure 3.2! `Python` integers are basically unbounded. However, the larger they get, the more memory they need and the longer it will take to compute with them.

3.2.2 Operations on Bit Strings

First Time Readers and Novices: You are encouraged to skip over this section. This section is about integers numbers from the perspective of bit strings and the bit string based operations that we can apply to them. If you are learning programming, then this part is not important now. You can circle back to it later.

All integer numbers can be represented as bit strings. In other words, a number $z \in \mathbb{Z}$ can be expressed as $b_0 2^0 + b_1 2^1 + b_2 2^2 + b_3 2^3 + b_4 2^4 \ldots$, where the b_i-values each are either 0 or 1. Then, $\ldots b_4 b_3 b_2 b_1 b_0$ is a bit string. If we represent integers with such strings of five bits, then the number 1 would have representation 00001, because it is equivalent to 2^0. In in Figure 3.3, we illustrate that the number 22 would be 10110 because $22 = 2^4 + 2^2 + 2^1$ and the number 15 would correspond to 01111, as $15 = 2^3 + 2^2 + 2^1 + 2^0$.

We can obtain the binary representation of integer numbers as text using the `bin` function in `Python`. As shown in Figure 3.4, `bin(22)` yields `'0b10110'`. Here, the `0b` prefix means that the following number is in binary representation. We can also enter numbers in binary representation in the console. Typing `0b10110` corresponds to the number `22`. Similarly, `bin(15)` yields `'0b1111'` and entering `0b1111` into the console corresponds to entering the number `15`.

In `Python`, we can compute the bitwise (i.e., binary) *or*, *and*, as well as the and *exclusive or* of this binary representation of integers using the `|`, `&`, and `^` operators, respectively. Binary *or* returns an integer in which all bits are set to 1 which were 1 in either of its two operands. `22 | 1` yields `23`, because the bit with value 1 is not set in 22 and the binary *or* sets it (effectively adding 1 to 22). The slightly more comprehensive example `22 | 15` sketched in Figure 3.3 gives us `31`, because $22 = 2^4 + 2^2 + 2^1$ and $15 = 2^3 + 2^2 + 2^1 + 2^0$, which `|` combines to $31 = 2^4 + 2^3 + 2^2 + 2^1 + 2^0$, i.e., each power of 2 that occurred in either of the two operands is present in the result. `bin(31)` yields `'0b11111'`.

Binary *and* returns the integer where only the bits remain 1 that were 1 in *both* operands. Applying binary *and* instead of *or*, i.e., doing `22 & 1` results in `0`, because, as said before, the bit 2^0 is not set in 22. `22 & 15`, yields `6`, because only 2^2 and 2^1 appear both in 22 and 15. Thus, `bin(6)` corresponds to `'0b110'`.

The *exclusive or*, which is often called *xor*, will set a bit to 1 if it is 1 in exactly one of the two operands. Therefore, `22 ^ 1` gives `23`, since only the bit with value 2^0 is set in 1 and the other bits that are 1 in 22 are not. `22 ^ 15` yields `25`, because 2^4, 2^3, and 2^0 occur only once in the two operators (whereas 2^2 and 2^1 occured in both of them). This is confirmed by typing `bin(25)`, which results in `'0b11001'`.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> bin(22)
'0b10110'
>>> 0b10110
22
>>> bin(15)
'0b1111'
>>> 0b1111
15
>>> 22 | 15
31
>>> bin(31)
'0b11111'
>>> 22 & 15
6
>>> bin(6)
'0b110'
>>> 22 ^ 15
25
>>> bin(25)
'0b11001'
>>> 22 << 1
44
>>> bin(44)
'0b101100'
>>> 22 >> 2
5
>>> bin(5)
'0b101'
>>> hex(22)
'0x16'
>>> 0x16
22
>>> oct(22)
'0o26'
>>> 0o26
22
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.4: Examples for the binary representation of integers and the operations that apply to it.

Finally, we can also shift bit strings to the left or right by i places. The former corresponds to multiplying with 2^i, the latter is the same as an integer division by 2^i. Shifting `22` by one bit position to the *left* – which is done by entering `22 << 1` – therefore results in `44`. We already know that `bin(22)` is `'0b10110'` and so it comes at no surprise that `bin(44)` is `'0b101100'` (notice the additional `0` that appeared on the right hand side). Shifting `22` by two bit positions to the *right* – which is done by entering `22 >> 2` – results in `5`. The `10` on the right hand side of the binary representation disappeared, as `bin(5)` is `'0b101'`.

Besides the binary representation of integer numbers, which is to the basis 2, there also exists the hexadecimal representation (base 16) and the octal representation (base 7). We can obtain the hexadecimal representation of 22 by computing `hex(22)` and get `0x16`, which corresponds to $1 * 16^1 + 6 * 1 = 22$. We can also enter hexadecimal numbers in the console like `0x16`, which yields `22`. The octal representation of 22 is obtained as `oct(22)`, which produces `0o26`, which, in turn, corresponds to $2 * 8^1 + 6$. Similarly, this octal number can be entered as `0o26`. With this, our excursion into binary maths ends and we now welcome back all first-time readers.

3.2.3 Summary

In conclusion, the integer type `int` represents whole numbers. Integers can be positive or negative or zero. All the primitive mathematical operations like addition, subtraction, multiplication, and division that you learned in school can be applied to integers. The normal arithmetic precedence rules (that you also have learned in school) apply. Parentheses can be used to group operations. Finally, `Python` also provides the same binary logic operators, working on the bit string representation of integers, that you may or or may not know from other programming languages as well.

Figure 3.5: The structure of an 64 bit / double precision IEEE Standard 754 floating point number [56, 60].

3.3 Floating Point Numbers

In the previous section, we have discussed integers in `Python`. One of the very nice features of the `Python` 3 language is that integers can basically become arbitrarily large. There is only the single type `int` and it can store any integer value, as long as the memory of our computer is large enough.

In an ideal world, we would have a similar feature also for real numbers. However, such a thing cannot be practically implemented. You will certainly remember the numbers $\pi \approx 3.141\,592\,653\,590\ldots$ and $e \approx 2.718\,281\,828\,459\ldots$ from highschool maths. They are transcendental [44, 66, 96], i.e., their fractional digits never end and nobody has yet detected an orderly pattern in them. Since these numbers are "infinitely long," we would require infinitely much memory to store them *if* we wanted to represent them *exactly*. So we don't and neither does `Python`. We cannot really represent the real numbers $\mathbb{R}$ exactly in the memory of our computers.

3.3.1 How Floating Point Numbers Work

But how does it work in `Python`? How can we deal with the fact that we cannot dynamically represent fractional numbers exactly even in typical everyday cases? With `float`, `Python` offers us one type for fractional numbers. This datatype represents numbers usually in the same internal structure as `doubles` in the `C` programming language [97], which, in turn, internally have a 64 bit IEEE Standard 754 floating point number layout [56, 60]. The idea behind this standard is to represent both very large numbers, like 10^{300} and very small numbers, like 10^{-300}. In order to achieve this, the 64 bits are divided into three pieces, as illustrated in Figure 3.5.

> **First Time Readers and Novices:** You just need to understand that `floats` have limited precision. You can jump right to the next section.

The first part, the so-called significand or mantissa, consists of 52 bits, represents the digits of the number. 52 bits can represent $52 \log_2 10 \approx 15$ to 16 decimal digits, meaning that we can represent numbers to a precision of about 15 digits. If we would just use 52 bits, then this would limit us to represent numbers maybe from 0 to $2^{52} - 1$ at a resolution of 1. Of course, we could also choose some other resolution, say 0.001. In this case, we could represent numbers from 0 to $0.001 * (2^{52} - 1)$ and the smallest non-zero number would be 0.001 instead of 1. Whatever fixed resolution we would choose, it would be good in some cases and bad in others.

Therefore, the second part of the 64 bit floating point number representation comes into play: The 11 bits of the exponent represent a power of 2 which is multiplied to the significand to get the actual number. In order to allow us to have both small and large numbers, this value must be able represent positive and negative exponents. Therefore, the stored value of the exponent is taken and a bias of 1023 is subtracted. Thus, a stored value of 2000 in the exponent fields leads to an actual exponent of $1050 - 1023 = 27$, which would mean that the significand is multiplied with 2^{27}, i.e., 134 217 728.

Finally, the sign bit in the floating point number dataformat indicates whether the number is positive or negative. Together, this allows us to represent numbers from $2.225\,073\,858\,507\,201\,4 * 10^{-308}$ to $1.797\,693\,134\,8623\,157 * 10^{308}$ with a resolution of approximately 15 digits. Of course, the same range also applies to negative numbers and 0 can be represented as well. Indeed, there are even special floating point values for infinity and errors. But more about this later.

Luckily, you will never really need to know these exact information. The important thing to remember is: Floating point numbers can represent a wide range of different values. Their range is large but still limited. They can represent integers and fractions. However, their accuracy is always limited to about 15 digits. In other words, regardless whether your `float` stores a very large or a very small number, you can have at most 15 digits of precision. For example, adding 1 to 10^{16} would

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 6 / 3
2.0
>>> 1.0 + 7
8.0
>>> 5 - 3.6
1.4
>>> 2 * 3.0
6.0
>>> 6.5 % 2
0.5
>>> 3.3 ** 0.5
1.816590212458495
>>> ((3.4 * 5.5) - 1.2) ** (4.4 / 3.3)
45.43432339119718
>>> from math import pi, e
>>> pi
3.141592653589793
>>> e
2.718281828459045
>>> from math import sin, cos, tan, log
>>> sin(0.25 * pi) ** 2
0.4999999999999999
>>> cos(pi / 3)
0.5000000000000001
>>> tan(pi / 4)
0.9999999999999999
>>> log(e ** 10)
10.0
>>> from math import asin, acos, atan
>>> asin(sin(0.925))
0.9250000000000002
>>> acos(cos(-0.3))
0.30000000000000016
>>> atan(tan(1))
1.0
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.6: Basic arithmetics with floating point numbers in Python.

still yield 10^{16}, because only 15 digits are "stored" and the 1 will just "fall off." You cannot represent numbers arbitrarily precisely.

3.3.2 Floating Point Arithmetic

Floating point numbers in Python can be distinguished from `ints` by having a decimal dot in their text representation, i.e., `5.0` is a `float` and `5` is an `int`. Let us now look at some examples for the basic arithmetic operations available for `floats` in Figure 3.6.

We already learned that the division operator `/` always yields a `float` result. Therefore `6 / 3` yields `2.0` instead of `2`.

The normal arithmetic operations like addition, subtraction, multiplication, division, and powers all work with `floats` as expected. Remember, however, that if `float` and `int` numbers are mixed, the results are always `floats`. Thus, `1.0 + 7` gives us `8.0` and `2 * 3.0` yields `6.0`. In other words, if one `float` occurs somewhere in an expression, it will "infect" everything that it touches to become a `float` too, even if the result could be represented as `int`. Some results cannot be integers anyway, for example `5 - 3.6` evaluates to `1.4`. The remainder (the modulo) of a division can also be computed for floating point numbers. The remainder of the division of 6.5 by 2, i.e., `6.5 % 2` is `0.5`.

The square root of 3.3 can be computed as $3.3^{0.5}$. We can write this as `3.3 ** 0.5`, which yields `1.816590212458495`. This brings us back to the previous section: $\sqrt{3.3}$ is not actually 1.816 590 212 458 495. It is an irrational number [10, 118] and irrational numbers cannot be expressed as fractions of integer numbers (by definition, otherwise they would be rational numbers). Since they cannot be expressed as integer fractions, we cannot write them in any way like $\frac{1\,816\,590\,212\,458\,495\ldots}{1\,000\,000\,000\,000\,000\ldots}$, regardless of how large a denominator we would pick. Hence, we cannot represent them exactly using discrete binary values of our computer's memory. Hence, the floating point representation cuts off

somewhere. And this somewhere is after 15 decimal places.

We can of course also write and compute more complex mathematical expressions. `((3.4 * 5.5)- 1.2)** (4.4 / 3.3)` corresponds to $((3.4*5.5)-1.2)^{\frac{4.4}{3.3}}$ and yields `45.43432339119718`. This is again not an exact value but a rounded value. We always need to keep this in mind.

Let us recall our initial example of the transcendental irrational numbers π and e. Certainly, these are very important constants that would be used in many computations. We can make them accessible in our code by importing them from the `math` module.[2] This can be done by typing `from math import pi, e`. When we then type `pi` and `e`, we can get to see their value in floating point representations: `3.141592653589793` and `2.718281828459045`, respectively. Again, these are not the exact values, but they are as close as we can get in this format.

Of course, π and e alone are not that much useful. If you reach back into your highschool days again, you will remember many interesting functions that are related to them. Let us import a few of them, again from the `math` module, via `from math import sin, cos, tan, log`. I think you can guess what these functions do.

From highschool, you may remember that $\sin\frac{\pi}{4} = \frac{\sqrt{2}}{2}$ and thus $\sin^2\frac{\pi}{4} = 0.5$. Let us compute this in Python by doing `sin(0.25 * pi)** 2`. Surprisingly, we get `0.4999999999999999` instead of `0.5`. The reason is again the limited precision of `float`, which cannot represent $\frac{\sqrt{2}}{2}$ exactly. Similarly, $\cos\frac{\pi}{3} = \frac{1}{2}$ but `cos(pi / 3)` yields `0.5000000000000001` and $\tan\frac{\pi}{4}$ expressed as `tan(pi / 4)` returns `0.9999999999999999` instead of 1. Then again, these values are incredibly close to the exact results. They are off by *less than* 10^{-15} so for all practical concerns, they are close enough. Sometimes, we even get the accurate result, e.g., when computing $\ln(e^{10})$ by evaluating `log(e ** 10)`, which results in `10.0`.

As final example of floating point arithmetics, let us import the inverse trigonometric functions by doing `from math import asin, acos, atan`. Obviously, $\arcsin\sin 0.925$ should be 0.925. Calculating `asin(sin(0.925))` indeed yields `0.9250000000000002`. Due to the periodicity of the trigonometric functions, $\arccos\cos -0.3$ is 0.3 and `acos(cos(-0.3))` results in `0.30000000000000016`. For $\arctan\tan 1$ we even get the exact result `1.0` by computing `atan(tan(1))`.

> **Best Practice 2** Always assume that any `float` value is imprecise. Never expect it to be exact [8, 107].

Due to the limited precision, it could be that you add two numbers $c = a + b$ but then find that $c - a \neq b$, because some digit was lost. This is obvious when adding a very small number to a very large number. We only have about 15 digits, so doing something like $10^{20} + 1$ will usually work out to just be 10^{20} in floating point arithmetics [107]. But digits could also be lost when adding numbers of roughly the same scale, because their sum could just be larger so that the 15-digit-window shifts such that the least-significant digit falls off [8]...

3.3.3 Back to Integers: Rounding

We already learned that a single `float` value inside a computation that otherwise uses `ints` basically forces the whole result to become a `float` as well. But maybe sometimes we want to have an `int` as result. Therefore, we need a "way back" from `float` to `int`. Python offers several functions for this.

The first and maybe most common one is `round`. This function accepts a `float` and rounds it to the nearest integer. If two integer numbers are equally close, it rounds it to the one that is even. This can be very surprising, as we learned in school that $x.5$ should be rounded to $x + 1$. This will only happen with `round` if $x + 1$ is even. We find examples for the behavior of `round` in Figure 3.7. `round(0.4)` yields the integer `0`, as expected. `round(0.5)` returns `0` as well, which one may not expect – but `0` is even and `1` would be odd. `round(0.6)` gives us the integer `1`. If we compute `round(1.4)`, we still get `1`. However, doing `round(1.5)` gives us `2`. This is because `2` is even.

Three more common functions to turn `floats` to `ints` are given in the `math` module. We can import them via `from math import floor, trunc, ceil`. (Again, we will learn later how `import` actually works. For now, just accept that it makes some functions available.)

[2] We will learn about these mechanism in detail later on.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> round(0.4)
0
>>> round(0.5)
0
>>> round(0.6)
1
>>> round(1.4)
1
>>> round(1.5)
2
>>> from math import floor, trunc, ceil
>>> floor(0.4)
0
>>> floor(0.5)
0
>>> floor(0.6)
0
>>> floor(-0.4)
-1
>>> trunc(0.4)
0
>>> trunc(0.6)
0
>>> trunc(-0.4)
0
>>> trunc(-0.6)
0
>>> ceil(0.4)
1
>>> ceil(-11.1)
-11
>>> ceil(-11.6)
-11
>>> ceil(11.6)
12
>>> trunc(11.6)
11
>>> int(0.9)
0
>>> int(-0.9)
0
>>> int(11.6)
11
>>>
```

Figure 3.7: Rounding `float` values to `int` values.

`floor` rounds the `float` it receives to the nearest integer which is less than or equal to. `floor(0.4)`, `floor(0.5)`, and `floor(0.6)` all yield `0`. `floor(-0.4)` gives us `-1`.

`trunc` just discards any fractional digits and only returns the integer part of a number. Hence, `trunc(-0.6)`, `trunc(-0.4)`, `trunc(0.4)`, and `trunc(0.6)` all yield `0`.

Finally, `ceil` rounds to the nearest integer which is greater than or equal to its argument. `ceil(0.4)` yields `1` and `-0.4` results in `0`. `ceil(-11.1)` and `ceil(-11.6)` both result in `11`. `ceil(11.6)` gives us `12`.

You can also try to directly convert any datatype (that supports it) to an `int`. This works by simply putting it as argument to the function `int`. With `floats`, this works exactly as invoking `trunc`: `int(0.9)` and `int(-0.9)` both give us `1`. `11.6` yields `11`.

With this, we have several ways to turn `floats` to `ints`. However, especially with the function `round`, we need to be careful. It does *not* work as one would expect!

3.3.4 The Scientific Notation

Earlier on, we wrote that `floats` can represent numbers as large as 10^{300} or as small as 10^{-300}. This leads to the question how it would print such values on the console and how we can read them. While it would be hugely impractical to write a 1 followed by 300 zeros to represent 10^{300}, it would also be *wrong*. We also already know the reason for this: A `float` is accurate to 15 decimals. So basically, the first 15 zeros would be correct, but the values of the other digits are actually *undefined*.

β	γ	$\equiv$	α
A.BCDEFG...	e+HIJ	$\equiv$	A.BCDEFG... $* 10^{HIJ}$
A.BCDEFG...	e-HIJ	$\equiv$	A.BCDEFG... $* 10^{-HIJ}$
-A.BCDEFG...	e+HIJ	$\equiv$	$-$A.BCDEFG... $* 10^{HIJ}$
-A.BCDEFG...	e-HIJ	$\equiv$	$-$A.BCDEFG... $* 10^{-HIJ}$

Figure 3.8: The structure of the scientific notation for floating point numbers in Python, which represent a value α in the form $\beta * 10^{\gamma}$.

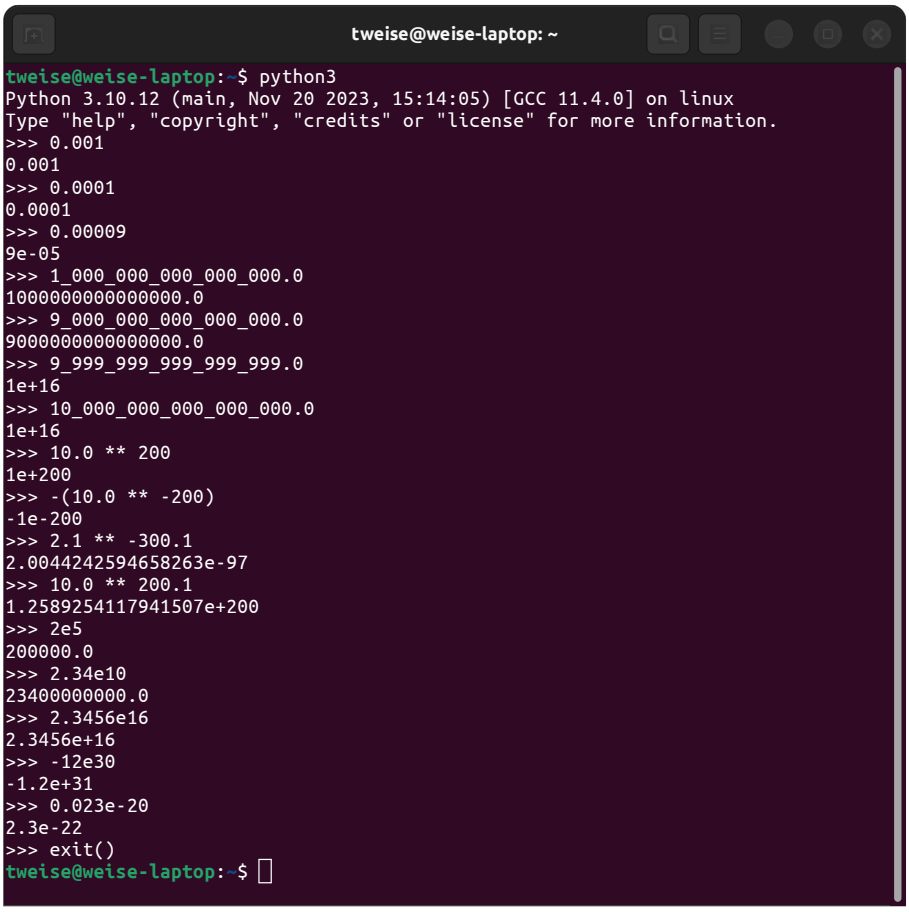


Figure 3.9: Examples for the scientific notation in Python.

Python, like many programming languages, solves this problem by using the *scientific notation* for floating point numbers. It uses this notation for any (absolute) `float` value smaller than 10^{-4} or larger than or equal to 10^{16}. Such numbers α are then represented in the form $\beta * 10^{\gamma}$ (such that $\beta * 10^{\gamma} = \alpha$, obviously). Since we cannot write this so nicely in a console, a lowercase e takes the place of the 10 and β and γ are written as normal text. In order to make sure that each number α has unique representation, it is defined that α must have exactly one non-zero leading digit, which may be followed by a decimal dot and then a fractional part. This notation is illustrated in Figure 3.8. *This notation only applies to `floats`, **not** `ints`.*

In Figure 3.9 we provide some examples for this notation. When we write `0.001` or `0.0001` in a Python console, the output is still this same number. However, `0.00009` is presented as `9e-05`, which stands for $9 * 10^{-5}$.Did you know that you are allowed to insert underscores (`_`) anywhere in a number (`int` or `float`)as a visual aid [15]? If not, you know now.

Best Practice 3 If you need to specify large integers or floats, using underscores (`_`) to separate groups of digits can be very helpful [15]. For example, `37_859_378` is much easier to read than `37859378`.

We can write 10^{15} as `float` by typing `1_000_000_000_000_000.0`. This equals the much less

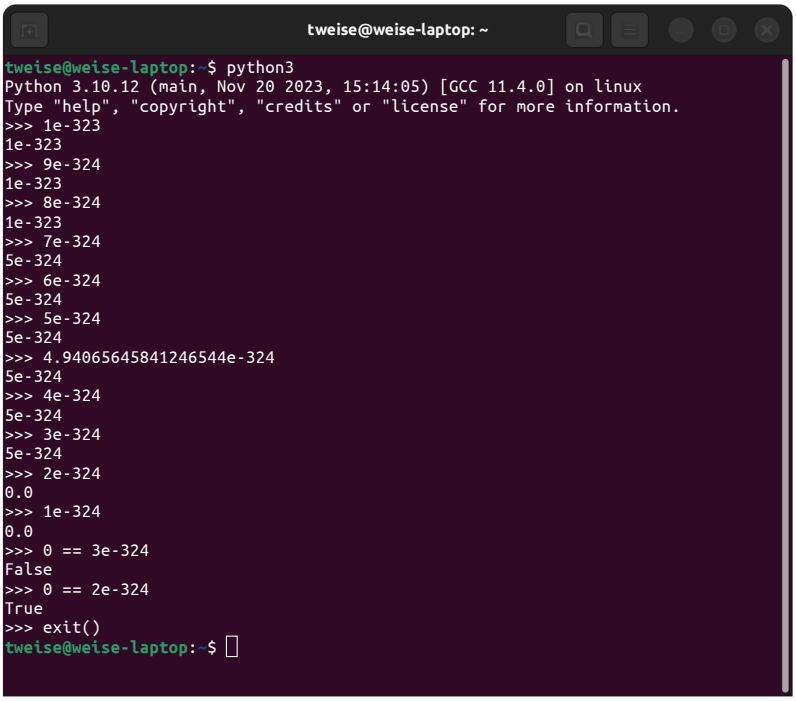


Figure 3.10: What happens with very small floating point numbers in Python?

readable `1000000000000000.0`. We can do the same for $9 * 10^{15}$ by writing `9_000_000_000_000_000.0` and get `9000000000000000.0`. However, if we write `9_999_999_999_999_999.0`, something interesting happens: We get `1e+16`, i.e., 10^{16}. This is because of the limited precision of the floating point numbers, namely the 15 digits often mentioned above. We cannot distinguish 10^{16} and $10^{16} - 1$ when using Python's `floats`. Indeed, writing `10_000_000_000_000_000.0` also yields `1e+16`.

This notation can also show really big numbers. For example 10^{200}, i.e., `10.0 ** 200`, shows up as `1e+200`. The really really small number -10^{-200} in turn, computed via `-(10.0 ** -200)`, is denoted as `-1e-200`.

The numbers are always scaled such that they begin with non-zero digit followed by the fractional part separated with a decimal dot, if need be. Examples for this are `2.1 ** -300.1` which yields `2.0044242594658263e-97` and `10.0 ** 200.1`, which turns up as `1.2589254117941507e+200`.

Of course, you can also input numbers in scientific notation. If you write `2e5`, this turns into `200000.0`. Of course, the number is stored as a `float` internally and this `float` does not know from which kind of text it was created. When it is turned back into text, it becomes a "normal number," because it is less than 10^{16}. And so does `2.34e10`, which becomes `23400000000.0`. `2.3456e16` however indeed remains `2.3456e16`.

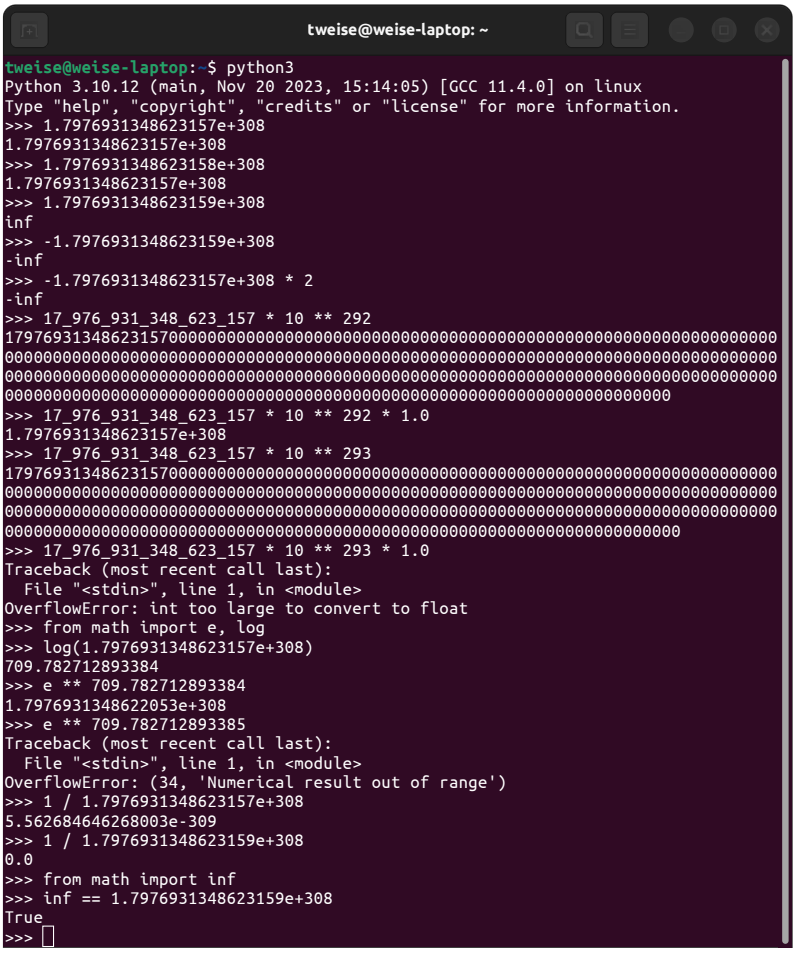
You can even violate the scientific notation a bit when entering numbers if you feel naughty. `-12e30`, for example, would better be written as `-1.2e+31`, which the Python console will do for you in its output. Similarly, `0.023e-20` becomes `2.3e-22`.

3.3.5 Limits and Special Floating Point Values: Infinity and "Not a Number"

We already learned that the floating point type `float` can represent both very small and very large numbers. But we also know that it is internally stored as chunk of 64 bits. So its range must be somehow finite. What happens if we exceed this finite range?

First the easy part: Now, we said that Python can store very small numbers, like 10^{-300}, in the `float` datatype. But how small really? Finding this out from the documentation is actually not so easy. Luckily, Java uses the same standard for its class `Double`. In its documentation [24], we find that the minimum value is 2^{-1074}, which is approximately $4.940\,656\,458\,412\,465\,44 * 10^{-324}$. So we would expect the smallest possible floating point value in Python to also be in this range.

In Figure 3.11, we take a look what happens if we approach this number. We use the scientific notation and begin to print the number `1e-323` (which is 10^{-323}). This number is correctly represented



```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 1.7976931348623157e+308
1.7976931348623157e+308
>>> 1.7976931348623158e+308
1.7976931348623157e+308
>>> 1.7976931348623159e+308
inf
>>> -1.7976931348623159e+308
-inf
>>> -1.7976931348623157e+308 * 2
-inf
>>> 17_976_931_348_623_157 * 10 ** 292
17976931348623157000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
>>> 17_976_931_348_623_157 * 10 ** 292 * 1.0
1.7976931348623157e+308
>>> 17_976_931_348_623_157 * 10 ** 293
17976931348623157000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000
>>> 17_976_931_348_623_157 * 10 ** 293 * 1.0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OverflowError: int too large to convert to float
>>> from math import e, log
>>> log(1.7976931348623157e+308)
709.782712893384
>>> e ** 709.782712893384
1.7976931348622053e+308
>>> e ** 709.782712893385
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OverflowError: (34, 'Numerical result out of range')
>>> 1 / 1.7976931348623157e+308
5.562684646268003e-309
>>> 1 / 1.7976931348623159e+308
0.0
>>> from math import inf
>>> inf == 1.7976931348623159e+308
True
>>>
```

Figure 3.11: What happens with very large floating point numbers in Python?

as `float` and shows up in the console exactly as we wrote it. However, if we go a bit smaller and enter `9e-324`, which is $9 * 10^{-324} = 0.9 * 10^{-323}$, we find that it again shows up in the console as `1e-323`. This is because the number is already subnormal [56, 60], i.e., uses only a few of the significand bits. The full precision of 15 digits is no longer available at this small scale. Thus `9e-324` and `1e-323` map to the same `float`. Converting this `float` to text yields `'1e-323'`. The same happens for `8e-324`, which also maps to `1e-323`. `7e-324`, however, is the same as `5e-324`. Matter of fact, `6e-324`, `5e-324`, `4e-324`, and `3e-324` all map to this same `float`.

It turns out that this number is already the smallest `float` that can be represented: `2e-324` simply becomes `0.0`. This value is simply too small to be represented as a 64 bit / double precision IEEE Standard 754 floating point number [56, 60]. The text `2e-324` that we enter into the Python console will therefore be translated to the `float` `0.0`. The comparison `2e-324 == 0.0` therefore results in `True`, while `3e-324 == 0.0` is still `False`.[3] So we learned what happens if we try to define very small floating point numbers: They become zero.

But what happens to numbers that are too big for the available range? Again, according to the nice documentation of Java, the maximum 64 bit double precision floating point number value is $(2-2^{-52}) * 2^{1023} \approx 1.797\,693\,134\,862\,315\,708 \cdots * 10^{308}$. We can enter this value as `1.7976931348623157e+308` and it indeed prints correctly in Figure 3.11. If we step it up a little bit and enter `1.7976931348623158e+308`, due to the limited precision, we again get `1.7976931348623157e+308`. However, if we try entering `1.7976931348623159e+308` into the Python console, something strange happens: The output is `inf`. `-1.7976931348623159e+308` gives us `-inf`. Multiplying this value by two, i.e., computing `-1.7976931348623157e+308 * 2`, still yields `-inf`. Intuitively, based on its name, one would assume that `inf` stands for "infinity" or ∞. However, it *actually* means *too big to represent as* `float` *or* ∞. If we enter numbers that are too big or exceed the valid range of `float` by multiplication, addition, subtraction, or division, we simply get `inf`. This does not actually mean "mathematical infinity," because,

[3] See Section 3.4 for more information on comparisons and the `bool` datatype with its values `True` and `False`.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from math import inf
>>> inf - 1
inf
>>> inf - 1e300
inf
>>> inf - inf
nan
>>> inf / 1e300
inf
>>> inf / inf
nan
>>> from math import nan
>>> nan + 1
nan
>>> nan + inf
nan
>>> nan == 1
False
>>> nan != 1
True
>>> 1 != 1
False
>>> nan == nan
False
>>> nan != nan
True
>>> inf == inf
True
>>> inf != inf
False
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.12: Not a Number, i.e., `nan`.

while `-1.7976931348623159e+308` is very big, it is not infinitely big. It simply means that the number is too big to put it into a 64 bit `float`.

Actually, the `int` type can represent larger numbers easily. `1.7976931348623157e+308` is equivalent to `17_976_931_348_623_157 * 10 ** 292`. This prints as a number with many zeros. Multiplying it with `1.0` yields a `float` with value `1.7976931348623157e+308`, exactly as expected. If we try a number ten times as large, i.e., `17_976_931_348_623_157 * 10 ** 293`, this is no problem with the `int`. But we can no longer convert it to a `float` by multiplying with `1.0`. Trying to do that with a ten times larger number, i.e., computing `17_976_931_348_623_157 * 10 ** 293 * 1.0` leads to an exception: The output is `OverflowError: int too large to convert to float`. An exception terminates the current flow of execution and signals an error. Later in Chapter 9, we will learn what `Exceptions` actually are and how to handle them properly and in Section 9.3.4, we will circle back to exactly this `OverflowError`.

The important thing to realize is that an overflow of a `float` may either lead to the `inf` value or to an error that stops your computation from continuing. As another example, let us again import the natural logarithm function `log` and the Euler's constant `e` from the `math` module by doing `from math import e, log`. We now can compute the natural logrithm from the largest possible `float` via `log(1.7976931348623157e+308)`. We get `709.782712893384`. Raising e to this power by doing `e ** 709.782712893384` leads to the slightly smaller number `1.7976931348622053e+308` due to the limited precision of the `float` type. However, if we try to raise e to a slightly larger power, and, for example, try to do `e ** 709.782712893385`, we again face an `OverflowError`.

We can also try to divide 1 by the largest `float` and do `1 / 1.7976931348623157e+308`. The result is the very small number `5.562684646268003e-309`. However, if we divide 1 by `1.7976931348623159e+308`, we get `0.0`. The reason is that `1.7976931348623159e+308` becomes `inf` and `1 / inf` is `0.0`.

Finally, `inf` also exists as constant in the `math` module. We can import it via `from math import inf`. And indeed, since the text `1.7976931348623159e+308` is parsed to a `float` value of `inf`, we find that `1.7976931348623159e+308 == inf` yields `True`.

Now, `inf` not actually being ∞ is a little bit weird. But it can get even stranger, as you will see in

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from math import isfinite, isinf, isnan, nan, inf
>>> isfinite(1e34)
True
>>> isfinite(inf)
False
>>> isfinite(nan)
False
>>> isinf(0.3)
False
>>> isinf(-inf)
True
>>> isnan(0.3455)
False
>>> isnan(inf)
False
>>> isnan(nan)
True
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.13: Checking for `nan` and `inf` via `isfinite`, `isinf`, and `isnan`.

Figure 3.12. `inf` is a number which is too big to be represented as `float` *or* ∞. So it is natural to assume that `inf - 1` remains `inf` and that even `inf - 1e300` remains `inf` as well. However, what is `inf - inf`? Mathematically speaking, this is very dodgy and $\infty - \infty$ as such is not a thing. Or *not a number*. Or `nan`, which stands for, well, Not a Number.

`nan` means that the result of a computation is neither a finite number or infinite. It is the result of shenanigans such as `inf - inf` or `inf / inf`. A `nan` value anywhere in a computation infects the result of the computation to also become `nan`. `nan + 1` remains `nan` and so does `nan + inf`.

The value `nan` is different from *any* value. `nan == 1` is `False` and `nan != 1` is `True` (`!=` checks for inequality). While all other float values are equal to themselves and – therefore – not different from themselves. Thus `1 != 1` is obviously `False`. In floating point mathematics, `inf == inf` is `True` and `inf != inf` is `False`[4]. However, `nan` is *really* different from really *anything*. Therefore, `nan == nan` is `False` and `nan != nan` is `True`!

Either way, the possible occurrence `inf` and `nan` in floating point computations is a cumbersome issue. If we want to further process results of a computation, we often want to make sure that it is neither `inf` nor `-inf` nor `nan`. This can be done via the function `isfinite`, which we can import from the `math` module, as you can see in Figure 3.13. `1e34` is a large number, but `isfinite(1e34)` is certainly `True`. `isfinite(inf)` and `isfinite(nan)` are both `False`. The function `isinf`, again from the `math` module, checks if a `float` is infinite. `isinf(0.3)` is therefore `False` whereas `isinf(-inf)` is `True`. Finally, we can check whether a `float` is `nan` via the `isnan` function from the `math` module. `isnan(0.3455)` and `isnan(inf)` are both `False`, whereas `isnan(nan)` is `True`.

3.3.6 Summary

In many situations, integer numbers are the way to go, for example when we count things or add discrete quantities. However, fractional numbers are also very common and we encounter them all the time in our daily life. Fractional numbers can be represented by the `float` datatype in `Python`.

`Python` offers us the very cool feature that basically arbitrary integer numbers can be represented by the `int` datatype. We can store very large numbers and are only limited by the memory available on our computer. Very large integer numbers are, however, something of a corner case ... they do not happen often.

Real numbers in $\mathbb{R}$ are a whole different beast. They include irrational numbers like $\sqrt{2}$ and transcendental numbers like π. These numbers are needed *often* and they have infinitely many fractional digits. Thus, there is no way to exactly represent them in computer memory exactly. Another problem is that we may need both very large numbers like 10^{300} and very small numbery like 10^{-300}.

Floating point numbers [52], provided as the `Python` datatype `float`, solve all of these problems

[4] Mathematicians may have mixed feelings about proclaiming that $\infty = \infty$ holds while $\infty - \infty$ is undefined...

(to some degree). They offer a precision of about 15 digits for a wide range of large and small numbers. 15 digits are more than enough for most applications. Many functions for floating point numbers, like logarithms and trigonometric functions, are offered by the `math` module.

Floating point numbers can be converted back to integers via rounding or truncating. Since writing out numbers such as 5.235^{212} in their full length would waste a lot of space and also make no sense, since only the highest 15 digits are actually stored and the rest would be random nonsense, the scientific notation was developed. It would represent this number as `5.235e+212`, which is quite readable if you know that the `e` in `eXXX` here just means 10^{XXX}.

However, when one deals with floating point numbers, a set of nasty problems can occur. For example, it could happen that we want to represent a number which is just *too big* of the available range. This number would be rendered as `inf`, which stands for both mathematical infinity ∞ and, well, "too big to be represented". In some cases, computations that would exceed the available range may also raise an `OverflowError`, which simply terminates them (we learn about this later). Numbers which are too tiny, on the other hand, simply become `0`. Finally, there are also situations where the result of a computation is neither too big nor too small and, yet, not an actual number. For example, a way to make mathematicians cry would be to try to compute `inf - inf` or `inf / inf`. The result is then `nan`, i.e., "Not a Number". The value `nan` has the interesting property that it is (to my knowledge) the only native `Python` value which is *different from itself*, i.e., `nan != nan`. The `math` module offers a set of functions to check whether a value is finite (`isfinite`), infinite (`isinf`), or `nan` (`isnan`).

When doing floating point computations, these are the issues that you should aware about. The precision and range is limited and strange things happen at these limits.

3.4 Boolean Values

Before, we already mentioned comparisons and their results, which can either be `True` or `False`. These two values constitute another basic datatype in `Python`: `bool`. They are fundamental for making decisions in a program, i.e., for deciding what to do based on data.

3.4.1 Comparisons

In the sections on `floats` and `ints`, we learned how to do arithmetics with real and integer numbers. You have learned these operations already in preschool. However, before you learn to calculate with numbers, you learned how to *compare* them. If we compare two numbers, the result is either `True`, if the comparison works our positively, or `False`, if it does not. `Python` supports six types of comparison:

- equal: $a = b$ corresponds to `a == b`,
- unequal: $a \neq b$ corresponds to `a != b`,
- less-than: $a < b$ corresponds to `a < b`,
- less-than or equal: $a \leq b$ corresponds to `a <= b`,
- greater-than: $a > b$ corresponds to `a > b`, and
- greater-than or equal: $a \geq b$ corresponds to `a >= b`.

How to use these operators is illustrated in Figure 3.14. It shows that `6 == 6` yields `True`, while `6 != 6` yields `False`. The expression `6 > 6` gives us `False`, but `6 >= 6` is `True`. `6 < 6` is also `False` while `6 <= 6` is, of course, `True`. While `5 > 6` is not `True`, `6 > 5` is. It is also possible to compare floating point numbers with integers and vice versa. `5.5 == 5` is `False`, while `5.0 == 5` is `True`.

Comparisons can also be chained: `3 < 4 < 5 < 6` is `True`, because `3 < 4` and `4 < 5` and `5 < 6`. `5 >= 4 > 4 >= 3`, however, is `False`, because while `5 >= 4` and `4 >= 3`, it is not true that `4 > 4`.

If we check the type of `True`, it yields `<class 'bool'>`, i.e., `bool`. The result of the expression `5 == 5` is a `bool` as well.

When talking about comparisons, there is one important, counter-intuitive exception to recall: The `nan` floating point value from Section 3.3.5. In Figure 3.12, we learned that `nan == nan` is `False` and `nan != nan` is `True`. This is the only primitive value (to my knowledge) which is not equal to itself.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 6 == 6
True
>>> 6 != 6
False
>>> 6 > 6
False
>>> 6 >= 6
True
>>> 6 < 6
False
>>> 6 <= 6
True
>>> 5 == 6
False
>>> 5 != 6
True
>>> 5 > 6
False
>>> 5 >= 6
False
>>> 6 > 5
True
>>> 6 >= 5
True
>>> 5 < 6
True
>>> 5 <= 6
True
>>> 6 <= 5
False
>>> 5.5 == 5
False
>>> 5.0 == 5
True
>>> 3 < 4 < 5 < 6
True
>>> 5 >= 4 > 4 >= 3
False
>>> type(True)
<class 'bool'>
>>> type(5 == 5)
<class 'bool'>
>>>
```

Figure 3.14: The results of basic comparisons are instances of `bool`.

a	b	a and b
False	False	False
False	True	False
True	False	False
True	True	True

(a) The truth table for the logical conjunction (*logical and*): `a and b`.

a	b	a or b
False	False	False
False	True	True
True	False	True
True	True	True

(b) The truth table for the logical disjunction (*logical or*): `a or b`.

a	not a
False	True
True	False

(c) The truth table for the logical negation (*logical not*): `not a`.

Figure 3.15: The truth tables for the Boolean operators `and`, `or`, and `not`.

3.4.2 Boolean Operators

The most common operations with Boolean values are the well-known Boolean logical operators `and`, `or`, and `not`. Their truth tables are illustrated in Figure 3.15.

- A Boolean conjunction, i.e., `and`, is `True` if and only both of its operands are also `True` and `False` otherwise, as shown in Figure 3.15a.
- A Boolean disjunction, i.e., `and`, is `True` if at least one of its two operands is `True` and `False` otherwise, as shown in Figure 3.15b.
- The Boolean negation, i.e., `not`, is `True` if its operand is `False`. Otherwise, it is `False`, as shown in Figure 3.15c.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> False and False
False
>>> False and True
False
>>> True and False
False
>>> True and True
True
>>> False or False
False
>>> False or True
True
>>> True or False
True
>>> True or True
True
>>> not True
False
>>> not False
True
>>> (True or False) and ((False or True) or (False and False))
True
>>> (5 < 4) or (6 < 9 < 8)
False
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.16: The `bool` values can be combined with the Boolean logical operators `and`, `or`, and `not`.

In Figure 3.16 we explore these three operators in the Python console. You can see that the operations can be used exactly as in the truth tables and yield the expected results. Additionally, you can of course nest and combine Boolean operators using parentheses. For example, `(True or False)and ((False or True)or (False and False))` resolves to `True and (True or False)`, which becomes `True and True`, which ultimately becomes `True`. You can also combine Boolean expressions like comparisons using the logical operators: `(5 < 4)or (6 < 9 < 8)` will be resolved to `(False)or (False)`, which becomes `False`.

3.4.3 Summary

Boolean values are very easy to understand and deal with. They can either be `True` or `False`. They can be combined using `and`, `or`, and `not`. And, finally, they are the results of comparison operators. Later, we will learn that Boolean decisions form the foundation for steering the control flow of programs.

3.5 Text Strings

The fourth important datatype in Python are text strings. Text strings are sequences of characters of an arbitrary length. In Python, they are represented by the datatype `str`. Indeed, we have already used it before, even in our very first example program back that simply printed `"Hello World"` in Listing 2.1 in Section 2.3. `"Hello World"` is such a text string.

3.5.1 Basic String Operations

As Figure 3.17 shows, there are two basic ways to specify a text string literal: Either enclosed by double quotes, e.g., `"Hello World!"` or enclosed by single quotes, e.g., `'Hello World!'`. The quotation marks are only used to delimit the strings, i.e., to tell Python where the string begins or ends. They are not themselves part of the string.

Best Practice 4 When defining a string literal, the double-quotation mark variant (`"..."`) may be preferred over the single-quotation mark variant (`'...'`); see also Best Practice 5.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> "Hello World!"
'Hello World!'
>>> 'Hello World!'
'Hello World!'
>>> "Hello" + ' ' + "World"
'Hello World'
>>> len("Hello")
5
>>> "Hello"[0]
'H'
>>> "Hello"[1]
'e'
>>> "Hello"[2]
'l'
>>> "Hello"[3]
'l'
>>> "Hello"[4]
'o'
>>> "Hello"[5]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
>>> "Hello"[6]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
>>> "Hello"[-1]
'o'
>>> "Hello"[-2]
'l'
>>> "Hello"[-3]
'l'
>>> "Hello"[-4]
'e'
>>> "Hello"[-5]
'H'
>>> "Hello"[-6]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
>>> "Hello"[0:3]
'Hel'
>>> "Hello"[1:3]
'el'
>>> "Hello"[2:]
'llo'
>>> "Hello"[1:-2]
'el'
>>> "Hello"[:-2]
'Hel'
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.17: Specifying string literals and indexing its characters.

One basic operation is string concatenation: `"Hello"+ ' ' + "World"` concatenates the three strings `"Hello"`, `" "`, and `"World"`. The result is `"Hello World"`. Notice how the singe space character string is needed, because `"Hello"+ "World"` would just yield `"HelloWorld"`.

Strings are different from the other datatypes we have seen so far. They are *sequences*, meaning that they are linear arrays composed of elements. These elements are the single characters, which correspond to letters, numbers, punctuation marks, white space, etc.

One basic set of things that we can do with strings is to extract these single characters. First, we need to know the length of a string. For this purpose, we can invoke the `len` function: `len("Hello")` is `5`, because there are five characters in "Hello". `len("Hello World!")` would give us `12`, because `"Hello"` has five characters, `"World!"` has six characters (the `"!"` does count!) and there is the single space character in the middle, so $5 + 6 + 1 = 12$. By the way, the empty string `""` has length `0`, i.e., `len("")` yields `0`.

Knowing the length of a string, we can now safely access its single characters. These characters are obtained using the square brackets `[]` with the character index inbetween. The character indexes start at 0. Therefore, `"Hello"[0]` returns the first character of `"Hello"` as a `str`, which is `"H"`. `"Hello"[1]` returns the second character, which is `"e"`. `"Hello"[2]` returns the third character, which is `"l"`. `"Hello"[3]` gives us the second `"l"`. Finally, `"Hello"[4]` gives us the fifth and last character, namely `"o"`. If we would try to access a character outside of the valid range of the string, say `"Hello"[5]`, this results in an `IndexError`. We learn later what errors are and how to handle them – for now, it is sufficient to know that they will stop your program. And rightly so, because `"Hello"` has only five characters and accessing the sixth one is not possible and would have an undefined result.

Negative indices, however, are permitted: The index `-1` just means "last character", so `"Hello"[-1]` yields the string `"o"`. The index `-2` then refers to the "second-to-last character", so `"Hello"[-2]` gives us `"l"`. The third character from the end, accessed via index `-3`, is again `"l"`. `"Hello"[-4]` gives us `"e"` and `"Hello"[-5]` gives us `"H"`. Of course, using a negative index that would bring us out of the string's valid range, such as `-6`, again yields an `IndexError`.

We can also obtain whole substrings by using index ranges, where the inclusive starting index and the *exclusive* end index are separated by a `:`. In other words, applying the index `[a:b]` to a string results in all characters in the index range from `a` to `b - 1`. Doing this is called *string slicing*. `"Hello"[0:3]` yields a string composed of the characters at positions 0, 1, and 2 inside `"Hello"`, i.e., `"Hel"`. The end index is always excluded, so the character at index 3 is not part of the result. If we do `"Hello"[1:3]`, we get `"He"`, because only the characters at indices 1 and 2 are included. If we do not specify an end index, then everything starting at the start index until the end of the string is included. This means that `"Hello"[2:]` will return all the text starting at index 2, which is `"llo"`. We can also use negative indices, if we want. Therefore, `"Hello"[1:-2]` yields `"el"` Finally, we can also omit the start index, in which case everything until right before the end index is returned. Therefore, `"Hello"[:-2]` will return everything from the beginning of the string until right before the second-to-last character. This gives us `"Hel"`. We will discussing slicing again later when discussing lists in Section 5.1.

Besides concatenating and extracting substrings, the `str` datatype supports many other operations. Here, we can just discuss the few most commonly used ones.

There are several ways to check whether one string is contained in another one. The first method is to use the `in` keyword. As Figure 3.18 shows, `"World" in "Hello World!"` yields `True`, as it checks whether `"World"` is contained in `"Hello World!"`, which is indeed the case. `"Earth" in "Hello World!"` is `False`, because `"Earth"` is not contained in `"Hello World!"`.

Often, however, we do not just want to know whether a string is contained in another one, but also *where* it is contained. For this, the `find` method exists. `"Hello World!".find("World")` tries to find the position of `"World"` inside `"Hello World!"`. It returns `6`, because the "W" of "World" is the seventh character in this string and the indices are zero-based. Trying to find the `"world"` in `"Hello World!"` yields `-1`, however. `-1` means that the string cannot be found. We learn that string operations are case-sensitive: `"World" != "world"` would be `True`. We also learn that we need to be careful not to use the result of `find` as index in a string directly before checking that it is `>= 0`! As you have learned, `-1` is a perfectly fine index into a string, even though it means that the string we tried to find was not found.

Sometimes, the text we are looking for is contained multiple times in a given string. For example, `"Hello World!".find("l")` returns `2`, because "l" is the third character in the string. However, it is also the fourth character in the string. `find` accepts an optional second parameter, namely the starting

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> "World" in "Hello World!"
True
>>> "Earth" in "Hello World!"
False
>>> "Hello World!".find("World")
6
>>> "Hello World!".find("world")
-1
>>> "Hello World!".find("l")
2
>>> "Hello World!".find("l", 3)
3
>>> "Hello World!".find("l", 4)
9
>>> "Hello World!".find("l", 10)
-1
>>> "Hello World!".rfind("l")
9
>>> "Hello World!".rfind("l", 0, 9)
3
>>> "Hello World!".rfind("l", 0, 3)
2
>>> "Hello World!".rfind("l", 0, 2)
-1
>>> "Hello World!".replace("Hello", "Hi")
'Hi World!'
>>> "Hello Hello World!".replace("Hello", "Hi")
'Hi Hi World!'
>>> " Hello World! ".strip()
'Hello World!'
>>> " Hello World! ".lstrip()
'Hello World! '
>>> " Hello World! ".rstrip()
' Hello World!'
>>> "Hello World!".lower()
'hello world!'
>>> "Hello World!".upper()
'HELLO WORLD!'
>>> "Hello World!".startswith("hello")
False
>>> "Hello World!".startswith("Hello")
True
>>> "Hello World!".endswith("Hello")
False
>>> "Hello World!".endswith("World!")
True
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.18: Some more basic string operations.

index where the search should begin. `"Hello World!".find("l", 3)` begins to search for `"l"` inside `"Hello World!"` starting at index 3. Right at that index, the second "l" is found, so that `3` is also returned. If we search for another "l" after that, we would do `"Hello World!".find("l", 4)`, which returns index 9, identifying the "l" in "World". After that, no more "l" can be found in the string, so `"Hello World!".find("l", 10)` results in a `-1`.

While `find` returns the first occurrence of a string in the supplied range, we sometimes want the last occurrence instead. If we want to search from the end of the string, we use `rfind`. `"Hello World!".rfind("l")` gives us `9` directly. If we want to search for the "l" before that one, we need to supply an inclusive starting and exclusive ending index of the range to be searched. `"Hello World!".rfind("l", 0, 9)` searches for any "l" from index 8 down to 0 and thus returns `3`. `"Hello World!".rfind("l", 0, 3)` gives us `2` and since there is no "l" before that, `"Hello World!".rfind("l", 0, 2)` yields `-1`.

Another common operation is to replace substrings with something else. `"Hello World!".replace("Hello", "Hi")` replaces all occurrences of "'Hello'" in "Hello World" with "Hi". The result is `"Hi World!"` and `"Hello Hello World!".replace("Hello", "Hi")` becomes `"Hi Hi World!"`.

Often, we want to remove all leading or trailing whitespace characters from a string. The

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> str(23)
'23'
>>> str(23.5)
'23.5'
>>> str(1 < 5) + " is True and " + str(1 + 5) + " = 6."
'True is True and 6 = 6.'
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.19: The `str` function converts objects to strings.

`strip` function does this for us: `" Hello World! ".strip()` returns `"Hello World!".strip()`, i.e., the same string, but with the leading and trailing space removed. If we only want to remove the spaces on the left-hand side, we use `lstrip` and if we only want to remove those on the right-hand side, we use `rstrip` instead. Therefore, `" Hello World! ".lstrip()` yields `"Hello World! "` and `" Hello World! ".rstrip()` gives us `" Hello World!"`.

In alphabet-based languages, we usually can distinguish between uppercase characters, such as "H" and "W", and lowercase, such as "e", "l", and "o". The method `lower` transforms all characters in a string to lowercase and `upper` translates them to uppercase instead. Thus `"Hello World!".lower()` returns `hello world!` whereas `"Hello World!".upper()` yields `"HELLO WORLD!"`.

As final functions, we can check whether a string begins or ends with another, we can use `startswith` and `endswith`, respectively. `"Hello World!".startswith("hello")` is `False` whereas `"Hello World!".startswith("Hello")` is `True`. `"Hello World!".endswith("Hello")` is `False`, too, but `"Hello World!".endswith("World!")` is `True`.

Of course, these were just a small selection of the many string operations available in Python. You can find more in the official documentation [130].

3.5.2 The str Function and f-strings

We now have learned the basic operations that can process strings. However, we did not yet learn one essential thing: How can we convert an object, like an `int` or `float` value to a `str`? Very often, we want to perform some complicated calculation that produces a number and then print this number.

In Python, you can convert many different objects to strings by passing them to the function `str`. As Figure 3.19 shows, passing the integer number `23` to `str`, i.e., invoking `str(23)`, yields the string `"23"`. Similarly, invoking `str` with the `float` value `23.5` produces the string `"23.5"`.

We can of course concatenate the results of such computations to form more comprehensive texts: `str(1 < 5)+ " is True and "+ str(1 + 5)+ "= 6."` produces `"True is True and 6 = 6."`, because `1 < 5` evaluated to `True`, which is converted to a string, and `1 + 5` gives us `6`, which, too, is converted to a string. However, converting data to strings like this is rather tedious.

Let us therefore discuss a very powerful and much more convenient gadget in Python's string processing toolbox: format strings, or f-strings for short [18, 45, 89, 128]. An f-string is like a normal string, except that it starts with `f"` instead of `"`. And, most importantly, it can contain other data and even complete expressions inside curly braces (`{...}`) which are then embedded into the string.

In Figure 3.20, we first consider the f-string `f"{12345678901234}is a really big integer."`[5]. This is basically a normal string, except that it contains an integer value. The opening curly brace ("{") right at its beginning signifies that some Python expression will begin which must be translated to a string. The actual expression, `12345678901234` is just a really big integer. The closing curly brace ("}") signifies the end of the expression. The Python interpreter evaluates all expressions inside such curly braces inside the f-string and then turns their results to strings (and removes the curly braces). This process is called *(string) interpolation*. `f"{12345678901234}is a really big integer."` simply becomes `f"12345678901234 is a really big integer."`

[5] The code that formats my inline Python examples sometimes eats spaces after { or }. Therefore, some of the strings presented here may look a bit off. In Figure 3.20, they are printed correctly, though.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> f"{12345678901234} is a really big integer."
'12345678901234 is a really big integer.'
>>> f"12345678901234 with thousand separator ',' is {12345678901234:,}."
"12345678901234 with thousand separator ',' is 12,345,678,901,234."
>>> f"{12345678901234} in hexadecimal notation is {12345678901234:x}."
'12345678901234 in hexadecimal notation is b3a73ce2ff2.'
>>> f"{12345678901234} in 0x-hexadecimal notation is {12345678901234:#x}."
'12345678901234 in 0x-hexadecimal notation is 0xb3a73ce2ff2.'
>>> f"{1234567890} in binary notation is {1234567890:b}."
'1234567890 in binary notation is 1001001100101100000001011010010.'
>>> f"{1234567890} in 0b-binary notation is {1234567890:#b}."
'1234567890 in 0b-binary notation is 0b1001001100101100000001011010010.'
>>> f"{5} + {4} = {5 + 4}"
'5 + 4 = 9'
>>> from math import pi
>>> f"pi is approximately {pi}."
'pi is approximately 3.141592653589793.'
>>> f"pi rounded to two decimals is {pi:.2f}."
'pi rounded to two decimals is 3.14.'
>>> f"1/321 as percentage with 2 decimals is {1/321:.2%}."
'1/321 as percentage with 2 decimals is 0.31%.'
>>> f"1.2345533e6 with thousand separator and 1 decimal is {1.2345533e6:,.1f}."
'1.2345533e6 with thousand separator and 1 decimal is 1,234,553.3.'
>>> from math import sin
>>> f"sin(0.25pi) is approximately {sin(0.25*pi):.5f}."
'sin(0.25pi) is approximately 0.70711.'
>>> f"{1.2359817e12} is {1.2359817e12:e} and approximately {1.2359817e12:.3g}."
'1235981700000.0 is 1.235982e+12 and approximately 1.24e+12.'
>>> f"Single braces without expression: {{ and }}."
'Single braces without expression: { and }.'
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.20: Python f-strings in action.

This first example was not very spectacular. But f-strings offer us several interesting means to format data. For example, we can add some formatting specifiers after the expression, separated by `:`. If our expression evaluates to an `int`, then we can specify a "thousand separator" after the `:`. In western languages, it is usually to group the digits of large numbers in groups of 3. In Chinese, they tend to use groups of 4 instead. To the best of my knowledge, we can only specify thousand separators, thought. This separator will be placed every three digits in the generated text. As example, `f"12345678901234 with thousand separator ',' is {12345678901234:,}."` turns into `"12345678901234 with thousand separator ',' is 12,345,678,901,234."`.

Back in Section 3.2.2, we learned that integers can also be represented in hexadecimal and binary format. f-strings conveniently offer this out-of-the-box, we just need to add a `:x` or a `:b` format specifier to the expression, respectively. For example, the f-string `f"{12345678901234}in hexadecimal notation is {12345678901234:x}."` becomes `"12345678901234 in hexadecimal notation is b3a73ce2ff2."` and the f-string `f"{1234567890}in binary notation is {1234567890:b}."` is turned to `"1234567890 in binary notation is 1001001100101100000001011010010."`. We can also add the `0x` and `0b` prefixes to the generated number strings by using the `:#x` and `:#b` format specifiers instead. Let us use the same examples again but instead with the #-prefixes in the format specifiers. The hexadecimal variant `f"{12345678901234}in 0x-hexadecimal notation is {12345678901234:#x}."` then becomes `"12345678901234 in 0x-hexadecimal notation is 0xb3a73ce2ff2."`. The binary formatting string `f"{1234567890}in 0b-binary notation is {1234567890:#b}."` turns into `"1234567890 in 0b-binary notation is 0b1001001100101100000001011010010."`.

But f-strings allow us to do even more. They can contain complete Python expressions. `f"{5}+ {4}= {5 + 4}"` is evaluated to `"5 + 4 = 9"`.

We can also access constants and variables from within the f-string. Let us again import the constant π from the `math` module by doing `from math import pi`. We can print it as string by typing `f"pi is approximately {pi}."` into the Python console. The result is the string `"pi is approximately 3.141592653589793."`

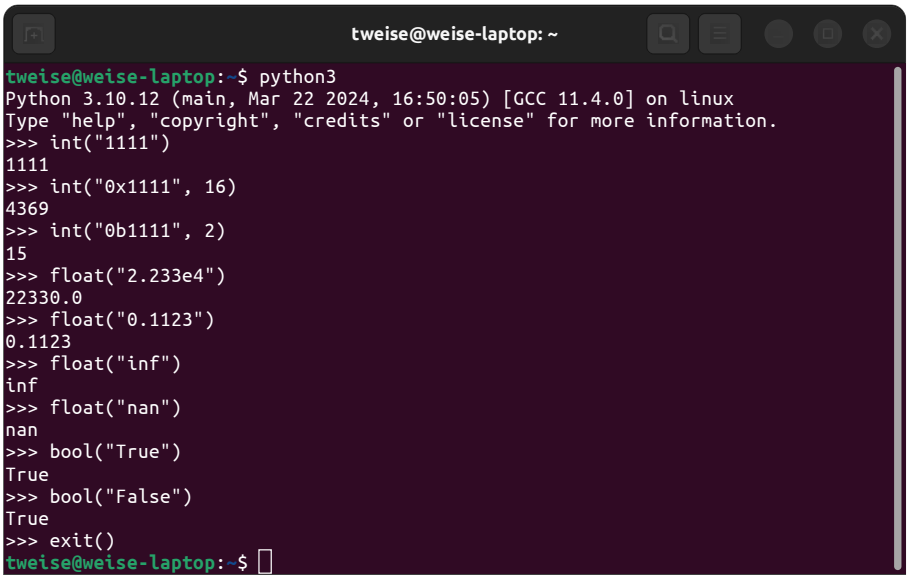


Figure 3.21: Converting strings to other datatypes.

In many situations, we do not want to see the full `float` value with all available significant digits. We can round it to two decimals by adding the `.2f` format specifier. Three digits would be `.3f` and so on... Anyway, `f"pi rounded to two decimals is {pi:.2f}."` gives us `"pi rounded to two decimals is 3.14."`.

Sometimes, we want to present a floating point value as a percentage. For example, $\frac{1}{321} = 0.003115265 \approx 0.31\%$. How can we convert such value to a nice string? By using the `:.2%` format specifier, which gives us a percentage with two decimals. `:.4%` would yield three decimals, and so on... Therefore `f"1/321 as percentage with 2 decimals is {1/321:.2%}."` turns into `"1/321 as percentage with 2 decimals is 0.31%."`.

We can also combine thousand separators and rounding to decimals. The format specifier `,.1f` will use the comma "," as thousand separator and print a floating point number rounded to one decimal. The f-string `f"1.2345533e6 with thousand separator and 1 decimal is {1.2345533e6:,.1f}."` thus is evaluated to `"1.2345533e6 with thousand separator and 1 decimal is 1,234,553.3."`.

Let us also insert a floating point arithmetic expression in an f-string. We therefore import the sine function `sin` from the `math` module via `from math import sin`. The f-string `f"sin(0.25pi)is approximately {sin(0.25*pi):.5f}."` computes the expression `sin(0.25*pi)` and presents its result rounded to five decimals via the `:.5f` format specifier. It therefore becomes `"sin(0.25pi)is approximately 0.70711."`.

We can also use the scientific notation. The format speciier `:e` simply prints a number in scientific notation back from Section 3.3.4. `:.3g` uses scientific notation, but only presents three digits, whereas `:.4g` would present four digits, and so on... `f"{1.2359817e12}is {1.2359817e12:e}and approximately {1.2359817e12:.3g}."` prints the same number three times. First, it is simply rendered as normal `float` value, then it is rendered in scientific notation, and then it is rendered in scientific notation, but rounded to three digits. This gives us `"235981700000.0 is 1.235982e+12 and approximately 1.24e+12."`.

The keen reader may have encountered a question at this stage: *"What do I do if I want to include a curly brace, say { or } inside my f-string?"* Well, if you include a single brace, it would be interpreted as start or end of an expression. If the following text makes sense as expression, it will be interpreted. If not, an error will occur. Either way, the curly brace would disappear. The solution is simple: If you need "{", just write "{{". It will be interpreted as a single "{" brace. If you need "}", just write "}}". It will be interpreted as a single "}" brace. `f"Single braces without expression: {{and }}."` simply becomes `"Single braces without expression: {and }."`

You are now able to convert the results of your computations to nice text.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("\"")
"
>>> print("\'")
'
>>> print("\\")
\
>>> print("\"\'\\")
"'\
>>> print("Hello\nWorld!")
Hello
World!
>>> print("Hello\r\nWorld!")
Hello
World!
>>> print("The horizontal tab is like a bigger space: '\t'.")
The horizontal tab is like a bigger space: '    '.
>>> print("Hello\
... World!")
HelloWorld!
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.22: Escaping special characters in Python strings.

3.5.3 Converting Strings to other Datatypes

While converting the other datatypes to strings is important to produce output, converting strings to the other datatypes is also important. If our programs accept input from the console, the command line, or from text files, we need to somehow translate these input strings to whatever datatype we actually need. Luckily, the datatypes we have discussed so far conveniently provide functions for doing so, and these functions are named like the datatypes themselves.

Figure 3.21 shows that we can convert the string `"1111"` to an integer `1111` simply by passing it to the function `int`. If we want to convert hexadecimal-formatted text such as the string `"0x1111"` instead, we need to tell the `int` function that the basis for conversion is 16. Doing `int("0x1111", 16)` yields `4369`, because $1 + 16^1 + 16^2 + 16^3 = 1 + 16 + 256 + 4096 = 4369$. Similarly, if we have a string in binary annotation, we pass 2 as second parameter to `int`. Calling `int("0b1111", 2)` returns `15`, because $1 + 2 + 4 + 8 = 15$.

We can also convert strings to floating point numbers by using the `float` function. This works for scientific notation (`float("2.233e4")` yields `22330.0`) as well as for "normal" floating point numbers (`float("0.1123")` yields the `float` `0.1123`). The `float` function also extends the special values that a floating point value can take one. Consequently, `float("inf")` gives us `inf` and `float("nan")` returns `nan`.

Finally, the function `bool` converts the strings `"True"` and `"False"` to `True` and `False`, respectively. With this, you are also able to convert strings to data that you can use as input for your computations.

3.5.4 Escaping Characters

First Time Readers and Novices: This section tells you how to include special characters in strings that you otherwise could not include, like quotation marks. You can skip this section and circle back to it when you need it.

The last example of f-strings brought up an interesting topic: "What do we do if we want to use a character in a string which we cannot use?" For example, if our string is delimited with double quotation marks `"`, then we cannot put the character "`"`" into it, because it would then be interpreted as the end of the string. On the other hand, if our string is delimited with single quotation marks `'`, then we cannot put the character "`'`" into it, because it would then be interpreted as the end of the string. Well, you can say, if I need a double quotation mark, then I will delimit my string with single quotation marks and vice versa. This is all good as long as you do not need *both* single and double quotation marks inside the string.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("""This is a multi-line string.
... I can hit enter to begin a new line.
... This linebreak is then part of the string.""")
This is a multi-line string.
I can hit enter to begin a new line.
This linebreak is then part of the string.
>>> print(f"""This is a multi-line f-string.
... It can contain expressions like {(5 ** 7) / 3123:.2f}
... or {(5 < 9) and ((5.5 / 3) < (2 * 11))}.""")
This is a multi-line f-string.
It can contain expressions like 25.02
or True.
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.23: Examples for multi-line strings.

The answer to this problem is *escaping*. The idea is very simple: If we need a certain quotation mark, then we simply put a backslash ("\") before it. The backslash tells the Python interpreter that the next character should be considered as a normal character and not be interpreted as any special character, like a string delimiter.

In Figure 3.22, we present several strings with escape sequences. For clarity reasons, we pass them to the `print` function to output them, which means that they show up undelimited in the console. A double quotation mark can be printed as `print("\"")`, i.e., as a string which is delimited and that then contains the escape sequence \". It then shows up as `"` in the output. A single quotation mark can be printed as `print("\'")`, i.e., via the escape sequence \'. It appears as `'` in the output.

If the use the backslash character "\" to escape characters which may otherwise have some special meaning ... then what do we do if we need a backslash inside of a string? Easy: We escape it. The escape sequence "\\" is converted to a single backslash and `print("\\")` writes `\` to the output. Knowing these sequences, we can now try to `print("\"\'\\")`. The result printed to the output then is `"'\`.

Another situation where escape sequences are nice is when we want to have strings that span over multiple lines. The newline sequence "\n" represents a "newline character" which causes the console to skip to the next line. `print("Hello\nWorld!")` will first print `Hello`, then end the current line and begin a new line, and then print `World!`. Notice that the newline character sequence "\n" is used in Linux and similar systems, whereas Windows uses "\r\n". Under Python, both always work, regardless under which operating system you are working, and you should *always* use "\n". However, only for the sake of completeness, I include the example `print("Hello\r\nWorld!")` as well, which produces the same output as `print("Hello\nWorld!")`.

You probably have pressed the tabulator key ⇥ on your keyboard at some time in the past. It produces something like a "longer space". If you want to include a horizontal tabulator in a string, the escape sequence "\t" is your friend: `print("The horizontal tab is like a bigger space: '\t'.")` yields `The horizontal tab is like a bigger space: '    '.`.

Finally, a backslash can also escape an actual newline in your string. If you have a string that is too long to write on a single line but you do not want to have a linebreak inside the actual string, you can simply put a backslash, hit Enter, and continue the typing the string. The linebreak will then be ignored entirely. Therefore, if you print `print("Hello\`, hit Enter, and then continue to write `World!")`, this produces the output `HelloWorld!`.

Escape sequences allow us to write arbitrary text in strings. We already learned the sequences "{{" and "}}" that were designed for f-strings only. The backslash-based escape sequence we discussed in this section work for both f-strings and normal strings.

3.5.5 Multi-Line Strings

Before we discussed that strings in Python are delimited either by `"` or `'` on each side. However, we can actually delimit them also delimit them with *three quotation marks* on each side, i.e., with either `"""`

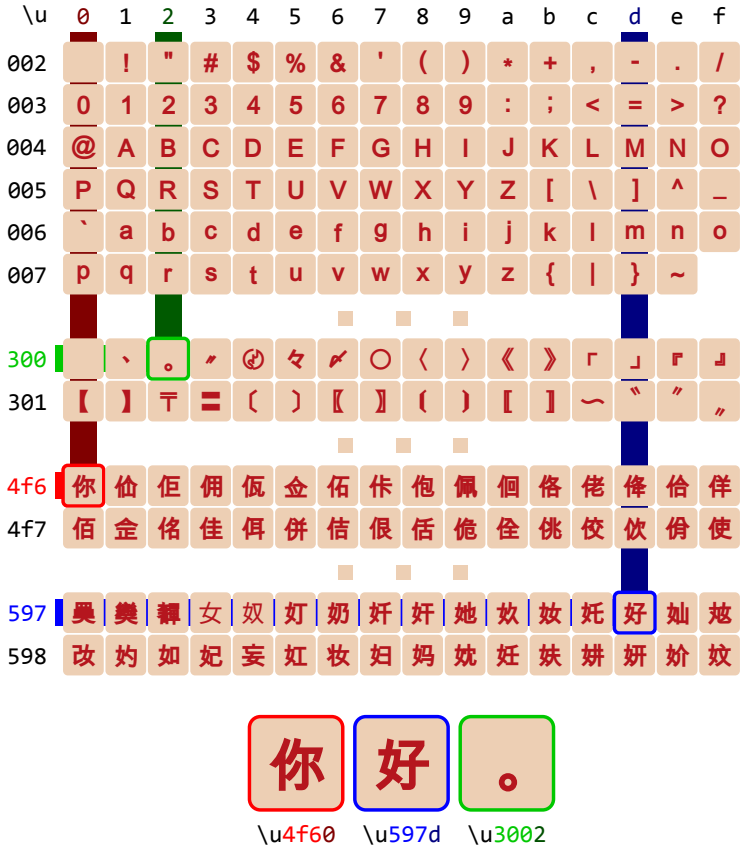


Figure 3.24: A subset of the Unicode character table including the Basic Lating characters as well as some Simplified Chinese characters.

or `'''`. Such string delimiters are used for multi-line strings. In such strings, you can insert linebreaks by hitting Enter completely normally. You can use the escape sequences from the previous section as well. The main use case are docstrings, which we will discuss later, see, e.g., Best Practice 12.

> **Best Practice 5** When defining a multi-line string literal, the double-quotation mark variant (`"""..."""`) is usually preferred over the single-quotation mark variant (`'''...'''`) [49, 143].

Figure 3.23 shows what happens if we print such a multi-line string. We first create the string by writing the three lines `This is a multi-line string.`, `I can hit enter to begin a new line.`, and `This linebreak is then part of the string.`. The first line begins with `"""` and the last one ends with `"""` as well. Passing this text to the `print` function, well, prints exactly this three-line string.

We can also have multi-line f-strings. These then simply start with `f"""`. The example in Figure 3.23 presents such a multi-line f-string with two expressions for (string) interpolation which spans over three lines.

3.5.6 Unicode and Character Representation

> **First Time Readers and Novices:** This section is for readers who want to learn how text is mapped to numbers in order to store it in a computer. First-time readers can safely skip over it.

The memory of our computers basically stores chunks of bits of certain fixed sizes, say, bytes that are composed of8 bit each. Usually, these are interpreted as integer numbers. While Python supports arbitrarily large integers, usually we deal with integers composed of 8 bytes, i.e., 64 bits. The `float` datatype in Python is also usually 8 bytes large, but these are interpreted differently in order to facilitate fractional numbers (see, e.g., Figure 3.5). But how does this work with text?

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("\u4f60\u597d\u3002")
你好。
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.25: "Hello" in Simplified Chinese and entered via Unicode escaped string.

Well, by mapping characters to numbers. A `str` is then nothing but a list of these numbers. The system then knows how to interpret these numbers as characters. Maybe the most well-known historical mapping is ASCII [3, 138], which, however, contained only latin characters, punctuation marks, numbers, and some control characters (like the newline and tab characters we learned when discussing string escaping). Since different languages use different characters, many different mappings have historically evolved and still exist today. In China, different mappings specialized to Chinese characters exist additionally, including the historical GB 2312 [26], GBK [23], or the newer GB 18030 [61]. Today, the vast majority of computers and systems understand one common standard that covers all languages: Unicode [62, 133, 137], the most frequently used mapping of characters to numbers. Therefore, `Python` uses Unicode as well.

Figure 3.24 illustrates a subset of the Unicode code table, including the Basic Latin characters, which are basically still compatible with ASCII, and some Simplified Chinese characters. Most Unicode characters can be identified by a number represented as four hexadecimal digits (mentioned back in Section 3.2.2). The rows Figure 3.24 are annotated with the first three of these digits and the columns with fourth and last hexadecimal digits.

`Python` allows us to enter Unicode characters via a special escape code starting with `\u` followed by these four digits. This is very useful. Imagine that you are sharing a program file with some colleagues. Depending on how their computer encodes text, the Basic Latin characters are usually always interpreted correctly. But some computers may misinterpret Unicode text as something else because the mix up the file encoding. If we use the `\u`-based escape, then we can represent *any* character as Basic Latin text sequence. It is also useful if we want to, e.g., enter Chinese text on a machine that does not have an IME or other corresponding tools, or text in any other kind of language where we do not have corresponding keys on the keyboard (see, e.g., Listing 4.3 later on).

Anyway, in Figure 3.25, we use the information obtained in Figure 3.24 to print the Chinese text "Ni Hao." standing for "Hello." as unicode escaped string. We found that the character for "Ni" has unicode number 4f60, "Hao" has 597d, and the big period has 3002. The string `"\u4f60\u597d\u3002"` then corresponds to the correct Chinese text.

3.5.7 Summary

Strings, i.e., instances of `str`, represent text. Text is obviously an important element of any program, not just because programs are usually written as text, but because they often receive text as input and send text back as output as well. Strings are therefore an essential element of programming. They are basically lists of characters. We can index them, i.e., extract portions of text, we can search inside a string to check whether and where it contains a certain substring, and we can manipulate strings, e.g., by replacing substrings.

It is also important that we can transform data such as `ints`, `floats`, or `bools` to strings. Because we actually *always* print them as strings to the console. The user cannot interpret the binary representation of such data, they only want text. While the function `str` can convert many different types of objects to strings, we often want to combine several different pieces of information to an output text. f-strings are the tool for that. They can render almost arbitrary data as nicely formatted strings and take care of things such as rounding or inserting thousand separators.

Converting strings to the other datatypes that we have discussed so far can, conveniently, be done by using functions of the same names: The function `int` converts its argument string to an instance of `int`. The function `float` converts its argument string to an instance of `float`. And the function `bool`, well, you guess it.

Sometimes we want to include characters in our strings that are dodgy. For example, if our string is delimited by `"` marking its begin and end, inserting such a `"` inside the string would be awkward.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ python3 -W ignore
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> None
>>> print(None)
None
>>> 1 is None
False
>>> "Hello World!" is None
False
>>> None is None
True
>>> print(print("Hello World!"))
Hello World!
None
>>> type(None)
<class 'NoneType'>
>>> exit()
tweise@weise-laptop:~$
```

Figure 3.26: Examples of using the value `None`.

Indeed, the `Python` interpreter would think that it marks the end of the string and then confuse the `"` marking the actual end as the beginning of a new string. Escape sequences solve this problem: We would just write `\"` instead of `"` if we want to insert a double quotation mark inside a string. Multi-line strings solve the problem that we sometimes want to enter text as strings that, well, spans multiple lines.

Finally, we learned that strings internally are Unicode character sequences. Unicode is a standard that maps the characters of all common languages to numbers. We can look up the number corresponding to a character in a Unicode code table. Usually, these are four-digit hexadecimal numbers, which we can then use with the special `\u` escape codes in `Python`. This allows us to basically represent arbitrary text from arbitrary languages using only the Basic Latin characters.

3.6 None

The last simple type we talk about is the `NoneType` and its one single value: `None`. Now you already learned the type `bool` which can take on only two different values, `True` and `False`. You have also learned that the type `float` has a special value called "Not a Number" and written as `nan`. So let's approach this new type from this direction.

`None` is used in scenarios where we want to specify that something does not have any value. It is not an integer, float, string, or bool. `None` is not equivalent to `0`, it is not equivalent to `nan`, and also different from the empty string `""`. It is just nothing.

Figure 3.26 illustrates some of the things we can do with `None`. If we write `None` into the `Python` console, then nothing happens. In the past, we just wrote values, such as `34` and, after we hit Enter, they would be printed again. Not so `None`. If we want to print `None`, we have to force it by using the function `print`. `print(None)` then indeed prints `None`.

The value `None` has many use cases. Unfortunately, most of them we have not yet learned about, so we will have to circle back to this later. For now, simply imagine that you want to write a program that step-by-step computes data. Variables that have not yet been computed could be set to `None` to signify that. If we had a `float`, we could try to set it to `nan` instead, but `nan` could also be the result of a computation. `None` would be much clearer, as no arithmetic calculation could ever return `None`.

For this to work we need to be able to check whether a value is `None`. And we can do this using the operator `is`.[6] `1 is None` gives us `False` and so does `"Hello World!"is None`. `None is None`, however, is `True`.

Also, functions that do not return any result do, in fact, return `None`. We already learned some functions. The function `len`, for example, can be used to compute the length of a string and then returns this value as `int`. The function `log` from the `math` module computes the natural logarithm and returns a `float`. `print`, however, just prints its parameter to the console ... and returns *nothing*. Well, not *nothing*. It returns `None`! We can test this by doing `print(print("Hello World!))`. The inner `print("Hello World!)` will print the string `"Hello World!"`. It will return `None`. This value `None`

[6] `is` is a bit similar to `==`, but instead of comparing whether the contents of an object are the same, it compares whether two values reference the same object.

is then passed into the outer `print` function, which thus essentially does `print(None)` and, thus, prints `None` to the console. We therefore see two lines of text appear, first `Hello World` and then `None`.

The type of `None` can be determined using the `type` function and, indeed, is `NoneType`.

A final use case for `None` is as default value for optional parameters of functions. But we will learn this much later. And thus, we conclude our short section about the value `None` at this point.

3.7 Summary

This section was far more exhausting than what I initially anticipated. I admit that. But I think we now have a solid understanding of the simple datatypes that `Python` offers to us and what we can do with them. We learned about integer numbers and how we can do arithmetics with them. We learned about floating numbers, which can represent fractions and which are limited in their precision. Boolean expressions, such as comparisons, can either be `True` or `False`. Strings are used to represent text and we learned how to convert the other types to and from them. Finally, `None` represents the absense of any value. Equipped with this knowledge, we now can embark to learn how to write programs that compute with these datatypes.

Chapter 4

Variables

We did already learn different simple types in Python as well as basic expressions, such as mathematical formulas or how to work with strings. We are still relatively far from writing programs, though. Basically, all we can do is expressions that fit on a single line.

For more complicated computations, we want to store and modify values. For this purpose, variables exist.

4.1 Defining and Assigning Variables

Like in mathematics, a variable in Python is basically a name with a value assigned to it. You can define a variable and assign its value by writing `name = value`. Here, `name` is the name of the variable and `value` be the value that we want to assign to that name.

4.1.1 A Simple Example of Variable Assignment and Comments in the Code

With this, we can now store intermediate results. This allows us, for the first time, two write programs that perform computations in multiple steps and that consist of multiple lines of code.

Best Practice 6 Comments help to explain what the code in programs does and are a very important part of the *documentation* of code. Comments begin with a `#` character, after which all text is ignored by the Python interpreter until the end of the current line. Comments can either occupy a complete line or we insert two spaces after the last code character in the line and then start the comment [143].

Listing 4.1 shows the source code of such a commented program. This program does not do anything useful, but it illustrates how variables can be used.

It begins by assigning the `int` value `1` to a variable named `int_var`. We could have chosen any other name as well, as long as it does not contain spaces, e.g., `my_value`, `cow`, `race_car`. But we chose `int_var`. The `=` assigns the value `1` to `int_value`. As Figure 4.1a illustrates, the value `1` will now be stored somewhere in memory and `int_var` is a name that points to this memory location.

We can use `int_var` just like any other value. For example, we can compute `2 + int_var` and pass the result to the `print` function. This will then print `3` to the `stdout` of our program. We can also use `int_var` in f-strings about which we leared back in Section 3.5.2. `f"int_var has value {int_var}."` will render to `"int_var has value 1."`.

Variables are called *variables* and not *constants* because we can change their value. Hence, we can update `int_var` and give it a new value. For example, we can do `int_var = (3 * int_var )+ 1`. As sketched in Figure 4.1b, this will update `int_var` to now hold the result of the computation `(3 * int_var)+ 1`. In this computation, the current (old) value of `int_var` is used. It therefore corresponds to computing `(3 * 1)+ 1`, which equals `4`. This value is stored somewhere in memory and `int_var` points to it. Doing `print(f"int_var is now {int_var}.")` will print `int_var is now 4.` to the `stdout`. The value `1` is now no longer referenced. Eventually, the Python interpreter could free the corresponding memory to use it for something else.

Ofcourse, we can have multiple variables. The command `float_var = 3.5` creates a variable named `float_var`. It also allocates a piece of memory, writes the floating point value `3.5` into

Listing 4.1: A Python program showing some examples for variable assignments. (stored in file `assignment.py`; output in Listing 4.2)

```python
# We define a variable named "int_var" and assign the int value 1 to it.
int_var = 1

# We can use the variable int_var in computations like any other value.
print(2 + int_var)  # This should print 2 + int_var = 2 + 1 = 3.

# We can also use the variable in f-strings.
print(f"int_var has value {int_var}.")  # prints 'int_var has value 1.'

# We can also change the value of the variable.
int_var = (3 * int_var) + 1  # int_var = (3 * 1) + 1 = 4
print(f"int_var is now {int_var}.")  # prints 'int_var is now 4.'

float_var = 3.5  # Ofcourse we can also use floating point numbers.
print(f"float_var has value {float_var}.")  # 'float_var has value 3.5.'

new_var = float_var * int_var  # new_var = 3.5 * 4 = 14.0 <- a float!
print(f"new_var = {new_var}.")
```

↓ `python3 assignment.py` ↓

Listing 4.2: The stdout of the program `assignment.py` given in Listing 4.1.

```
3
int_var has value 1.
int_var is now 4.
float_var has value 3.5.
new_var = 14.0.
```

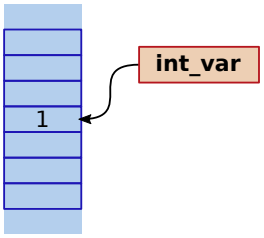
it, and lets `float_var` point to that piece of memory, as illustrated in Figure 4.1c. We can use this variable in an f-string as well: `print(f"float_var has value {float_var}.")` is interpolated to `"float_var has value 3.5."`.

In a final step, we create a third variables with the name `new_var` by computing `new_var = float_var * int_var`. The result is `3.5 * 4`, i.e., `14.0`, `float` value. Figure 4.1c illustrates this variable assignment step. Finally, `print(f"new_var = {new_var}.")` then prints `new_var = 14.0.`.

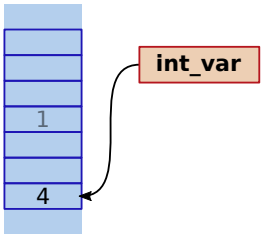
The standard output stream (stdout) of the complete program is given in Listing 4.2. For your convenience, we also showed the results when executing the program in PyCharm or the Ubuntu terminal in Figure 4.2. They are obviously identical.

Notice that we wrote the names of variables in a certain style, which is somewhat standard in Python programming. For the sake of creating readable code that fits nicely together with code from other projects...

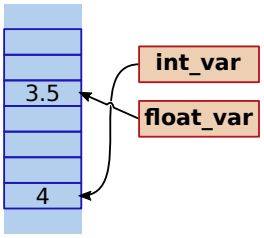
Best Practice 7 Variable names should be lowercase, with words separated by underscores [143].



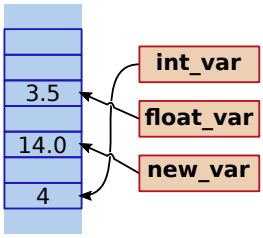
(a) `int_var = 1`



(b) `int_var = (3 * int_var)+ 1`

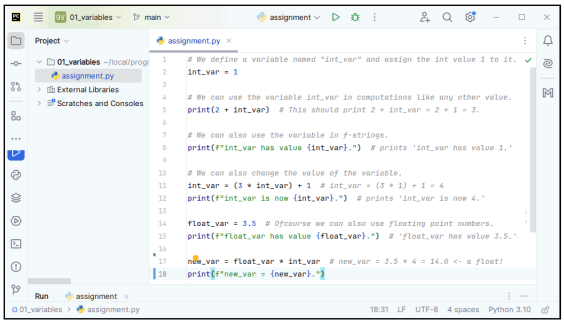


(c) `float_var = 3.5`

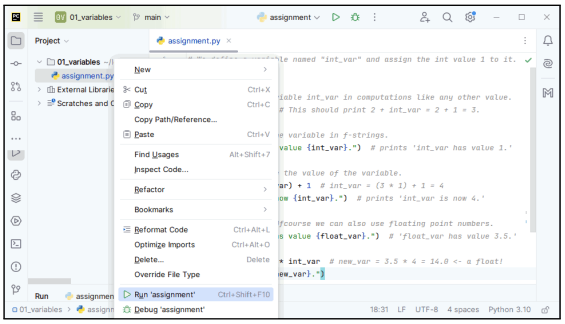


(d) `new_var = float_var * int_var`

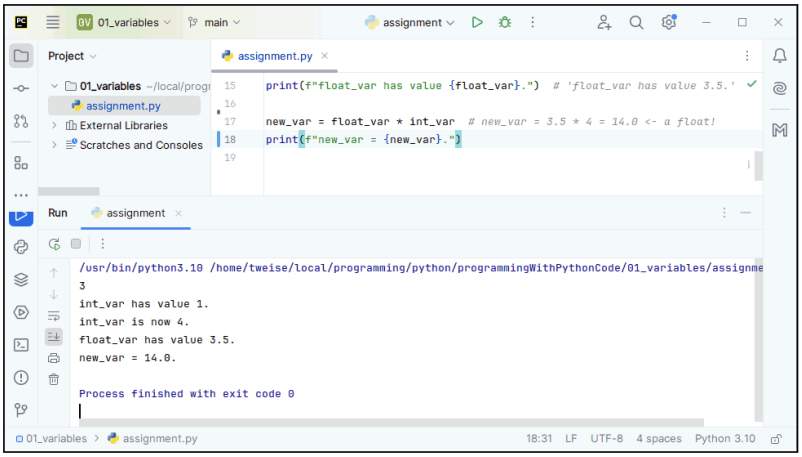
Figure 4.1: Illustrations of the variable assignments in Listing 4.1 in the same order in which they appear in the program: Variables are basically names that point to objects which are located somewhere in memory.



(a) The file `assignment.py` opened in PyCharm.



(b) Left-clicking on Run 'assignment' in the pop-up menu after right-clicking on `assignment.py`, or directly pressing Ctrl + ⇧ + F10, to run the program.



(c) The output of the program `assignment.py` in PyCharm.

(d) The output of the program `assignment.py` in the Ubuntu terminal (which you can open via Ctrl + Alt + T).

Figure 4.2: Running the program `assignment.py` from Listing 4.1 in PyCharm (Figures 4.2a to 4.2c) or the Ubuntu terminal (Figure 4.2d).

4.1.2 LIU Hui's Method and the Approximation of π

Let us now come to a more serious example. I am not good at mathematics, but I really like mathematics anyway, so we will go with a mathematics example: approximating π. The number π is the ratio of the circumference of a circle and its diameter. A we already mentioned before in Section 3.3, it is transcendental, a never-ending and never-repeating sequence of digits. We can compute it to a certain precision, e.g., as the `float` constant `pi` with value `3.141592653589793`. But we can never really write it down.

Well, we I say "we can compute it", then the question "How?" immediately arises. One particularly ingenious answer was given by the Chinese mathematician LIU Hui (刘徽) somewhere in the third century AD [98] in his commentary to the famous Chinese mathematics book *Jiu Zhang Suanshu* (九章算术) [31, 33, 70, 98, 129]. In Figure 4.3, we show how π, i.e., the ratio of the circumference and the diameter of a circle can be approximated by inscribing regular e-gons into a circle. The corners of the e-gons lie on the circle.

We start with a hexagon ($e = 6$) where the radius r is equal to the radius of the circle. All the e edges s_6 of this hexagon then have length r as well. It is easy to see that the circumference of the hexagon is $U = e * s_6 = 6 * r$. The diameter of the circle is $D = 2r$. Assuming that the circumference of the hexagon is an approximation of the circumference of the circle, we could approximate π as $\pi \approx \frac{U}{D}$. For $e = 6$ edges, this gives us $\pi_6 = \frac{6r}{2r} = 3$.

Now this is a very coarse approximation of π. We can get closer to the actual ratio if we would use more edges, i.e., higher values of e. The ingenious idea of LIU Hui is to use e-gons with $e = 3 * 2^n$. For $n = 1$, we get the hexagon with $e = 6$. For $n = 2$, we double the edges and have a dodecagon with $e = 12$ edges. But how do we get the edge length s_{12} of this dodecagon?

We can get it from the edge length s_6 and radius r of the hexagon. If we use the same six corners for the hexagon and dodecagon and connect the newly added six corners with the center of the circle, then these connections will separate each edge of the hexagon exactly in half and do so at a 90° angle, as shown again in Figure 4.3. Here, the new side length s_{12} is the hypotenuse of a right-angled triangle with base $\frac{s_6}{2}$ and height y. To get the height y, we can use that $r = x + y$ and the fact that there is a second right-angled triangle here, namely the one with base x, height $\frac{s_6}{2}$, and hypotenuse r. This gives us $x^2 + \left(\frac{s_6}{2}\right)^2 = r^2$. Let's make things easier by choosing $r = 1$. We get $x^2 = 1 - \left(\frac{s_6}{2}\right)^2 = 1 - \frac{{s_6}^2}{4}$ and, hence, $y = 1 - \sqrt{1 - \frac{{s_6}^2}{4}}$. With this we can move on to ${s_{12}}^2 = y^2 + \left(\frac{s_6}{2}\right)^2$, which we can resolve to ${s_{12}}^2 = \left(1 - \sqrt{1 - \frac{{s_6}^2}{4}}\right)^2 + \frac{{s_6}^2}{4}$. Using $(a - b)^2 = a^2 - 2ab + b^2$ and applying it to the first term, we get ${s_{12}}^2 = 1 - 2\sqrt{1 - \frac{{s_6}^2}{4}} + \left(1 - \frac{{s_6}^2}{4}\right) + \frac{{s_6}^2}{4}$. This then gives us ${s_{12}}^2 = 2 - 2\sqrt{1 - \frac{{s_6}^2}{4}} - \frac{{s_6}^2}{4} + \frac{{s_6}^2}{4}$, which we can further refine to ${s_{12}}^2 = 2 - 2\sqrt{1 - \frac{{s_6}^2}{4}}$. We can pull th 2 from outside the root into the root by multiplying everything inside by $2^2 = 4$ and get ${s_{12}}^2 = 2 - \sqrt{4 - {s_6}^2}$. Thus, we have the really elegant $s_{12} = \sqrt{2 - \sqrt{4 - {s_6}^2}}$.

As new approximation of π_{12}, we now have $\frac{12 * s_{12}}{2r} = 6 * s_{12} = 6\sqrt{2 - \sqrt{4 - {s_6}^2}} = 6\sqrt{2 - \sqrt{4 - 1}} = 6\sqrt{2 - \sqrt{3}} \approx 3.105828539$. This is already quite nice. We can actually repeat this step to get to s_{24}. And we could continue this process by again doubling the number the edges. Repeating the above

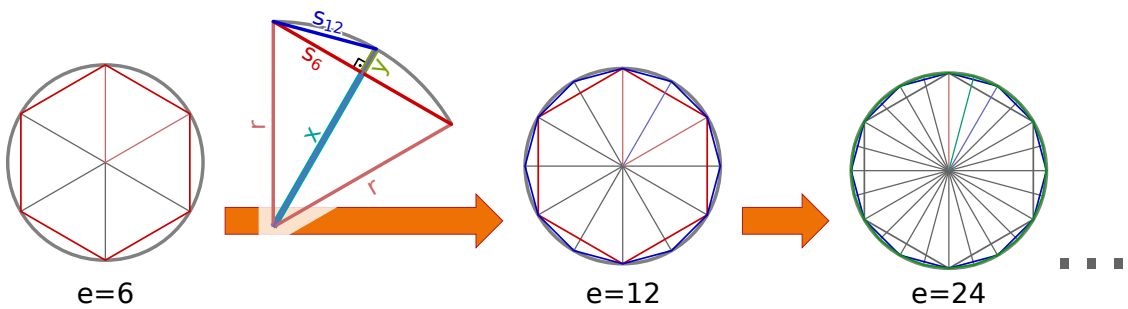


Figure 4.3: Approximating the ratio of the circumference and the diameter of a circle, i.e., π, by inscribing regular $3 * 2^n$-gons.

Listing 4.3: A Python program showing several steps of the approximation of π using the method of LIU Hui. (stored in file `pi_liu_hui.py`; output in Listing 4.4)

```python
from math import pi, sqrt

print(f"We use Liu Hui's Method to Approximate \u03c0\u2248{pi}.")
e = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s = 1.0  # the side length: Initially 1, meaning the radius is also 1.
print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")

e *= 2  # We double the number of edges...
s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")

e *= 2  # We double the number of edges.
s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")

e *= 2  # We double the number of edges.
s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")

e *= 2  # We double the number of edges.
s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")

e *= 2  # We double the number of edges.
s = sqrt(2 - sqrt(4 - (s ** 2)))
print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
```

↓ `python3 pi_liu_hui.py` ↓

Listing 4.4: The stdout of the program `pi_liu_hui.py` given in Listing 4.3.

```
We use Liu Hui's Method to Approximate π≈3.141592653589793.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050416 give us π≈3.1058285412302498.
24 edges, side length=0.2610523844401031 give us π≈3.132628613281237.
48 edges, side length=0.13080625846028635 give us π≈3.139350203046872.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

calculations and observing Figure 4.3, we get the equation:

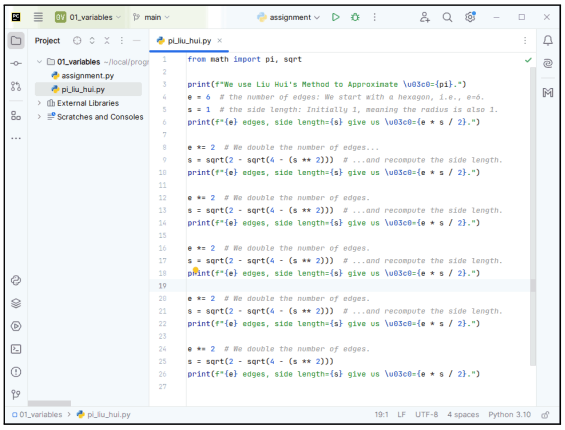
$$s_{2e} = \sqrt{2 - \sqrt{4 - s_e^2}} \tag{4.1}$$

$$\pi_{2e} = \frac{e}{2} s_{2e} \tag{4.2}$$

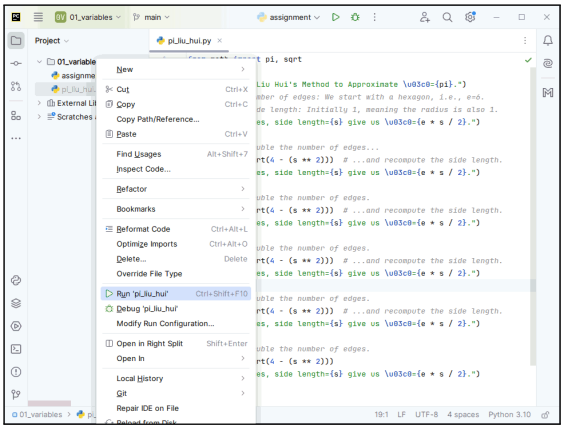
Now that we have learned some programming, we do no longer need to type the numbers and computation steps into a calculator. Instead, we can simply write them into a program, as illustrated in Listing 4.3. We begin by setting the number of edges `e = 6` and the side length to `s = 1`, still choosing $r = 1$. In each iteration of the approximation, we simply set `e *= 2`, which is equivalent to `e = e * 2`, to double the number of edges. We compute `s = sqrt(2 - sqrt(4 - (s ** 2)))` having imported the `sqrt` function from the `math` module. We print the approximated value of π as `e * s / 2`. Notice how elegantly we use the unicode characters π and $\approx$ via the escapes `\u03c0` and `\u2248`, respectively, from back in Section 3.5.6 (and how nicely it indeed prints the greek character π in the stdout in Listing 4.4). Either way, since Equations 4.1 and 4.2 are always the same, we can simply copy-paste the lines of code for updating `s`, `e`, and printing the approximated value of π several times.

Listing 4.4 shows the standard output stream (stdout) produced by this program. Indeed, each new approximation comes closer to π. For 192 edges, we get the approximation `3.1414524722853443`.

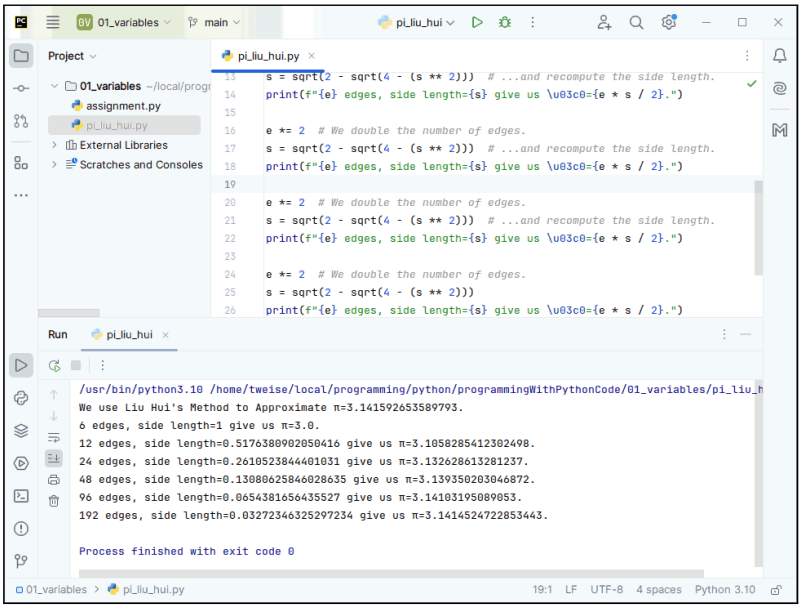
Given that the constant `pi` from the `math` module is `3.141592653589793`, we find that the first four digits are correct and that the number is only off by only 0.0045%! For your convenience, we also showed the results when executing the program in `PyCharm` or the `Ubuntu` terminal in Figure 4.4. They are obviously identical. Therefore, in the future, we will no longer add these screenshots and only print code and output pairs like Listings 4.3 and 4.4.



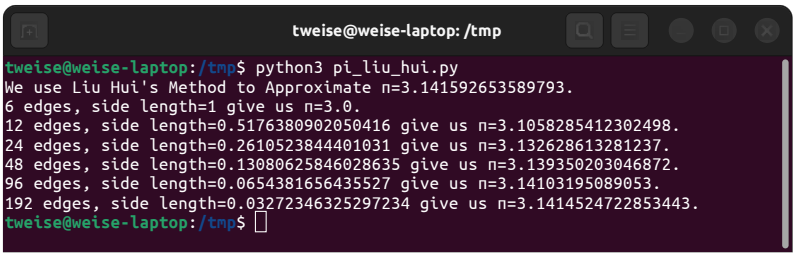
(a) The file `pi_liu_hui.py` opened in PyCharm.



(b) Left-clicking on Run 'pi_liu_hui' in the pop-up menu after right-clicking on `pi_liu_hui.py`, or directly pressing Ctrl + ⇧ + F10, to run the program.



(c) The output of the program `pi_liu_hui.py` in PyCharm.



(d) The output of the program `pi_liu_hui.py` in the Ubuntu terminal (which you can open via Ctrl + Alt + T).

Figure 4.4: Running the program `pi_liu_hui.py` from Listing 4.1 in PyCharm (Figures 4.2a to 4.2c) or the Ubuntu terminal (Figure 4.2d).

4.2 Multiple Assignments and Value Swapping

In Python, we can assign values to multiple variables at once. In this case, we separate both the variable names and the values with commas. The first line (`a, b = 5, 10`) in Listing 4.5 assigns the values `5` and `10`, respectively, to the variables `a` and `b`, respectively. After this assignment step, `a == 5` and `b == 10` holds. `print(f"a=a, b=b")` therefore prints *a=5, b=10*.

This method can also be used to *swap* the values of two variables. Writing `a, b = b, a` looks a bit strange but it basically means "the *new* value of `a` will be the *present* value of `b` and the *new* value of `b` will be the *present* value of `a`." The line is therefore basically equivalent to first storing `a` in a temporary variable `t`, then overwriting `a` with `b`, and finally copying the value of `t` to `t`. But it accomplishes this in single line of code instead of three. `print(f"a=a, b=b")` thus now prints *a=10, b=5*.

Best Practice 8 Swapping of variable values can best be done with a multi-assignment statement, e.g., `a, b = b, a`.

The same concept of multiple assignments works for arbitrarily many variables. `z, y, x = 1, 2, 3` assigns, respectively, `1` to `z`, `2` to `y`, and `3` to `x`. `print(f"x=x, y=y, z=z")` thus yields `x=3, y=2, z=1`.

We can also swap multiple values. `x, y, z = z, y, x` assigns the present value of `z` to become the new value of `x`, the present value of `y` to also be the new value of `y`, and the present value of `x` to become the new value of `z`. `print(f"x=x, y=y, z=z")` now gives us `x=1, y=2, z=3`.

Listing 4.5: A Python program assigning multiple values to multiple variables and using the same method to swap variable values. (stored in file `multi_and_swap.py`; output in Listing 4.6)

```
a, b = 5, 10
print(f"a={a}, b={b}")

a, b = b, a
print(f"a={a}, b={b}")

z, y, x = 1, 2, 3
print(f"x={x}, y={y}, z={z}")

x, y, z = z, y, x
print(f"x={x}, y={y}, z={z}")
```

↓ `python3 multi_and_swap.py` ↓

Listing 4.6: The stdout of the program `multi_and_swap.py` given in Listing 4.5.

```
a=5, b=10
a=10, b=5
x=3, y=2, z=1
x=1, y=2, z=3
```

4.3 Variable Types and Type Hints

4.3.1 Variable Types

A variable is basically a name pointing to an object. Each object has a type and we already learned about several of these datatypes in Chapter 3. We can obtain the type of an object stored in variable `var` by invoking `type(var)`. Listing 4.7 shows a program that does just that and the output of that program is given in Listing 4.7. It is obvious that the type of a variable that holds an integer value is `int`, the type of a variable that holds a floating point number is `float`, and so on. There really is not much to say about that.

4.3.2 Types and Confusion

Well, actually, there is. You see, when you declare a variable in a language like C, you have to specify its *type*. You are then permitted to only assign values that have exactly this type to the variable. In Python, you do not need to specify a type and you can assign whatever you want to a variable. This has the advantage that the code is shorter (because you do not need to write the type), looks more elegant, and programming becomes easier. At first glance. However, there are also problems. Let's take a look at Listing 4.9. We declare a variable named `int_var` and store the integer `8` in it. Then we update `int_var` by computing `int_var = int_var / 3`. Back in Section 3.2, you learned that the `//` operator performs an integer division with an `int` result, whereas the division using the `/` operator always returns a `float`. This means that our variable `int_var` now contains a `float`, which is also visible in the output in Listing 4.10.

From the perspective of Python, this is totally fine. The program executes and the output appears without error. However, from the perspective of programming, Listing 4.9 is *wrong*. Imagine that this was not just some random example without meaning. Imagine that this was a part of a really useful program. Imagine that you got this program from some source and try to understand it. If you read this program, then you find that a variable named `int_var` contains a `float`. This is not forbidden, but when reading the code, it must strike you as odd.

Indeed, there are at least two possible explanations for this: On the one hand, maybe, the original author of this code mistakenly mixed up the `/` operator for the `//` (see Best Practice 1). Maybe they

Listing 4.7: An example of the types of variables. (stored in file `variable_types.py`; output in Listing 4.8)

```
int_var = 1 + 7  # Create an integer variable holding an integer number.
print(type(int_var))  # This prints "<class 'int'>".

float_var = 3.0  # 3.0 is a float and it is stored in float_var.
print(type(float_var))  # This prints "<class 'float'>".

str_var = f"float_var={float_var}"  # Render an f-string into str_var.
print(type(str_var))  # This prints "<class 'str'>".

bool_var = (1 == 0)  # 1 == 0 is False, so a bool is stored in bool_var.
print(type(bool_var))  # This prints "<class 'bool'>".

none_var = None  # We create none_var which, well, holds None.
print(type(none_var))  # This prints "<class 'NoneType'>".
```

↓ `python3 variable_types.py` ↓

Listing 4.8: The stdout of the program `variable_types.py` given in Listing 4.7.

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
```

Listing 4.9: An example of the confusing variable types. (stored in file `variable_types_wrong.py`; output in Listing 4.10)

```
int_var = 1 + 7  # Create an integer variable holding an integer number.
print(type(int_var))  # This prints "<class 'int'>".

int_var = int_var / 3  # / returns a float, but we may expect an int?
print(type(int_var))  # This now prints "<class 'float'>".
```

↓ `python3 variable_types_wrong.py` ↓

Listing 4.10: The stdout of the program `variable_types_wrong.py` given in Listing 4.9.

```
<class 'int'>
<class 'float'>
```

Listing 4.11: The results of static type checking with Mypy of the program given in Listing 4.9. (This is actually output generated by the script Listing 10.1 on page 175.)

```
$ mypy variable_types_wrong.py --no-strict-optional --check-untyped-defs
variable_types_wrong.py:4: error: Incompatible types in assignment (
    ↪ expression has type "float", variable has type "int")  [assignment]
Found 1 error in 1 file (checked 1 source file)
# mypy 1.13.0 failed with exit code 1.
```

wanted to do an integer division and accidentally did a floating point division. Depending on what the code later on (in our imaginary larger program) does, it could be very hard to find such an error.

On the other hand, maybe, the author fully well wanted to do a floating point division and expected a `float` to be stored in `int_var`, but chose a misleading name. Choosing this name, however, can be very dangerous: What if another programmer continues to work on this code and, based on the variable's name, expects it to contain an `int` whereas it actually contains a `float`. This could again lead to all sorts of strange errors later on in her code.

Best Practice 9 The names we use in program code should clearly reflect our intentions.

If the programmer followed Best Practice 9, then the code would clearly contain a bug, namely a mix-up between the `/` operator for the `//`. Regardless of what is true, you will certainly agree that something is wrong with this program. And most certainly, it was just a small oversight, maybe even just a typo. Unfortunately, since you are not the author of the program, you do not know what is wrong. This code will cause some problem down the line. Many such problems exist in many software projects and they indeed are hard to find [71]. So here, the lenience of Python of allowing us to not specify types comes back to bite us.

Everyone of us makes such mistakes. It is impossible to completely avoid them. Luckily, there are two things that we can do to prevent such situations from passing our "quality control":

1. Use static type checking tools to find such potential errors in our code.
2. Use type hints to annotate variables with their types to *a)* make our intention clearer and *b)* support type checking tools.

And if you are in one of my classes, you better do both. And now we will learn how to do that.

4.3.3 Static Type Checking

A first step to avoiding any type-related errors in programs is, ofcourse, careful programming. The second step is to use tools that check whether your program code contains ambiguities or errors. In languages like C, the compiler will take care of that for you.

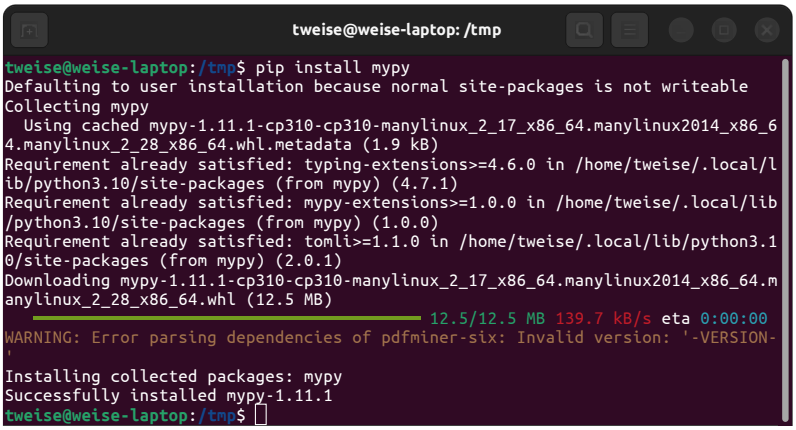


Figure 4.5: Installing Mypy in a Ubuntu terminal via pip.

Listing 4.12: The results of static type checking with Mypy of the program given in Listing 4.7. (This is actually output generated by the script Listing 10.1 on page 175.)

```
$ mypy variable_types.py --no-strict-optional --check-untyped-defs
Success: no issues found in 1 source file
# mypy 1.13.0 succeeded with exit code 0.
```

In Python, which allows for dynamic typing and is an interpreted language, we will use a tool like Mypy [83]. You can install this tool by opening a terminal. Under Ubuntu, you therefore press Ctrl + Alt + T, and under Windows, you press ⊞ + R, type in cmd, and hit ↵. Then type in pip install mypy and hit Enter. The Mypy tool will be installed as illustrated in Figure 4.5.

We can now apply the tool to the program from Listing 4.9. All we have to do is invoke it in the terminal, giving the program to be checked as argument as well as some additional parameters. In Listing 4.11, we invoke mypy variable_types_wrong.py --no-strict-optional --check-untyped-defs, where variable_types_wrong.py is the (very fitting) name of the program to check. Indeed, Mypy tells us that something dodgy is going on in the fourth line of that program, i.e., int_var = int_var / 3. It will fail with an exit code of 1. Programs usually return 0 as exit code if everything went well and some non-zero value if something went wrong. And something went wrong, because Mypy found the error.

Useful Tool 1 Mypy [83] is a static type checking tool for Python. This tool can warn you if you, e.g., assign values to a variable that have a different type than the values previously stored in the variable, which often indicates a potential programming error. It can be installed via pip install mypy, as illustrated in Figure 4.5 on page 62. You can then apply Mypy using the command mypy fileToScan.py. We use the Bash script given in Listing 10.1 on page 175 to apply Mypy to the example programs in this book.

If we instead apply Mypy the completely fine (albeit useless) program Listing 4.7, it will tell us that there is no error, as illustrated in Listing 4.12. So we now have one tool at our hands with which we can check our source code for type-related problems. Notice that this program just checks the source code. It does not change the code and it does not execute our program. It just reads in the code and looks for type-related errors. This will obviously have no impact on the programs performance or speed. It also cannot fix the errors, as it cannot what the programmer actually intended to do. But knowing that line 4 in Listing 4.9 is probably wrong will help the programmer to fix that error or oversight before passing the program on to someone else.

Best Practice 10 Every program should pass static type checking with tools such as Mypy (see Useful Tool 1). Any issue found by the tools should be fixed. In other words, type check the program. If there is an error, fix the error and *type check it again*. Repeat this until no errors are found anymore.

Listing 4.13: Listing 4.9, but with the variable explicitly hinted as `int`. (stored in file `variable_types_wrong_hints_1.py`; output in Listing 4.14)

```
int_var: int = 1 + 7   # Create a variable hinted as integer.
print(type(int_var))   # This prints "<class 'int'>".

int_var = int_var / 3   # / returns a float, but we may expect an int?
print(type(int_var))   # This now prints "<class 'float'>".
```

↓ `python3 variable_types_wrong_hints_1.py` ↓

Listing 4.14: The stdout of the program `variable_types_wrong_hints_1.py` given in Listing 4.13.

```
<class 'int'>
<class 'float'>
```

Listing 4.15: Listing 4.9, but with the variable explicitly hinted as either `int` or `float` and named appropriately. (stored in file `variable_types_wrong_hints_2.py`; output in Listing 4.16)

```
my_var: int | float = 1 + 7   # A variable hinted as either int or float.
print(type(my_var))   # This prints "<class 'int'>".

my_var = my_var / 3   # / returns a float, which is OK.
print(type(my_var))   # This now prints "<class 'float'>".
```

↓ `python3 variable_types_wrong_hints_2.py` ↓

Listing 4.16: The stdout of the program `variable_types_wrong_hints_2.py` given in Listing 4.15.

```
<class 'int'>
<class 'float'>
```

4.3.4 Type Hints

When we discussed Listing 4.9 in Section 4.3.2, we stated that there could be two reasons for the error in the code: Either, the author accidentally mixed-up two datatypes or operators (`/` vs. `//`) or they chose a misleading name for their variable `int_var`. The problem that any type checking tool faces is that it cannot know the intention of the programmer. It can find that line 4 four is probably wrong, because the variable `int_var`, which former contained an `int`, now gets a `float` value assigned to it.

Oddly enough, the problem of guessing the intention of the programmer does not exist in a statically typed language like pglsC. Here, we *need* to define the type of every variable before assigning a value to it. Therefore, if the programmer would have wanted `int_var` to strictly be an integer, they would have declared it as an integer variable. If they wanted to store `floats` in it, they would have declared it as a `float` variable. The compiler would have seen any malpractice right away and could tell us that either line 4 is wrong or our initial assignment of an `int` to the variable in line 1.

This is where the dynamic typing and lenience of Python comes back to bite us. It is very convenient for small projects, but as soon as the projects get bigger, it creates a mess. Remember that "real programs" are much more complex than Listing 4.9. Imagine wading through thousands of lines of code to figure out what type a variable has, and, while doing so, remember that Python permits overwriting the contents of a variable with objects of an entirely different type whenever it pleases us.

Realizing that dynamic typing can be a blessing but also a problem, *optional* type hints were introduced into the Python language [114, 142]. We can now declare the type of a variable if we want. This solves the above problem basically entirely and allows us to tell type checking tools our intention.

When we declare and assign a variable, `my_var = 1` for example, and want to define it as integer variable, for example, we simply write `my_var: int = 1`. Of course, a type checker will already see that `my_var` should be an integer variable when we assign the integer `1` to it. However, by writing `: int` after its name, we also clearly establish our intention. And this makes a difference.

If the author of Listing 4.9 had used type hints, they could have written their program differently, as illustrated in Listings 4.13 and 4.15. Both programs as well as the original one produce exactly the same

Listing 4.17: The results of static type checking with Mypy of the program given in Listing 4.13.

```
$ mypy variable_types_wrong_hints_1.py --no-strict-optional --check-untyped
    ↪ -defs
variable_types_wrong_hints_1.py:4: error: Incompatible types in assignment
    ↪ (expression has type "float", variable has type "int")  [assignment]
Found 1 error in 1 file (checked 1 source file)
# mypy 1.13.0 failed with exit code 1.
```

Listing 4.18: The results of static type checking with Mypy of the program given in Listing 4.15.

```
$ mypy variable_types_wrong_hints_2.py --no-strict-optional --check-untyped
    ↪ -defs
Success: no issues found in 1 source file
# mypy 1.13.0 succeeded with exit code 0.
```

output if we execute them with the Python interpreter, since type hints are ignored by the interpreter. However, if we type-check them with Mypy, we get two different results, namely Listings 4.17 and 4.18, respectively.

In Listing 4.13, we annotate `int_var` as an integer variable by writing `int_var: int`. While the name of the variable already indicates that we want it to hold integers, we now have formally specified this intention. The Mypy type checker tells is in Listing 4.17 basically the same as it stated In Listing 4.11, namely that assigning a `float` to this variable is wrong.

However, if the intention of the author wanted that the variable could hold either an `int` or a `float`, they could have annotated it with the type hint `int | float`. The `|` here is not the "bit-wise or" we learned back in Section 3.2.2, but instead indicates that both `int`s and `float`s are acceptable values for a variable [106].[1] This annotation is used in Listing 4.15. A side-effect of using this type is that, when writing this code, its author would have realized that the name `int_var` would not be an appropriate name. They would probably have used something like `my_var` instead. Applying Mypy to this program yields the output Listing 4.18, which indicates that everything is OK.

Using type hints together with type checking would have prevented the problems in Listing 4.9 from the start. Either, its author would have discovered the mistake of computing a `float` value instead of an `int`. Or they would have more clearly communicated their intention by marking the variable to either hold an `int` or a `float`. And this comes at no cost at all when executing the program, because type hints are only checked by programmers and tools, but not by the interpreter.

Best Practice 11 Always use type hints.

For the sake of completeness, let us also annotate Listing 4.7 with type hints as a small exercise. The variable `int_var`, in which we want to store the integer value `8`, will be annotated with `: int`. The variable `float_var`, in which we want to store the floating point number `3.0`, will be annotated with `: float`. The variable `str_var`, in which we want to store the string `"float_var=3.0"`, will be annotated with `: str`. The variable `bool_var`, in which we want to store the value `False`, will be annotated with `: bool`. And, finally, the variable `none_var`, in which we want to store `None`, will be annotated with `: None`. This produces Listing 4.19, which we can check with Mypy and obtain Listing 4.20.

It is very obvious at this point that including the type of the variable in the variable name is no needed. It is not helpful for any tool. If we want to convey our intention about the type to another programmer who may be reading our code, then type hints are a much better choice. Different from variable names, they also can be interpreted by type checkers.

With type hints, we have brought the advantages of a statically typed programming language to Python. Since they are optional, a programmer can choose whether and where to use them. However,

[1] Technically speaking, the type hint specification in PEP484 [142] allows all variables annotated with `float` to also accept `int` values in section The Numeric Tower. Thus annotating the variable only with `: float` would have been sufficient. However, since `int` is actually not a subclass of `float` in Python, this can be confusing in some cases. Writing `int | float` gives me the opportunity to introduce the `|` operator . . . so this is how we do it.

Listing 4.19: A variant of Listing 4.7 which has been improved by adding type annotations. (src)

```
int_var: int = 1 + 7  # Create an integer variable holding an integer.
print(type(int_var))  # This prints "<class 'int'>".

float_var: float = 3.0  # 3.0 is a float and it is stored in float_var.
print(type(float_var))  # This prints "<class 'float'>".

str_var: str = f"float_var={float_var}"  # Render an f-string.
print(type(str_var))  # This prints "<class 'str'>".

bool_var: bool = (1 == 0)  # 1 == 0 is False, so a bool is stored.
print(type(bool_var))  # This prints "<class 'bool'>".

none_var: None = None  # We create none_var which, well, holds None.
print(type(none_var))  # This prints "<class 'NoneType'>".
```

Listing 4.20: The results of static type checking with Mypy of the program given in Listing 4.19.

```
$ mypy variable_types_hints.py --no-strict-optional --check-untyped-defs
Success: no issues found in 1 source file
# mypy 1.13.0 succeeded with exit code 0.
```

if you are a student attending one of my courses, consider them as mandatory. Their advantage is that they allow us to find many (though by far not all) logical errors. They make the code easier to read and easier to understand. Therefore, from my perspective, Best Practice 11 *always* applies.

4.4 Object Equality and Identity

We use variables to references objects in memory. When we compare two variables, we actually compare the objects they reference. It is clear that two variables can reference objects that are either equal or not equal. They can also reference the same object. These two concepts – equality and identity – must be distinguished.

And this can be done rather easily. Imagine that you buy a green jacket, let's call it A. Then you buy the exactly same jacket again, for a second time, and call the second jacket B. Now A and B are equal, but they are still two different objects.

Figure 4.6 illlustrates this issue. If we have two variables `a=1.4142135623730951` and `b=1.4142135623730953`, then they have different values and reference different objects (namely, the two different memory cells holding the two different `float` values). In this case, `a == b` and `a is b` will both return `False`.

Now we can also have another variable `c`, which may reference a `float` object that holds the same value as the one referenced by `a`. This value would be stored somewhere else in memory. Then, `a == b` is `True`, because the variables have equal values. However, `a is b` would be `False`, because they still reference different objects.

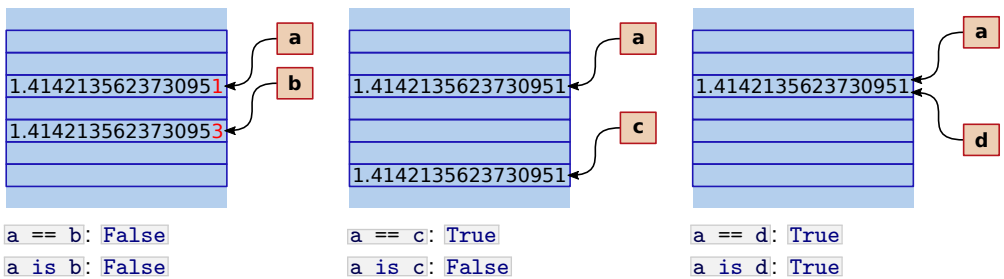


Figure 4.6: A illustration of the concepts of equality (`==`) and identity (`is`) from the perspective of variables.

Listing 4.21: An example of the difference between equality and identity. (stored in file `identity_1.py`; output in Listing 4.22)

```
# We define sqrt_2 to be a constant with the value of square root of 2.
sqrt_2: float = 1.4142135623730951  # We set the value of the variable.
print(f"sqrt_2 = {sqrt_2}")  # We print the value of the variable

# We can also compute the square root using the `sqrt` function.
from math import sqrt  # Import the root function from the math module.
sqrt_2_computed: float = sqrt(2.0)  # Compute the square root of 2.0.
print(f"sqrt_2_computed = {sqrt_2_computed}")  # Print the value.

# Let's compare the computed and the constant value:
print(f"are they equal: {sqrt_2 == sqrt_2_computed}")
print(f"are they the same object: {sqrt_2 is sqrt_2_computed}")
```

↓ `python3 identity_1.py` ↓

Listing 4.22: The stdout of the program `identity_1.py` given in Listing 4.21.

```
sqrt_2 = 1.4142135623730951
sqrt_2_computed = 1.4142135623730951
are they equal: True
are they the same object: False
```

If I would declare a variable `d = a`, then `d` would point to exactly the same `float` object as `a`. Now, both `a == b` and `a is b` are `True`.

A simple example of the difference between equality and identity is given in Listing 4.21. Here, we declare a `float` variable and store in it the value of $\sqrt{2}$ (as exactly as it can be represented by a Python `float`, that is), which just so happens to be `1.4142135623730951`. We then import the function `sqrt` from the `math` module and compute `sqrt(2.0)`. We store the result of this in a second variable, namely `sqrt_2_computed`. We find that `sqrt_2 == sqrt_2_computed` is `True`, whereas `sqrt_2 is sqrt_2_computed` is `False`. The two values are clearly the same, however, they are stored at two different memory locations. The interpreter did not know that the same value as the result of `sqrt(2.0)` was already stored somewhere in memory. Therefore, it allocated the necessary memory for the result of the `sqrt` operation and stored it there. We now have two variables that reference two objects which both hold the same value.

First Time Readers and Novices: In the rest of this subsection, we discuss (albeit superficially) how objects are cached and reused. First-time readers are welcome to skip over the rest of this subsection.

A slightly more involved example of the issue of object equality and identity is given in Listing 4.23. You see, when the Python interpreter parses the program code file, it will allocate the memory and store the values of constants and literals. Listing 4.23, we declare `a = "Hello World!"` and then `b = "Hello World!"`. In other words, `a` and `b` will receive the same value. As the output in Listing 4.24 shows, the Python interpreter is clever enough not to store this value twice in memory. Instead, `a` and `b` will point to the same object, i.e., `a is b` holds. Interestingly, even if we subsequently do `c = "Hello "+ "World!"`, the interpreter is still clever enough to recognize that this constant expression yields the same result already stored in `a`. It basically computes the result of the concatenation of the two constant strings and discovers this identity. Therefore, `a is c` will also be `True`.

If we use a variable in such an expression, it does no longer work, though: First setting `d = "Hello"` and then setting `d = d + "World!"` will yield an equal result (`a == d` is `True`), but it will be stored somewhere else in memory (`a is d` is `False`). The interpreter, at least in its current version, cannot infer value identities of results of computations that involve values that are not constants.

Well, sometimes it can. It seems that Python, like Java, caches a set of small integers.[2] It is obvious that we need small integer numbers again and again and again in our programs. Allocating

[2] Some sources say that all integers between `-5` and `256` are cached. However, this is highly implementation and configuration specific and may be entirely different on your machine.

Listing 4.23: An second example of the difference between equality and identity. (stored in file `identity_2.py`; output in Listing 4.24)

```python
# String and integer literals and identity.
a: str = "Hello World!"
b: str = "Hello World!"
print(f"Are 'a' and 'b' the same object: {a is b}")

c: str = "Hello " + "World!"
print(f"Are 'a' and 'c' the same object: {a is c}")

d: str = "Hello"
d = d + " World!"
print(f"Are 'a' and 'd' the same object: {a is d}")
print(f"Are 'a' and 'd' equal objects: {a == d}")

e: int = 10
mul: int = 5
f: int = (e * mul) // mul
print(f"Are 'e' and 'f' the same object: {e is f}")

g: int = 1_000_000_000_000_000_000
h: int = (g * mul) // mul
print(f"Are 'g' and 'h' the same object: {g is h}")
print(f"Are 'g' and 'h' equal objects: {g == h}")
```

↓ `python3 identity_2.py` ↓

Listing 4.24: The stdout of the program `identity_2.py` given in Listing 4.23.

```
Are 'a' and 'b' the same object: True
Are 'a' and 'c' the same object: True
Are 'a' and 'd' the same object: False
Are 'a' and 'd' equal objects: True
Are 'e' and 'f' the same object: True
Are 'g' and 'h' the same object: False
Are 'g' and 'h' equal objects: True
```

a new objects whenever we use the values `1` or `0` would be wasteful. In Listing 4.23, we calculate the result of the computation `f = (e * mul)// mul`, where `e` is a variable with value `10` and `mul` is a variable with value `5`. The result is obviously `10` as well, i.e., a small integer. And it is identical to `e`, i.e., `e is f` is `True` – despite begin the result of a computation including only variables.

We now replace `10` with a large value, say we set `g = 1_000_000_000_000_000_000`. We compute `h = (g * mul)// mul`. Of course, `g == h` is `True`. However, `g is h` is `False`. Because this integer value is too large to be cached and a new object is generated.

4.5 Summary

Variables allow us to store and access data. After gaining this ability, we finally can begin to write "real" programs. We do no longer just execute single commands in the Python console. Instead, we can write Python program files (with the name suffix `.py`) that perform computations in multiple steps. Our implementation of the ideas of LIU Hui (刘徽) to approximate the ratio of the circumference of a circle to its diameter, i.e., the number π, Listing 4.3, was a first example. Here, we refined the approximation in several steps and ended up with some pretty good estimate.

The elegant syntax of Python allows us to assign and reassign variables. We can swap the values of two variables `a` and `b` by writing `a, b = b, a`.

However, the elegant and simply syntax also has drawbacks. It does not force us to define the type of a variable. As a result, it does not prevent us from first storing an integer inside a variable and then storing a string into it. This can lead to problems and confusion. The Python interpreter does not care about such potential issues and will happy obey any command we give to it. Therefore, we need

additional tools: Type checkers like Mypy help us to detect such potential problems. They can tell us if we try to store something in a variable which is of a different type than the previous usage of the variable. These programs are applied to the program files, read them, and look for errors.

These type checkers cannot guess whether the previous usage was wrong or our attempt to overwrite the value with one of a different type. Or maybe all is good and we intended from the beginning to allow the variable to store different kinds of objects. To allow us to create some clarity, type hints were introduced. We can annotate variables with type hints that, basically, specify the type of objects we want to be stored in the variables. This gives the dynamically typed Python language the same expressiveness of statically typed languages like Java. The Python interpreter does not care about this and, again, happily ignores all such type hints and obeys all of our commands. Static type checking tools, like Mypy, however, now can see our intend and provide better error messages when they detect an issue.

As final topic in this section, we tackled the issue of equality versus identity. Two variables `x` and `y` are identical if they point to the same object. Then, `x is y` and `y is x` will be `True`. Even if this is not the case, i.e., `x is y` is `False`, then the variables do not point to one and the same object. However, they could still point to two objects which have the same value. Nothing can prevent me from storing `"Hello!"` multiple times at multiple different locations in memory. I can let `x` and `y` point to two different locations both storing the same string `"Hello!"`. Then, `x is y` may be `False`, but `x == y` could be `True`.

Having discussed all of these issues, we can not depart from the interesting topic of variables.

Chapter 5

Collections

We already learned about simple datatypes, like integer and floating point numbers, strings, and Boolean values. We also learned how we can use variables to store instances (objects) of such datatypes. However, in many cases, we do not just want to store a single object. Often, we want to store and process *collections* of objects [20, 27, 32]. `Python` offers us four basic types of collections:

The first one, *lists*, are mutable sequences of objects. We can create a list `my_list` composed of the three strings `"a"`, `"b"`, and `"c"` by writing `my_list = ["a", "b", "c"]`. Like the characters in strings, we can access the elements of lists using square brackets. For instance, `my_list[0]` would return the first element of `my_list`, namely `"a"`. In Section 5.1 we learn more about lists and we also get to know a new static code analysis tool that can help us to detect programming issues.

The second type of collections is formed by *tuples*. Tuples are similar to lists, with two main differences: First, tuples are immutable, which means once a tuple is created, it cannot be changed anymore. Second, the semantics of tuples allows for them to contain objects of different types, whereas lists should only contain elements of a single type. Tuples are created using parentheses, so `my_tuple = (1, 2, "z")` creates a tuple `my_tuple` containing the two integers `1` and `2` as well as the string `z`. The elements can be accessed using square brackets, so `my_tuple[1]` gives us the second element, namely `2`. We learn more about tuples in Section 5.2.

The mathematical notion of sets is implemented in the `Python` datatype `set`. A set can contain each element at most once. The methods for modifying sets and for checking whether elements are contained in them are particularly fast. Sets are created using curly braces, i.e., `my_set = {1.2, 2.3, 4.5, 2.3}` would create the set `my_set` with the three numbers `1.2`, `2.3`, and `4.5`. Notice that `2.3` would appear only once in the set. Sets cannot be indexed, but like lists and tuples they support the `in` operator and `1.2 in my_set` returns `True` while `1.3 in my_set` is `False`. We will discuss sets in Section 5.3.

Finally, dictionaries are mappings between keys and values, similar to hash tables in other programming languages. They, too, are created using curly braces which, however, contain key-value pairs. In other words, `my_dict = {"pi": 3.1416, "e": 2.7183, "phi": 1.618}` creates a dictionary which maps the strings `"pi"`, `"e"`, and `"phi"` to the values `3.1416`, `2.7183`, and `1.618`, respectively. The value of a key can be retrieved using the square-bracket indexing, i.e., `my_dict["e"]` would return `2.7183`. They are discussed in Section 5.4.

5.1 Lists

A `list` is a mutable sequence of objects which can be accessed via their index [120]. They work very similar to the strings we already discussed in Section 3.5, but instead of characters, they can contain any kind of objects and they can be modified.

5.1.1 Basic Functionality and Examples

In Listing 5.1, we provide some first examples for using lists. A list can be defined by simply writing its contents, separated by `,` inside square brackets `[...]`. `["apple", "pear", "orange"]` creates a list with three elements, namely the strings `"apple"`, `"pear"`, and `"orange"`. If we want to store a list in a variable, then we can use the type hint `list[elementType]` where `elementType` is to be replaced with the type of the list elements [81]. `fruits: list[str] = ["apple", "pear", "orange"]` therefore

Listing 5.1: A first example for using lists in Python: creating, indexing, printing of and appending elements and other lists to lists. (stored in file `lists_1.py`; output in Listing 5.2)

```python
"""An example of creating, indexing, and printing lists."""

fruits: list[str] = ["apple", "pear", "orange"]  # Create List.
print(f"We got {len(fruits)} fruits: {fruits}")  # Print length and list.

fruits.append("cherry")  # Append one element at the end of a list.
print(f"There now are {len(fruits)} fruits: {fruits}")

vegetables: list[str] = ["onion", "potato", "leek"]  # Create list.
print(f"The vegetables are: {vegetables}.")  # Print the list.

food: list[str] = []  # Create an empty list.
food.extend(fruits)  # Append all elements of `fruits` to `food`.
food.extend(vegetables)  # Append all elements of `vegetables` to `food`.
print(f"Fruits and vegetables: {food}")  # Print the new list.
print(f"len(food) = {len(food)}")  # Print the length of list `food`.
print(f"food[0] = {food[0]}")  # Print the first element of `food`.
print(f"food[1] = {food[1]}")  # Print the second element of `food`.
print(f"food[2] = {food[2]}")  # Print the third element of `food`.
print(f"food[-1] = {food[-1]}")  # Print the last element of `food`.
print(f"food[-2] = {food[-2]}")  # Print the second-to-last element.
print(f"food[-3] = {food[-3]}")  # Print the third-to-last element.

del food[1]  # Delete the element at index 1 from list `food`.
print(f"Food is now: {food}.")  # Print the list again.
```

↓ `python3 lists_1.py` ↓

Listing 5.2: The stdout of the program `lists_1.py` given in Listing 5.1.

```
We got 3 fruits: ['apple', 'pear', 'orange']
There now are 4 fruits: ['apple', 'pear', 'orange', 'cherry']
The vegetables are: ['onion', 'potato', 'leek'].
Fruits and vegetables: ['apple', 'pear', 'orange', 'cherry', 'onion', '
    ↪ potato', 'leek']
len(food) = 7
food[0] = apple
food[1] = pear
food[2] = orange
food[-1] = leek
food[-2] = potato
food[-3] = onion
Food is now: ['apple', 'orange', 'cherry', 'onion', 'potato', 'leek'].
```

creates the list `fruits` with the contents listed above. It also tells any automated type checking tool and other programmers that we intent that only `str` values should be stored inside the list.

The length of a list is can be obtained using the `len` function. `len(fruits)` will therefore return the value `3`. We can use lists in f-strings just like any other datatype. The string representation of `fruits` which then would be used is simply `"['apple', 'pear', 'orange']"`.

We can add single elements to a list by using the `append` method. Invoking `fruits.append("cherry")` will append the string `"cherry"` to the list `fruits`. The list then equals `["apple", "pear", "orange", "cherry"]` and has `len(fruits)== 4`.

Of course we can have multiple lists in a program. In Listing 5.1, we now create the second list `vegetables` with the three elements `"onion"`, `"potato"`, and `"leek"`.

An empty list is created with expression `[]`, which consists of just the square brackets with no contents inside. We can append *all* the elements of one collection to a list by using the `extend` method. We start with the empty list `food` and then invoke `food.extend(fruits)`. Now all the

Listing 5.3: A second example of using lists in Python: inserting and deleting elements, sorting and reversing lists. (stored in file `lists_2.py`; output in Listing 5.4)

```python
"""An example of creating, modifying, sorting, and copying lists."""

numbers: list[int] = [1, 7, 56, 2, 4]  # Create the list.
print(f"The numbers are: {numbers}.")  # Print the list.

print(f"is 7 in the list: {7 in numbers}")  # Check if 7 is in the list.
print(f"is 2 NOT in the list: {2 not in numbers}")  # the opposite check
print(f"7 ist at index {numbers.index(7)}.")  # Search for number 7.
print(f"2 ist at index {numbers.index(2)}.")  # Search for number 2.

numbers.insert(2, 12)  # Insert the number 12 at index 2...
print(f"After inserting 12, the numbers are: {numbers}.")  # and print.

numbers.remove(56)  # Remove the number 56 from the list.
print(f"After removing 56, numbers are: {numbers}.")  # Print the list.

numbers.sort()  # Sort the list `numbers` in place.
print(f"The sorted numbers are: {numbers}.")  # Print the list.

numbers.reverse()  # Reverse the order of the list elements.
print(f"The reversed numbers are: {numbers}.")  # And print the list.

cpy: list[int] = list(numbers)  # Create a copy of the list `numbers`.
print(f"cpy == numbers: {cpy == numbers}.")  # Indeed, `cpy == numbers`.
print(f"cpy is numbers: {cpy is numbers}.")  # No, `cpy is not numbers`.

del cpy[0]  # We change `cpy`, but `numbers` remains unchanged.
print(f"cpy == numbers: {cpy == numbers}.")  # Now, `cpy != numbers`.
print(f"cpy is numbers: {cpy is numbers}.")  # And `cpy is not numbers`.
print(f"cpy is not numbers: {cpy is not numbers}.")  # indeed, it is not.
```

↓ `python3 lists_2.py` ↓

Listing 5.4: The stdout of the program `lists_2.py` given in Listing 5.3.

```
The numbers are: [1, 7, 56, 2, 4].
is 7 in the list: True
is 2 NOT in the list: False
7 ist at index 1.
2 ist at index 3.
After inserting 12, the numbers are: [1, 7, 12, 56, 2, 4].
After removing 56, numbers are: [1, 7, 12, 2, 4].
The sorted numbers are: [1, 2, 4, 7, 12].
The reversed numbers are: [12, 7, 4, 2, 1].
cpy == numbers: True.
cpy is numbers: False.
cpy == numbers: False.
cpy is numbers: False.
cpy is not numbers: True.
```

contents of the list `fruits` are appended to `food`. We then invoke `food.extend(vegetables)`, which will add all the elements from the list `vegetables` to `food` as well. `fruits` and `vegetables` remain unchanged during this procedure, but `food` now contains all of their elements as well. It contains all seven fruits and vegetables and its `len(food)` is therefore `7`.

We can access the elements of a list by their index, again in the same way we access the characters in a string. `food[0]` returns the first element of the list `food`, which is `"apple"`. `food[1]` returns the second element of the list `food`, which is `"pear"`. And so on. We can also access the elements using the end of the list as reference: `food[-1]` returns the last element of the list `food`, which is `"leek"`. `food[-2]` returns the second-to-last element of the list `food`, which is `"potato"`. And so on.

Listing 5.5: A third example of using lists in Python: slicing, adding, and multiplying lists. (stored in file `lists_3.py`; output in Listing 5.6)

```python
"""An example of more operations with lists."""

l1: list[int] = [1, 2, 3, 4]  # create first list
l2: list[int] = [5, 6, 7]  # create second list
l3: list[int] = l1 + l2  # l3 = concatenation of l1 and l2.
print(f"l3 = l1 + l2 == {l3}")  # [1, 2, 3, 4, 5, 6, 7]

l4: list[int] = l2 * 3  # l4 = l2, repeated three times.
print(f"l4 = l2 * 3 == {l4}")  # [5, 6, 7, 5, 6, 7, 5, 6, 7]

l5: list[int] = l4[2:-2]  # l5 = l4 from index 2 to 3rd from end
print(f"l5 = l4[2:-2] == {l5}")  # [7, 5, 6, 7, 5]

l6: list[int] = l4[1::2]  # start at index 1, take every 2nd element
print(f"l6 = l4[1::2] == {l6}")  # [6, 5, 7, 6]

# Start copying l4 at last element, move backwards take every 2nd
# element, and stop right before index=3.
l7: list[int] = l4[-1:3:-2]
print(f"l7 = l4[-1:3:-2] == {l7}")  # [7, 5, 6]

l7[1] = 12  # Modify the slice l7 originally from l4.
print(f"l4={l4}, l7={l7}")  # Shows that l4 remains unchanged.

a, b, c = l2  # store the three elements of l2 into variables
print(f"a={a}, b={b}, c={c}")  # a=5, b=6, c=7
```

↓ `python3 lists_3.py` ↓

Listing 5.6: The stdout of the program `lists_3.py` given in Listing 5.5.

```
l3 = l1 + l2 == [1, 2, 3, 4, 5, 6, 7]
l4 = l2 * 3 == [5, 6, 7, 5, 6, 7, 5, 6, 7]
l5 = l4[2:-2] == [7, 5, 6, 7, 5]
l6 = l4[1::2] == [6, 5, 7, 6]
l7 = l4[-1:3:-2] == [7, 5, 6]
l4=[5, 6, 7, 5, 6, 7, 5, 6, 7], l7=[7, 12, 6]
a=5, b=6, c=7
```

Finally, elements can also be deleted from the list by their index. `del food[1]` deletes the second element from the list `food`. The second element is `pear` and if we print `food` again, it has indeed disappeared.

In Listing 5.3, we illustrate some more operations on lists. We begin again by creating a list, this time of numbers: `numbers: list[int] = [1, 7, 56, 2, 4]` creates (and type-hints) a list of five integers. If we want to know whether an element is included in a list, we can use the `in` operator. `7 in numbers` returns `True` if `7` is located somewhere inside the list `numbers` and `False` otherwise. The `not in` operator inquires the exact opposite: `2 not in numbers` becomes `True` if `2` is *not* in the list `numbers` and is `False` if it is in the list. If we want to know at which index a certain element in the list is located, we can use the `index` method. `numbers.index(7)` will search where the number `7` is located inside `numbers`. Since it is the second elements and indices start at 0, it returns `1`. Similarly, `numbers.index(7)` returns `3`, because `numbers[3] == 2`.

The `insert` method allows us insert an element at a specific index. The elements which are currently at that or higher indices are moved up one slot. `numbers.insert(2, 12)` will insert the number `12` at index `2` into the list `numbers`. The element `56` which currently occupies this spot is moved to index `3`, which means that the `2` located at this place is moved to index `4`, which means that the value `4` which right now is stored in this location will move to index `5`. The list `numbers` now looks like this: `[1, 7, 12, 56, 2, 4]`.

If we want to remove a specific element from the list without knowing its location, the `remove` method will do the trick. `numbers.remove(56)` searches through the list `numbers` for the element `56` and, once it finds it, deletes it. The list becomes `[1, 7, 12, 2, 4]`.

We can sort a list inplace by using the `sort` method. `numbers.sort()` sorts the list `numbers`, which then becomes `[1, 2, 4, 7, 12]`. Similarly, we can reverse a list, i.e., make the last element become the first, the second-to-last element the second, and so on, by using the method `reverse`. Reversing the list `numbers` after we sorted it will turn it into `[12, 7, 4, 2, 1]`.

If we want to create a list copy of an existing sequence, we can just invoke the constructor `list` directly. `cpy: list[int] = list(numbers)` creates the new list `cpy` which has the same contents as `numbers`. This means that `cpy == numbers` will be `True`, because `cpy` is an exact copy of `numbers`. `cpy is numbers`, however, is `False`. They are not the same object.

We can change `cpy` by deleting its first element via `del cpy[0]`. `numbers` will be unaffected by this and stays unchanged. Now, both `cpy == numbers` and `cpy is numbers` will be `False`. By the way, in the same manner as `not in` is the opposite of the `in` operator, `not is` is the opposite of `is`: `cpy is not numbers` yields `True` because `cpy` is not the same object as `numbers`.

In Listing 5.5, we continue our journey through the magical land of Python `lists`. You can add two lists `a` and `b` using `+`, i.e., do `c = a + b`. The result is a new list which contains all elements of the first list `a` followed by all the elements from the second list `b`. Therefore, the expression `[1, 2, 3, 4] + [5, 6, 7]` results in `[1, 2, 3, 4, 5, 6, 7]`. Similarly, if you multiply a list by an integer `n`, you get a new list which equals the old list `n` times concatenated. `[5, 6, 7] * 3` therefore yields `[5, 6, 7, 5, 6, 7, 5, 6, 7]`.

In Section 3.5.1, we discussed string slicing. Lists can be sliced in pretty much the same way [120]. When slicing a list `l` or a string, you can provide either two or three values in the square brackets, i.e., either do `l[i:j]` or `l[i:j:k]`. If `j < 0`, then it is replaced with `len(l) - j`. In both the two and three indices case, `i` is the inclusive start index and `j` is the exclusive end index, i.e., all elements with index `m` such that `i <= m < j`. In other words, the slice will contain elements from `l` whose index is between `i` and `j`, including the element at index `i` but *not* including the element at index `j`. If a third index `k` is provided, the it is the step length. `l[i:j:k]` selects all items at indices `m` where `m = i + n*k`, `n >= 0` and `i <= m < j`. If `i` is omitted, i.e., if `:j:k` is provided, then `i=0` is assumed. If `j` is omitted, i.e., if `i::k` is provided, then `j=len(l)` is assumed.

If you have a list `l4 = [5, 6, 7, 5, 6, 7, 5, 6, 7]`, then the slice `l4[2:-2]` will return a new list which contains all the elements of `l4` starting from index `2` and up to (excluding) the second-to-last element. The slice `l4[1::2]` starts at index `1`, continues until the end of the list, and adds every second element. This results in `[6, 5, 7, 6]`. As final example, consider the slice `l4[-1:3:-2]`. It will begin creating the new list at the last element. The step-length is `-2`, so it will move backwards and add every second element to the new list. It stops adding elements before reaching index `3`. Therefore, the result will be the new list `l7 = [7, 5, 6]`.

Notice that the slices we create are independent copies of ranges of the original lists. The list `l7` is a slice from the list `l4`. If we modify it, e.g., set `l7[1] = 12`, then we set the second element of `l7` to `12`. `l7` becomes `[7, 12, 6]`. Now, the second element of `l7` originally is the seventh element of `l4`, namely the `5` located at index `6`, which is equivalent to index `-3`. You may wonder whether this element now also has changed. It did not. `l4` remains unchanged by any operation on the independent copied slice `l7`.

An interesting functionality is also list unpacking. In Section 3.5.1, the list `l2` contains the three elements `[5, 6, 7]`. If we know the number of elements in the list in our program, then we can assign them to exactly the same number of variables. `a, b, c = l2` creates and assigns values to three variables `a=5`, `b=6`, and `c=7` by unpacking the list `l2`.

5.1.2 An Example of Errors and a new Tool

Now, in the previous chapter, we learned that static code analysis tools can help us to discover subtle problems in our programs. Obviously, when dealing with more complex datastructures like lists, there are also more potential problems, more mistakes that one could make. Let us look at the very short example Listing 5.7. The program consists of only two lines, `my_list: list[str] = list([1, 2, 3])` and `print(my_list)`. It does not have any *error* in the strict sense. We can execute it just fine and it will produce the output `[1, 2, 3]` as shown in Listing 5.8.

Listing 5.7: A program processing lists which exhibits some subtle errors and inefficiencies. (stored in file `lists_error.py`; output in Listing 5.8)

```
my_list: list[str] = list([1, 2, 3])
print(my_list)
```

↓ `python3 lists_error.py` ↓

Listing 5.8: The stdout of the program `lists_error.py` given in Listing 5.7.

```
[1, 2, 3]
```

Listing 5.9: The results of static type checking with Mypy of the program given in Listing 5.7.

```
$ mypy lists_error.py --no-strict-optional --check-untyped-defs
lists_error.py:1: error: List item 0 has incompatible type "int"; expected
    ↪ "str"  [list-item]
lists_error.py:1: error: List item 1 has incompatible type "int"; expected
    ↪ "str"  [list-item]
lists_error.py:1: error: List item 2 has incompatible type "int"; expected
    ↪ "str"  [list-item]
Found 3 errors in 1 file (checked 1 source file)
# mypy 1.13.0 failed with exit code 1.
```

Figure 5.1: Installing Ruff in a Ubuntu terminal via pip.

However, upon closer inspection, we discover some issues. In a first step, we would apply Mypy (as Useful Tool 1) to check for problems with the types of variables. And indeed, Listing 5.9 shows us *three* errors! We defined `my_list` as a list of strings by using the type hint `list[str]`. However, we then set its value to be a list of three integer numbers (hence, three errors). As promised in the title of this section, we will also use another tool to analyze this program: Ruff.

Useful Tool 2 Ruff is a very fast Python linter that checks the code for all kinds of problems, ranging from formatting and style issues over missing documentation to performance problems and potential errors [88]. It can be installed via `pip install ruff` as shown in Figure 5.1 on page 74. You can then apply Ruff using the command `ruff check fileToScan.py`. We provide a script for using Ruff with a reasonable default configuration in Listing 10.2 on page 176.

Let us apply Ruff to the program `lists_error.py` given in Listing 5.7, which produces the output Listing 5.10. Ruff finds two errors in this file: First, it complains that any python file should start with multi-line string specifying the purpose of the file. The use of such docstrings makes it easier for other programmers to understand what is done by which file in projects that are composed of multiple Python scripts.

Listing 5.10: The results of linting with Ruff of the program given in Listing 5.7. (We used the script given in Listing 10.2 on page 176 to apply Ruff.)

```
$ ruff check --target-version py312 --select=A,AIR,ANN,ASYNC,B,BLE,C,C4,COM
    ↪ ,D,DJ,DTZ,E,ERA,EXE,F,FA,FIX,FLY,FURB,G,I,ICN,INP,ISC,INT,LOG,N,NPY,
    ↪ PERF,PIE,PLC,PLE,PLR,PLW,PT,PYI,Q,RET,RSE,RUF,S,SIM,T,T10,TD,TID,TRY
    ↪ ,UP,W,YTT --ignore=A005,ANN001,ANN002,ANN003,ANN101,ANN204,ANN401,
    ↪ B008,B009,B010,C901,D203,D208,D212,D401,D407,D413,INP001,N801,
    ↪ PLC2801,PLR0904,PLR0911,PLR0912,PLR0913,PLR0914,PLR0915,PLR0916,
    ↪ PLR0917,PLR1702,PLR2004,PLR6301,PT011,PT012,PT013,PYI041,RUF100,S,
    ↪ T201,TRY003,UP035,W --line-length 79 lists_error.py
lists_error.py:1:1: D100 Missing docstring in public module
lists_error.py:1:22: C410 Unnecessary `list` literal passed to `list()` (
    ↪ remove the outer call to `list()`)
  |
1 | my_list: list[str] = list([1, 2, 3])
  |                      ^^^^^^^^^^^^^^^ C410
2 | print(my_list)
  |
  = help: Remove outer `list` call

Found 2 errors.
No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes`
    ↪ option).
# ruff 0.7.2 failed with exit code 1.
```

Best Practice 12 Each Python file should start with a string describing its purpose [49]. This can either be a single line, like a headline, or a longer text. In the second case, the first line must be a headline, followed by an empty line, followed by the rest of the text. Either way, it must be a string delimited by `"""..."""` [49, 143].

Additionally, Ruff finds that writing `list([1, 2, 3])` is actually useless waste of speed and memory: It basically creates a list via `[1, 2, 3]` and then immediately makes a copy of it via the `list` function wrapped around the list specification. We can leave this outer call to `list` away.

In Listing 5.11 we implement the three recommendations from the two tools. We change the type hint of the list to `list[int]`, which solves the type confusion that Mypy discovered. We remove the useless copying of the list as Ruff recommended. Finally, we add a proper docstring at the top of the file, in which we even document the changes we applied. The output Listing 5.12 of the new program remains the same. But now, both tools are satisfied, as shown in Listings 5.13 and 5.14. And our program is much clearer and faster.

Best Practice 13 Use many static code analysis tools and use them always. They can discover a wide variety of issues, problems, or potential improvements. They can help you to keep your code clean and to enforce a good programming style. Do not just apply them, but also *implement* their suggestions where possible.

Listing 5.11: The corrected version of Listing 5.7, taking into account the information given by Mypy in Listing 5.9 and Ruff in Listing 5.10. (stored in file `lists_fixed.py`; output in Listing 5.12)

```
"""
A fixed version of the original, erroneous program.

The original program was only two lines, namely:

> my_list: list[str] = list([1, 2, 3])
> print(my_list)

There were three errors:

1. mypy will detect that we store integers in a list of str.
2. ruff finds the missing docstring at the program head.
3. ruff finds that writing [1, 2, 3] is better than list([1, 2, 3]).

We now fix it here.
"""
my_list: list[int] = [1, 2, 3]
print(my_list)
```

↓ `python3 lists_fixed.py` ↓

Listing 5.12: The stdout of the program `lists_fixed.py` given in Listing 5.11.

```
[1, 2, 3]
```

Well, this was only a two-line program. But ask yourself: Did you spot the incorrect type hint when you read the program? Did you see that we actually created a list and then copied it instead of using it directly? (The docstring I give you, no chance of seeing that as we did not mention it before.) Imagine that your job would be to work on a program with thousands of lines that was developed by a colleague. Wouldn't you love it if that colleague had thoroughly documented and type-hinted and checked their code? Be that colleague.

Listing 5.13: The results of static type checking with Mypy of the program given in Listing 5.11.

```
$ mypy lists_fixed.py --no-strict-optional --check-untyped-defs
Success: no issues found in 1 source file
# mypy 1.13.0 succeeded with exit code 0.
```

Listing 5.14: The results of static type checking with Ruff of the program given in Listing 5.11.

```
$ ruff check --target-version py312 --select=A,AIR,ANN,ASYNC,B,BLE,C,C4,COM
    ↪ ,D,DJ,DTZ,E,ERA,EXE,F,FA,FIX,FLY,FURB,G,I,ICN,INP,ISC,INT,LOG,N,NPY,
    ↪ PERF,PIE,PLC,PLE,PLR,PLW,PT,PYI,Q,RET,RSE,RUF,S,SIM,T,T10,TD,TID,TRY
    ↪ ,UP,W,YTT --ignore=A005,ANN001,ANN002,ANN003,ANN101,ANN204,ANN401,
    ↪ B008,B009,B010,C901,D203,D208,D212,D401,D407,D413,INP001,N801,
    ↪ PLC2801,PLR0904,PLR0911,PLR0912,PLR0913,PLR0914,PLR0915,PLR0916,
    ↪ PLR0917,PLR1702,PLR2004,PLR6301,PT011,PT012,PT013,PYI041,RUF100,S,
    ↪ T201,TRY003,UP035,W --line-length 79 lists_fixed.py
All checks passed!
# ruff 0.7.2 succeeded with exit code 0.
```

5.2 Tuples

Tuples are very similar to lists, with three differences: First, they are immutable. You cannot add, delete, or change the elements of a tuple. Second, on a semantic level, lists are intended to hold objects of the same type. The type hint `list[int]` indicates that we want something to be list of integer numbers. While the Python interpreter permits us to ignore this and still store arbitrary objects in that list, this would violate the idea behind lists. Tuples, on the other hand, are designed to contain elements of different types. Since they cannot be changed, it will always be clear which element of which type is at which location. Three, tuples are defined using parentheses instead of square brackets, i.e., with `(...)`.

Best Practice 14 When you need to use an indexable sequence of objects, use a list if you intent to modify this sequence. If you do not intent to change the sequence, use a tuple.

Listing 5.15 shows us some of the things we can do with tuples. We create a tuple `fruits` to hold three strings. The proper type hint for this is `tuple[str, str, str]` [81]. The line `fruits: tuple[str, str, str] = ("apple", "pear", "orange")` thus creates a tuple `fruits` which contains three strings, namely `"apple"`, `"pear"`, and `"orange"` We also annotated the variable with a type hint informing any static code analysis tool that we indeed intend to store three strings in the tuple. Notice that the tuple is defined using parentheses, whereas a list with the same content would have been defined using square brackets, e.g., as `fruits: list[str] = ["apple", "pear", "orange"]`.

If we are not sure about the actual number of elements that we will put in a tuple but know that they are all of the same type, we can use the ellipsis `...` in the type hint. `tuple[str, ...]` denotes a tuple that can receive an arbitrary amount of strings. The line `veggies: tuple[str, ...] = ("onion", "potato", "leek", "garlic")` is therefore correct and defined a tuple `veggies` consisting of four strings.

The elements of tuples can be accessed using the normal square brackets, exactly like list elements. `veggies[0]` gives us the first element of the tuple `veggies`, namely `"onion"`. `veggies[1]` gives us the second element of the tuple `veggies`, namely `"potato"`. Negative indices also work and index the tuple from the end. `veggies[-1]` gives us the last element of the tuple `veggies`, namely `"garlic"`. `veggies[-2]` gives us the second-to-last element of the tuple `veggies`, namely `"leek"`.

Tuples support many of the same non-modifying operators as lists. The `in` and `not in` operators both work. We quickly test only the former by asking `'pear' in fruits`, which evaluates to `True` and `'pear' in veggies`, which is `False`. The `index`, too, is available and `fruits.index('apple')` returns `0` because `"apple"` is the very first element of `fruits`.

In Listing 5.17 we explore tuples containing elements of multiple types. The type hint `tuple[str, int, float]` states that we want to define a tuple where the first element is a string, the second element is an integer number, and the third element is a floating point number. The line `mixed: tuple[str, int, float] = ("apple", 12, 1e25)` stores such a tuple in the variable `mixed`. The first element of the tuple is the string `"apple"`. The second element is the integer `12` and the third element is the floating point number `1e25`, i.e., 10^{25}. With the line `other: tuple[str, int, float] = ("pear", 1, 1.2)` we create another such tuple.

We now want to create a list of such tuples. This list should, of course, also be annotated with the proper type hint. This is easy: The type hint for our kind of tuples is `tuple[str, int, float]`. The type hint for a list is `list[elementType]`, where `elementType` is the type of the elements. If we want to create a list containing tuples of our kind, then the proper type hint would be `list[tuple[str, int, float]]`. Having realized this, we can now create the list `tuples`. We use the square bracket syntax for this. As first element, we put the tuple stored in the variable `mixed`.

Listing 5.21: The results of static type checking with Mypy of the program given in Listing 5.19.

```
$ mypy tuples_3.py --no-strict-optional --check-untyped-defs
tuples_3.py:11: error: Unsupported target for indexed assignment ("tuple[
    ↪ int, list[int]]")  [index]
Found 1 error in 1 file (checked 1 source file)
# mypy 1.13.0 failed with exit code 1.
```

Listing 5.15: A first example of using tuples in Python: creating, indexing, and printing of tuples. (stored in file `tuples_1.py`; output in Listing 5.16)

```python
"""An example of creating, indexing, and printing tuples."""

fruits: tuple[str, str, str] = ("apple", "pear", "orange")
print(f"We got {len(fruits)} fruits: {fruits}")

veggies: tuple[str, ...] = ("onion", "potato", "leek", "garlic")
print(f"The vegetables are: {veggies}.")  # Print the tuple.

print(f"veggies[0] = {veggies[0]}")  # first element of `veggies`.
print(f"veggies[1] = {veggies[1]}")  # second element of `veggies`.
print(f"veggies[-1] = {veggies[-1]}")  # last element of `veggies`.
print(f"veggies[-2] = {veggies[-2]}")  # second-to-last element.

print(f"is pear in fruits: {'pear' in fruits}")
print(f"is pear in veggies: {'pear' in veggies}")
print(f"apple is at index {fruits.index('apple')} in fruits.")
```

↓ `python3 tuples_1.py` ↓

Listing 5.16: The stdout of the program `tuples_1.py` given in Listing 5.15.

```
We got 3 fruits: ('apple', 'pear', 'orange')
The vegetables are: ('onion', 'potato', 'leek', 'garlic').
veggies[0] = onion
veggies[1] = potato
veggies[-1] = garlic
veggies[-2] = leek
is pear in fruits: True
is pear in veggies: False
apple is at index 0 in fruits.
```

Then we simply define another tuple `("pear", -2, 4.5)` inline. The third element is the tuple stored in variable `other`. As last element, we again define a tuple `("pear", -2, 3.3)`. Listing 5.18 shows that we can print this list of tuples just like any other list (using an f-string).

There is an order defined on tuples. Two tuples `x` and `y` are compared elementwise lexicographically. If the first element of `x` is less than the first element of `y`, then `x < y`. If the first element of `x` is greater than the first element of `y`, then `x > y`. If the first elements of both tuples are the same, the comparison continues at the second element, and so on. This means that we can sort our list of tuples by writing `tuples.sort()`. As a result, the tuple with the string `"apples"` remains at the first position. All other tuples have `"pear"` as first element and thus are "greater." Two of them have the integer `-2` as the second element and thus are located before the tuple with `1` as second element. Among these two tuples, the one with the smallest third element comes first.

Finally, this example program shows us that we can unpack tuples just like lists. `a, b, c = mixed` stores `"apple"` in `a`, `12` in `b`, and `1e25` in `c`. Interestingly, we can also create tuples the other way around. We can leave away the parentheses when creating tuples in cases where no confusion is possible. `mixed = "x", 4, 4.5` has the same effect as `mixed = ("x", 4, 4.5)`.

Let us finally explore the immutability of tuples in Listing 5.17. As stated before, we are not permitted to add, remove, insert, delete, or overwrite any element of a tuple. What happens if we try create a tuple that contains a list? The line `mt: tuple[int, list[int]] = (1, [2])` does just this. It creates a new tuple `mt` whose first element is the integer number `1` and whose second element is a list of integers, namely `[2]`. We can access this list via `mt[1]`. While the tuple cannot be changed, this list can! `mt[1].append(2)` will append the number `2` to the list, which now is `[2, 2]`. Printing `f"mt == mt"` therefore yields `mt == (1, [2, 2])`.

What we are never permitted to do is to change the contents of the tuple itself. Trying to do `mt[1] = [3, 4]` will raise a `TypeError`. This is an exception which, unless properly caught and handled, will terminate the program immediately. Listing 5.20 shows us this. While the first two `print` commands

Listing 5.17: A second example of using tuples in Python: tuples with elements of different types and tuple unpacking. (stored in file `tuples_2.py`; output in Listing 5.18)

```
"""An example of creating tuples of mixed types."""

mixed: tuple[str, int, float] = ("apple", 12, 1e25)  # mixed types
print(f"The mixed tuple is {mixed}.")  # print the tuple

other: tuple[str, int, float] = ("pear", 1, 1.2)  # second such tuple
print(f"the other tuple: {other}.")  # print  it as well

tuples: list[tuple[str, int, float]] = [  # create a list of 4 tuples
    mixed, ("pear", -2, 4.5), other, ("pear", -2, 3.3)]
print(f"tuples list: {tuples}.")  # print that list

tuples.sort()  # sort the list
print(f"sorted tuples list: {tuples}.")

a, b, c = mixed  # we unpack the tuple
print(f"a={a}, b={b}, c={c}")

mixed = "x", 4, 4.5
print(f"mixed is now: {mixed}")
```

↓ `python3 tuples_2.py` ↓

Listing 5.18: The stdout of the program `tuples_2.py` given in Listing 5.17.

```
The mixed tuple is ('apple', 12, 1e+25).
the other tuple: ('pear', 1, 1.2).
tuples list: [('apple', 12, 1e+25), ('pear', -2, 4.5), ('pear', 1, 1.2), ('
    ↪ pear', -2, 3.3)].
sorted tuples list: [('apple', 12, 1e+25), ('pear', -2, 3.3), ('pear', -2,
    ↪ 4.5), ('pear', 1, 1.2)].
a=apple, b=12, c=1e+25
mixed is now: ('x', 4, 4.5)
```

still succeed, the program crashes when we lay our hands on the tuple directly. The Python interpreter is kind enough to print the error as well as where it happened (see also later in Chapter 9). This would help the programmer to find the problem.

Had the programmer used Mypy to check the code before running it, however, they would have found the error already. The output of Mypy given in Listing 5.21 shows us that a tuple is an *Unsupported target for indexed assignment*. And indeed it is.

Best Practice 15 Only put immutable objects into tuples. Mutable objects inside tuples makes the tuples modifiable as well, while other programmers may assume that they are immutable. This can lead to strange errors down the line.

In summary, tuples are like lists, just immutable and they are semantically conceptualized to contain objects of different types. Careful with immutability, though, as it can only be guaranteed if the tuple itself consists of only immutable objects.

Listing 5.19: A third example of using tuples in Python: testing the immutability property. (stored in file `tuples_3.py`; output in Listing 5.20)

```python
"""An example of testing the immutability of tuples."""

# Create a tuple consisting of an immutable object (the integer '1') and
# a mutable object (the list [2]).
mt: tuple[int, list[int]] = (1, [2])
print(f"mt == {mt}")  # This prints mt == (1, [2])

mt[1].append(2)  # We can actually change the list inside the tuple.
print(f"mt == {mt}")  # This prints mt == (1, [2, 2])

mt[1] = [3, 4]  # However, this will fail with an TypeError exception.
print(f"mt == {mt}")  # ...and we never reach this part.
```

↓ `python3 tuples_3.py` ↓

Listing 5.20: The stdout and standard error stream (stderr) as well as the exit code of the program `tuples_3.py` given in Listing 5.19.

```
mt == (1, [2])
mt == (1, [2, 2])
Traceback (most recent call last):
  File "{...}/02_collections/tuples_3.py", line 11, in <module>
    mt[1] = [3, 4]  # However, this will fail with an TypeError exception.
    ~~~~~
TypeError: 'tuple' object does not support item assignment
# 'python3 tuples_3.py' failed with exit code 1.
```

5.3 Sets

Lists and tuples are classical sequence-based datastructures. Another classical datastructure is a *set*. Lists and tuples can contain an element arbitrarily often. Sets implement the mathematical notation of sets. Different from tuples and lists, they can contain each element at most once. To ensure this property, sets, like tuples, should only contain immutable objects. If you could change the object, then there would be no way for the set to ensure that no two of its objects can be the same.

Best Practice 16 Only put immutable objects into sets.

Different from tuples, sets themselves are not immutable. Furthermore, sets are *unordered* collections [121]. Therefore, also different from tuples and lists, they cannot be indexed, i.e., the square-bracket notation does not work with sets.

Best Practice 17 Sets are unordered. Never expect anything about how the objects you put into a set are actually stored there.

At this point, you may ask: *What do I need sets for?* Adding and removing objects and checking whether an object is contained in a collection ... lists can do all of that as well. Plus lists are ordered and indexable. So what is the advantage of sets? Sets are fast. In Python, sets are implemented based on the concept of hash tables and thus inherit the computational performance of these datastructures [29, 73, 126]. The operation for checking whether one object is contained in a set can be done in $\mathcal{O}(1)$ in average, whereas lists need $\mathcal{O}(n)$ in average, with n being the list length [4, 57, 95]. This means that checking whether an element is contained in a set costs approximately a constant number of CPU cycles, whereas doing the same for a list takes a number of cycles roughly linear in the length of the list. Lists thus can be preferred if we either only have very few elements to check or if we need to access elements using integer indices. If we have more than just very few elements and/or need to perform set operations such as those discussed in the following text, sets are the way to go.

Listing 5.22: A first example of using sets in Python: creating, modifying, and converting sets. Since sets are unordered, printing them can yield a different result each time a program is executed (see Best Practice 17). (stored in file `sets_1.py`; output in Listing 5.23)

```python
"""An example of creating, modifying, and converting sets."""

upper: set[str] = {"A", "G", "B", "T", "V"}  # Some uppercase letters...
print(f"some uppercase letters are: {upper}")  # Print the set.

upper.add("Z")  # Add the letter "Z" to the set.
upper.add("A")  # The letter "A" is already in the set.
upper.add("Z")  # The letter "Z" is already in the set.
print(f"some more uppercase letters are: {upper}")  # Print the set.

upper.update(["K", "G", "W", "Q", "W"])  # Try to add 5 letters.
print(f"even uppercase letters are: {upper}")  # Print the set.

lower_tuple: tuple[str, ...] = ("b", "i", "j", "c", "t", "i")
lower: set[str] = set(lower_tuple)  # Convert a tuple to a set.
print(f"some lowercase letters are: {lower}")  # Print the set 'lower'.
lower.remove("b")  # Delete letter b from the set of lower case letters.
print(f"lowercase letters after deleting 'b': {lower}")  # Print the set.

letters: set[str] = set(lower)  # Copy the set of lowercase characters.
letters.update(upper)  # Add all uppercase characters.
print(f"some letters are: {letters}")  # Print the set 'letters'.

# Create a sorted list containing all elements of the set letters.
# Warning: Strings are sorted such that uppercase characters come before
# lowercase characters.
letters_list: list[str] = sorted(letters)
print(f"the sorted list of letters is: {letters_list}")
```

↓ `python3 sets_1.py` ↓

Listing 5.23: The stdout of the program `sets_1.py` given in Listing 5.22.

```
some uppercase letters are: {'A', 'V', 'G', 'B', 'T'}
some more uppercase letters are: {'A', 'V', 'Z', 'G', 'B', 'T'}
even uppercase letters are: {'G', 'A', 'V', 'Q', 'W', 'T', 'K', 'Z', 'B'}
some lowercase letters are: {'c', 'b', 'j', 't', 'i'}
lowercase letters after deleting 'b': {'c', 'j', 't', 'i'}
some letters are: {'W', 'G', 'c', 'T', 'A', 'K', 'V', 'j', 'Q', 't', 'Z', '
    ↪ B', 'i'}
the sorted list of letters is: ['A', 'B', 'G', 'K', 'Q', 'T', 'V', 'W', 'Z
    ↪ ', 'c', 'i', 'j', 't']
```

In Listing 5.22, we provide a first example of creating and working with sets. Sets can be created by using curly braces, i.e, `{...}`. They can be type-hinted using the notation `set[elementType]` where `elementType` is the type of elements to be stored in the set [81]. The line `upper: set[str] = {"A", "G", "B", "T", "V"}` creates the variable `upper`, which points to a set of the five uppercase latin characters `"A"`, `"G"`, `"B"`, `"T"`, and `"V"`. The type hint `set[str]` states that this is a set that shall contain only strings.

With the method `add`, we can add new elements to a set. Invoking `upper.add("Z")` will add the string `"Z"` to the set `upper`. The following call to `upper.add("A")` does nothing, because the string `"A"` is already contained in the set. Similarly, invoking `upper.add("Z")` again also does nothing, since `"Z"` is now already a set member as well.

We can also add a batch of elements to the set using the `update` method. `upper.update(["K", "G", "W", "Q", "W"])` attempts to add all five elements in the list `["K", "G", "W", "Q", "W"]` to the set `upper`. However, `"W"` appears twice and hence, is only added once. `"G"` is already a member of the set and therefore not added.

Listing 5.24: A second example of using sets in Python: creating sets and set operations (as illustrated in Figure 5.2). Since sets are unordered, printing them can yield a different result each time a program is executed (see Best Practice 17). (stored in file `sets_2.py`; output in Listing 5.25)

```
"""An example of creating sets and set operations."""

odd: set[int] = {1, 3, 5, 7, 9, 11, 13, 15}  # a subset of odd numbers
print(f"some odd numbers are: {odd}")  # Print the set.
print(f"is 3 \u2208 odd: {3 in odd}")  # Check if 3 is in the set odd.
print(f"is 2 \u2209 odd: {2 not in odd}")  # Check if 2 is NOT in odd.

prime: set[int] = {2, 3, 5, 7, 11, 13}  # a subset of the prime numbers
print(f"some prime numbers are: {prime}")  # Print the set.

set_or: set[int] = odd.union(prime)  # Create a new set as union of both
print(f"{len(set_or)} numbers are in odd \u222A prime: {set_or},")

set_and: set[int] = odd.intersection(prime)  # Compute the intersection.
print(f"{len(set_and)} are in odd \u2229 prime: {set_and},")

only_prime: set[int] = prime.difference(odd)  # Prime but not odd
print(f"{len(only_prime)} are in prime \u2216 odd: {only_prime},")

# Get the numbers that are in one and only one of the two sets.
set_xor: set[int] = odd.symmetric_difference(prime)
print(f"{len(set_xor)} are in (odd \u222A prime) "
      f"\u2216 (odd \u2229 prime): {set_xor}, and")

only_odd: set[int] = odd.difference(prime)  # Odd but not prime
print(f"{len(only_odd)} are in odd \u2216 prime: {only_odd},")
odd.difference_update(prime)  # delete all prime numbers from odd
print(f"after deleting all primes from odd, we get {odd}")
```

↓ `python3 sets_2.py` ↓

Listing 5.25: The stdout of the program `sets_2.py` given in Listing 5.24.

```
some odd numbers are: {1, 3, 5, 7, 9, 11, 13, 15}
is 3 ∈ odd: True
is 2 ∉ odd: True
some prime numbers are: {2, 3, 5, 7, 11, 13}
9 numbers are in odd ∪ prime: {1, 2, 3, 5, 7, 9, 11, 13, 15},
5 are in odd ∩ prime: {3, 5, 7, 11, 13},
1 are in prime \ odd: {2},
4 are in (odd ∪ prime) \ (odd ∩ prime): {1, 2, 9, 15}, and
3 are in odd \ prime: {1, 9, 15},
after deleting all primes from odd, we get {1, 9, 15}
```

We can also create a set by passing a sequence into the constructor `set`. To demonstrate this, we first create the tuple `lower_tuple` to hold the lowercase characters `("b", "i", "j", "c", "t", "i")`. The command `lower: set[str] = set(lower_tuple)` creates the set `lower`, type hints it as set of strings, and lets it contain all the elements of `lower_tuple`. `lower_tuple` contains the letter `"i"` twice, but it will appear only once in the set that we have created. Notice: Just calling `set()` without argument (or with an empty collection inside) creates an empty set. We can delete an element of a set using the `remove` function: `lower.remove("b")` deletes the element `"b"` from the set `lower`.

We now copy the set `upper` by invoking `set(upper)` and obtain the new set `letters`. We add all the elements of set `lower` to the set `letters` by calling `letters.update(lower)`.

We can also convert the set `letters` to a list or tuple. `list(letters)` will achieve the former, `tuple(letters)` `tuple` the latter. However, since sets are unordered [121, 122] and the order in which the elements would appear in the created list or tuple is not defined. They may appear in a strange or unexpected order. Here we therefore use the function `sorted`, which accepts an arbitrary collection

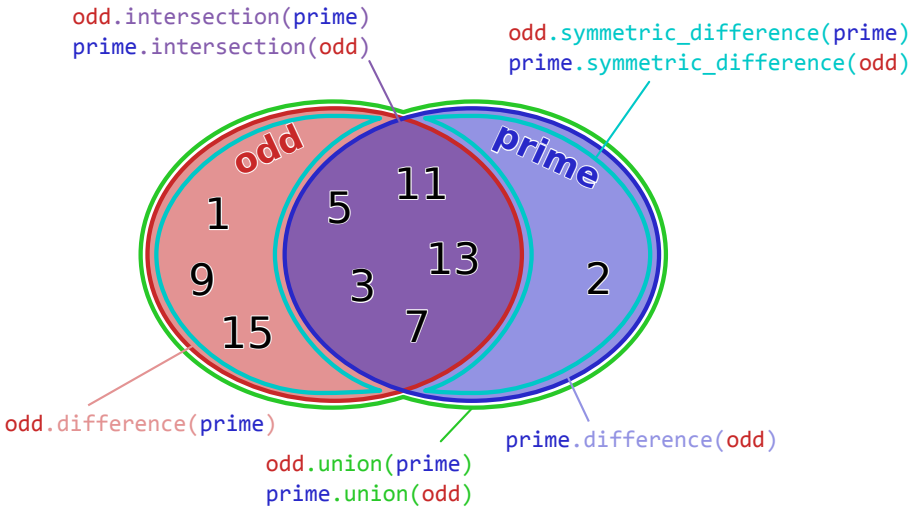


Figure 5.2: An illustration of the set operations in Python as used in Listing 5.24.

of objects as input and returns a sorted list. `sorted(letters)` therefore creates a list where all the elements of letters appear in a sorted fashion. Notice that the natural order of letters is alphabetically, but uppercase letters come before lowercase letters (which sometimes may be confusing as well). Either way, we obtain `letters_list`, which contains all the strings in `letters` in this default sorting order.

Best Practice 18 Careful when sorting or comparing strings: The default order is *uppercase characters before lowercase characters*, i.e., `"A" < "a"`. If you want that upper- and lowercase characters are treated the same (e.g., that `"A"` is considered as equal to `"a"`), as is the case in dictionary ordering, i.e., if you want to sort a collection `my_text` of strings in a case-insensitive manner, use `sorted(my_text, key=str.casefold)`.

In Listing 5.24 we illustrate several basic operations with sets, some of which are visualized in Figure 5.2. This time, we begin by defining a set `odd` containing the first eight odd integers. Sets, like lists and tuples, support both the `in` and `not in` operators. In the context of sets, they are equivalent to the $\in$ and $\notin$ operations from, well, set theory. We illustrate this in our small example using the unicode escape sequences `\u2208` and `\u2209` for both characters (see back in Section 3.5.6). Both `3 in odd` and `2 not in odd` are `True`.

We then define a set `prime` containing the six smallest prime numbers. We can compute the joint set of `prime` and `odd`, i.e., `prime` $\cup$ `odd` by using the `union` method. Both `prime.union(odd)` and `odd.union(prime)` will create a new set containing all the elements from both sets, namely the first eight odd integers and the number `2`. Similarly, the `intersection` method computes the intersection of two sets. Both `prime.intersection(odd)` and `odd.intersection(prime)` compute `odd` $\cap$ `prime`, which consists of the five numbers `3, 5, 7, 11, 13`. If you want to get the set of numbers that are either in `odd` or in `prime` but not in both sets, the method `symmetric_difference` does the job. Both `prime.symmetric_difference(odd)` and `odd.symmetric_difference(prime)` return `{1, 2, 9, 15}`. The elements of `prime` that are *not* in `odd` can be computed using `prime.difference(odd)`. The result, `prime` $\setminus$ `odd`, contains a single element, namely the number `2`. Similarly, we can create the new set `only_odd` containing all the elements of `odd` that are *not* in `prime` by using `odd.difference(prime)`. The result, `odd` $\setminus$ `prime`, contains a the three elements `1`, `9`, and `15`. Notice that all of these operations create new and independent sets. If we want to delete all elements that occur in some container from an existing set, i.e., modify the set, then we use the method `difference_update`. `odd.difference_update(prime)` modifies the set `odd` by deleting all elements from `prime` from it. This operation did not create a new set, but change the existing one. `odd == only_odd` would now hole. It is similar to the `update` method, but deletes elements instead of adding them.

Listing 5.26: An example of using dictionaries in Python. (stored in file `dicts_1.py`; output in Listing 5.27)

```python
"""An example of dictionaries."""

num_str: dict[int, str] = {  # create and type hint a dictionary
    2: "two", 1: "one", 3: "three", 4: "four"}  # the elements
print(f"num_str has {len(num_str)} elements: {num_str}")  # print dict
print(f"I got {num_str[2]} shoes and {num_str[1]} hat.")  # get element

print(f"the keys are: {list(num_str.keys())}")  # print the keys
print(f"the values are: {list(num_str.values())}")  # print the values
print(f"the items are: {list(num_str.items())}")  # the key-value pairs

num_str[5] = "fivv"  # insert (or update) the value of a key
print(f"after adding key 1 num_str is now {num_str}")
num_str[5] = "five"  # update the value of a key
print(f"after updating key 1 num_str is now {num_str}")

del num_str[4]  # delete a key
print(f"after deleting key 4 num_str is now {num_str}")
# get the value of a key, then delete it
print(f"popping key 5 gets us {num_str.pop(5)}")

str_num: dict[str, int] = {}  # create empty dictionary
str_num.update({"one": 1, "three": 3, "two": 2, "four": 4})
print(f"{num_str[1]} + {num_str[2]} = {str_num['three']}")
```

↓ `python3 dicts_1.py` ↓

Listing 5.27: The stdout of the program `dicts_1.py` given in Listing 5.26.

```
num_str has 4 elements: {2: 'two', 1: 'one', 3: 'three', 4: 'four'}
I got two shoes and one hat.
the keys are: [2, 1, 3, 4]
the values are: ['two', 'one', 'three', 'four']
the items are: [(2, 'two'), (1, 'one'), (3, 'three'), (4, 'four')]
after adding key 1 num_str is now {2: 'two', 1: 'one', 3: 'three', 4: 'four
    ↪ ', 5: 'fivv'}
after updating key 1 num_str is now {2: 'two', 1: 'one', 3: 'three', 4: '
    ↪ four', 5: 'five'}
after deleting key 4 num_str is now {2: 'two', 1: 'one', 3: 'three', 5: '
    ↪ five'}
popping key 5 gets us five
one + two = 3
```

5.4 Dictionaries

Dictionaries in Python are containers that store key-value pairs. The value associated with a given key is accessed via the `[...]`-based indexing. The keys are unique and can be of an arbitrary type, as long as they are immutable. The concept of dictionary is also known as hash table or hash map in other languages or domains. Sets and dictionaries in Python are implemented using similar datastructures [4], namely hash tables [29, 73, 126], and thus exhibit similar performance.

Listing 5.26 shows some examples of how we can use dictionaries. A dictionary can be created again using the `{...}` syntax, however, this time, we place comma-separated `key-value` pairs within the curly braces. The type hints for dictionaries is `dict[keyType][valueType]`, where `keyType` is the type for the keys and `valueType` is the type for the values [81]. The line `num_str: dict[int, str] = {2: "two", 1: "one", 3: "three", 4: "four"}` therefore creates a new dictionary and stores it in the variable `num_str`. The type-hint of the variable states that it can point only to dictionaries that have integers as keys and strings as values. The contents of the dictionary are the key-value pairs `2: "two"`, `1: "one"`, `3: "three"`, and `4: "four"`. This means that the value

string `"one"` is stored under the key `1`, the value string `"two"` is stored under the key `2`, and so on.

We can use dictionaries in f-strings like any other Python object. The function `len` can give us the number of elements in a dictionary. For our dictionary `num_str` with four key-value pairs, it returns `4`. The elements of the dictionary can be accessed by their keys using square brackets. Therefore `num_str[2]` will return the string `"two"` associated with key `2` and `num_str[1]` returns the string `"one"`.

We can access the keys, values, and key-value pairs of a dictionary as collections using the `keys`, `values`, and `items` functions, respectively. In the example Listing 5.26, we convert them to lists: `list(num_str.keys())` gives us all the keys in the dictionary `num_str` as a `list`, which thus equals `[2, 1, 3, 4]`. Similarly, `list(num_str.values())` gives us all the values in the dictionary `num_str` as a `list`, which thus equals `["two", "one", "three", "four"]`. Finally, `num_str.items()` returns the key-value pairs a sequence of tuples and wrapping this into a list yields `[(2, "two"), (1, "one"), (3, "three"), (4, "four")]`. You may notice that all of these sequences have the same order of elements that was used when we created the dictionary. They key-value pair `2: "two"` comes before `1: "one"`. This is because dictionaries, different from sets, are ordered datastructures. Their elements appear in all sequenced versions always in the same order in which they were originally inserted into the dictionary [36].

Dictionaries can be modified. We can add an entry simply by assigning a value to a key. `num_str[5] = "fivv"` assigns the value `"fivv"` to the key `5`. This association is now part of the dictionary. Oops, we noticed a typo: We wanted to add `"five"`, not `"fivv"`. `num_str[5] = "five"` will overwrite this faulty assignment. All keys in a dictionary are unique, so there can only be one value associated with key `5`. This also means that dictionary keys must obeye the same requirement as set elements:

Best Practice 19 Dictionary keys must be immutable.

We can also delete key-value pairs. This is done by invoking `del dict[key]`, which deletes the key `key` and its associated value from the dictionary. The method `pop` allows us to get the value of a given key and then immediately deletes the key-value pair from the dictionary. `num_str.pop(5)` will return `"five"` and remove the key-value pair `5: "five"` from the dictionary. Afterwards, requesting `num_str[5]` would lead to an which would terminate the program (unless properly caught).

As stated before, the types for keys and values in a dictionary can be chosen arbitrary, with the limitation that keys must be immutable. We now create the empty dictionary `str_num: dict[str, int] = {}`.[1] Notice that only due to the type hint `dict[str, int]`, other programmers and static type-checking tools can know at this point that we intend to use strings as keys and integers as values in this new dictionary. The `update` method allows us to append the values of an existing dictionary to a given dictionary. `str_num.update({"one": 1, "three": 3, "two": 2, "four": 4})` therefore stores four key-value pairs into our new dictionary. This time, the keys are indeed strings and the values are integers. The slightly convoluted f-string `f"{num_str[1]}+ {num_str[2]}= {str_num['three']}"` evaluates to `"one + two = 3"`. The first two values are looked up in the `num_str` dictionary under keys `1` and `2`, respectively. The last value is obtained from `str_num` under key `"three"`.

5.5 Summary

If you have managed to fight your way through the book until this point, then you can already do quite a few things. You can use the computer like a fancy calculator by evaluating numerical expressions, which you learned in Chapter 3. By utilizing variables as discussed in Chapter 4, you can realize some simple algorithms like approximating π. In this section, you learned about compound datastructures that can store multiple values. You learned about mutable and immutable sequences of values, namely lists and tuples. You also learned that Python offers you the mathematical notion of sets as well as dictionaries, which are key-value mappings. You now have the basic knowledge of the most commonly used datatypes in Python. You know operations to manipulate and query them.

[1] Using the `dict()` without arguments has the same effect, but several linters discourage this and encourage using `{}` instead.

However, when learning about these structures, you may have felt some eerie awkwardness. Something did not feel right. If I can only construct a list step by step, then what is the advantage compared to just using many variables? The missing ingredient are control flow statements. Statements that allow us to iterate over a list. Statements that allow us to perform an action A if an element is contained in a set, for example, and an action B otherwise. The next big part of this book will close this gap and provide you with all the tools necessary to write "real" programs.

Part II

Control Flow Statements

With the things we learned so far, we can write simple linear programs. These programs perform instruction after instruction in exactly the same order in which we write them down. If we want something done twice, we have to write it down twice. Our programs have no way to adapt their behavior based on their input data. They cannot do one thing if a variable has a certain value and do something else otherwise. They do not branch or loop. Now we will learn how to write programs that *do* branch or loop. We will learn statements that change the *control flow*.

Definition 5.1 (Control Flow) The *control flow* is the order in which the statements in a program are executed.

Chapter 6

Conditional Statements

Conditional statements allow us to execute a piece of code if a certain Boolean expression evaluates to `True`. This allows us to let the program make choices, to do one thing in one situation and another thing in a different situation. Conditionals are therefore the most fundamental control flow statement.

6.1 The `if` Statement

A conditional statement is basically a piece of program code which will execute a set of instructions if a condition is met. The condition is provided as the kind of Boolean expressions that we already discussed back in Section 3.4. The syntax for this in `Python` is very simple:

```
if booleanExpression:
    conditional statement 1
    conditional statement 2
    # ...

normal statement 1
normal statement 2
# ...
```

The first line begins with `if` statement, followed by a Boolean expression, followed by a colon (`:`). If – and only if – the Boolean expression evaluates to `True`, then the *indented* block of statements below the `if` are executed. Notice that each of these conditionally executed statements is indented by four spaces compared to the `if`. After the body block of the `if` statement, the normal program code resumes without additional indentation. The statements that are not indented anymore will be executed regardless of the outcome of the Boolean expression.

Best Practice 20 Blocks are indented by four spaces.

Let's use this construct to check whether a year is a leap year. According to the Gregorian calendar, a year y is a leap year if y is divisible by 4 but not by 100; or if it is divisible by 400 [39]. Listing 6.1 implements this rule. We begin by storing the year that we want to investigate in the integer variable `year`. But how can we check the condition?

From Section 3.2 you may remember the modulo division operator `%`: Applying it to two values `a` and `b`, it will return the remainder of the division `a // b`. If the remainder is 0, then `a` can be divided by `b` evenly. For example, `10 % 5 == 0` whereas `10 % 6 == 4` and `10 % 3 == 1`. So to check whether `year` can be divided by 4, we just need to check whether `(year %4) == 0`. Furthermore, `year` should not be divisible by 100, so we also check `(year %100) != 0`, i.e., whether the remainder of the division of `year` by 100 is *not* 0. Well, if that turns out to be `False`, the year could still be a leap year is it was divisible by 400. So the final condition to be checked is `(year %400) == 0`. Now we piece all of this together using the logical `and` and `or` operators and get the condition `(((year %4) == 0) and ((year %100) != 0)) or ((year %400) == 0)`.

All what is left to do is put an `if` in front of the condition and a `:` after it, and we got the conditional check. If it evaluates to `True`, we will print `f"{year}is a leap year."`. We copy this code

Listing 6.1: An example of using the `if` statement. (stored in file `if_example.py`; output in Listing 6.2)

```
"""
An example of using the ‘if‘ statement.

A year is a leap year if it is divisible by 4 but not divisible by 100
or if it is divisible by 400.
We can use the modulo operator ‘%‘ to check this.
‘a % 100‘ will be ‘0‘ if and only ‘a‘ is a multiple of ‘100‘.
The Boolean statements can be combined with ‘and‘ and ‘or‘ and grouped
using parentheses.
"""

year: int = 2024  # the year 2024 should be a leap year
if (((year % 4) == 0) and ((year % 100) != 0)) or ((year % 400) == 0):
    print(f"{year} is a leap year.")  # This line will be executed.

year = 1900  # the year 1900 is not a leap year
if (((year % 4) == 0) and ((year % 100) != 0)) or ((year % 400) == 0):
    print(f"{year} is a leap year.")  # This line is never reached.

print("End of year checking.")  # This text is always printed.
```

↓ `python3 if_example.py` ↓

Listing 6.2: The stdout of the program `if_example.py` given in Listing 6.1.

```
2024 is a leap year.
End of year checking.
```

twice and check the years `2024` (the year when I began writing this book) and `1900` (some other year). Notice that the second `if` starts at the same level of indentation of the first `if`. This means that it is executed regardless of the outcome of the first conditional. After all of this code, we print the text `End of year checking.` to the console. This last line of code is again not indented and therefore will be executed regardless of the two conditional statements before it. The output of our program given in Listing 6.2 confirms that 2024 is indeed a leap year, while 1900 was not.

6.2 The `if...else` Statement

The ability to perform some action if a given expression evaluates to `True` is already nice. However, often we want to perform some action if the expression evaluates to `True` and another action otherwise. Well, we could already do this. We could write the same `if` statement twice, but in the second one we would just wrap `not (...)` around the condition. This seems to be quite verbose and verbosity leads to confusion. So we are offered a better alternative: the `if ... else` statement, which looks like this:

```
if booleanExpression:
    conditional statement 1
    conditional statement 2
    # ...
else:
    alternative statement 1
    alternative statement 2
    # ...

normal statement 1
normal statement 2
# ...
```

The statement begins like a normal `if` statement: `if`, followed by a Boolean expression, followed by a colon (`:`) marks the condition. The following block – where each statement is indented by four spaces

Listing 6.3: An example of using the `if ... else` statement. (stored in file `if_else_example.py`; output in Listing 6.4)

```
"""An example of using the ‘if-else‘ statement."""

year: int = 2024  # the year 2024 should be a leap year
if (((year % 4) == 0) and ((year % 100) != 0)) or ((year % 400) == 0):
    print(f"{year} is a leap year.")  # This line will be executed.
    print("yes, it really is.")  # This line will be executed as well.
else:  # Not leap year.
    print(f"{year} is not a leap year.")  # This line is never reached.

year = 1900  # the year 1900 is not a leap year
if (((year % 4) == 0) and ((year % 100) != 0)) or ((year % 400) == 0):
    print(f"{year} is a leap year.")  # This line is never reached.
else:  # Not leap year.
    print(f"{year} is not a leap year.")  # This line will be executed.
    print("Believe you me, it indeed is not.")  # This line will, too.

print("End of year checking.")  # This text is always printed.
```

↓ `python3 if_else_example.py` ↓

Listing 6.4: The stdout of the program `if_else_example.py` given in Listing 6.3.

```
2024 is a leap year.
yes, it really is.
1900 is not a leap year.
Believe you me, it indeed is not.
End of year checking.
```

– is executed only if the Boolean expression evaluates to `True`. Then, the `else` line follows. Notice that it is at the same indentation level as the original `if`. All the statements in the following block are again indented by four spaces. They are only executed if the Boolean expression used in the `if` line evaluated to `False`. This now allows us to, for example, print one text if a year is a leap year and print another text otherwise. In Listing 6.3 we show exactly this functionality, together with the fact that there can be multiple statements both in the `if` and the `else` block.

Well, if there can be multiple statements in these blocks and `if` and `if...else` are also both *statements*, then can we nest them? Can we place an `if` statement inside the body of another `if` statement?

Yes, we can, and we explore this in Listing 6.5. Here, we have three numbers `a`, `b`, and `c`. We want to know the maximum, i.e., the largest number stored in either `a`, `b`, or `c`. For this purpose, we first check if `a > b`. If this is `True`, then `b` cannot contain the largest number and it must be stored in either `a` or `c`. Therefore, inside the body of that first `if` statement, we only have to check if `a > c`. Notice how this second `if` and its whole associated `else` and body are indented by another four spaces compared to the outer `if`. If this turns out to be `True`, we print the f-string `f"{a} is the greatest number."`. If this is `False`, then it could either be that `a < c` or that `a == c`. Either way, printing the f-string `f"{c} is the greatest number."` will yield the correct result. Now we need to return to the outer `if` and tackle its alternative branch, the `else` part. What happens if `a > b` did not evaluate to `True`? Well, this case, either `b > a` or `b == a`, which means that it is sufficient to compare `b` with `c` to get the result. Therefore, we again indent a new `if`, this time with the condition `b > c`. If that one is `True`, then the largest value is equal to the one stored in `b` and we can print `f"{b} is the greatest number."`. If that was `False`, we come to the `else` branch. Here, we know that `b >= a` and that either `c > b` or `c >= b`. Therefore, we again print the f-string `f"{c} is the greatest number."`.

Just for fun, I included two more lines in Listing 6.5 that show two useful functions we did not yet learn before: `max` and `min`. Both receive a sequence of values and return the largest and smallest value, respectively. Therefore, the same work as with our nested `if` can be achieved by computing `max(a, b, c)`. The minimum is obtained using `min(a, b, c)`.

Listing 6.5: An example of using nested `if ... else` statements. (stored in file `if_else_nested.py`; output in Listing 6.6)

```
"""An example of using the nested ‘if-else‘ statements."""

a: int = 13  # the first number
b: int = 7  # the second number
c: int = 9  # the third number

if a > b:
    if a > c:  # This means that a > b and a > c.
        print(f"{a} is the greatest number.")
    else:  # This means that a > b and c >= a.
        print(f"{c} is the greatest number.")
else:  # This means that b >= a.
    if b > c:  # This means that b >= a and b > c.
        print(f"{b} is the greatest number.")
    else:  # This means that b >= a and c >= b.
        print(f"{c} is the greatest number.")

# Some side information: The max and min function are very useful.
print(f"The maximum is {max(a, b, c)}.")  # max: get maximum of a sequence
print(f"The minimum is {min(a, b, c)}.")  # min: get minimum of a sequence
```

↓ `python3 if_else_nested.py` ↓

Listing 6.6: The stdout of the program `if_else_nested.py` given in Listing 6.5.

```
13 is the greatest number.
The maximum is 13.
The minimum is 7.
```

6.3 The `if...elif...else` Statement

In some cases, we need to query a sequence of alternatives in such a way that `else` blocks would be nested over `else` blocks over `else` blocks, and so on. Since everytime we nest a conditional statement into another one we have to add four spaces of indentation, this would quickly fill the horizontal of our screens and look rather ugly. Therefore, the `elif` statement has been developed, which can replace an `else` containing just another `if`. The syntax of a combined `if ... elif` looks like this:

```
if booleanExpression 1:
    conditional 1 statement 1
    conditional 1 statement 2
    # ...
elif booleanExpression 2:  # such block can be placed arbitrarily often
    conditional 2 statement 1
    conditional 2 statement 2
    # ...
else:  # this, too, is optional
    alternative statement 1
    alternative statement 2
    # ...

normal statement 1
normal statement 2
# ...
```

Notice that we can use arbitrarily many `elifs` and, optionally, one `else`. Only if the condition of the `if` evaluates to `False`, the condition of the first `elif` is checked. Only if both the conditions of the `if` and the first `elif` returned `False`, the condition of the second `elif` is checked. Only if both the conditions of the `if` and the first and second `elif` returned `False`, the condition of the third `elif` is

Listing 6.7: An example of using the `if ... elif` statement. (stored in file `if_elif_example.py`; output in Listing 6.8)

```
"""An example of ‘elif‘ using human age groups."""

age: int = 42  # the age of the person
phase: str | None = None  # the life phase, to be computed

if age <= 3:  # If the age is no more than 3 years...
    phase = "infancy"  # then the person is in their infancy.
elif age <= 6:  # If (NOT age <= 3) and (age <= 6) ...
    phase = "early childhood"  # then they are in their early childhood.
elif age <= 8:  # If (NOT age <= 3) and (NOT age <= 6) and (age <= 8)
    phase = "middle childhood"
elif age <= 11:  # If ... (NOT age <= 8) and (age <= 11)
    phase = "late childhood"
elif age <= 20:  # If ... (NOT age <= 11) and (age <= 20)
    phase = "adolescence"
elif age <= 35:  # If ... (NOT age <= 20) and (age <= 35)
    phase = "early adulthood"
elif age <= 50:  # If ... (NOT age <= 35) and (age <= 50)
    phase = "midlife"
elif age < 80:  # If ... (NOT age <= 50) and (age < 80)
    phase = "mature adulthood"
else:  # otherwise, i.e., if age >= 8
    phase = "late adulthood"

print(f"A person of {age} years is in their {phase}.")
```

↓ `python3 if_elif_example.py` ↓

Listing 6.8: The stdout of the program `if_elif_example.py` given in Listing 6.7.

```
A person of 42 years is in their midlife.
```

checked. And so on. Only if all the conditions of the `if` and all `elifs` were `False`, the body of the `else` is executed (if any).

We use this syntax to classify a person's current phase of life by their age in Listing 6.7. First, we define the `age` as an integer variable and pick a value, say 42. We want to fill a string in the variable `phase` that describes the current phase of life of that person. Initially, nothing is filled in, so we set `phase = None`. (We used the type hint `str | None` to specify that the variable can either be a string or `None` [106].) Anyway, we start with the first conditional and write `if age <= 3`. If this evaluates to `True`, we want to set `phase = "infancy"`, meaning that the person is a child in the earliest phase of life. If this is `True`, the complete nested `if...elif...else` block ends and no other conditions are checked. If this is not the case, i.e., if `age <= 3` evaluates to `False`, i.e., if `age > 3`, the next condition is checked. Instead of writing `else` followed by another (indented) `if`, we can just write `elif age <= 6` (without additional indentation). The `elif` basically functions as the `else` followed by an (indented) `if`. It checks whether the person's age is less than or equal to 6 (after it is already clear that `age <= 3` is `False`). If `age <= 6` holds, we call the life phase `"early childhood"`. If this is `True`, the complete nested `if...elif...else` block ends and no other conditions are checked. If this is not the case, i.e., if `age <= 6` evaluates to `False`, we move on to the next `elif`. Only then, `elif age <= 8` is executed and if its condition evaluates to `True`, we refer to the lift phase as `"middle childhood"`. If `age <= 8` does not hold, we move to the next `elif`. And so on. If even `age < 80`, the condition of the last `elif`, is `False`, the `else` block is executed. It will set `phase = "late adulthood"`. Finally, our program prints the life phase using an f-string.

Now, if you followed the book so far, you have noticed that I am promoting using static code analysis tools. For example, we introduced `Ruff` in Useful Tool 2 back in Section 5.1.2. Of course, we should apply such tools to all of our code. If we apply `Ruff` to Listing 6.5, it will produce the output shown in Listing 6.9. This is an example of how a linter can help us to improve the code quality and make our programs more compact.

Listing 6.9: The results of linting with Ruff of the program given in Listing 6.5.

```
$ ruff check --target-version py312 --select=A,AIR,ANN,ASYNC,B,BLE,C,C4,COM
    ↪ ,D,DJ,DTZ,E,ERA,EXE,F,FA,FIX,FLY,FURB,G,I,ICN,INP,ISC,INT,LOG,N,NPY,
    ↪ PERF,PIE,PLC,PLE,PLR,PLW,PT,PYI,Q,RET,RSE,RUF,S,SIM,T,T10,TD,TID,TRY
    ↪ ,UP,W,YTT --ignore=A005,ANN001,ANN002,ANN003,ANN101,ANN204,ANN401,
    ↪ B008,B009,B010,C901,D203,D208,D212,D401,D407,D413,INP001,N801,
    ↪ PLC2801,PLR0904,PLR0911,PLR0912,PLR0913,PLR0914,PLR0915,PLR0916,
    ↪ PLR0917,PLR1702,PLR2004,PLR6301,PT011,PT012,PT013,PYI041,RUF100,S,
    ↪ T201,TRY003,UP035,W --line-length 79 if_else_nested.py
if_else_nested.py:12:1: PLR5501 [*] Use `elif` instead of `else` then `if`,
    ↪  to reduce indentation
   |
10 |        else:  # This means that a > b and c >= a.
11 |            print(f"{c} is the greatest number.")
12 | / else:  # This means that b >= a.
13 | |      if b > c:  # This means that b >= a and b > c.
   | |____^ PLR5501
14 |            print(f"{b} is the greatest number.")
15 |        else:  # This means that b >= a and c >= b.
   |
   = help: Convert to `elif`

Found 1 error.
[*] 1 fixable with the `--fix` option.
# ruff 0.7.2 failed with exit code 1.
```

Listing 6.10: An example of using the nested `if ... elif` statement based on the recommendations of the Ruff linter applied to Listing 6.5. (stored in file `if_elif_nested.py`; output in Listing 6.11)

```python
"""An example of using the nested `if-else` statements and `elif`."""

a: int = 13  # the first number
b: int = 7  # the second number
c: int = 9  # the third number

if a > b:  # This means that a > b
    if a > c:  # This means that a > b and a > c.
        print(f"{a} is the greatest number.")
    else:  # This means that a > b and c >= a.
        print(f"{c} is the greatest number.")
elif b > c:  # This means that b >= a and b > c.
    print(f"{b} is the greatest number.")
else:  # This means that b >= a and c >= b.
    print(f"{c} is the greatest number.")
```

↓ `python3 if_elif_nested.py` ↓

Listing 6.11: The stdout of the program `if_elif_nested.py` given in Listing 6.10.

```
13 is the greatest number.
```

If we implement its recommendation, we obtain Listing 6.10. We can compact the `else ... if` branch into an `elif` statement and the nested `else` can be moved one indentation level up. This program achieves in 15 lines for what Listing 6.5 needs 16, and it is easier to read, too. (In this program, I left the additional example of the `min` and `max` function away.)

Best Practice 21 Prefer `elif` over nested `else ... if` constructs.

Listing 6.12: An example of using the nested `if ... else` statements that could be inlined. See Listing 6.15 for the more compact inlined variant. (stored in file `if_else_could_be_inline.py`; output in Listing 6.13)

```
"""An example of if-elif-else expressions that could be inlined."""

number: int = 100  # the number

# Numbers with an absolute value less than ten are small.
# If their absolute value is larger than ten, they are large.
size: str
if abs(number) < 10:  # just a random threshold for this example...
    size = "small"
else:
    size = "large"

# Numbers can be positive, negative, or unsigned (0 is unsigned).
sign: str
if number < 0:
    sign = "negative"
elif number > 0:
    sign = "positive"
else:
    sign = "unsigned"

print(f"The number {number} is {size} and {sign}.")  # Print the result.
```

↓ `python3 if_else_could_be_inline.py` ↓

Listing 6.13: The stdout of the program `if_else_could_be_inline.py` given in Listing 6.12.

```
The number 100 is large and positive.
```

6.4 The Inline `if...else` Statement

A very common use case of `if...else` statements is to assign values to variables. In Listing 6.12 we display such a situation. We want to write some code that tells us whether a number is large or small and positive or negative. For this purpose, we first store the number in an integer variable `number`. As example, we choose the value `100`. For the sake of this example, let's assume that a `number` whose absolute value |`number`| is less than ten should be small. The absolute value of a number in Python can be computed using the function `abs`. Hence, we can build the condition `if abs(number)< 10:` and store the string `"small"` in the variable `size` if the condition evaluates to `True` and, otherwise, store `"large"` in `size`. Similarly, we can define the string variable `sign`. We store `"negative"` in `sign` if `number < 0`, `"positive"` if `number > 0`, and `"unsigned"` otherwise. Finally, we can print the result of these conditions using an f-string.

Now you notice that in the `if...else` statement, all we did was to assign values to the same variable `size`. Likewise, in the `if...elif...else` statement, each branch *only* assigned one value to the same variable `sign`. Python offers us a more compact syntax for this, and Ruff can again give us some idea about that in Listing 6.14: The inline `if...then...else` statement, which has the following syntax:

```
variable = valueA if conditionForUsingValueA else valueB

variable = valueA if conditionForUsingValueA else (valueB if
    ↪ conditionForUsingValueB else valueC)
```

In the first variant, the value `valueA` will be assigned to `variable` if `conditionForUsingValueA` evaluates to `True`. Otherwise, `valueB` is assigned. This statement can again arbitrarily nested, as we show in the second variant: Here, again, value `valueA` will be assigned to `variable` if `conditionForUsingValueA`

Listing 6.14: The results of linting with Ruff of the program given in Listing 6.12.

```
$ ruff check --target-version py312 --select=A,AIR,ANN,ASYNC,B,BLE,C,C4,COM
    ↪ ,D,DJ,DTZ,E,ERA,EXE,F,FA,FIX,FLY,FURB,G,I,ICN,INP,ISC,INT,LOG,N,NPY,
    ↪ PERF,PIE,PLC,PLE,PLR,PLW,PT,PYI,Q,RET,RSE,RUF,S,SIM,T,T10,TD,TID,TRY
    ↪ ,UP,W,YTT --ignore=A005,ANN001,ANN002,ANN003,ANN101,ANN204,ANN401,
    ↪ B008,B009,B010,C901,D203,D208,D212,D401,D407,D413,INP001,N801,
    ↪ PLC2801,PLR0904,PLR0911,PLR0912,PLR0913,PLR0914,PLR0915,PLR0916,
    ↪ PLR0917,PLR1702,PLR2004,PLR6301,PT011,PT012,PT013,PYI041,RUF100,S,
    ↪ T201,TRY003,UP035,W --line-length 79 if_else_could_be_inline.py
if_else_could_be_inline.py:8:1: SIM108 Use ternary operator `size = "small"
    ↪  if abs(number) < 10 else "large"` instead of `if`-`else`-block
   |
 6 |   # If their absolute value is larger than ten, they are large.
 7 |   size: str
 8 | / if abs(number) < 10:  # just a random threshold for this example...
 9 | |     size = "small"
10 | | else:
11 | |     size = "large"
   | |_________________^ SIM108
12 |
13 |   # Numbers can be positive, negative, or unsigned (0 is unsigned).
   |
   = help: Replace `if`-`else`-block with `size = "small" if abs(number) <
       ↪ 10 else "large"`

Found 1 error.
# ruff 0.7.2 failed with exit code 1.
```

Listing 6.15: An example of using the inline `if ... else` expression to shorten Listing 6.12, which incorporates the suggestion by Ruff in Listing 6.14. (stored in file `inline_if_else.py`; output in Listing 6.16)

```python
"""An example of using the inline if-else expression."""

number: int = 100  # the number

# Numbers with an absolute value less than ten are small.
# If their absolute value is larger than ten, they are large.
size: str = "small" if abs(number) < 10 else "large"

# Numbers can be positive, negative, or unsigned (0 is unsigned).
sign: str = "negative" if number < 0 else (
    "positive" if number > 0 else "unsigned")

print(f"The number {number} is {size} and {sign}.")  # Print the result.
```

↓ `python3 inline_if_else.py` ↓

Listing 6.16: The stdout of the program `inline_if_else.py` given in Listing 6.15.

```
The number 100 is large and positive.
```

evaluates to `True`. Otherwise, `valueB` is used if `conditionForUsingValueB` evaluates to `True` and, again otherwise, `valueC` is used.

Best Practice 22 When your `if...then...else` statement only assigns values to variables, use the inline variant discussed in Section 6.4, as it is more compact.

We apply this syntax to refine Listing 6.12 and obtain the much more compact Listing 6.15. Both

programs are equivalent, but the second one only has 13 instead of 22 lines of code. Notice again how a linter can help us to refine and compactify our code.

6.5 Summary

With the statements we discussed in this section, you are now able to create a program that makes decisions based on data. Before this, we could only perform straightforward computations and calculate the results of simple functions. Now our variables can receive the result of a function A if the input meets a condition B and otherwise the result of a function C. This is already quite nice. For example, we can now implement and hard-code decision trees [116, 125] and Listing 6.7 is basically an example of that. Still, the instructions in our programs are still executed in the sequence in which we wrote them down. While our control flow can now branch, it cannot perform anything more fancy and advanced ... like looping back upon itself...

Chapter 7

Loops

When we are working with sequences of data, we do not just want to perform an action on one data element. We instead often want to apply the actions repetitively to all data elements. Loops allow us to do just that, to perform the same actions multiple times.

7.1 The `for` Loop Statement

The most basic such sequence in Python may be the `for` loop, which has the following pattern:

```
for loopVariable in sequence:
    loop body statement 1
    loop body statement 2
    # ...

normal statement 1
normal statement 2
# ...
```

The keyword `for` is followed by a loop variable. Then comes the keyword `in`, the `sequence` we want to iterate over, and finally a colon (`:`). This variable will iteratively take on the values in the `sequence`. The loop body statements in the following, indented block are executed for each of these values. Each time the body of the loop is executed is called an *iteration* of the loop. After the loop, we leave one blank line followed by the code that will be executed after the loop completes.

In its most simple form, the `for` loop is applied to a `range` of integer numbers. Ranges are sequences which work basically like slices (see Sections 3.5.1 and 5.1.1). `range(5)` will give us a sequence of integers starting with 0 and reaching up to right *before* 5, i.e., the integer range 0..4. `range(6, 9)` gives the sequence of integers starting with 6 and stopping right *before* 9, i.e., the integer range 6..8. Finally, `range(20, 27, 2)` results in a sequence of integers that begins at 20, increments by 2 in each step, and ends right before 27. This is the sequence $(20, 22, 24, 26)$. `ranges`, like slices, can also have negative increments: The `range(40, 30, -3)` starts with 40 and stops before reaching 30 and decrements by 3 in each step. This is equivalent to the set $(40, 37, 34, 31)$.

In Listing 7.1, we loop over exactly these ranges. In this listing, we try to create a dictionary (see Section 5.4) where some integer numbers are mapped to their squares. We use four `for` loops to fill this dictionary with data. In each of these first four `for` loops, we use `i` as the loop variable.

When iterating over the `range(5)` in the first loop, `i` will hold the value `0` in the first iteration (= execution of the loop body). The loop body `squares[i] = i * i` will thus effectively be `squares[0] = 0` and thus store the value `0` under key `0` into the dictionary `squares`. In the second iteration, `i` will hold the value `1`. Then, the body `squares[i] = i * i` will effectively be `squares[1] = 1`. In the third iteration, `i` will hold the value `2` and the body will perform `squares[2] = 4`. Next, `i = 3` and `squares[3] = 9` will be executed and in the laste iteration of the first loop, we store `squares[4] = 16`.

In the second loop, which uses `range(6, 9)`, `i` will take on the values `6`, `7`, and `8`, one by one. The dictionary `squares` will thus be extended with the values `squares[6] = 36`, `squares[7] = 49`, and `squares[8] = 64`. In the third loop, iterating over `range(20, 27, 2)`, the following updates will be performed one by one `squares[20] = 400`, `squares[22] = 484`, `squares[24] = 576`, and `squares[26] = 676`.

Listing 7.1: An example of using the `for` loop over a `ranges` of integer numbers. (stored in file `for_loop_range.py`; output in Listing 7.2)

```
"""Apply a for loop over a range."""

# We will construct a dictionary holding square numbers.
squares: dict[int, int] = {}  # Initialize ‘squares‘ as empty dict.

for i in range(4):  # i takes on the values 0, 1, 2, and 3 one by one.
    squares[i] = i * i  # Stores 0: 0, 1: 1, 2: 4, 3: 9.

for i in range(6, 9):  # i takes on the values 6, 7, and 8 one by one.
    squares[i] = i * i  # Stores 6: 36, 7: 49, 8: 64.

for i in range(20, 27, 2):  # i takes on the values 20, 22, 24, and 26.
    squares[i] = i * i  # Stores 20: 400, 22: 484, 24: 576, 26: 676.

for i in range(40, 30, -3):  # i takes on the values 40, 37, 34, and 31.
    squares[i] = i * i  # Stores 40: 1600, 37: 1369, 34: 1156, 31: 961.

print(squares)  # Print the dictionary.

for _ in range(3):  # Iterate the loop three times. Ignore counter ‘_‘.
    print("Hello World!")  # We don't need the counter.
```

↓ `python3 for_loop_range.py` ↓

Listing 7.2: The `stdout` of the program `for_loop_range.py` given in Listing 7.1.

```
{0: 0, 1: 1, 2: 4, 3: 9, 6: 36, 7: 49, 8: 64, 20: 400, 22: 484, 24: 576,
    ↪ 26: 676, 40: 1600, 37: 1369, 34: 1156, 31: 961}
Hello World!
Hello World!
Hello World!
```

In the fourth loop, `i` takes on the values of the sequence `range(40, 30, -3)`, which has the negative step length `-3`. `i` therefore first becomes `40`, then `37` in the second iteration, then `34`, and, finally, `31`. We then print the dictionary and get the expected output in Listing 7.2.

Best Practice 23 If we do not care about the value of a variable (or parameter), we should name it `_` [74]. This information is useful for other programmers as well as static code analysis tools.

At the end of Listing 7.1 we show this special case: We want to print `"Hello World!"` three times. Instead of copying the line `print("Hello World!")` three times, we put it in a loop. However, nowhere in the loop body we care about the value of the loop variable. We thus simply call it `_`. If we would not call it that, then another programmer seeing our code (or a static code analysis tool for that matter) could be confused as to why we do not use the loop variable. Always remember that "real" code could be much more complicated, and any semantic hint we can include to convey our intentions will be helpful.

With these new tools at hand, we can revisit our program Listing 4.3 for approximating π from back in Section 4.1.2. In this program, we executed the same code five times. Instead of doing this, we can put this into a loop, which reduces the lines of code from over 25 to about 10 in Listing 7.3. The outputs in Listings 4.4 and 7.4 are exactly the same.

Listing 7.3: A variant of Listing 4.3 which uses a `for` loop instead of five copies of the same instructions. (stored in file `for_loop_pi_liu_hui.py`; output in Listing 7.4)

```
"""We execute Liu Hui's method to approximate pi in a loop."""
from math import pi, sqrt

print(f"We use Liu Hui's Method to Approximate \u03c0\u2248{pi}.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.

for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 for_loop_pi_liu_hui.py` ↓

Listing 7.4: The stdout of the program `for_loop_pi_liu_hui.py` given in Listing 7.3.

```
We use Liu Hui's Method to Approximate π≈3.141592653589793.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050416 give us π≈3.1058285412302498.
24 edges, side length=0.2610523844401031 give us π≈3.132628613281237.
48 edges, side length=0.13080625846028635 give us π≈3.139350203046872.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

7.2 The `continue` and `break` Statements

Loops often have complex bodies, maybe containing conditional statements or other loops. It is not an uncommon situation that, after performing some computations in the body of the loop, we already know that we can continue directly with the next iteration instead of executing the remainder of the loop body. Sometimes we also find that we can entirely stop with the loop and continue with whatever

Listing 7.5: An example of the `continue` and `break` statements in a `for` loop. (stored in file `for_loop_continue_break.py`; output in Listing 7.6)

```
"""Here we explore the `break` and `continue` statements."""

for i in range(15):  # i takes on the values from 0 to 14 one by one.
    s: str = f"i is now {i}."  # Create a string with the value of i.
    if i > 10:  # If i is greater than 10, then...
        break  # ...abort the loop altogether, do not execute next line.
    if 5 <= i <= 8:  # If i is in the range of 5..8, then...
        continue  # ...skip the rest of the loop body, do next iteration.
    print(s)  # We get here if neither continue nor break were called.

print("All done.")  # We always get here.
```

↓ `python3 for_loop_continue_break.py` ↓

Listing 7.6: The stdout of the program `for_loop_continue_break.py` given in Listing 7.5.

```
i is now 0.
i is now 1.
i is now 2.
i is now 3.
i is now 4.
i is now 9.
i is now 10.
All done.
```

instructions come after it, even if we did not yet exhaust the sequence over which we are iterating. The former can be achieved using the `continue` and the latter with the `break` statement.

An example of both statements is given in Listing 7.5. Here, we iterate the variable `i` over the 15 values from `0` to `14`. In the loop body, we first create a string `s` with the current value of `i` via the f-string `f"i is now {i}."` The very last instruction of the loop body, `print(s)`, prints this string.

While `i` would go from `0` to `14`, we actually want to abort the loop as soon as `i` becomes greater than `10`. Instead of modifying the `range` over which we loop (which would be reasonable), we here want to use the `break` statement. We therefore wrap it into the conditional `if i > 10:`, meaning that it will only be executed if `i > 10`. As soon as `break` is reached, the loop will immediately be aborted. No further instruction in the loop body is executed and no further iteration is performed. Instead, the process will continue after the loop, with the line `print("All done.")`.

If `i > 10` did not hold, the rest of the loop body is executed. For the case that `i` $\in 5..8$, we now want to directly jump to the next loop iteration by invoking the `continue` statement. This means that if `i == 5`, the `continue` statement lets the control directly return to the head of the loop, which will set `i = 6`. This will happen until `i == 9`. The condition `5 <= i <= 8` is *not* met for all `i` $\in 0..4 \cup 9..15$. The next line, namely the `print(s)`, can only be reached in these cases. Of course, we already know that we will never even reach this code as soon as `i == 11`.

As a result the program will print the string `s` only for `i` $\in 0..4 \cup \{9, 10\}$ before finally outputting `All done.`. This can be observed in the program output collected in Listing 7.6. With `break` and `continue`, we now have two tools that can help us to either abort any loop prematurely or to abort the current iteration of the loop prematurely (and continue with the next one, if any), respectively.

7.3 Nesting Loops

Like conditional statements, `for` loops can be arbitrarily nested. Let us explore this by computing the list of all prime numbers less than 200 in Listing 7.7.

Definition 7.1 (Prime Number) A prime number $p \in \mathbb{N}_1$ is a positive integer $p > 1$ (greater than one) that has no positive integer divisors other than 1 and p itself [30, 113, 146].

In our example program, we want to store all of these prime numbers in the list `primes`. We know that `2` is a prime number and the only even one, so we directly initialize `primes = [2]`, i.e., we set `primes` to initially be a list only containing the integer `2`. This will allow us to later on only consider the odd numbers in the range 3..200. Just out of interest, we will count the total number of divisions that we need to perform until we have the full list of primes in the variable `n_divisions`.

To find all primes in the integer set 2..200, we let the loop variable `candidate` iterate over `range(3, 200, 2)`. This is the sequence of integer numbers starting `3` increasing with step length `2` and stopping right before `200`. We know that even numbers (except 2) are not prime and we know that 200 is not a prime number either, so this should be OK. Therefore, `candidate` will iteratively become `3`, `5`, `7`, ..., and eventually `195`, `197`, and `199`. For each value of `candidate`, we begin with the assumption that it is prime and then try to prove the opposite. We set `is_prime = True` then will try to find a divisor, in which case we will set `is_prime = False`. If we cannot find a divisor of `candidate`, then `is_prime` will remain `True` and we can add `candidate` to the list `primes`. At least, this is the plan.

We will implement this idea with a nested inner loop. Since the loop variable `candidate` will always be odd, only odd numbers can be potential divisors. Obviously, only integers greater than or equal to `3` are potential divisors. We also only need to explore whether `check` is a divisor of `candidate` if it is not bigger than $\sqrt{\texttt{candidate}}$. If we had three integer numbers a, b, and c such that $a = b * c$ and $b > \sqrt{a}$, then it must be that $c < \sqrt{a}$. Since $a = \sqrt{a} * \sqrt{a}$ holds by definition, it would be impossible that $a = b * c$ if both $b > \sqrt{a}$ and $c \geq \sqrt{a}$. Thus, if $b > \sqrt{a}$ was a divisor of a, then c would also be a divisor of a that we would have found before reaching b in the loop since $c < \sqrt{a} < b$.

Now, most integer numbers do not have integer square roots. Since integer divisors cannot have fractions anyway, it is sufficient for us to use $\lfloor\sqrt{\texttt{candidate}}\rfloor$. In Python, such a "truncated" integer square root of an integer number $\lfloor\sqrt{a}\rfloor$ can be computed using the `isqrt` function from the `math` module, which we `import` at the top of our program.

Listing 7.7: Computing a list of all primes from 2..200 using nested `for` loops. (stored in file `for_loop_nested_primes.py`; output in Listing 7.8)

```
"""Compute all primes less than 200 using two nested for loops."""

from math import isqrt  # the integer square root == int(sqrt(...))

primes: list[int] = [2]  # the list for the primes; We know 2 is prime.
n_divisions: int = 0  # We want to know how many divisions we needed.

for candidate in range(3, 200, 2):  # ...all odd numbers less than 200.
    is_prime: bool = True  # Let us assume that `candidate` is prime.

    for check in range(3, isqrt(candidate) + 1, 2):  # ...odd numbers
        n_divisions += 1  # Every test requires one modulo division.
        if candidate % check == 0:  # modulo == 0: division without rest
            is_prime = False  # If check divides candidate evenly, then
            break  # candidate is not a prime. We can stop the inner loop.

    if is_prime:  # If True: no smaller number divides candidate evenly.
        primes.append(candidate)  # Store candidate in primes list.

# Finally, print the list of prime numbers.
print(f"After {n_divisions} divisions: {len(primes)} primes {primes}.")
```

↓ `python3 for_loop_nested_primes.py` ↓

Listing 7.8: The `stdout` of the program `for_loop_nested_primes.py` given in Listing 7.7.

```
After 252 divisions: 46 primes [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37,
    ↪  41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107,
    ↪ 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179,
    ↪ 181, 191, 193, 197, 199].
```

We can therefore iterate a second, inner loop variable `check` over `range(3, isqrt(candidate)+ 1, 2)`. If `candidate <= 3`, then this loop will never be executed because no number `check` with $3 \leq$ `check` $<$ `candidate` exists. Then, `is_prime` will remain `True` and we will add `candidate` to `primes` further down the outer loop body. If `candidate > 3`, then `check` will go from $3, \ldots, \lfloor\sqrt{\texttt{candidate}}\rfloor$.

In the body of our inner loop, we try to find out whether `check` is an integer divisor of `candidate`. We can do this by computing the remainder of the division `canddiate`/`check`. Back in Section 3.2.1 we learned the modulo division operator `%`, which does exactly that. If `candidate %check` is `0`, then we can divide `candidate` by `check` without remainder. Then, `candidate` is divisible by `check` and cannot be a prime number. We thus can set `is_prime = False`. We can also immediately exit the inner loop using the `break` statement, because once we know that `candidate` is not a prime number, we do not need to check further potential divisors. (Notice that we also count all the modulo division operations we perform by doing `n_divisions += 1` at the beginning of the inner loop.)

After the inner loop, if `is_prime` is still `True`, we add `candidate` to `primes` by invoking the `append` method of the list. Once we have completed the outer loop as well, we print the number `n_divisions` of divisions we have performed, the number `len(primes)` of prime numbers we have discovered and, finally, the list `primes` itself. The output in Listing 7.8 tells us that, after performing 252 divisions, we could identify 46 prime numbers inside 2..200.

7.4 Loops over Sequences

When introducing the `for` loop, we stated that it iterates over sequences of data. We learned that `ranges` are the most basic such sequences. However, in Chapter 5 we learned about collection datatypes, namely `lists`, `tuples`, `sets`, and `dicts`. The elements of all of these datatypes can be accessed sequentially, too. This means that a `for` loop can iterate over them as well!

In Listing 7.9 we illustrate how this is done: It works exactly as if we were using `ranges`. In the

Listing 7.9: An example iterating over the elements in different collection datastructures using `for` loops. (stored in file `for_loop_sequence.py`; output in Listing 7.10)

```
"""Iterate over several different containers with ‘for‘ loops."""

txt: list[str] = []  # We will collect the output text in this list.

lst: list[int] = [1, 2, 3, 50]  # Create a list with 4 integers.
for i in lst:  # i takes on the values 1, 2, 3, and 50.
    txt.append(f"i={i}")  # We store "i=1", "i=2", "i=3", and "i=50"

tp: tuple[float, ...] = (7.6, 9.4, 8.1)  # Create a tuple with 3 floats.
for f in tp:  # i takes on the values 7.6, 9.4, and 8.1.
    txt.append(f"f={f}")  # We store "f=7.6", "f=9.4", and "f=8.1"

st: set[str] = {"u", "v", "w"}  # Create a set with 3 strings.
for s in st:  # s takes on the values "u", "v", and "w" (unordered!).
    txt.append(f"s={s!r}")  # We store "s='u'", "s='v'", and "s='w'".

dc: dict[float, bool] = {1.1: True, 2.5: False}  # Create a dictionary.
for k in dc:  # Iterate over the keys in the dictionary == 1.1 and 2.5.
    txt.append(f"k={k}")  # We store "k=1.1" and "k=2.5".
for v in dc.values():  # Iterate over the values in the dictionary.
    txt.append(f"v={v}")  # We store "v=True" and "v=False".
for k, v in dc.items():  # Iterate over the key-value combinations.
    txt.append(f"{k}: {v}")  # Store "1.1: True" and "2.5: False"

# Merge text into single string with separator ", " and print it.
print(", ".join(txt))
```

↓ `python3 for_loop_sequence.py` ↓

Listing 7.10: The `stdout` of the program `for_loop_sequence.py` given in Listing 7.9.

```
i=1, i=2, i=3, i=50, f=7.6, f=9.4, f=8.1, s='w', s='u', s='v', k=1.1, k
    ↪ =2.5, v=True, v=False, 1.1: True, 2.5: False
```

program, we will build a list `txt` with strings that we will later write to the output.

First, we want to iterate over a list `lst` containing the four integers `[1, 2, 3, 50]`. This is done by simply writing `for i in lst`. Here, `i` is the loop variable and it will step-by-step take on all the values in `lst`, one by one. In the body of this loop, we invoke `txt.append(f"i={i}")`. The f-string will evaluate to `"i=1"` in the first iteration, to `"i=2"` in the second, to `"i=3"` in the third, and to `"i=50"` in the fourth and last iteration of this loop. These strings are appended to the list `txt` via the `append` method.

Then we move on and want to iterate over a tuple `tp`, which contains three floating point numbers, namely `(7.6, 9.4, 8.1)`. This works exactly in the same way: `for f in tp` will let the loop variable `f` take on the values `7.6`, `9.4`, and finally `8.1`. We again append their textual representation to the list `txt`, this time using the f-string `f"f={f}"`.

As third example, we create the set `st` containing the three strings `{"u", "v", "w"}`. We can do this again by simply writing `for s in st`, which lets the loop variable `s` take on the values `"w"`, `"u"`, and `"v"`, *in an arbitrary order*. Remember that sets in Python are unordered (see Best Practice 17), which means that if we run the program twice, we may get different results. Either way, we can iterate over all the values in the set `st`. We again want to store these values as nicely formatted strings in `txt`. This time we use the f-string `f"s={s!r}"`. Notice the `!r` format specifier: It will print the *representation* of the string `s`, which basically means to put quotation marks (and to potentially escape otherwise unprintable characters, see Section 3.5.4). Thus, we will add `"s='v'"`, `"s='u'"`, and `"s='w'"` to `txt` (in an arbitrary order).

As fourth and final example, we create a dictionary `dc` that maps floating point numbers to Boolean values, containing the two entries, namely `{1.1: True, 2.5: False}`. Now a dictionary is a bit special:

It maps keys to values. When working with the whole dictionary datastructure `dc` as a collection, then we can access it in three ways:

1. Iterating over `dc` directly lets us view all the keys in the dictionary `dc`. This is equivalent to iterating over `dc.keys()`.
2. Iterating over `dc.values()` lets us access all the values in the dictionary `dc`.
3. Iterating over `dc.items()` lets us access all key-value pairs in the dictionary `dc` as tuples.

In our example program, we apply all of these methods. We iterate over the keys in `dc` via `for k in dc`, which lets `k` take on the values `1.1` and `2.5`, one after the other. We iterate over the values in `dc` via `for v in dc.values()`, which lets `v` take on the values `True` and `False`, one after the other.

Finally, we iterate over all key-value pairs. Look closely at this one: We *could* write `for t in dc.items()`, which would let a loop variable `t` take on the tuple values `(1.1, True)` and then `2.5: False`. However, back in Section 5.2, we learned about tuple unpacking. So instead, we write `for k, v in dc.items()`. This is a shortcut for writing `for t in dc.items()` followed by `k, v = t`. It directly unpacks the tuples in the sequence `dc.items`, meaning that we get pairs of `k=1.1`, `v=True` and `k=2.5`, `v=False`. All of them are again appended to our text list `txt`.

After all of these loops, we have ended up with a list `txt` containing 16 strings. We want to combine all of them to a single string, using `", "` as a separator. We could do this in another loop. However, Python offers us a much more efficient shorthand for this: The method `join` of the class `str`.

For any string `z`, `z.join(seq)` accepts a sequence `v` of strings. It then concatenates all the strings in `seq`, placing `z` as separator between any two strings. Thus, invoking `", ".join(txt)` produces `i=1, i=2, i=3, i=50, f=7.6, f=9.4, ...`. This text is finally is written to the output via the `print` function, giving us the results shown in Listing 7.10.

Let us now use this new ability to iterate over collection datastructures to improve upon our prime number enumeration program Listing 7.7. Back when writing that program, we immediately stored `2` as a prime into our list `primes`. We then only tried the odd numbers less than 200 as potential prime number `candidates`. For each potential prime number `candidate`, we tested all (odd) numbers `check` from the range `range(3, isqrt(candidate)+ 1, 2)` as possible divisors.

When thinking about this, we realize that actually, only the prime numbers in this range are interesting. For example, we do never need to check whether `candidate` is divisible by 9, because we will have already checked whether it can be divided by 3. We also never need to check whether we can divide it by 55, because we will have already checked 5.

Therefore, in our new and more efficient program Listing 7.11, we replace the head of the inner loop with `for check in primes[1:]:`. `primes[1:]` is a slice of the list `primes` containing all but the first element (see Section 5.1). This first element is `2`, and since all `candidates` are odd, we do not need to try whether they can be divided by `2`. The outer loop will step-by-step add prime numbers to `primes`. Actually, for each value of the loop variable `candidate`, `primes` will contain all prime numbers which are smaller than `candidate`. We only need to check those which are less than or equal to `isqrt(candidate)`, so we store this value in a new variable `limit` to avoid re-computing it in the inner loop. The `break` statement lets us exit the inner loop as soon as we hit this limit.

The lists of primes that our programs generate in Listings 7.8 and 7.12 are exactly the same. However, our new program needs to perform only 224 divisions instead of 252.

Listing 7.11: Computing a list of all primes from 2..200 using nested `for` loops. Compared to Listing 7.7, this program is more efficient because it only tests primes as divisors. (stored in file `for_loop_sequence_primes.py`; output in Listing 7.12)

```python
"""Compute all primes less than 200, with a for loop over a sequence."""

from math import isqrt  # the integer square root == int(sqrt(...))

primes: list[int] = [2]  # the list for the primes; We know 2 is prime.
n_divisions: int = 0  # We want to know how many divisions we needed.

for candidate in range(3, 200, 2):  # ...all odd numbers less than 200.
    is_prime: bool = True  # Let us assume that `candidate` is prime.
    limit: int = isqrt(candidate)  # Get the maximum possible divisor.

    for check in primes[1:]:  # We only test with the odd primes we got.
        if check > limit:  # If the potential divisor is too big, then
            break  # we can stop the inner loop here.
        n_divisions += 1  # Every test requires one modulo division.
        if candidate % check == 0:  # modulo == 0: division without rest
            is_prime = False  # If check divides candidate evenly, then
            break  # candidate is not a prime. We can stop the inner loop.

    if is_prime:  # If True: no smaller number divides candidate evenly.
        primes.append(candidate)  # Store candidate in primes list.

# Finally, print the list of prime numbers.
print(f"After {n_divisions} divisions: {len(primes)} primes {primes}.")
```

↓ `python3 for_loop_sequence_primes.py` ↓

Listing 7.12: The stdout of the program `for_loop_sequence_primes.py` given in Listing 7.11.

```
After 224 divisions: 46 primes [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37,
    ↪  41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107,
    ↪ 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179,
    ↪ 181, 191, 193, 197, 199].
```

7.5 Enumerating over Sequences

Assume that we have a list `data` of integer values `v`. If we want to iterate over all the values `v` in `data`, then `for v in data` would do the trick. However, what if we also want to know the current index `i` for each data element during the iteration? Then this construct no longer works.

We basically have two choices: As first option, we can iterate over `data` by accessing the elements by their index `i` directly. In this case, we would use a `range` that goes from `0` to `len(data)- 1`. The head of the loop now looks like this: `for i in range(len(data))`. The data values `v` are then accessed via `data[i]`. This is illustrated in Listing 7.13.

The drawback of this method is that it will *only* work for `lists`, because among the collection datastructures discussed so far, only `lists` are directly indexable. Therefore, the second option to achieve our goal would be to use an additional integer variable `i` which we would initialize as `i: int = 0` before the loop. We would then increment it as `i += 1` at the bottom of the loop body, where `i += 1` is equivalent to `i = i + 1`. This would work with all collection classes.

Both options are a bit iffy, as they introduce more and less readable additional code. There is a third option, though. And because I advocate using static code analysis tools, I let another such tool discover this option for us.

Listing 7.13: Enumerating over the index-value pairs of a list (inefficiently). (stored in file `for_loop_no_enumerate.py`; output in Listing 7.14)

```
"""Enumerate the index-value pairs in a list."""

data: list[int] = [1, 2, 3, 5]
for i in range(len(data)):
    print(f"data[{i}]={data[i]}")
```

↓ `python3 for_loop_no_enumerate.py` ↓

Listing 7.14: The stdout of the program `for_loop_no_enumerate.py` given in Listing 7.13.

```
data[0]=1
data[1]=2
data[2]=3
data[3]=5
```

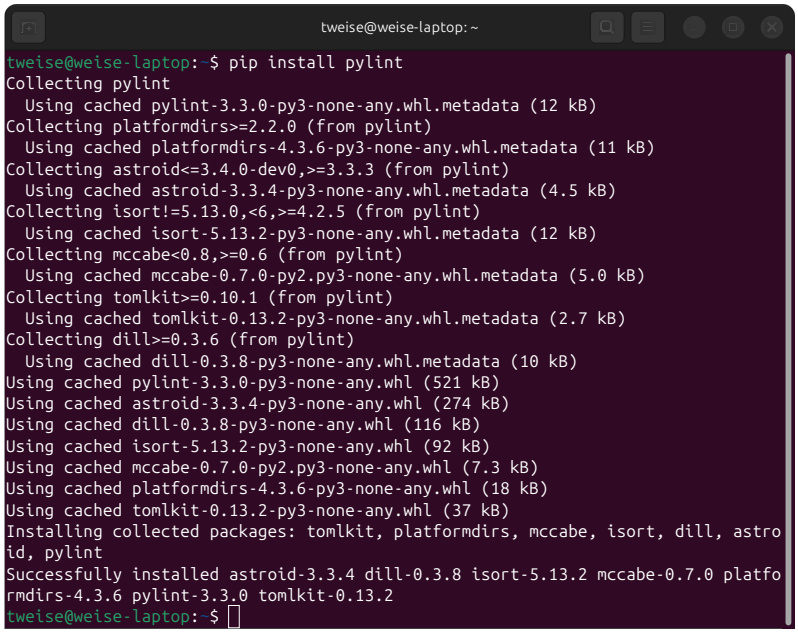


Figure 7.1: Installing `Pylint` in a `Ubuntu` terminal via `pip`.

Useful Tool 3 `Pylint` is a Python linter that analyzes code for style, potential errors, and possible improvements [110]. It can be installed via `pip install pylint` as shown in Figure 7.1 on page 106. You can then apply `Pylint` using the command `pylint fileToScan.py`. We provide a script for using `Pylint` with a reasonable default configuration in Listing 10.3 on page 177.

In Listing 7.15, `Pylint` suggest that we should iterate over the list `data` using the `enumerate` function. `enumerate` accepts a collection as parameter and creates a sequence of index-value tuples. Since we can unpack such tuples while iterating over this sequence (see Listing 7.9), the head of our loop now takes on the form `for i, v in enumerate(data)`. Notice that the index `i` comes first and the value `v` comes second in this unpacked-tuple-based enumeration. This is implemented in Listing 7.16. And it would work exactly the same, regardless whether `data` was a list, set, tuple, or dictionary.

Now, above we used the `Pylint` linter to discover that we could improve our code by using this concept. At the time of this writing, `Ruff` and `Mypy` did not find this potential improvement. Maybe their future versions will. Regardless, it becomes clear that using more static analysis tools is always a good idea, as stated back in Best Practice 13.

Listing 7.15: The results of static code analysis using the `Pylint` linter for the program given in Listing 7.13.

```
$ pylint for_loop_no_enumerate.py --disable=C0103,C0302,C0325,R0801,R0901,
    ↪ R0902,R0903,R0911,R0912,R0913,R0914,R0915,R1702,R1728,W0212,W0238,
    ↪ W0703
************* Module for_loop_no_enumerate
for_loop_no_enumerate.py:4:0: C0200: Consider using enumerate instead of
    ↪ iterating with range and len (consider-using-enumerate)

------------------------------------
Your code has been rated at 6.67/10

# pylint 3.3.1 failed with exit code 16.
```

Listing 7.16: Enumerating over the index-value pairs of a list using the `enumerate` function. (stored in file `for_loop_enumerate.py`; output in Listing 7.17)

```python
"""Enumerate the index-value pairs in a list using enumerate."""

data: list[int] = [1, 2, 3, 5]
for i, v in enumerate(data):
    print(f"data[{i}]={v}")
```

↓ `python3 for_loop_enumerate.py` ↓

Listing 7.17: The `stdout` of the program `for_loop_enumerate.py` given in Listing 7.16.

```
data[0]=1
data[1]=2
data[2]=3
data[3]=5
```

7.6 The `while` Loop

Old clay tablets show that the Babylonians were able to approximate $\sqrt{2}$ maybe as far back as 4000 years ago [46, 119]. The mathematician Hero(n) of Alexandria, who lived in the first century CE, specified an abstract algorithm for computing the square root of numbers which, today, is known as Heron's Method [75, 119].

Given a number a and aiming to find the square root $\sqrt{a}$ of a, this algorithm starts with a guess x_0, let's say $x_0 = 1$. In each iteration i, it will compute a new guess as follows [75, 119]:

$$x_{i+1} = \frac{1}{2}\left(x_i + \frac{a}{x_i}\right) \tag{7.1}$$

We can roughly imagine that the algorithm works as follows: If x_i was too big, i.e., $x_i > \sqrt{a}$, then $\frac{a}{x_i} < \sqrt{a} < x_i$. If x_i was too small, i.e., $x_i < \sqrt{a}$, then $\frac{a}{x_i} > \sqrt{a} > x_i$. By computing the average of x_i and $\frac{a}{x_i}$ as the next guess, we hope to approach $\sqrt{a}$. If $x_i = \sqrt{a}$, then $\frac{a}{x_i} = \sqrt{a}$ by definition and $x_{i+1} = x_i$. Showing that this actually works and that the error gets smaller over time is more complicated [119].

If we want to implement this algorithm, we will naturally need a loop of some sort. Clearly, we perform the same computation again and again. However, a `for` loop will not do: We do not know the number of steps that we will need in advance. Of course, we could try to pick a very very huge number and then `break` the loop when the guesses converg ... but that is just ugly. The `while` comes to rescue:

```
while booleanExpression:
    loop body statement 1
    loop body statement 2
    # ...

normal statement 1
normal statement 2
# ...
```

In the `while` loop, the body is executed as long as a Boolean expression in the head of the loop evaluates to `True`. We now use this new construct to implement Heron's Method in Listing 7.18 and use it to compute the square roots of 0.5, 2, and 3.

We begin the program with an outer `for` loop that iterates a variable `number` over the `float` values `0.5`, `2.0`, and `3.0`. We want to apply the algorithm to each of these values. We use two variables `guess` be the current guess of what $\sqrt{\texttt{number}}$ could be and `old_guess` be the previous guess.

We initialize `guess` with `1.0` and `old_guess` with a different value, say `0.0`. Our `while` loop should keep iterating as long as `guess != old_guess`. Of course, if we could represent real numbers at infinite precision, we would never reach `guess == old_guess` for any `number` with an irrational square root. However, the `float` datatype has limited precision (see Section 3.3.1). Therefore, at least in our examples, `guess != old_guess` indeed eventually becomes `False`.

Best Practice 24 Do not use (un)equality comparisons of `floats` as loop termination criteria, as they may lead to endless loops. There can always be inputs that cause endless oscillations between values or the appearance of `nan` values (see Section 3.3.5).

The only reason why we did it here is that it looks nice and is easy to read as a functional and yet brief example of a `while` loop.[1] Anyway, the loop condition necessitates us to store some value different from `1.0` in `old_guess` initially (and we picked `0.0`). In the loop, first the current guess becomes the

[1] To enforce that no oscillation between two neighboring `float` values could occur, one could use the condition `abs(old_guess - guess)/ max(old_guess, guess)>= 1e-15` instead. This makes the loop stop when the relative difference between `guess` and `old_guess` becomes less than $10^{-}15$.

Listing 7.18: We compute the square root of a number using Heron's Method [75, 119] implemented as py `while` loop. (stored in file `while_loop_sqrt.py`; output in Listing 7.19)

```python
"""Using a ‘while‘ loop to implement Heron’s Method."""

from math import sqrt  # math.sqrt is as exact with float as possible.

for number in [0.5, 2.0, 3.0]:  # The three numbers we want to test.
    guess: float = 1.0  # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.

    while old_guess != guess:  # Repeat until nothing changes anymore.
        old_guess = guess  # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.

    actual: float = sqrt(number)  # The "actual" computed sqrt value.
    print(f"\u221A{number}\u2248{guess}, sqrt({number})={actual}")
```

↓ `python3 while_loop_sqrt.py` ↓

Listing 7.19: The `stdout` of the program `while_loop_sqrt.py` given in Listing 7.18.

```
√0.5≈0.7071067811865475, sqrt(0.5)=0.7071067811865476
√2.0≈1.414213562373095, sqrt(2.0)=1.4142135623730951
√3.0≈1.7320508075688772, sqrt(3.0)=1.7320508075688772
```

old guess via `old_guess = guess`. Then we update the guess as specified in Equation 7.1, by setting `guess = 0.5 * (guess + number / guess)`.

Finally, we print the result of the computation, and for the sake of comparison, we also print the output of the `sqrt` function of the `math` module. As you can see, our algorithm delivers almost the same result. It works quite well, at very high precision. Also, notice how we used the Unicode escape method from Section 3.5.6 to represent the characters $\sqrt{\cdot}$ and $\approx$ as `\u221A` and `\u2248` to get them printed on the console.

7.7 The `else` Statement at the Bottom of Loops

Now, you have learned before that we can leave a loop body immediately by calling the `break` statement. Let's say that we want to perform a certain action *after* the loop if the loop has completed normally, i.e., if `break` was not invoked. We can do this by declaring a Boolean variable denoting whether the loop has completed normally before the loop and initializing it with `True`. If we invoke `break`, then we would first set this variable to `False`. After the loop, we could place an `if` to check if the variable is still `True` and then execute the action. This is totally fine, but Python offers us a much less verbose method: Using the `else` statement at the bottom of the loop, which works for both `for` and `while` loops:

```
for loopVariable in sequence:
    loop body statement 1
    loop body statement 2
    # ...
else:
    code executed if break not invoked 1
    # ...
normal statement 1
normal statement 2
# ...
```

```
while booleanExpression:
    loop body statement 1
    loop body statement 2
    # ...
else:
    code executed if break not invoked 1
    # ...
normal statement 1
normal statement 2
# ...
```

We now use this construct to implement a binary search [11, 53, 73]. Binary search works is an algorithm that finds the index of an element in a *sorted* sequence `data` of values. The core concept of binary search is that we consider a segment S of the list in which the element E we search may be contained. In each step of the algorithm, we want to reduce the size of this segment by ruling out the *half* in which E cannot be. We do this looking at the element M right in the middle. Now, the whole sequence `data` and, hence, also the segment S, are sorted. If M is bigger than E, then E must be in the lower half, i.e., in the sub-segment from the start of S to right before M. If M is smaller than E, then E must be in the upper half, i.e., in the sub-segment starting right after M and reaching until the end of S. Otherwise, we must have found the element. This means in one step we have effectively halved the size of S. If $n =$ `len(data)`, then we can do this at most $\log_2 n$ times and the time complexity of binary search is in $\mathcal{O}(\log n)$ [11, 53, 73].

In Listing 7.20, we want to find the indices of some characters in the alphabetically sorted string `data = "abdfjlmoqsuvwyz"`. Of course, there is the `find` method that can do this, but we want to take advantage of the fact that the characters in `data` are sorted. We search for the six characters `"a"`, `"c"`, `"o"`, `"p"`, `"w"`, and `"z"`. Four of them are in `data`, but `"c"` and `"p"` are not. To search them anyway, we let a variable `search` iterate over the list `["a", "c", "o", "p", "w", "z"]`.

Listing 7.20: We implement binary search [11, 53, 73] using a `while` loop with a `break` statement. (stored in file `while_loop_search.py`; output in Listing 7.21)

```
"""Using a ‘while‘ loop to implement Binary Search."""

data: str = "abdfjlmoqsuvwyz"  # A string of sorted characters.

for search in ["a", "c", "o", "p", "w", "z"]:  # Search six characters.
    upper: int = len(data)  # *Exclusive* upper index.
    lower: int = 0  # Lowest possible index = 0 (inclusive).
    while lower < upper:  # Repeat until lower >= upper.
        mid: int = (lower + upper) // 2  # Works ONLY in Python 3 :-).
        mid_str: str = data[mid]  # Get the character at index mid.
        if mid_str < search:  # If mid_str < search, then clearly...
            lower = mid + 1  # ...the index of search must be < mid.
        elif mid_str > search:  # If mid_str > search, then clearly...
            upper = mid  # ...the index of search must be > mid.
        else:  # If neither (mid_str < search) nor (mid_str > search)...
            print(f"Found {search!r} at index {mid} in {data!r}.")
            break  # Exit while loop and skip over while loop's else.
    else:  # executed if the while condition is False; not after break
        print(f"Did not find {search!r} in {data!r}.")
```

↓ `python3 while_loop_search.py` ↓

Listing 7.21: The stdout of the program `while_loop_search.py` given in Listing 7.20.

```
Found 'a' at index 0 in 'abdfjlmoqsuvwyz'.
Did not find 'c' in 'abdfjlmoqsuvwyz'.
Found 'o' at index 7 in 'abdfjlmoqsuvwyz'.
Did not find 'p' in 'abdfjlmoqsuvwyz'.
Found 'w' at index 12 in 'abdfjlmoqsuvwyz'.
Found 'z' at index 14 in 'abdfjlmoqsuvwyz'.
```

Now in the inner loop, we implement the binary search. This search will maintain and update two indices `lower` and `upper`. `lower` is the inclusive lower end of the segment in which `search` could be contained. It is therefore initialized with `0`. `upper` is the exclusive upper end of the segment in which `search` could be contained. We initialize it with `len(data)`; as it is *exclusive*, it will be 1 bigger than the largest valid index `len(data)- 1`. Our segment is not empty, i.e., contains at least one element, as long as `lower < upper`. This is therefore the loop condition of the inner loop.

Inside the binary search loop, we first compute the mid index as `mid = (lower + upper)// 2`[2]. We obtain the value `mid_str` as the single character at that index via `mid_str = data[mid]`.

We know that if `mid_str < search`, then our character `search` cannot be located at any index in the range 0..`mid`. So in this case, we can update the *inclusive* index `lower` to become `mid + 1`. Otherwise, if `mid_str > search`, then we know that `search` could not possibly located anywhere in the range `mid`..`len(data)- 1`. We thus would set the *exclusive* index `upper` to `mid`, which excludes all items starting at index `mid` from further consideration.

Now if neither `mid_str < search` nor `mid_str > search` were `True`, it must be that `mid_str == search`. This means that we found the location of `search` – it is at the index `mid`. Therefore, we print this result to the output. (Notice that the `!r` format specifiers in the f-string we use add the nice single quotes around `search` and `data`.) After printing the information, we exit the `while` loop using the `break` statement.

Now, there is the possibility that we cannot find `search` in `data` because it is simple in there. In this case, we will never print the output and also not leave the loop with `break`. In each iteration that does not end with `break`, we will either increase `lower` or decrease `upper`. Thus, eventually `lower < upper` will become `False`. This is when the `else` statement at the bottom of the loop is executed. Then and

[2] Interestingly, this works only because Python 3 has integers of infinite range (see Section 3.2). In programming languages like C or Java where integer types have limited ranges, we need to do `mid = lower + (upper - lower)// 2` [53].

only then we print that we did not find the string.

7.8 Summary

With this, we depart from the subject of loops. We have learned two ways to execute code iteratively: The `for` loop iterates of sequences of objects, which can either be `ranges` of numbers or arbitrary collections. The `while` loop permits us to specify an arbitrary Boolean expression as loop condition. In the bodies of both loops, we can jump to the next iteration at any time using the `continue` statement or we can exit the loops entirely using the `break` statement. Finally, placing an `else` statement at the bottom of the loop allows us to execute some code when the loop completes regularly, i.e., not via `break`.

We now have some nice tools in our hands. We can create code that branches and conditionally performs actions via `if`-`else`. And we can repeatedly perform actions via `for` and `while`. Our examples also have become more elaborate and interesting. We can now approximate π with arbitrarily many steps of the LIU Hui's approach. We can implement Heron's Method to compute the square root of a number and we perform binary search over arbitarily large (sorted) data. We can do quite a lot!

What we cannot yet do is to have a block of code that we want to re-use in *different* places.

Chapter 8

Functions

Functions are blocks of code that can be invoked from anywhere else in a program. You already learned many functions, from the basic `print` routine that just prints the value of its parameter to the output to the `sqrt` function from the `math` module which computes the square root. Now you will learn how to make your own functions.

8.1 Defining and Calling Functions

The syntax for defining our own functions in Python is as follows:

```
def my_function(param_1: type, param_2: type, ...) -> result_type:
    """
    Short sentence describing the function.

    The title of the so-called docstring is a short sentence stating
    what the function does. It can be followed by several paragraphs of
    text describing it in more detail. Then follows the list of
    parameters, return values, and raised exceptions (if any).

    :param param_1: the description of the first parameter (if any)
    :param param_2: the description of the second parameter (if any)
    :returns: the description of the return value (unless `-> None`).
    """
    body of function 1
    body of function 2
    return result  # if result_type is not None we return something


normal statement 1
normal statement 2
my_function(argument_1, argument_2)  # we can call the function like this
```

A function in Python is created by using the `def` keyword, followed by the name of the function.

> **Best Practice 25** Function names should be lower case, with underscores separating multiple words if need be [143].

`function!parameters`Then follows an opening and a closing parenthesis, i.e., `(...)`. A function can have parameters through which we can pass values to it. Inside the function, these parameters act like variables. The values of these variables can be passed in when we call (invoke, execute) the function.

> **Definition 8.1 (Parameter)** A function *parameter* is a variable defined inside the function that receives its value when the function is called.

Notice that, just like variables, all such parameters should be annotated with type hints (see Section 4.3). Functions can return results (like the `sqrt` function of the `math` module does) or return nothing (like `print`). If they return a result, the type of this result is specified via the type hint `-> result_type`. The function header ends with a colon (`:`).

Best Practice 26 All parameters and the return value of a function should be annotated with type hints. From my perspective: *A function without type hints is wrong.*

Then, indented by four spaces, follows the function body.

Best Practice 27 The body of a function is indented with four spaces.

This can be an arbitrary block of code, which may contain all the things we already learned. The function can, at any point, be left using the `return` statement. If the function is supposed to return a value `result`, then this is done via `return result`. Notice that, like the `break` statement in loops, we can place `return` at any location we want. We can also have multiple `return` values at different places in the function.

The function `my_function` then can be called from anywhere in the code by writing `my_function(value_1, value_2, ...)`, where `value_1` is passed in as value of `param_1`, `value_2` is passed in as value of `param_2`, and so on. This follows the same pattern of function calls that we already used in many of our examples.

Definition 8.2 (Argument) An *argument* is the actual value given for a function parameter when the function is called.

Between the header of a function and its body, we always need to place a so-called docstring, which is a multi-line string (see Section 3.5.5). This string consists of a title line shortly describing what the function does. After an empty line, we can (but not necessarily need to) place a paragraph of text providing a more detailled discussion. Then follows the list of parameters, each in the syntax `:param parameter_name: description`. Then follows the return value description (if the function returns something) in the form .

Best Practice 28 Each function should be documented with a docstring. If you work in a team or intend to place your code in public repositories like on GitHub, then this very very much increases the chance that your code will be used correctly. From my perspective: *A function without docstring is wrong.*

Best Practice 29 After the function and its body are defined, leave *two* blank lines before writing the next code [143].

After all of this long introduction, let us finally come to some example. Let's implement the factorial function as, well, function. The factorial is defined as follows [22, 38]

$$a! = \begin{cases} 1 & \text{if } a = 0 \\ \prod_{i=1}^{a} i & \text{otherwise, i.e., if } a > 0 \end{cases} \tag{8.1}$$

where $\prod_{i=1}^{a} i$ stands for the product $1 * 2 * 3 * \cdots * (a-1) * a$. We will implement this function in Python call it `factorial` in Listing 8.1. It should take a single parameter `a` as parameter. `a` will be type-hinted as integer and the result of our function will be an integer as well.

The body of this function is straightforward. We begin by initializing a variable `product` with the value `1`. Then, we need a loop that iterates a variable i over all integers less than or equal to `a`. We want to multiply these values to `product`. Well, we can skip over `i` $= 1$, because that would be useless. So we will use a `for` loop iterating `i` over the `range(2, a + 1)`. This effectively starts `i` at `2`. Since the upper limit `a + 1` of the `range` is always *exclusive*, the last value for `i` will be `a`. Notice that we really use `a` like a normal variable that was assigned a value.

Listing 8.1: Implementing a function computing the factorial of a positive integer number. (stored in file `def_factorial.py`; output in Listing 8.2)

```python
"""Implementing the factorial as a function."""


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


for j in range(10):  # Test the `factorial` function for `i` in 0..9`.
    print(f"The factorial of {j} is {factorial(j)}.")
```

↓ `python3 def_factorial.py` ↓

Listing 8.2: The `stdout` of the program `def_factorial.py` given in Listing 8.1.

```
The factorial of 0 is 1.
The factorial of 1 is 1.
The factorial of 2 is 2.
The factorial of 3 is 6.
The factorial of 4 is 24.
The factorial of 5 is 120.
The factorial of 6 is 720.
The factorial of 7 is 5040.
The factorial of 8 is 40320.
The factorial of 9 is 362880.
```

Anyway, inside the loop body, we compute `product *= i`, which is equivalent to `product = product * i`. After the loop, `product` holds `a`!. So we can return it as the result of the function, by writing `return product`.

We can now compute the factorial of any number `x` by calling `factorial(x)`. After the function body, we leave two empty lines. And then we compute the factorials of the numbers from 1 to 9 in a `for`-loop and print them by using f-strings. Inside this loop and in the f-string, we can use the function `factorial` exactly like any other function we used before, like `sqrt` or `print`. It may be an interesting side information at the end of this example that the factorial can actually be computed *faster* than using this product form, see, e.g. [87].

Functions can have a more than one parameter or no parameter at all. They can return one value or return nothing at all. Functions can also be called from other functions. Let us investigate these options by investigating another interesting mathematical operation: The computation of the greatest common divisor, also known as `gcd`.

This can be done using the Euclidean algorithm [17, 41, 47], going back to Euclid of Alexandria ($E\acute{\upsilon}\kappa\lambda\varepsilon\acute{\iota}\delta\eta\varsigma$) who flourished about 300 BCE. The greatest common divisor of two numbers positive $a \in \mathbb{N}_1$ and $b \in \mathbb{N}_1$ is the greatest number $g \in \mathbb{N}_1 = \texttt{gcd}(a,b)$ such that $a \bmod g = 0$ and $b \bmod g = 0$, where $\bmod$ is the modulo division operator equivalent to Python's `%`. This means that g divides both a and b without remainder. If $a = b$, then obviously $\texttt{gcd}(a,b) = a = b$ as well. Otherwise, we know that $a = ig$ for some $i \in \mathbb{N}_1$ and $b = jg$ for some $j \in \mathbb{N}_1$. If we assume, without loss of generality, that $a > b$. Then, $c = a - b = (i-j)g$ and it will be clear that $c \bmod g = (a-b) \bmod g = (i-j)g \bmod g = 0$ as well, i.e., that $\texttt{gcd}(a,b) = \texttt{gcd}(a-b,b) = g$. Similarly, $d = a \bmod b = ig \bmod (jg) = ig - \lfloor i/j \rfloor * jg = g(i - j\lfloor i/j \rfloor)$ is still divisible by g without

Listing 8.3: Implementing the Euclidean Algorithm as a function and calling it from another function. (stored in file `def_gcd.py`; output in Listing 8.4)

```python
"""Euclidian Algorithm for the Greatest Common Divisor as a function."""

from math import gcd as math_gcd  # Use math's gcd under name `math_gcd`.


def gcd(a: int, b: int) -> int:  # 2 `int` parameters and `int` result
    """
    Compute the greatest common divisor of two numbers `a` and `b`.

    :param a: the first number
    :param b: the second number
    :return: the greatest common divisor of `a` and `b`
    """
    while b != 0:  # Repeat in a loop until `b == 0`.
        a, b = b, a % b  # the same as `t = b`; `b = a % b`; `b = t`.
    return a  # If `b` becomes `0`, then the gcd is in `a`.


def print_gcd(a: int, b: int) -> None:  # `-> None` == returns nothing
    """
    Print the result of the gcd of `a` and `b`.

    :param a: the first number
    :param b: the second number
    """
    print(f"gcd({a}, {b})={gcd(a, b)}, math_gcd={math_gcd(a, b)}.")
    # Notice: no `return` statement. Because we return nothing.


print_gcd(1, 0)
print_gcd(0, 1)
print_gcd(765, 273)
print_gcd(24359573700, 35943207300)
```

↓ `python3 def_gcd.py` ↓

Listing 8.4: The stdout of the program `def_gcd.py` given in Listing 8.3.

```
gcd(1, 0)=1, math_gcd=1.
gcd(0, 1)=1, math_gcd=1.
gcd(765, 273)=3, math_gcd=3.
gcd(24359573700, 35943207300)=2148300, math_gcd=2148300.
```

remainder as $d \bmod g = 0$. This means that `gcd`$(a \bmod b, b) =$ `gcd`$(a, b) = g$, too.

Since both d and c are less than a, we could replace a with either of them. In particular, d will be less than both a and b, so we could store b in a and replace b with d. We would repeat the computation until reaching $b = 0$, at which point a will be g. Matter of fact, by choosing the module-based update, we do not even need to assume that $a > b$. Because if $b > a$, then $a \bmod b = a$ and we would just switch a and b in this step. If $a = b$, then $a \bmod b = 0$ and we would immediately terminate after the first step and return a as the greatest common divisor.

This algorithm is implemented in Listing 8.3 as function `gcd`. `gcd` has two integer parameters, `a` and `b`, and returns another `int`. Its body is surprising short: We use a `while` loop that iterates as long as `b > 0`. After the loop, we `return a` as the result. If `b == 0` holds at the beginning, the loop will never be executed and `a` is returned as-is, which is correct: `gcd`$(a, 0) = a$ for all $a \in \mathbb{N}_1$.

However, if `b > 0`, we enter the loop's body, which is a single line of code: the tuple-unpacking command `a, b = b, a %b` (see Section 5.1). This line first completely evaluates the right-hand side. This creates a tuple where the first value is `b`. The second value is `a % b`. The tuple is then unpacked

and stored in the variables `a` and `b`. `a` will thus receive the value that `b` had during the evaluation of the right-hand side. `b` will receive the previously computed value of `a % b`. In other words, `b` is stored in `a` and the remainder of the division of `a` by `b` is stored in `b`. Clearly, `b` will become smaller in each iteration and since it can never become negative, it will eventually reach `0` and the loop will terminate. Similarly, the `gcd` is never "lost" during the loop and will thus be the value in `a` at the end. And this value is returned.

So this was a function with two parameters and one return value. Let us now implement a second function, this time with no return value. `print_gcd` accepts again two parameters `a` and `b` and returns nothing. Instead, it will print the `gcd` nicely using `print` and an f-string. Since the `math` module also provides a function names `gcd` for computing, well, the greatest common divisor, we want to compare the result of our function with this one.

Of course, we cannot have two functions named `gcd` in the same context. So we import the function from the `math` module *under a different name*: `from math import gcd as math_gcd` makes the `gcd` function from the module `math` available under the name `math_gcd`. And we use it in the f-string in `print_gcd` under that name.

Finally, we confirm that `gcd` and `math_gcd` compute the same result for four test cases at the bottom of our program. Now that all is said and done, it should be mentioned that the Euclidean Algorithm has a particularly efficient binary variant which is faster than our implementation in Listing 8.3. This binary variant may have been developed in China in the first century CE [17].

8.2 Functions in Modules

You may not have noticed it, but we just made a very big step in our programming skills. We moved from simple programs which only consist of one big block of code to modular programs. We can now reuse code. And we can distribute code over multiple files. This clearly is a concern for any application larger than a simple script: How can we avoid writing our applications as a single, huge, and unstructured file which would be impossible to maintain in the long run? How can we divide our application into smaller units that we can test, improve, and maintain separately and maybe use and reuse in different contexts? A big part of the answer to this question are *modules* and *packages* [131].

For all intents and purposes within this book, a *module* is a `Python` file and a *package* is a directory wherein the file is located. As described in [131], modules do not necessarily need to be files and packages can probably be created otherwise as well, but let us keep it simple here.

Indeed, we have already worked with modules, most prominently the `math` module. This module is basically a collection of mathematical functions. Since we have implemented several mathematical functions by ourselves, let us put some of them in a module as well.

In Listing 8.5 we do just that. We create the `Python` file `my_math.py` and place two functions into it: The function `factorial` from Listing 8.1 and a new function called `sqrt`. The `sqrt` function basically encapsulates our code from back in Listing 7.18, where we implemented the Heron's Method to compute the square root, as a function. Now, `number`, the input of this algorithm, comes in as a parameter.

Best Practice 30 Package and module names should be short and lowercase. Underscores can be used to improve readability. [143]

Our new module `my_math` does not look very special or different from what we did so far. The one difference that we notice, however, is that it "does nothing". In the file, we define two functions, but we do not actively call them, we do not use them for anything. This is the purpose of this module: It just provides the functions. We will use them elsewhere.

Listing 8.6 is where we use them: We write a program, i.e., another `Python` file, named `use_my_math.py`. In this file, we want to use our two functions `factorial` and `sqrt` from the module `my_math`. For this purpose, we have to tell the `Python` interpreter where it can find these two functions. We do this by writing `from my_math import factorial, sqrt`. The meaning of the line is quite obvious: There is a module `my_math` from which we want to import, i.e., make available, two functions, namely `factorial` and `sqrt`.

Now, the `Python` interpreter knows a lot of modules. Several modules ship with any `Python` installation, like `math`. Others are installed via a package manager like `pip` [63] (we will eventually

Listing 8.5: The module `my_math`, which provides two mathematics functions, namely `sqrt`, implementing the algorithm of Heron to compute the square root from Listing 7.18, and `factorial`, copied from Listing 8.1. (src)

```
"""A module with mathematics routines."""


def factorial(a: int) -> int:  # 1 `int` parameter and `int` result
    """
    Compute the factorial of a positive integer `a`.

    :param a: the number to compute the factorial of
    :return: the factorial of `a`, i.e., `a!`.
    """
    product: int = 1  # Initialize `product` as `1`.
    for i in range(2, a + 1):  # `i` goes from `2` to `a`.
        product *= i  # Multiply `i` to the product.
    return product  # Return the product, which now is the factorial.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v == number`.
    """
    guess: float = 1.0  # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while old_guess != guess:  # Repeat until nothing changes anymore.
        old_guess = guess  # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

discuss this later). The `my_math` module is found because it is in the same directory as the program `use_my_math`.

If we had placed the `my_math.py` file into a sub-directory named `math_pack` instead, then we would import our functions from `math_pack.my_math` instead, where `math_pack` would be called a *package*. Of course, we could also create another level of directories, say we could have directory `utils`, containing directory `math_pack`, containing our file `my_math.py`. In this case, we would import our functions like `from utils.math_pack.my_math import`.... The names of package and modul are separated by a `.` when importing from them. This allows us to nicely and hierarchically structure our projects into modules and packages for different purposes.

In Listing 8.6 we can use both `sqrt` and `factorial` exactly as if we had defined them in this program. We first print a few values for `sqrt` and `factorial` and also show that we can compute the result of the square root of a factorial. We also just copy the code from Listing 7.3, where we use LIU Hui's method to approximate π – but this time, we use our own implementation of the square root function instead of the one from the `math` module. Interestingly, the sixth and last approximation step in Listing 8.7 shows exactly the same result as in Listing 7.4.

Listing 8.6: A program using the functions `sqrt` and `factorial` from the module `my_math` given in Listing 8.5. (stored in file `use_my_math.py`; output in Listing 8.7)

```
"""Using the mathematics module."""

from my_math import factorial, sqrt  # Import our two functions.

print(f"6!={factorial(6)}")  # Use the ‘factorial‘ function.
print(f"\u221A3={sqrt(3.0)}")   # Use the ‘sqrt‘ function.
print(f"\u221A(6!*1.5)={sqrt(factorial(6) * 1.5)}")  # Use both.

print("We now use Liu Hui’s Method to Approximate \u03c0.")
e: int = 6  # the number of edges: We start with a hexagon, i.e., e=6.
s: float = 1.0  # the side length: Initially 1, i.e., radius is also 1.
for _ in range(6):
    print(f"{e} edges, side length={s} give us \u03c0\u2248{e * s / 2}.")
    e *= 2  # We double the number of edges...
    s = sqrt(2 - sqrt(4 - (s ** 2)))  # ...and recompute the side length.
```

↓ `python3 use_my_math.py` ↓

Listing 8.7: The stdout of the program `use_my_math.py` given in Listing 8.6.

```
6!=720
√3=1.7320508075688772
√(6!*1.5)=32.863353450309965
We now use Liu Hui’s Method to Approximate π.
6 edges, side length=1.0 give us π≈3.0.
12 edges, side length=0.5176380902050417 give us π≈3.10582854123025.
24 edges, side length=0.2610523844401035 give us π≈3.1326286132812418.
48 edges, side length=0.13080625846028637 give us π≈3.139350203046873.
96 edges, side length=0.0654381656435527 give us π≈3.14103195089053.
192 edges, side length=0.03272346325297234 give us π≈3.1414524722853443.
```

8.3 Unit Testing

Structuring our code into functions and modules has several advantages. We can reuse code and we can divide big application into smaller pieces, both of which make it easier to understand what our program is doing. Or is *supposed* to be doing. Because errors happen in programming. They happen often and they happen naturally.

But dividing code into functions and placing it into modules has yet another advantage: We can test these separate functions separately. Imagine that you have a big application using lots of components, code for reading and writing files, code for making mathematical computations, and so on. It is not eay to figure out how to check that the big application behaves exactly as it is supposed to. There will be many combinations of inputs and environments and conditions that we would need to test. Even if we would find that the application behaves, maybe only in some cases, a bit strange, it would be hard to figure out which of its many components has an error.

However, we can also do something else: We can test each component separately, too. It is much easier to check whether our `factorial` and `sqrt` functions behave as we expect them than to do this for a whole complex program.

Definition 8.3 (Unit Testing) *Unit Testing* is a software testing technique where separate components or functions of an application are tested in isolation.

The goal of unit testing is to ensure that each unit of the software performs as expected. Since tests can be developed along (or even before!) the single functions are implemented, potential errors can be found early in the development process. As the unit tests focus on smaller, well, units, they can be less complex and easier to understand. The consequent usage of unit tests supports a modular and cleaner programming style, as it forces the developer to divide bigger programs into smaller pieces that can be invoked and tested in separation. Finally, unit testing is especially useful if an application is

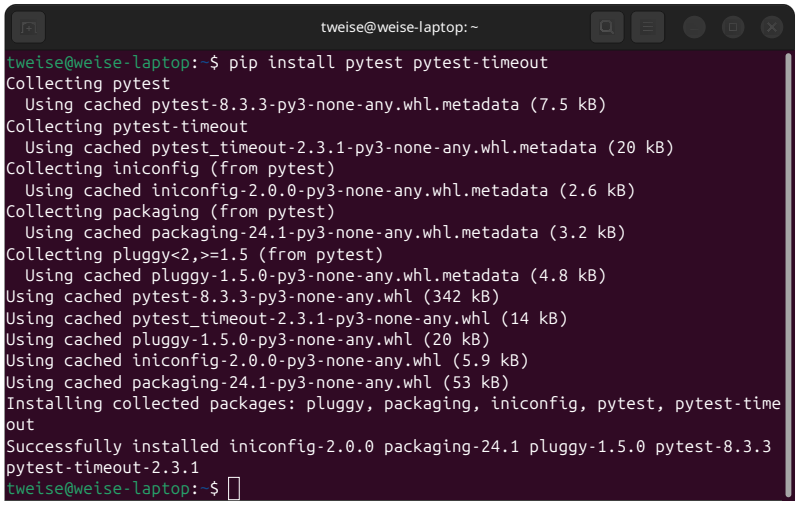


Figure 8.1: Installing `pytest` in a `Ubuntu` terminal via `pip`.

developed, improved, and maintained over a long time: If test cases are preserved and re-tested every time the software changes, then we can discover if a change that we applied to an older piece of code breaks a unit test, i.e., may lead to an unwanted change of behavior our program that we did not anticipate. Especially with its increased automation, unit testing in software development has steadily gained importance during the past decades [115, 134, 147] and is an important cornerstone of `Python` software development [35, 99, 101].

Useful Tool 4 `pytest` is a `Python` is a `Python` framework for writing and executing software tests. It can be installed via `pip install pytest pytest-timeout` as shown in Figure 8.1 on page 119. You can then apply `pytest` using the command `pytest --timeout=toInS file(s)`, where `toInS` should be replaced with a reasonable timeout in seconds and `file(s)` is one or multiple files with test cases. We provide a script for using `pytest` with a reasonable default configuration in Listing 10.4 on page 178.

You will ask yourself how testing and especially the reuse of test cases works. It is, actually, fairly simple: We have our actual program code in one or multiple `Python` files. We have our test code in some other `Python` files. For the same of clarity, if the actual code is in a file `my_code.py`, then we could the code for the tests into a file called `test_my_code.py`.

In Listing 8.8 we do just that. We create a file `test_my_math.py` and into this file, we want to put the code for testing our module `my_math`. This testing code is defined in form of functions. The names of these functions must start with `test_`.

Now our module `my_math` provides two functions, `factorial` and `sqrt`. At the top of our new tests module `test_my_math`, we `import` both of these functions. Naturally, we would create corresponding test functions and call them `test_factorial` and `test_sqrt`. These functions have no parameters and no return values.

Tests are often defined in the form of several assertions assertions:

```
assert booleanExpr  # raises AssertionError if not booleanExpr
```

An assertion is defined by the keyword `assert` followed by an arbitrary Boolean expression. If the Boolean expression evaluates to `True`, then nothing happens. If it evaluates to `False`, then an `AssertionError` is raised. This error will cause the test function to fail immediately. If the complete test function runs through without raising any `Exception`, then the test has succeeded. If it fails or raises an `Exception` at any point during its execution, the test has failed.

Now we create the function `test_factorial` to test the `factorial` function. For example, we know that $0! = 1$. So it makes sense to define `assert factorial(0)== 1`. The Boolean expression here is `factorial(0)== 1`, which is obviously only `True` if `factorial(0)` evaluates to `1`. We define similar cases for $1!$, $2!$, and $3!$. Obviously, we cannot write down a complete list of all possible inputs.

Listing 8.8: A small unit test suite for Listing 8.5. (src)

```python
"""Testing our mathematical functions."""

from math import inf, isnan, nan  # some float value-checking functions

from my_math import factorial, sqrt  # Import our two functions.


def test_factorial() -> None:
    """Test the function `factorial` from module `my_math`."""
    assert factorial(0) == 1  # 0! == 1
    assert factorial(1) == 1  # 1! == 1
    assert factorial(2) == 2  # 2! == 2
    assert factorial(3) == 6  # 3! == 6
    assert factorial(12) == 479_001_600  # 12! == 4790'016'00
    assert factorial(30) == 265_252_859_812_191_058_636_308_480_000_000


def test_sqrt() -> None:
    """Test the function `sqrt` from module `my_math`."""
    assert sqrt(0.0) == 0.0  # The square root of 0 is 0.
    assert sqrt(1.0) == 1.0  # The square root of 1 is 1.
    assert sqrt(4.0) == 2.0  # The square root of 4 is 2.
    s3: float = sqrt(3.0)  # Get the approximated square root of 3.
    assert abs(s3 * s3 - 3.0) <= 5e-16  # sqrt(3)^2 should be close to 3.
    assert sqrt(1e10 * 1e10) == 1e10  # 1e10^2 = 1e10 * 1e10
    assert sqrt(inf) == inf  # The square root of +inf is +inf.
    assert isnan(sqrt(nan))  # The root of not-a-number is still nan.
```

Listing 8.9: The output of the unit tests in Listing 8.8: While the test of `factorial` succeeds, our `sqrt` function fails for input `0.0`.

```
$ pytest --timeout=10 --no-header --tb=short test_my_math.py
========================== test session starts ==========================
collected 2 items

test_my_math.py .F                                                  [100%]

=============================== FAILURES ================================
_______________________________ test_sqrt ________________________________
test_my_math.py:20: in test_sqrt
    assert sqrt(0.0) == 0.0  # The square root of 0 is 0.
my_math.py:28: in sqrt
    guess = 0.5 * (guess + number / guess)  # The new guess.
E   ZeroDivisionError: float division by zero
======================== short test summary info ========================
FAILED test_my_math.py::test_sqrt - ZeroDivisionError: float division ...
====================== 1 failed, 1 passed in 0.03s ======================
# pytest 8.3.3 with pytest-timeout 2.3.1 failed with exit code 1.
```

But these are the four smallest ones for which the factorial is defined. By testing also 12!, we have checked `factorial` for a mid-sized argument. We then also create a test case for a fairly large number, say 30!. We have computed this value using an online tool that offers arbitrary precision. The result of this function is more than $265 * 10^{30}$, which is fairly beyond the range of 64 bit integers (and thus showcases that integers in `Python` 3 have an unlimited range, as stated back in Section 3.2). This test case validates whether our `factorial` function works well for large numbers, too. If our `factorial` function passes all of these tests, we can be fairly certain that it is implemented correctly. There could still be errors, though. For example, we did not test 4!. But at least it looks unlikely that 3! and 12! "work," but not 4!.

For the `sqrt` function, we create similar test function and call it `test_sqrt`. Reasonable test cases are `assert sqrt(0.0)== 0.0`, `assert sqrt(1.0)== 1.0`, and `assert sqrt(4.0)== 2.0`. We would also expect that `sqrt(x)* sqrt(x)== x` for different `x`. However, we have to account for the limited precision of the datatype `float` discussed in Section 3.3.1. Even if we would get as close to `sqrt(x)` for some `x` as possible, we can only represent 15 to 16 digits. Therefore, we have to give a little bit of wiggle room when we compute `s3 = sqrt(3.0)` and expect that `s3 * s3 == 3.0`. We do this by writing `assert abs(s3 * s3 - 3.0)<= 5e-16`, i.e., by assuming that the difference between `3.0` and `s3 * s3` is not bigger than the very small number $5 * 10^{-16}$. On the other hand, the square root of `1e10 * 1e10` should be representable exactly, namely as `1e10`.

The datatype `float` also offers us two special values, which both can be imported from the `math` module and which were discussed in Section 3.3.5: `inf` stands for positive infinity $+\infty$ and `nan` stands basically for "undefined", a value that you may get if you try to compute something like `inf - inf`. Our `sqrt` function has to understand and correctly handle these values as well. `sqrt(inf)` should again return `inf`. `sqrt(nan)` should return `nan`. However, we cannot do `assert sqrt(nan)== nan`, since `==` will yield `False` for any value if at least one `nan` is involved (see again Section 3.3.5). For testing whether `sqrt(nan)` yield `nan`, we use the function `isnan` from the `math` module. This function returns `True` for `isnan(nan)` and `False` otherwise.

With this, we have covered most reasonable inputs that either `factorial` or `sqrt` could receive. We have defined what we would expect as output for these inputs. These expectations are implemented as test cases. Regardless of how `factorial` or `sqrt` are implemented, they should pass these test cases. Otherwise, they are wrong.

Running our test cases with `pytest` yields the output in Listing 8.9. This output tells us that `test_factorial` ran through without any issue. `test_sqrt` however failed with a `ZeroDivisionError` that was raised inside our `sqrt` function. This happened when we tried to compute `sqrt(0.0)`.

When we look back at our `my_math` module in Listing 8.6, we find that our initial guess for the square root is `1`. In each step, we then compute `guess = 0.5 * (guess + number / guess)`. If `number` is `0.0`, then this effectively means `guess = 0.5 * guess`. We have learned that the precision of `floats` is finite and that positive values smaller than `5e-324` will just become `0.0`. This will happen here too, and `guess` will thus eventually become `0.0`. In the next iteration, this leads to us trying to divide `0.0 / 0.0`, causing the `ZeroDivisionError`.

In order to fix this problem, we introduce the check `if number <= 0.0` into our `sqrt` function in Listing 8.11. If this new conditional evaluates to `True`, we return `0.0` directly. Now we do not consider the case that a negative number is passed into `sqrt` at this point. Here, we would ideally `raise` an error by ourselves, but we will learn only later how to do that.[1]

We apply the same tests to the new version of the `sqrt` function in Listing 8.11. The output of the test case, provided in Listing 8.12, now indicates another error: We a timeout!

You see, when invoking `pytest` with the option `--timeout=10` – which only works if the package `pytest-timeout` is installed – we limit the maximum runtime of our test suite to ten seconds. Ten seconds is a reasonable time for *this book* as we actually run all the scripts automatically during the book building process. In practical situations, you will usually choose a larger time limit.

Best Practice 31 Always attach a timeout to your unit tests. This timeout can be generous, maybe one hour, but it will serve as sentinel against either endless loops, deadlocks, or other congestion situations which all would be practical test failures. Timeouts protect automated builds or continuous integration systems from clogging.

[1] We learn it in Chapter 9 and there we will revisit our `sqrt` function in Listing 9.1, too.

Listing 8.10: An improved variant of Listing 8.5 dealing with the failing test case `0.0` discovered in Listing 8.9. (src)

```
"""A second version of our module with mathematics routines."""

# factorial is omitted here for brevity


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v == number`.
    """
    if number <= 0.0:  # Fix for the special case `0`:
        return 0.0  # We return 0; for now, we ignore negative values.

    guess: float = 1.0  # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while old_guess != guess:  # Repeat until nothing changes anymore.
        old_guess = guess  # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Listing 8.11: We use the same small unit test suite given in Listing 8.8 for `sqrt` in Listing 8.10. (src)

```
"""Testing our second version of the `my_math` module."""

from math import inf, isnan, nan  # some float value-checking functions

from my_math_2 import sqrt  # Get our 2nd square root implementation.

# test_factorial() is omitted for brevity


def test_sqrt() -> None:
    """Test the function `sqrt` from module `my_math_2`."""
    assert sqrt(0.0) == 0.0  # The square root of 0 is 0.
    assert sqrt(1.0) == 1.0  # The square root of 1 is 1.
    assert sqrt(4.0) == 2.0  # The square root of 4 is 2.
    s3: float = sqrt(3.0)  # Get the approximated square root of 3.
    assert abs(s3 * s3 - 3.0) <= 5e-16  # sqrt(3)^2 should be close to 3.
    assert sqrt(1e10 * 1e10) == 1e10  # 1e10^2 = 1e10 * 1e10
    assert sqrt(inf) == inf  # The square root of +inf is +inf.
    assert isnan(sqrt(nan))  # The root of not-a-number is still nan.
```

Listing 8.12: The output of the unit tests in Listing 8.11: This time, we hit the timeout because of an endless loop!

```
$ pytest --timeout=10 --no-header --tb=short test_my_math_2.py
========================== test session starts ==========================
collected 1 item

test_my_math_2.py F                                                  [100%]

================================ FAILURES =================================
_______________________________ test_sqrt ________________________________
test_my_math_2.py:18: in test_sqrt
    assert sqrt(inf) == inf  # The square root of +inf is +inf.
my_math_2.py:18: in sqrt
    while old_guess != guess:  # Repeat until nothing changes anymore.
E   Failed: Timeout >10.0s
======================== short test summary info ========================
FAILED test_my_math_2.py::test_sqrt - Failed: Timeout >10.0s
========================== 1 failed in 10.02s ===========================
# pytest 8.3.3 with pytest-timeout 2.3.1 failed with exit code 1.
```

Listing 8.13: An improved variant of Listing 8.10 dealing with the failing test case `inf` discovered in Listing 8.12. (src)

```python
"""A third version of our module with mathematics routines."""

from math import isfinite

# factorial is omitted here for brevity


def sqrt(number: float) -> float:
    """
    Compute the square root of a given `number`.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v == number`.
    """
    if number <= 0.0:  # Fix for the special case `0`:
        return 0.0  # We return 0; for now, we ignore negative values.
    if not isfinite(number):  # Fix for case `+inf` and `nan`:
        return number  # We return `inf` for `inf` and `nan` for `nan`.

    guess: float = 1.0  # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while old_guess != guess:  # Repeat until nothing changes anymore.
        old_guess = guess  # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

The time limit protected us here. Our `sqrt` function goes into an endless loop if `number = inf`. Initially, we set `guess = 1.0`. In the first iteration of the loop in `sqrt`, `guess = 0.5 * (guess + number / guess)` makes `guess` become `inf`. However, in the second iteration, we have `number / guess` becoming `inf / inf` which yields `nan`. From now on, all calculations yields `nan`. Since `nan != nan` is `True`, the loop never ends.

We finally solve this problem in Listing 8.13. Here, we add a new condition: `if not isfinite(number)`, we return `number` as-is. `isfinite` is another function from the `math` module. It takes one parameter and it returns `True` if it is a finite number and `False` if it is `inf`, `-inf`, or `nan`. This means that condition can only become `True` for `inf` or `nan`, as we already return `0.0` if

Listing 8.14: We use the same small unit test suite given in Listing 8.8 for `sqrt` in Listing 8.13. (src)

```
"""Testing our third version of the `my_math` module."""

from math import inf, isnan, nan  # some float value-checking functions

from my_math_3 import sqrt  # Get our 3rd square root implementation.

# test_factorial() is omitted for brevity


def test_sqrt() -> None:
    """Test the function `sqrt` from module `my_math_3`."""
    assert sqrt(0.0) == 0.0  # The square root of 0 is 0.
    assert sqrt(1.0) == 1.0  # The square root of 1 is 1.
    assert sqrt(4.0) == 2.0  # The square root of 4 is 2.
    s3: float = sqrt(3.0)  # Get the approximated square root of 3.
    assert abs(s3 * s3 - 3.0) <= 5e-16  # sqrt(3)^2 should be close to 3.
    assert sqrt(1e10 * 1e10) == 1e10  # 1e10^2 = 1e10 * 1e10
    assert sqrt(inf) == inf  # The square root of +inf is +inf.
    assert isnan(sqrt(nan))  # The root of not-a-number is still nan.
```

Listing 8.15: The output of the successful unit tests in Listing 8.14.

```
$ pytest --timeout=10 --no-header --tb=short test_my_math_3.py
========================== test session starts ==========================
collected 1 item

test_my_math_3.py .                                                [100%]

=========================== 1 passed in 0.01s ===========================
# pytest 8.3.3 with pytest-timeout 2.3.1 succeeded with exit code 0.
```

`number <= 0.0`. Thus, we return `inf` if the input of our function is `inf` and `nan` if the input is `nan`. These are also the results that we would expect.

In Listing 8.14, we apply our unit tests to this new version of our `sqrt` function. As you can see in the output provided in Listing 8.15, the tests now complete successfully. This was a good example of how tests can help us to spot errors in our code. When looking at Listing 8.6, we certain assumed that all of our functions were implemented correctly. However, pytest has helped us to spot two erros. We then fixed these errors.

Best Practice 32 A function which is not unit tested is *wrong*.

Best Practice 33 A good unit test for a given function should cover both expected as well as extreme cases. For a parameter, we should test both the smallest and largest possible argument values, as well as values from its normally expected range.

Best Practice 34 A good unit test for a function should cover all branches of the control flow inside the function. If a function does one thing in one situation and another thing in another situation, then both of these scenarios should have associated unit tests.

8.4 Function Arguments: Default Values, Passing them by Name, and Constructing them

After the brief introduction into unit testing, let us now come to a lighter topic: passing arguments to functions.We have already seen examples for this. Our `gcd` function from back in Listing 8.3 has two

Listing 8.16: Implementing the probability density function (PDF) of the normal distribution as function with default argument values. (src)

```
"""The Probability Density Function (PDF) of the Normal Distribution."""

from math import exp, pi, sqrt


def pdf(x: float, mu: float = 0.0, sigma: float = 1.0) -> float:
    """
    Compute the probability density function of the normal distribution.

    :param x: the coordinate at which to evaluate the normal PDF
    :param mu: the expected value or arithmetic mean, defaults to `0.0`.
    :param sigma: the standard deviation, defaults to `1.0`
    :return: the value of the normal PDF at `x`.
    """
    s2: float = 2 * (sigma ** 2)  # stored for reuse
    return exp((-((x - mu) ** 2)) / s2) / sqrt(pi * s2)  # compute pdf
```

parameters `a` and `b` and we can invoke it by writing the values of these parameters in parentheses. `gcd(12, 4)` will invoke `gcd` and assign `12` to `a` and `4` to `b`.

We can also let parameters have so-called *default values*. If a parameter has a default value, then we can either specify the value of the parameter when calling the function *or* we can simply omit it, i.e., not assign a value to it. In the latter case, the parameter will then have the default value. From inside the function, this looks the same as if we passed in the default value.

As a simple example, let us implement the probability density function (PDF) of the normal distribution. You may remember from high school math that this function, let's call it f, defined as

$$f(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{-(x-\mu)^2}{2\sigma^2}} \tag{8.2}$$

Here, μ is the arithmetic mean, i.e., the expected value, of the distribution and σ is its standard deviation (making σ^2 its variance). x is the variable of this function. This function describes typical bell-shaped curve of the normal distribution as sketched in Figure 8.2. Implementing this function as a, well, function in Python is straightforward. Listing 8.16 offers the function `pdf` with three parameters: `x`, `mu`, and `sigma`, which represent x, μ, and σ.

Now, the two parameters μ and σ of f (respectively `mu` and `sigma` of `pdf`) represent the general normal distribution. The *standard* normal distribution has $\mu = 0$ and $\sigma = 1$, i.e., is centered around the mean 0 and has a standard deviation (and variance) of 1. We therefore define the *default* values for `mu` to be `0.0` and for `sigma` to be `1.0`. This is done directly in the header of the function. Instead of writing `x: float, mu: float, sigma: float`, we write `x: float, mu: float = 0.0, sigma: float = 1.0`.

In Listing 8.17, we import our new `pdf` function and use it inside a program. When calling `pdf`, we can omit the values of the parameters with default values, in which case they will take on their default values. For example, invoking `pdf(0.0)` if equivalent to calling `pdf(0.0, 0.0, 1.0)`. We also can specify some of the parameters with default values while omitting others. For instance, the function call `pdf(2.0, 3.0)` is the same as `pdf(2.0, 3.0, 1.0)`. Obviously, we must always specify the first parameter (`x`), because it has no default value.

Best Practice 35 Default parameter values must always be immutable.

The default value of a function must always be immutable. If you would pass in, e.g., a `list`, then the function could modify the list and the next call to this function would then receive this modified list. Even worse, if the function was to return the list, it could be modified outside of the function. The behavior of such code could become arbitrarily hard to debug.

Back to business. What would we do if we want to specify the value of the parameter `sigma` of our function, but leave `mu` at its default value? We can do this by passing in values by parameter name:

Listing 8.17: Using the PDF of the normal distribution implemented in Listing 8.16. (stored in file `use_normal_pdf.py`; output in Listing 8.18)

```python
"""Use the Probability Density Function of the Normal Distribution."""

from normal_pdf import pdf  # import our function

print(f"f(0,0,1) = {pdf(0.0)}")  # x is given, default used otherwise.
print(f"f(2,3,1) = {pdf(2.0, 3.0)}")  # x and mu are given, sigma=1.0.
print(f"f(-2,7,3) = {pdf(-2.0, 7.0, 3.0)}")  # all three are given.
print(f"f(-2,0,3) = {pdf(-2.0, sigma=3.0)}")  # x and sigma given.
print(f"f(0,8,1.5) = {pdf(mu=8.0, x=0.0, sigma=1.5)}")  # unordered...

# We call the function using a dictionary of parameter values.
args_dict: dict[str, float] = {"x": -2.0, "sigma": 3.0}
print(f"f(-2,0,3) = {pdf(**args_dict)}")  # notice the double "*" ("**")

# We call the function using a tuple of parameter values.
args_tup: tuple[float, float, float] = (-2.0, 7.0, 3.0)
print(f"f(-2,7,3) = {pdf(*args_tup)}")  # notice the single "*"

# We call the function using a list of values, but leave one default.
args_lst: list[float] = [2.0, 3.0]
print(f"f(2,3,1) = {pdf(*args_lst)}")  # notice the single "*"
```

↓ `python3 use_normal_pdf.py` ↓

Listing 8.18: The stdout of the program `use_normal_pdf.py` given in Listing 8.17.

```
f(0,0,1) = 0.3989422804014327
f(2,3,1) = 0.24197072451914337
f(-2,7,3) = 0.0014772828039793357
f(-2,0,3) = 0.10648266850745075
f(0,8,1.5) = 1.7708679390146084e-07
f(-2,0,3) = 0.10648266850745075
f(-2,7,3) = 0.0014772828039793357
f(2,3,1) = 0.24197072451914337
```

`pdf(-2.0, sigma=3.0)` passes in `-2.0` for `x` and `3.0` for `sigma`. It does not specify any value for `mu`, leaving it at its default value, which renders the call equivalent to `pdf(-2.0, 0.0, 3.0)`. This passing in of arguments by specifying `parameterName=value` also allows us to specify the arguments in arbitrary order. `pdf(mu=8.0, x=0.0, sigma=1.5)` is an example of this. Don't do such things, though.

Listing 8.17 provides also another interesting way to call a function in Python. As we have established by now, the parameters of a function have names. If we write something like

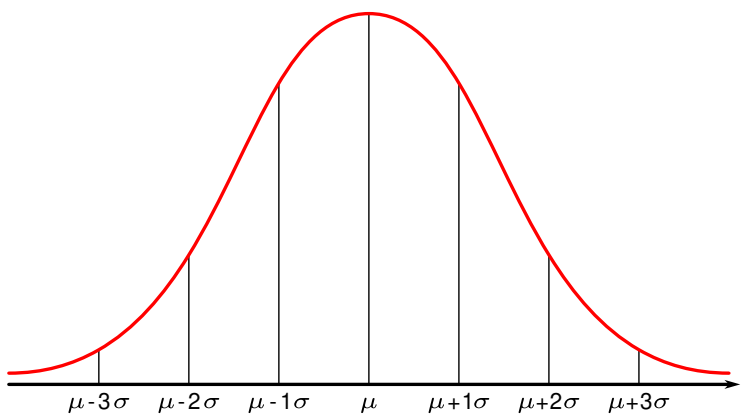


Figure 8.2: A sketch of the probability density function of the normal distribution given in Equation 8.2.

`mu=8.0, x=0.0, sigma=1.5` to assign arguments, this looks very similar to the way we created dictionary constants back in Section 5.4 and Listing 5.26. Calling `pdf(-2.0, sigma=3.0)` is equivalent to writing `pdf(x=-2.0, sigma=3.0)`.

We can create a dictionary with the values `{"x": -2.0, "sigma": 3.0}`. Let's call this dictionary `args_dict`. Can we now somehow pass in these values to `pdf`? We indeed can: We just have to write `pdf(**args_dict)`. Doing this will unpack the dictionary `args_dict` and pass all the values under their assigned names in as arguments to their corresponding parameters. `pdf(**args_dict)` is thus equivalent to `pdf(x=-2.0, sigma=3.0)`. Two things are to notice here: First, the double `*` (called wildcard, star, or asterisk) *before* the dictionary, i.e., the `**` is telling `Python` to unpack the dictionary this way. Second, default argument values still apply here, i.e., `mu` will have value `0.0` in this function call.

Similarly, maybe we do not care about the parameter names but want pass them in by position, as we have always done in the past. Then, we can construct a sequence, e.g., a `list` or `tuple` with the parameter values. Of course, `lists` and `tuples` do not store key-value relationships, only values at positions. We could create a tuple `args_tuple` with the value `(-2.0, 7.0, 3.0)`. Then, invoking `pdf(*args_tuple)` will basically fill in the three values in their into the parameters, i.e., will be equivalent to `pdf(-2.0, 7.0, 3.0)`. This time, only a single wildcard `*` is placed before `args_tuple`. We can also pass the parameters in by "unpacking" a list. In our example Listing 8.17, we create the list `args_list = [2.0, 3.0]`. Calling `pdf(*args_list)` then is the same as writing `pdf(2.0, 3.0)`, which, in turn, is identical to `pdf(2.0, 3.0, 1.0)`. Again, parameters with default values do not need to be supplied.

At first glance, the use of all of the above is not entirely clear. What do we need default parameter values for? Well, in some cases, you may want to enable a user to "customize" your functions. A typical example is the `plot` method of the `Axes` object provided by popular `Matplotlib` library. You will normally provide a sequence of x- and y-coordinates to this function it will draw a line which goes through all the points specified this way. However, you can also optionally specify a color for the line, markers to be painted at the points, line dash style, a label, colors and sizes for the markers, a z-order to be used if multiple lines are drawn, and so on, and so on. The use of default arguments allows the function call to be relatively simple in most cases, while still allowing the user to do more complex formatting if need be.

From this example, we can also directly extrapolate a use case for building the arguments of a function in a dictionary. Imagine that you write an own function that uses one of the plotting methods of `Matplotlib`. Let's say that your function does a plot call where it provides ten parameter values. However, you have one special case where you need to provide one more parameter, maybe a line dash style that you otherwise do not need to provide. Then, you could have some `if` in your code that branches to do the ten-parameter-call in one case and the eleven-parameter-call in the other. This means that a rather complex function call appears twice in a very similar manner. If you instead construct the parameters in a dictionary and in the `if` branch just add the eleventh parameter if need be, your code will become much simpler.

8.5 Functions as Parameters or Variables, `Callable`, and `lambdas`

We have just learned that we can basically construct a function call by placing the parameter values into collection objects and then invoke the function by "unpacking" the collection using either `*` (for position-based parameters) or `**` (for dictionaries). But there is one more interesting thing that we can do with functions. You see, in `Python`, all things are objects. References to objects can be stored by variables or passed in as function arguments. Functions are objects too. This means that we can also store functions in variables or pass them as argument to other functions!

At first glance, this sounds awfully odd. Why would someone like to pass a function as parameter to another function? At second glance, there are a wide variety of situations where we would actually want to do that. Having done so many mathematics-based examples in this book, let's pick one such situation arising in maths.

In high school, you have learned about integration and differentiation. A *definite integral* is the formal calculation of the area under a function, using infinitesimal stripes of the region. The second fundamental theorem of calculus states: Given a function $f(x)$ which is continuous over the real interval $[a, b]$ and its antiderivative $F(x)$, the definite integral $\int_a^b f(x)\,dx = F(b) - F(a)$ (where antiderivative means that $F' = f$). In other words, if we want to get the area beneath $f(x)$ over the

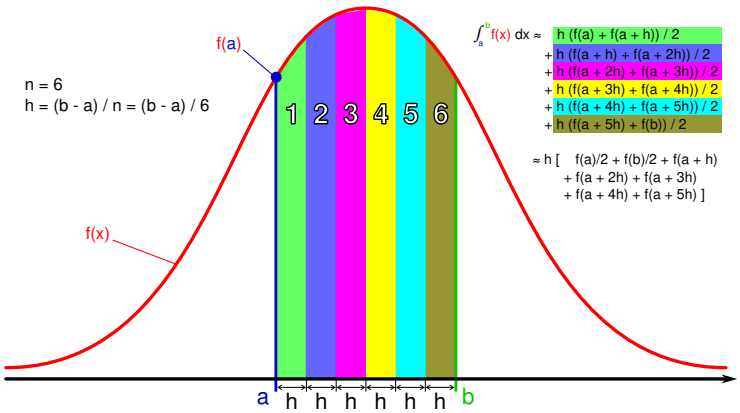


Figure 8.3: A sketch of the trapezoid method for approximating definite integrals.

interval $[a, b]$, we would first obtain the antiderivative and then simply calculate $F(b) - F(a)$. Obtaining the antiderivative involves pen and paper symbolic maths. We cannot really program such maths at this stage in this book as an example. But we can go back to the original definition of the definite integral, namely that it equals the area beneath the function obtained by using infinitesimal (read: immeasurably small) strips of the region.

While we cannot really use "immeasurably small" strips, we could use "fairly small" ones to approximate the right result. If we have a function $f(x)$, which would obviously be a parameter of our program, as well as the interval limits a and b, we could divide the range $[a, b]$ into n strips. For each strip, we could approximate the area beneath $f(x)$, add up the n areas, and have a rough approximation of the total area. n could be another parameter for our integration approach: The larger n, the smaller will the strips get, the more accurate should our estimate become (and the longer it will take to get it).

The trapezoid method is one very simple implementation of this idea [40]. As illustrated in Figure 8.3, it treats the strips as right trapezoids. The baseline of the trapezoid is a piece of the x-axis with length $h = (b - a)/n$. Clearly, $n * h = b - a$ and thus, n trapezoids of equal base length form the range of the x-axis under $f(x)$. The baseline first trapezoid starts right at $x = a$ and extends to $x = a + h$. The second trapezoid starts at $x = a + h$ and extends to $x = a + 2h$. The last trapezoid starts at $x = a + (n - 1)h$ and extends to $x = a + nh = b$. Each trapezoid has two sides meeting with its baseline in right angles. The length of these sides are the values of $f(x)$ at the corresponding x-coordinate. The area of the i-th trapezoid is thus $h[f(a + (i - 1)h) + f(a + ih)]/2$. Summing up the n areas yields an approximation of the definite integral, as sketched in Figure 8.3.

Each value $f(a + ih)$ except for $i = 0$ and $i = n$ appears twice in the sum and each time is halved. Instead of computing these values twice, dividing them by two, and then adding them, we can simply compute them only once. Our implementation `integrate` of this approach, given in Listing 8.19, does exactly that.

The interesting part of our `integrate` method, however, is not the approximation of the integral. It is the header. More precisely, it is the first parameter: `f: Callable[[float], float |int]`. `Callable` is the type hint for anything that can be called, i.e., functions [5]. Like the type hint for lists and tuples, it can be parameterized following the scheme [91]:

```
Callable[[parameterType1, parameterType2, ...], resultType]
```

In other words, inside the `Callable[...]`, we first provide the list of parameter types, then a comma `,` followed by the return type. `f: Callable[[float], float |int]` thus states that our function expects, as first parameter, *another function* `f`. `f` must accept one parameter of type `float`. Its return type is `float |int`, i.e., it should return either a `float` or `int` [106]. Notice that the type `Callable` is provided by the module `typing`[2], so we need to import it first if we want to use it.

Additionally, our `integrate` function accepts two parameters `a` and `b` which must be of type `float |int`, i.e., can either be an `float` or `int`. Their default values are `0.0` and `1.0`, meaning

[2] [5] states that this import is now deprecated and we should use `collections.abc.Callable` instead. In the past, this created some errors for me, so for now we stick with using `typing`.

Listing 8.19: Using the trapezoid method to numerically approximate definite integrals, as sketched in Figure 8.3. (stored in file `integral.py`; output in Listing 8.20)

```python
"""Numerical integration using the trapezoid method."""

from math import pi, sin
from typing import Callable

from normal_pdf import pdf


def integrate(
        f: Callable[[float], float | int], a: int | float = 0.0,
        b: int | float = 1.0, n: int = 100) -> float:
    """
    Integrate the function ‘f‘ between ‘a‘ and ‘b‘ using trapezoids.

    :param f: the function to integrate: in float, out float or int
    :param a: the lower end of the range over which to integrate
    :param b: the upper end of the range over which to integrate
    :param n: the number of trapezoids to use for the approximation
    :return: the approximated integration result
    """
    result: float = 0.5 * (f(a) + f(b))  # Initialize with start + end.
    h: float = (b - a) / n  # The base length of the trapezoids.
    for i in range(1, n):  # The steps between start and end.
        result += f(a + h * i)  # Add f(x) between trapezoids.
    return result * h  # Multiply result with base length.


print(f"\u222b1dx|0,1 \u2248 {integrate(lambda _: 1, n=7)}")
print(f"\u222bx²-2dx|-1,1 \u2248 {integrate(lambda x: x * x - 2, -1)}")
print(f"\u222bsin(x)dx|0,\u03C0 \u2248 {integrate(sin, b=pi, n=200)}")
print(f"\u222bf(x,0,1)dx|-1,1 \u2248 {integrate(pdf, -1)}")
print(f"\u222bf(x,0,1)dx|-2,2 \u2248 {integrate(pdf, -2, 2)}")
```

↓ `python3 integral.py` ↓

Listing 8.20: The stdout of the program `integral.py` given in Listing 8.19.

```
∫1dx|0,1 ≈ 1.0
∫x²-2dx|-1,1 ≈ -3.3332
∫sin(x)dx|0,π ≈ 1.9999588764792162
∫f(x,0,1)dx|-1,1 ≈ 0.6826733605403601
∫f(x,0,1)dx|-2,2 ≈ 0.9544709416896361
```

that we will integrate over $[0, 1]$ if they are not specified. As final parameter, we expect an integer `n`, by default `100`, which corresponds to the number of trapezoids that we shall use for the approximation.

Let's now test how well our trapezoid-based integration works. First, let's compute the definite integral $\int_0^1 1\,dx$. For this purpose, we need to pass the function $f(x) = 1$ as the `f` parameter of `integrate`. We could do this by writing:

```python
def const_1(x: float) -> float:
    return 1.0
```

or

```python
def const_1(_: float) -> float:
    return 1.0
```

Listing 8.21: The results of static type checking with Mypy of the program given in Listing 8.19.

```
$ mypy integral.py --no-strict-optional --check-untyped-defs
Success: no issues found in 1 source file
# mypy 1.13.0 succeeded with exit code 0.
```

where the _ indicates that we are actually going to ignore this parameter (see Best Practice 23). We could then invoke `integrate(const_1)`.

However, there also is a more compact way to specify functions that we are only going to use once: The so-called `lambdas` [77], which have the structure

```
lambda param1, param2, ...: return value
```

Due to the `:` in the notation, we cannot annotate `lambdas` with type hints. As a `lambda` expression is very small and only used once, this does not pose a serious problem. Either way, since we need one parameter for our 1-returning-function and since we do not care about the value of this parameter, we can simply write:

```
integrate(lambda _: 1.0)
```

`lambdas` are functions that we only want use in a single place. This is clearly the case for a function that ignores its parameter and always returns `1.0`. So we pass this expression into our `integrate` function as value of the parameter `f`.

Obviously, the area under the constant function 1 over the range $[0, 1]$ is also 1. Using `n=7` trapezoids, we get exactly this result. In the output of our program, we write `∫1dx|0,1` where the `|0,1` denotes the limits $[0, 1]$ of the interval over which we integrate (and we use the unicode escape `"\u222b"` to represent the $\int$ character in the f-strings).

Having passed this simple sanity test, let's try to compute the beneath under the function $g(x) = x^2 - 2$ over the interval $[-1, 1]$, i.e., $\int_{-1}^{1} x^2 - 2\,dx$. The antiderivative of $g(x)$ is $G(x) = \frac{1}{3}x^3 - 2x + c$ and $G(1) - G(-1) = [\frac{1}{3} - 2] - [-\frac{1}{3} + 2] = \frac{2}{3} - 4 = -3\frac{1}{3} = -3.\overline{3}$. We pass the function $g(x)$ as `lambda` expression into our `integrate` function by writing `lambda x: x*x - 2`. We also need to specify a value for the parameter `a`, namely `-1`. Using the default value of `n=100` steps, our method returns `-3.3332`, which is fairly close to $-3.\overline{3}$.

Let us now integrate the sinus function over the interval $[0, \pi]$. We can import `sin` from the `math` module and pass it into our function for `f`. We also need to specify `b = pi`, which we, too, have imported from `math`. This time, we use `n = 200` trapezoids to approximate the definite integral. The antiderivative of $\sin x$ is $-\cos x + c$, so the expected output would be $[-\cos \pi] - [-\cos 0] = [-(-1)] - [-1] = 2$. Indeed, our function delivers about 1.99996, which is, again, fairly close.

Let us, as final example, also integrate the probability density function (PDF) of the normal distribution. We implemented this function and called it `pdf` back in Listing 8.16, so we just need to `import` it from there. Notice that this function actually had three parameters, `x`, `mu`, and `sigma`. The latter two had the default values `mu = 0.0` and `sigma = 1.0`. If these values are used, the function computes the PDF of the *standard* normal distribution.

Interestingly, although we demand that the function supplied to `integrate` should only have one parameter, we can still pass in `pdf` due to its second and third parameter being optional. Even Mypy accepts this function as valid parameter, as its output (or better, the lack of it) in Listing 8.21 suggests.

The standard normal distribution has standard deviation $\sigma = 1$ and mean $\mu = 0$. If we integrate it over the interval $[-1, 1]$, we basically compute the probability that a standard normally distributed random variable X is drawn from the interval $[\mu - \sigma, \mu + \sigma]$, i.e., from within one standard deviation from the mean. As you may still know from high school math, this probability is roughly 68.26%. Similarly, the probability that such a random variable is sampled from within $[-2, 2]$, i.e., not more than two standard deviations away from the mean, is about 95.44%. The output of our little integrator function in Listing 8.20 fits perfectly to this.

As a final clarification, let us mention that trapezoid-based approximation of definite integrals is by no means the best approach. Others, like Simpson's rule, are much better [40]. Still, as an example of functions that work on other functions, it was suitable.

8.6 Summary

Functions are the central building block needed to create modular code. They allow us to put code into units with clearly defined interfaces. The input of a function are its parameters (if any). The output is its return value (if any). Both parameters and return value can be annotated with type hints. The description of what the function does as well as what the parameters and the return value mean, goes into the docstring.

This clear definition and separation from the rest of the code has several advantages. First, multiple programmers can work together on a project. They can work on different functions, which we can place into different modules (`Python` files). If we did not have functions available, this would be impossible.

Second, by using proper type hints for the function parameters and return value as well as proper docstrings, the behavior of functions is easier to understand by fellow programmers. Type checkers like `Mypy` can also verify whether functions are called with correct parameters more easily. The code does not just become more modular, but easier to maintain. It can be checked by static code analysis tools as well.

Third, functions allow us to reuse pieces of code in different locations. If we want to compute the definite integral of two different functions, we do not have to implement two routines for integration. We can make one and use it in two different locations.

Fourth, if we have a code with clearly defined input and output that exists in isolation from other code, then we can *test* this code in isolation as well. It may be extremely difficult to test a complete program. However, it may be rather straightforward to test a small function. Unit tests, for example using the `pytest` framework, allow us to do just that. It is also not hard to imagine that unit tests can compound our trust into our code: In the previous sections, we developed a function `pdf` for computing the probability density function of the normal distribution and a function `integrate` for approximating definite integrals. Assume that we properly unit tested these two functions and shown for both of them that they produce the expected results for a wide range of different inputs. It would be easy to see that this then would also give us a certain confidence that using `integrate` to compute a definite integral over a certain input range of `pdf` should be correct, too. This could be verified with some additional unit tests. Working our way up from testing small components to larger and larger building blocks of an application gives us a good chance to have only few bugs in the final product.

These are a lot of advantages of using functions. But functions in `Python` also come with lots of nice features as well.

For example, we can specify default values for function parameters. This allows us to create functions that can be invoked using only few arguments in the normal case and that can be customized with more parameters in special cases. Since functions themselves are objects, they can be stored in variables and parameters as well. The proper type hint for such variables or parameters is defined using the `Callable` type. Functions that accept `Callable`s as parameter can also accept `lambda`s. `lambda`s are an abbreviated way to define functions, usually in a single line and only for a single use.

Chapter 9

Exceptions

So far, we have mainly focused on writing correct code. We try to create code that is free of errors.

When we execute our programs, then there are at least two things that can go wrong. On one hand, we can never really be sure that our program code is free of programming mistakes. The larger a project gets, the more likely it is that there are some bugs hidden somewhere in the code. Thorough unit testing can reduce the likelihood of bugs, but it cannot entirely prevent them.

On the other hand, our programs do not exist all by themselves in a vacuum. They receive input, maybe from files, maybe from the user, maybe from sensors. This input may be wrong.

Coarsely, we can group both kinds of problems together as situations that were not anticipated, that are exceptions from the intended program flow. We already have encountered such situations. For example, trying to access a character of a string at an index greater than or equal to the length of the string will lead to an `IndexError`, as we saw in Section 3.5.1. The attempt to modify an element of a `tuple` is punished with a `TypeError` in Section 5.2. Back in Section 3.3.5, we saw that trying to compute something like `(10 ** 400)* 1.0` will yield an `OverflowError`, as the integer 10^{400} is too large to be converted to a `float` during the multiplication with `1.0`.

Clearly, some of these errors may result from programming mistakes. But they could just as well result from invalid data being entered in the input of the program.

9.1 Introduction

When we create a new function or program, we have to face the question: *What should we do if we receive incorrect input?* We can imagine three different approaches:

1. We simply ignore the issue. If the input of our program or function is faulty, then the output will be wrong, too. This is often called Garbage In–Garbage Out (GIGO) [105]. Converting the integer `10 ** 400` to a `float` could just yield `inf`, for example.

2. We try to sanitize the input. For example, our `factorial` function in Listing 8.1 expects an integer as input. If someone were to pass in the floating point value `2.4` instead, we could round it to `2` and return the result. Matter of fact, our `sqrt` function implemented in Listing 8.10 returns `0.0` if we pass in a negative number.

3. We can guard the function by raising an `Exception` [48, 90, 124].

The latter is what `Python` does in the examples mentioned initially: While it could simply ignore if we try to overwrite an element of a `tuple`, it instead raises an `TypeError`, for example. Personally, I am also a fan of this approach.

If we would follow the GIGO paradigm, then faulty data will propagate. Maybe the output of our function is fed as input into another function, whose output is then piped into another function, and so on. An error could then will lead to some crash down the line. If functions that are written under the assumption that GIGO is OK are paired with such that perform input sanitization, errors could remain unnoticed. The erroneous results could then become part of some actual, real-life decisions and designs. And even if found out, it will be extremely hard to discover where things went wrong in the long chain of computations and function calls.

Input sanitization could cover some error that happened earlier. This could even cause more errors, because it will allow other programmers to call our code with wrong values. Input sanitization encourages sloppy programming.

This leaves raising an `Exception`. But what does that actually mean? Raising an `Exception` means two things:

1. We store information about the error and information about the current execution state (current line of code and the function call hierarchy) in an object (the `Exception`).
2. The control flow immediately leaves the currently executed block of instructions as well as all calling blocks or functions. It jumps up in the call hierarchy until reaching code that handles the raised `Exception` is reached. If no such code exists, the current process is terminated with an exit code different from `0`.

In other words, raising an `Exception` is a way to exit the current control flow and to signal an error that must either explicitly be handled by code or will lead the process to terminate. In my opinion, this is the best way to handle incorrect input or other erroneous situations for several reasons:

1. It clearly and explicitly shows that an error has happened, *where* it happened, *when* it happened, and, to some degree, *why* it happened. This makes it much easier to find out whether the error is caused by invalid input or by a programming mistake.
2. It prevents GIGO from occurring. Thus, it prevents a faulty situation or corrupted data from propagating out of the current context. If an error occurs, this the raised `Exception` takes down the current path of execution and this stops the contaminated control flow.
3. It forces programmers to explicitly deal with the error condition. An error cannot be simply ignored. Indeed, someone who calls our code might write code that ignores or discards the `Exceptions` that we raised. But they have to do so *explicitly* in their code. Thus, they have to *intentionally* deal with the possible error condition. It cannot happen that an error gets overlooked. An `Exception` that is not handled will lead to the termination of the process.
4. One might argue: "But what if the process crashes because an `Exception` is not handled?" The answer may be: What is worse? That an error causes the current process to crash unexpectedly or that all future results after the error are wrong and, even worse, are wrong *unnoticed*?

Best Practice 36 Errors should not be ignored and input should be sanitized. Instead, the input of our functions should be checked for validity (where reasonable). `Exceptions` should be raised as early as possible if any unexpected situation occurs.

9.2 Raising Exceptions

Let us now first look at an example of how we can signal an error condition in our code. For this purpose, we re-visit the square root function that `sqrt` we implemented back in Listing 8.10 in Section 8.3. In that implementation, we realized that certain input values such as `inf`, `-inf`, `nan`, and `0.0` deserve special treatment. We also found that nothing could stop a user to pass a negative number as input to our `sqrt` implementation. We did not yet have any means to deal with nonsense in a reasonable way, so we decided to just return `0.0` in that case.

Obviously, this is a bad idea because passing a negative number to `sqrt` can only mean two things: Either the programmer who did that does not know that a square root is. Or the negative input came as the result from another computation and that computation somehow was wrong. In the first case, we should somehow make it explicit to the programmer that the square root of a negative number is not defined and that they should question their approach to mathematics. In the second case, we should rather stop the computation right there and then before the incorrect results propagate and do some damage elsewhere. In both cases, returning `0.0` is not a good idea.

Instead, we want to signal this error explicitly. Our function should raise an `Exception`. We therefore create a new implementation of `sqrt` in Listing 9.1. Before writing the actual Python code, we will express the new behavior in the docstring of our function:

Listing 9.1: A new variant of the `sqrt` function from back in Listing 8.10 that raises an `ArithmeticError` if its input is non-finite or negative. (src)

```
"""A ‘sqrt‘ function raising an error if its result is not finite."""

from math import isfinite  # A function that checks for ‘inf‘ and ‘nan‘.


def sqrt(number: float) -> float:
    """
    Compute the square root of a given ‘number‘.

    :param number: The number to compute the square root of.
    :return: A value ‘v‘ such that ‘v * v == number‘.
    :raises ArithmeticError: if ‘number‘ is not finite or less than ‘0.0‘
    """
    if (not isfinite(number)) or (number < 0.0):  # raise error
        raise ArithmeticError(f"sqrt({number}) is not permitted.")
    if number <= 0.0:  # Fix for the special case ‘0‘:
        return 0.0  # We return 0; for now, we ignore negative values.
    if not isfinite(number):  # Fix for case ‘+inf‘ and ‘nan‘:
        return number  # We return ‘inf‘ for ‘inf‘ and ‘nan‘ for ‘nan‘.

    guess: float = 1.0  # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while old_guess != guess:  # Repeat until nothing changes anymore.
        old_guess = guess  # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess
```

Best Practice 37 Any function that may raise an exception should explain any exception that it explicitly raises in the docstring. This is done by writing something like `:raises ExceptionType: why` where `ExceptionType` is to be replaced with the type of the exception raised and `why` with a brief explanation why it will be raised.

In our case, any input x for which $\sqrt{x}$ would be either undefined or not finite should lead to an error. Since this error is of arithmetic nature, we will raise an `ArithmeticError`. Our docstring therefore contains the line `:raises ArithmeticError: if ‘number‘ is not finite or less than ‘0.0‘`. Any other programmer using our code therefore can easily see what kind of `Exceptions` our code could raise.

In the new `sqrt` implementation Listing 9.1 we first check whether either `not isfinite(number)` or `number < 0.0` holds. The `isfinite` function is offered by the `math` module and returns `True` if its argument is either `inf`, `-inf`, or `nan`. The complete expression becomes `True` if the input of the function is not finite or negative. We thus check whether the output of the `sqrt` function would not be a finite number. If this is indeed the case, we `raise ArithmeticError(f"sqrt({number})is not permitted.")`.

`raise` is the keyword used to, well, raise an `Exception`, i.e., to signal an error. `ArithmeticError` then creates the object with the information about the exception. We can pass a string as parameter to this function, and we here chose to pass in an f-string which contains the value of the number with which our `sqrt` function was called. This line of code will force the control flow to immediately exit our `sqrt` function. The `Exception` object "raises" up until it is either caught (which we discuss later) or the process itself is terminated.

The latter can be observed in Listing 9.2. In the program, we iteratively apply our new `sqrt` function to the values inside a `tuple` using a `for` loop. We write the results of these computations to the `stdout` using f-string and `print`.[1] The first five numbers are fine and `sqrt` returns the proper

[1] The `flush=True` parameter passed to `print` forces the output to be immediately written and not cached. This is only useful to keep the output in order in this example, because the `stdout` and `stderr` are captured and presented in Listing 9.3 and the order of the text could otherwise get mixed up.

Listing 9.2: Using the new variant of the `sqrt` function from Listing 9.1 that raises an `ArithmeticError` if its input is non-finite or negative. (stored in file `use_sqrt_raise.py`; output in Listing 9.3)

```
"""Using the sqrt function which raises errors."""

from math import inf, nan  # Import infinity and not-a-number from math.

from sqrt_raise import sqrt  # Import our sqrt function.

# Apply our protected square root function to several values.
for number in (0.0, 1.0, 2.0, 4.0, 10.0, inf, nan, -1.0):
    # We get an error when reaching `inf`.
    print(f"\u221A{number}\u2248{sqrt(number)}", flush=True)

print("The program is now finished.")  # We never get here.
```

↓ `python3 use_sqrt_raise.py` ↓

Listing 9.3: The stdout and stderr as well as the exit code of the program `use_sqrt_raise.py` given in Listing 9.2.

```
√0.0≈0.0
√1.0≈1.0
√2.0≈1.414213562373095
√4.0≈2.0
√10.0≈3.162277660168379
Traceback (most recent call last):
  File "{...}/06_exceptions/use_sqrt_raise.py", line 10, in <module>
    print(f"\u221A{number}\u2248{sqrt(number)}", flush=True)
                                 ^^^^^^^^^^^^
  File "{...}/06_exceptions/sqrt_raise.py", line 15, in sqrt
    raise ArithmeticError(f"sqrt({number}) is not permitted.")
ArithmeticError: sqrt(inf) is not permitted.
# 'python3 use_sqrt_raise.py' failed with exit code 1.
```

results. However, the latter three numbers, `inf`, `nan`, and `-1.0` all would cause an error.

We collect the output of the program in Listing 9.3. As can be seen, for `0.0`, `1.0`, `2.0`, `4.0`, and `10.0`, the results are printed as anticipated. However, when the `for` loop reaches `inf`, the program is terminated an the so-called stack trace is written to the output.

It begins with the line `Traceback (most recent call last):`. In the first line following this text, the source code file and the index of the line in that file where the `Exception` was originally raised are printed. Notice that the actual path to the files will be different depending on where the source code of the examples is located. We replaced the variable part of the path with "{...}". The remaining part of the path, `06_exceptions/use_sqrt_raise.py`, clearly points out the calling program as the culprit Listing 9.2. Indeed, the following line of text identifies that instruction in Listing 9.2 that caused the error and even marks the offending function invocation by underlining it with `^^^^^^^^^^^^`. Below that, we get to see the context of our `sqrt` function: First, the path to its module is given (ending in `06_exceptions/sqrt_raise.py`) and it is pointed out that the `Exception` was raised in line 15. This line of code is then also given, and it indeed is the one starting with `raise ArithmeticError`. The stack trace therefore shows us exactly where the error happened and from where the code causing the error was called.

After the stack trace, we can see information about the error printed that we passed in: `ArithmeticError: sqrt(inf)is not permitted.` We made this message by ourselves using an f-string when raising the `Exception`. We did this so that it tells the user that `sqrt` was called with the argument `inf` that we did not permit.

The above information allows us to pretty much identify the source of the problem. It shall be stated here that new programmers often ignore the stack trace. They see that a program produces and error and then try to figure out why by looking at their code. They often do not read the stack trace or the error information below it.

Best Practice 38 The stack trace and error information printed on the Python console in case of an uncaught Exception are essential information to identify the problem. They should *always* be read and understood before trying to improve the code.

In our original tuple of inputs that we iteratively passed to sqrt, the last three elements are inf, nan, and -1.0. The call to sqrt with inf as argument was performed and failed. After that, no further output has been generated. Indeed, the control flow has left the for loop and the process has been terminated with exit code 1, as the output in Listing 9.3 shows.

Terminating the process may seem rash, but it is not. If a programmer used our sqrt function incorrectly, then this will force them to fix their error. If the input inf was the result of corrupted data, another erroneous computation, or an input mistake by the user, then terminating the program prevented this error from propagating. For both scenarios, the stack trace and error output gives clear information about what went wrong and where.

9.3 Handling Exceptions

Of course, we do not want that all possible unexpected error conditions will immediately terminate our process. For example, maybe we programmed a program for painting a picture, the user painted a picture and wants to store it, but enters a wrong destination. It would be annoying if the program would immediately crash. In this section, we discuss how errors can be handled. We use examples that are specifically constructed to cause certain errors to illustrate how we can deal with them.

9.3.1 The try...except Block

The try-except clause exists as primary approach to recover from errors. We place the code that may raise an Exception into an indented block that is prefixed by try:. After this block, we write the handlers for specific exception types. For example, if we assume that a block could raise an ArithmeticError, we could write except ArithmeticError as ae:. The code inside this except block would be executed if and only if indeed an ArithmeticError was raised somewhere in the try block. In this case, the ArithmeticError would be available as local variable ae in this block. Of course, multiple different types of Exceptions may be raised, so we can have multiple except blocks. This looks like this:

```
try:  # Begin the try-except block.
    # Code that may raise an exception or that
    # calls a function that may rise one.
except ExceptionType1 as ex1:  # One exception type that can be caught.
    # Code the handles exceptions of type ExceptionType1.
except ExceptionType2 as ex2:  # Another exception type (optional).
    # Code the handles exceptions of type ExceptionType2 that are not
    # instances of ExceptionType1.

next statement  # Executed only if there are no uncaught Exceptions.
```

Let us now try the try-except block. Back in Section 3.5.1, we learned that text strings offer the method r.find(s) that searches a string s inside r and returns the first index where it is encountered. If s cannot be found in r, -1 is returned instead. There exists a very simply operation r.index(s), which works exactly like find, but instead of returning -1, it will raise a ValueError if s cannot be found in r. This is quite useful in cases where we know that s must be contained in r and if it is not, then that is an error.

In Listing 9.4, we explore this behavior. Our string r is "Hello World!". In the try-block, we place a for loop which lets a variable s iteratively take on three values. In its first iteration, s = "Hello" and r.index(s) will yield 0. This is printed to the output. In the second iteration, s = "world", which cannot be found since searching in strings is case-sensitive. r.index(s) will therefore raise an Exception.

The except block after the try block is executed if anywhere inside the try block a ValueError is raised. In this case, this ValueError becomes available in variable ve. In this block, we simply print the error.

Listing 9.4: The `index` function of a string raises a `ValueError` if it cannot find the given substring. Here we catch this error in a `try`...`except` block. (stored in file `try_except_str_index.py`; output in Listing 9.5)

```
"""Demonstrate `try...except` by looking for text in a string."""

r: str = "Hello World!"  # This is the string we search inside.

try:  # If this block raises an error, we continue at `except`.
    for s in ["Hello", "world", "!"]:  # The strings we try to find.
        print(f"{s!r} is at index {r.index(s)}.")
except ValueError as ve:  # ValueError is raised if `s` isn't in `text`.
    print(f"Error: {ve}")  # Error, as "world" is not in "Hello World!".

print("The program is now finished.")  # We get here after except block.
```

↓ `python3 try_except_str_index.py` ↓

Listing 9.5: The stdout of the program `try_except_str_index.py` given in Listing 9.4.

```
'Hello' is at index 0.
Error: substring not found
The program is now finished.
```

After the block, we print `"The program is now finished."`. This code is executed only if no uncaught `Exception` has left the `try`-`except` block. The output of the program given in Listing 9.5 shows that this is indeed the case: We first get the results of the successful search for `"Hello"`, followed by the output for the failed search. The last line then is `The program is now finished.`

Notice that the third value in the loop, `"!"`, never gets assigned to the variable `s`. The `try` block is immediately terminated as soon as any `Exception` occurs. If the `Exception` can be handled by a corresponding `except` block, then this block is executed. Otherwise, the whole process will be terminated. Either way, even if a fitting `except` block exists, the code after the failing instruction in the `try` block will not be executed.

While we can catch and handle `Exceptionss`, it is important to only handle reasonable errors. For example, suppose your program should be writing text to a file. It is totally acceptable that this may fail for a variety of file system related reasons, like insufficient space on the device, an access rights violation, or an incorrect file name. Such errors may be handled with a corresponding `except` block and reasonable actions may be taken. If, on the other hand, a `ZeroDivisionError` would occur during our attempt to write the file, then this indicates that something else went really wrong. Such an error is not OK in this context. So we should only try to catch errors that are meaningful and that we anticipate in a given context. Any other error should indeed cause our program to crash, to prevent more problems later. A crashed program is the clearest indicator to the user that something is wrong and that actions on their side are required, after all.

Best Practice 39 Only `Exceptions` should be caught by `except` blocks that we can meaningfully handle. The `except` block is not to be used to just catch any `Exception`, to implement GIGO, or to try to sanitize erroneous input.

In Listing 9.6, we revisit our new `sqrt` function. This function will `raise` an `ArithmeticError` if its argument is non-finite or negative. This time, we want to compute $\sqrt{\frac{1}{0}}$ and thus aim to store the result of `sqrt(1 / 0)` in a variable `sqrt_of_1_div_0`. We first declare the variable as a `float`. Then, in a `try`-`except` block, we perform the actual computation: `sqrt_of_1_div_0 = sqrt(1 / 0)`. Knowing that `sqrt` might raise an `ArithmeticError`, we provide a corresponding `except` block. However, we also know that `1 / 0` looks a bit dodgy, as we also try to intercept a potential `ZeroDivisionError` error. As you can see, we can have two independent `except` clauses.

So, which one will be executed? Certainly, $\frac{1}{0}$ is not finite, so `sqrt` will raise an `Exception`. Then again, $\frac{1}{0}$ cannot be computed at all, so maybe we get a `ZeroDivisionError` error instead? We find that the `except` block for `ZeroDivisionError` is executed. The reason is that in order to invoke

Listing 9.6: The handling of multiple errors, namely `ZeroDivisionError` and `ArithmeticError`, as well as what happens if a variable remains unassigned due to an error (a `NameError` is raised). (stored in file `try_multi_except.py`; output in Listing 9.7)

```python
"""Demonstrate `try...except` with multiple exceptions and NameError."""

from sqrt_raise import sqrt  # Import our sqrt function.

sqrt_of_1_div_0: float  # Declare this variable, but do not assign it.

try:  # If this block raises an error, we continue at `except`.
    # sqrt_of_1_div_0 only gets assigned if sqrt(1 / 0) succeeds...
    sqrt_of_1_div_0 = sqrt(1 / 0)  # Which error will this produce?
except ZeroDivisionError as de:  # A division by zero?
    print(f"We got a division-by-zero error: {de}.", flush=True)
except ArithmeticError as ae:  # Or an ArithmeticError?
    print(f"We got an arithmetic error: {ae}.", flush=True)

print("Now we try to print the value of sqrt_of_1_div_0.", flush=True)
print(sqrt_of_1_div_0)  # Does not work: sqrt_of_1_div_0 is not assigned
print("The program is now finished.")  # We never get here.
```

↓ `python3 try_multi_except.py` ↓

Listing 9.7: The `stdout` and `stderr` as well as the exit code of the program `try_multi_except.py` given in Listing 9.6.

```
We got a division-by-zero error: division by zero.
Now we try to print the value of sqrt_of_1_div_0.
Traceback (most recent call last):
  File "{...}/06_exceptions/try_multi_except.py", line 16, in <module>
    print(sqrt_of_1_div_0)  # Does not work: sqrt_of_1_div_0 is not
        ↪ assigned
          ^^^^^^^^^^^^^^^
NameError: name 'sqrt_of_1_div_0' is not defined
# 'python3 try_multi_except.py' failed with exit code 1.
```

`sqrt(1 / 0)`, the Python interpreter must first compute the result of `1 / 0`. This computation raises `ZeroDivisionError` and `sqrt` is never called.

This leads us to the question: If `sqrt` is never called, then what will be assigned to `sqrt_of_1_div_0`? The answer is: Nothing. In order to perform the assignment, we would need the result of `sqrt(1 / 0)`. But since this result never becomes available, the assignment is never performed.

This means that when we try to `print(sqrt_of_1_div_0)` after the `try`-`except` block, the value of `sqrt_of_1_div_0` is undefined. The variable `sqrt_of_1_div_0` never really received any value at all. Trying to access it will `raise` a `NameError`. While we did declare it, the Python interpreter does not know this variable yet, as it does not have a value. Our program will terminate, because the `NameError` is never caught anywhere. Instead, the stack trace and error information will be printed in Listing 9.7.

Best Practice 40 If an `Exception` is raised, be aware that the control flow will immediately leave the current block. The statement in which the `Exception` was raised will not be completed but aborted right away. Therefore, no variable assignments or other side-effects can take place anymore and it is possible that variables remain undefined.

9.3.2 Exceptions in Exception Handlers

What happens if an `Exception` is raised *inside* an `except` block? In Listing 9.8, we try to compute `sqrt_of_1_div_0 = sqrt(1 / 0)` twice. First in the `try` block, exactly as in Listing 9.6 and then again

Listing 9.8: An example of `Exceptions` being raised inside an `except` block. (stored in file `try_except_nested_1.py`; output in Listing 9.9)

```python
"""Demonstrate ‘try...except‘ with an exception raised in ‘except‘."""

from sqrt_raise import sqrt  # Import our sqrt function.

sqrt_of_1_div_0: float  # Declare this variable, but do not assign it.

try:  # If this block raises an error, we continue at ‘except‘.
    # sqrt_of_1_div_0 only gets assigned if sqrt(1 / 0) succeeds...
    sqrt_of_1_div_0 = sqrt(1 / 0)  # This produces a ZeroDivisionError.
except ZeroDivisionError as de:  # Catch an ZeroDivisionError.
    print(f"We got a division-by-zero error: {de}.", flush=True)
    sqrt_of_1_div_0 = sqrt(1 / 0)  # Let's try it again!

print("Now we try to print the value of sqrt_of_1_div_0.", flush=True)
print(sqrt_of_1_div_0)  # Does not work: sqrt_of_1_div_0 is not assigned
print("The program is now finished.")  # We never get here.
```

↓ `python3 try_except_nested_1.py` ↓

Listing 9.9: The `stdout` and `stderr` as well as the exit code of the program `try_except_nested_1.py` given in Listing 9.8.

```
We got a division-by-zero error: division by zero.
Traceback (most recent call last):
  File "{...}/06_exceptions/try_except_nested_1.py", line 9, in <module>
    sqrt_of_1_div_0 = sqrt(1 / 0)  # This produces a ZeroDivisionError.
                              ~~^~~
ZeroDivisionError: division by zero

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "{...}/06_exceptions/try_except_nested_1.py", line 12, in <module>
    sqrt_of_1_div_0 = sqrt(1 / 0)  # Let's try it again!
                              ~~^~~
ZeroDivisionError: division by zero
# 'python3 try_except_nested_1.py' failed with exit code 1.
```

in the `except` block that handles the `ZeroDivisionError` that this will cause. So inside an `except` block for handling `ZeroDivisionErrors`, another `ZeroDivisionError` will be raised.

The result is shown in Listing 9.9: The `except` block is terminated immediately and text is written to the `stderr` indicating why. While we handled the original `ZeroDivisionError`, another error occurred. The output first presents the stack trace of the `Exception` that we were handling. It then informs us that `During handling of the above exception, another exception occurred:`. Then it prints the stack trace of the new `Exception` that occurred inside the `except` block. Since there is no code for handling this error, our process terminates with exit code 1.

It is of course possible that an error that we normally can handle may cause another error. Therefore, like any other Python code blocks, we can also nest `try`-`except` blocks. Listing 9.10 shows exactly this. By placing the second `sqrt` invocation located in the `except` block into yet another `try`-`except` block, we can catch the `ZeroDivisionError` error.

Listing 9.10: An example of nested `try`-`except` blocks, as an improvement over Listing 9.8. (stored in file `try_except_nested_2.py`; output in Listing 9.11)

```
"""Demonstrate nested `try...except` blocks."""

from sqrt_raise import sqrt  # Import our sqrt function.

sqrt_of_1_div_0: float  # Declare this variable, but do not assign it.

try:  # If this block raises an error, we continue at `except`.
    # sqrt_of_1_div_0 only gets assigned if sqrt(1 / 0) succeeds...
    sqrt_of_1_div_0 = sqrt(1 / 0)  # This produces a ZeroDivisionError.
except ZeroDivisionError as de:  # Catch an ZeroDivisionError.
    print(f"We got a division-by-zero error: {de}.", flush=True)
    try:  # Nesting try-except blocks is totally fine.
        sqrt_of_1_div_0 = sqrt(1 / 0)  # Let's try it again!
    except ZeroDivisionError as de:  # Another ZeroDivisionError?
        print(f"Yet another division-by-zero error: {de}.", flush=True)

print("The program is now finished.")  # This time, we do get here.
```

↓ `python3 try_except_nested_2.py` ↓

Listing 9.11: The stdout of the program `try_except_nested_2.py` given in Listing 9.10.

```
We got a division-by-zero error: division by zero.
Yet another division-by-zero error: division by zero.
The program is now finished.
```

9.3.3 The `try...except...else` Block

What can we do if we need the result of a computation inside a `try` block but only can use it if the `try` block completely succeeds? One possible solution in such a situation is the `try`-`except`-`else` block. The difference to the `try`-`except` block is only that an `else` block follows, which is executed if and only if no `Exception` occurred.

```
try:  # Begin the try-except block.
    # Code that may raise an exception or that
    # calls a function that may rise one.
except ExceptionType1 as ex1:  # One exception type that can be caught.
    # Code the handles exceptions of type ExceptionType1.
# ... maybe more `except` blocks
else:
    # Code executed if and only if no Exception occurred.

next statement  # Executed only if there are no uncaught Exceptions.
```

In Listing 9.12, we present an example of the `try`-`except`-`else` block. The example is structured a bit similar to Listing 9.6. We again use our own `sqrt` function, this time attempting to compute $\sqrt{\frac{1}{0}}$ and then $\sqrt{3}$, each within its own block. Back in Listing 9.6, we got a `NameError` because we wanted to access the value of a variable that we assigned in the `try` block. The `try` block failed. While we did catch and process the `ZeroDivisionError` in an `except` block, the variable was never assigned. So accessing the variable later cause the error.

This time, we place the code for accessing the variables into `else` blocks. These blocks are *only* executed if the `try` block succeeds. Hence, the variables are guaranteed to be exist and have values properly assigned to them if the `else` blocks are reached. In case of $\sqrt{\frac{1}{0}}$, the `else` block is not reached. In case of $\sqrt{3}$, it is and its corresponding `print` instruction is executed. Listing 9.12 shows that this program can properly finish.

Listing 9.12: An example of the `try`-`except`-`else` block which is structured a bit similar to Listing 9.6 but avoids the `NameError` by placing the access to the variables into the `else` blocks. (stored in file `try_except_else.py`; output in Listing 9.13)

```
"""Demonstrate 'try...except..else'."""

from sqrt_raise import sqrt  # Import our sqrt function.

sqrt_of_1_div_0: float  # Declare this variable, but do not assign it.

try:  # If this block raises DivisionByZero, we continue at 'except'.
    sqrt_of_1_div_0 = sqrt(1 / 0)  # This is a DivisionByZero!
except ZeroDivisionError as de:  # We catch and print the ZeroDivisionError
    ↪ .
    print(f"We got a division-by-zero error: {de}.", flush=True)
else:  # This code is not executed because an exception occurred.
    print(f"\u221A(1/0)\u2248{sqrt_of_1_div_0}")  # Never reached.

sqrt_3: float  # Declare this variable, but do not assign it.
try:  # If this block raises ArithmeticError, we continue at 'except'.
    sqrt_3 = sqrt(3.0)  # This will work just fine and raise no error.
except ArithmeticError as ae:  # Or an ArithmeticError?
    print(f"We got an arithmetic error: {ae}.", flush=True)  # Not done.
else:  # This code is executed because no exception occurs.
    print(f"\u221A3\u2248{sqrt_3}")  # Always executed.

print("The program is now finished.")  # We do get here.
```

↓ `python3 try_except_else.py` ↓

Listing 9.13: The stdout of the program `try_except_else.py` given in Listing 9.12.

```
We got a division-by-zero error: division by zero.
√3≈1.7320508075688772
The program is now finished.
```

9.3.4 The `try...finally` Block

We do know that errors may occur in a piece of code. These may be errors that we can reasonably expect to potentially happen. Such errors we will process with corresponding `except` blocks. Then, there might be unanticipated errors for which we cannot define a reasonable `except` block. In the latter case, the program should terminate and the stack trace should be printed.

However, there may be situations where we do not just want to immediately quit be perform some necessary actions beforehand. A typical example of this is if we are currently writing contents to a file. Let's say we are writing a table of data row by row into a text file, and suddenly some unexpected error occurs. If we terminate immediately without closing the file, then the contents of the complete file could be lost. If we close the file before terminating, then at least the data that was successfully written so far will be preserved. By terminating the program, we would still indicate to the user that there is some serious problem that needs attention. But at least we would not destroy the data that was correctly produced.

For this purpose, the `try`-`finally` block exists. Basically, we can add a `finally` block that contains the code that should always be executed. While we definitely need a `try` block that contains that code that may cause an error, we can optionally add `except` blocks to handle certain `Exceptions` and an `else` block to be executed if no error occurs.

Listing 9.16: The results of static type checking with Mypy of the program given in Listing 9.14.

```
$ mypy try_except_else_finally.py --no-strict-optional --check-untyped-defs
try_except_else_finally.py:26: error: Argument 1 to "divide_and_print" has
    ↪ incompatible type "str"; expected "int | float"  [arg-type]
Found 1 error in 1 file (checked 1 source file)
# mypy 1.13.0 failed with exit code 1.
```

```
try:  # Begin the try-except block.
    # Code that may raise an exception or that
    # calls a function that may rise one.
except ExceptionType1 as ex1:  # The **optional** except blocks.
    # Code the handles exceptions of type ExceptionType1.
# ... maybe more `except` blocks
else:  # The **optional** else block.
    # Code executed if and only if no Exception occurred.
finally:  # The **optional** finally block.
    # Code will always be executed, even if exceptions are uncaught.

next statement  # Executed only if there are no uncaught Exceptions.
```

Let us explore this in another artificial example in Listing 9.14. Here, we create the function `divide_and_print` that accepts two parameters `a` and `b` which can either be integers or floating point numbers. In a `try` block, the function attempts to divide `a` by `b` and `print` the result using an f-string. Since we do not know exactly what values `b` can take on beforehand, we anticipate that a `ZeroDivisionError` may occur. In the corresponding `except` block, we would then print a message that explains the situation. In this case, the division result would not be printed because the `try` block would terminate when the f-string is interpolated.

We also attempt to catch a possible `TypeError`. Such an error would occur if the function is invoked by an argument that is neither an `int` nor a `float` and that does not support divison. This is a typical example for an error that we should *not* attempt to catch. This error could only appear if another programmer was using our function incorrectly. We only process this error here for the sake of the example and print an appropriate message in the `except` block.

In the example, we also have an `else` block which notifies us that no error occurred. The code in this block is only executed if no `ZeroDivisionError` and no `TypeError` and also no other `Exception` was raised.

We finish the division and error handling part of the function with a `finally` block. This block will be executed if no `Exception` was raised anywhere, but also if a `ZeroDivisionError`, `TypeError`, or other `Exception` were raised. Even if an error was raised *inside* one of the `except` blocks, this code would be executed. It will print that the division code was completed.

Then, in the last line of the function after the whole `try`-`except`-`else`-`finally` blocks, we print yet another message. This code outside the blocks is reached only if either no error occurred at all or if the error was handled by one of the two `except` blocks (without yet another error).

In Listing 9.15, we show the `stdout` and `stderr` for invoking this function with several different arguments. For `divide_and_print(10, 5)`, both the division result in the `try` block as well as the messages from the `else` block, the block, and from the very end of our function are printed. `divide_and_print(3, 0)` will cause a `ZeroDivisionError`. Therefore, the `print` instruction in the `try` block is not completed as interpolating the f-string already fails. The first `except` block, which handles the `ZeroDivisionError`, is executed and prints its message. The `else` block is not reached but the `finally` block prints its message. Since the error was properly handled, the message in the `print` instruction at the end of the function is written to the output as well.

Invoking `divide_and_print("3", 0)` means that we intentionally ignore the type hints in the function definition. The Python interpreter allows us to do this without complaint, as type hints are only hints and strict requirements (as in other programming languages like Java or C). However, the division `a / b` will fail since the string `"3"` does not support a division operation. It raises a `TypeError`, which is subsequently caught by our second `except` block. This means that the `else` block is again

not reached. The `finally` block is executed still and so is the `print` at the bottom of our function, since the `TypeError` was properly handled. By the way, had we applied the static type checker `Mypy` to Listing 9.14, it would have informed us that we here try to call `divide_and_print` with an invalid argument, as shown in Listing 9.16.

Finally, we attempt to compute `divide_and_print(10 ** 313, 1.0)`, i.e., to calculate $\frac{10^{313}}{1}$. At first glance, this looks totally fine. However, back in Section 3.3.5, we learned about the limits of the datatype `float`. Indeed, the `int` 10^{313} is outside the range of numbers that a `float` can represent. By trying to divide it by `1.0`, we force it to be converted to `float` first. This will raise an `OverflowError`. We do not have an `except` block for handling `OverflowErrors`. This means that, of course, no message is printed by the `try` block, none of our `except` blocks are reached, and the `else` block is not executed either. The `finally` block, however, *is* executed and prints its message to the output. Since we did not catch the `OverflowError`, the code after our blocks at the bottom of our function is not executed. Instead, our function is terminated immediately after the `finally` block completes. Since there is no `try`-`except` block able to catch `OverflowErrors` wrapped around the function call, the whole `Python` interpreter terminates as well. It again prints the stack trace, which informs us which error occured and where it happened.

Listing 9.14: An example for the `try`-`except`-`else`-`finally` block. (stored in file `try_except_else_finally.py`; output in Listing 9.15)

```python
"""Demonstrate the try-except-else-finally clause."""


def divide_and_print(a: int | float, b: int | float) -> None:
    """
    Divide the numbers ‘a‘ and ‘b‘ and print the result.

    :param a: the dividend
    :param b: the divisor
    """
    try:  # We try to do some computation that may cause an Exception.
        print(f"{a} / {b} = {a / b}", flush=True)  # Divide and print.
    except ZeroDivisionError as zd:  # Is b == 0 ?
        print(f"We got a ZeroDivisionError when doing {a} / {b}: {zd}.")
    except TypeError as te:  # Has one of the values the wrong type?
        print(f"We got a TypeError when computing {a} / {b}: {te}.")
    else:  # Only called when no Exception occurred.
        print(f"No error occurred when computing {a} / {b}.")
    finally:  # This code is always called.
        print(f"We are finally done with division-by-{b}.", flush=True)
    print(f"This code comes after all the {a} by {b} division code.")


divide_and_print(10, 5)            # This works just fine.
divide_and_print(3, 0)             # This yields a ZeroDivisionError.
divide_and_print("3", 0)           # This yields a TypeError.
divide_and_print(10 ** 313, 1.0)   # Causes an uncaught OverflowError!
```

↓ `python3 try_except_else_finally.py` ↓

Listing 9.15: The stdout and stderr as well as the exit code of the program `try_except_else_finally.py` given in Listing 9.14.

```
10 / 5 = 2.0
No error occurred when computing 10 / 5.
We are finally done with division-by-5.
This code comes after all the 10 by 5 division code.
We got a ZeroDivisionError when doing 3 / 0: division by zero.
We are finally done with division-by-0.
This code comes after all the 3 by 0 division code.
We got a TypeError when computing 3 / 0: unsupported operand type(s) for /: 'str' and 'int'.
We are finally done with division-by-0.
This code comes after all the 3 by 0 division code.
We are finally done with division-by-1.0.
Traceback (most recent call last):
  File "{...}/06_exceptions/try_except_else_finally.py", line 27, in <module>
    divide_and_print(10 ** 313, 1.0)  # Causes an uncaught OverflowError!
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "{...}/06_exceptions/try_except_else_finally.py", line 12, in divide_and_print
    print(f"{a} / {b} = {a / b}", flush=True)  # Divide and print.
                         ~~^~~
OverflowError: int too large to convert to float
# 'python3 try_except_else_finally.py' failed with exit code 1.
```

9.4 Testing Exceptions

Back in Section 8.3, we introduced the concept of unit tests and show how the tool `pytest` can be used to test our functions. We stated in Best Practice 34 that we should cover all the branches of the control flow inside a function with unit tests. One kind of branch that is often overlooked are `Exceptions` and extension handling [85].

If our function raises a certain `Exception` in some case, then we should have a unit test that checks if this `Exceptions` is actually raised as it should. Now, any `Exception` raised by a unit test will cause the test to fail. This seems to contradict our goal to intentionally raise the exceptions. Luckily, `pytest` offers us a device for this.

Let us expand our Listing 8.11 for testing the `sqrt` function to now also test for `ArithmeticErrors`. If we want to check whether a function indeed raises a certain `Exception` of type `ExpectedExceptionType` for a given input, then we can wrap the corresponding function call into a `with raises(ExpectedExceptionType):`. We will learn the exact structure and functioning of `with` blocks later. They are closely related to `Exception` handling, but require knowledge of some other topics that we did not yet cover. For now, let us just ignore how the `with` statement works and just focus on how it is used for testing.

The `pytest` module offers a so-called context manager [28] named `raises`. This is basically a function that can be invoked in the `with` statement and that tells the `pytest` system that, in the following, indented, block, we expect that a certain `Exception` must be raised. If such an `Exception` is not raised, the test fails. If it is raised, the text succeeds.

In particular, we can provide the `Exception` class as well as a regular expression and [76, 92, 94, 112] (regex) via the parameter `match` to `raises`. If the block does not raise an `Exception` of the expected type, the test fails. If it does raise a corresponding `Exception`, then it will be converted to a string. The string is then matched against the regex and if it indeed matches, the test succeeds. If it does not match, then the test fails.

But what is a regex? We will learn this later as well, in a part of the book that is not yet written. For now, let's say that a regex is a string pattern that can be compared with another string. For example, the `"sqrt.* is not permitted."` that we use in Listing 9.17 is such a regex. All the "normal" characters in this string must match exactly gainst characters in the text it is compared against. The only thing that makes it different from a normal string is the `.*` that it contains. The dot (`.`) can be replaced with any other character and the star (`*`) says that arbitrarily many such arbitrary characters are allowed. `"sqrt.* is not permitted."` would thus match against `"sqrt is not permitted."`, `"sqrt(1)is not permitted."`, `"sqrt(inf)is not permitted."`, but not `"sqrt is wrong."`. Therefore, our produced error message must follow this pattern for the test to work. And, as you can find in the `pytest` output in Listing 9.18, for the values `v in [-1.0, inf, -inf, nan]`, no error is reported.

Listing 9.17: A unit test checking that our new variant of the `sqrt` function given in Listing 9.1 properly raises an `ArithmeticError` if its input is non-finite or negative. (src)

```
"""Testing our sqrt function that raises an error for invalid inputs."""

from math import inf, nan  # some maths constants

from pytest import raises  # Needed checking that exceptions are raised.

from sqrt_raise import sqrt  # Import our new sqrt function.


def test_sqrt() -> None:
    """Test the new ‘sqrt‘ function that raises errors."""
    assert sqrt(0.0) == 0.0  # The square root of 0 is 0.
    # ... We skip the other tests here for space reasons.

    for v in [-1.0, inf, -inf, nan]:
        with raises(ArithmeticError, match="sqrt.* is not permitted."):
            sqrt(v)  # The square root of v is not permitted
```

Listing 9.18: The output of the unit tests in Listing 9.17: The `ArithmeticErrors` are correctly raised.

```
$ pytest --timeout=10 --no-header --tb=short test_sqrt_raise.py
========================= test session starts =========================
collected 1 item

test_sqrt_raise.py .                                                 [100%]

========================== 1 passed in 0.01s ==========================
# pytest 8.3.3 with pytest-timeout 2.3.1 succeeded with exit code 0.
```

Listing 9.19: Two unit tests checking the original variant of the `sqrt` function from back in Listing 8.11 that does not raise errors. (src)

```python
"""Testing the square root implementation that does not raise errors."""

from pytest import raises


def sqrt(number: float) -> float:
    """
    Compute the square root of a `number`, but do not raise errors.

    :param number: The number to compute the square root of.
    :return: A value `v` such that `v * v == number`.
    """
    if number <= 0.0:  # Fix for the special case `0`:
        return 0.0  # We return 0; for now, we ignore negative values.
    guess: float = 1.0  # This will hold the current guess.
    old_guess: float = 0.0  # 0.0 is just a dummy value != guess.
    while old_guess != guess:  # Repeat until nothing changes anymore.
        old_guess = guess  # The current guess becomes the old guess.
        guess = 0.5 * (guess + number / guess)  # The new guess.
    return guess


def test_sqrt_1() -> None:
    """Test the `sqrt` variant that does not itself raise Exceptions."""
    with raises(ArithmeticError):  # We can also test without `match`.
        sqrt(-1.0)  # This is not permitted, but no Exception is raised.


def test_sqrt_2() -> None:
    """Test the `sqrt` variant that does not itself raise Exceptions."""
    with raises(ArithmeticError, match="sqrt.* is not permitted."):
        sqrt(10 ** 320)  # OverflowErrors are ArithmeticErrors.
```

Listing 9.20: The output of the unit tests in Listing 9.19: No error was raised in it, so the first fails and the error message in the second test does not fit, so it fails as well.

```
$ pytest --timeout=10 --no-header --tb=short test_sqrt.py
========================= test session starts =========================
collected 2 items

test_sqrt.py FF                                                    [100%]

=============================== FAILURES ================================
_____________________________ test_sqrt_1 ______________________________
test_sqrt.py:25: in test_sqrt_1
    with raises(ArithmeticError):  # We can also test without `match`.
E   Failed: DID NOT RAISE <class 'ArithmeticError'>
_____________________________ test_sqrt_2 ______________________________
test_sqrt.py:32: in test_sqrt_2
    sqrt(10 ** 320)  # OverflowErrors are ArithmeticErrors.
test_sqrt.py:19: in sqrt
    guess = 0.5 * (guess + number / guess)  # The new guess.
E   OverflowError: int too large to convert to float

During handling of the above exception, another exception occurred:
test_sqrt.py:31: in test_sqrt_2
    with raises(ArithmeticError, match="sqrt.* is not permitted."):
E   AssertionError: Regex pattern did not match.
E    Regex: 'sqrt.* is not permitted.'
E    Input: 'int too large to convert to float'
======================== short test summary info ========================
FAILED test_sqrt.py::test_sqrt_1 - Failed: DID NOT RAISE <class 'Arith...
FAILED test_sqrt.py::test_sqrt_2 - AssertionError: Regex pattern did n...
========================== 2 failed in 0.02s ===========================
# pytest 8.3.3 with pytest-timeout 2.3.1 failed with exit code 1.
```

For these values, our `sqrt` function does indeed raise the expected `Exceptions`, with the expected error messages. The unit test succeeds.

In Listing 9.19, we write a similar test. This time, we use basically the original version of our `sqrt` function which does not by itself raise any `Exception`. This function is directly implemented in the listing, in the same way as we did in Listing 8.10, and again called `sqrt`. We also write two unit tests into the new file `test_sqrt.py`. Both are intentionally designed to show what happens if either an expected `Exception` is not raised (`test_sqrt_1`) or if its error message does not match the `match` argument (`test_sqrt_2`).

`test_sqrt_1` simply feeds the value `-1.0` to `sqrt`. It expects that an `ArithmeticError` should be raised. It does not provide a specific `match` regex against which the error message should be compared. The output in Listing 9.20 shows us that this test fails. If the code inside the `with raises(ArithmeticError):` does not actually raise an `ArithmeticError`, the test fails.

The second test case, `test_sqrt_2`, is a bit more interesting. It specifies `with raises(ArithmeticError, match="sqrt.* is not permitted.")`, just like the tests in Listing 9.17 for the `sqrt` function that does raise `ArithmeticErrors`. No we do know that our original `sqrt` from Listing 8.10 does not raise such errors ... by itself. However, we try to compute `sqrt(10 ** 320)`. This very large integer values will eventually be converted to a `float` inside our function. This will fail, because it is too large, as we just discussed back in Section 9.3.4. This failure results in an `OverflowError`. `OverflowErrors` are a special case of `ArithmeticError`. So indeed, this does fulfill our testing requirement: If an `OverflowErrors` is also a `ArithmeticError`, so the right type of `Exception` was raised. However, its error message will not fit to our `match` argument. Therefore, the unit test still fails. The output of Listing 9.20 explains this clearly.

With `pytest`, we can now therefore:

1. Test whether a function computes the expected output for selected (correct) inputs. If it does not, the test fails.

2. Test whether a function does not raise an unexpected `Exception` for the selected (correct) inputs. If it does raise one, the test fails.

3. Raises the expected `Exceptions` for selected (wrong) inputs with the correct error messages. If it does not, the test fails. We do this by using the `with raises(...)` statement.

This allows us to cover both the expected and correct use of our function with tests as well as unexpected incorrect uses. We can then be confident that our code is unlikely to cause harm, neither due to bugs created by ourselves nor due to accidental misuse by other programmers due to misunderstandings (which would immediately signaled to them by `Exceptions`). Notice that good unit testing goes hand in hand with good documentation in docstrings, because good docstrings reduce the chance of such misunderstandings.

Best Practice 41 It is important to cover both the reasonable expected use of our functions as well as unexpected use with incorrect arguments with test cases. The latter case should raise `Exceptions`, which we should verify with unit tests.

9.5 Built-in Exceptions

[19]

Backmatter

Best Practices

When writing programs in any programming language, there are always many best practices that should be followed. Best practices are methods and styles that have crystallized over decades of programming experience. They help to avoid mistakes, increase the code maintainability, and improve performance.

Best Practice 1: Always be careful with which division operator you use for `ints`. If you need an integer result, make sure to use `//`. Remember that `/` always returns a `float` (and see Best Practice 2), even if the result is a whole number.

Best Practice 2: Always assume that any `float` value is imprecise. Never expect it to be exact [8, 107].

Best Practice 3: If you need to specify large integers or floats, using underscores (`_`) to separate groups of digits can be very helpful [15]. For example, `37_859_378` is much easier to read than `37859378`.

Best Practice 4: When defining a string literal, the double-quotation mark variant (`"..."`) may be preferred over the single-quotation mark variant (`'...'`); see also Best Practice 5.

Best Practice 5: When defining a multi-line string literal, the double-quotation mark variant (`"""..."""`) is usually preferred over the single-quotation mark variant (`'''...'''`) [49, 143].

Best Practice 6: Comments help to explain what the code in programs does and are a very important part of the *documentation* of code. Comments begin with a `#` character, after which all text is ignored by the `Python` interpreter until the end of the current line. Comments can either occupy a complete line or we insert two spaces after the last code character in the line and then start the comment [143].

Best Practice 7: Variable names should be lowercase, with words separated by underscores [143].

Best Practice 8: Swapping of variable values can best be done with a multi-assignment statement, e.g., `a, b = b, a`.

Best Practice 9: The names we use in program code should clearly reflect our intentions.

Best Practice 10: Every program should pass static type checking with tools such as `Mypy` (see Useful Tool 1). Any issue found by the tools should be fixed. In other words, type check the program. If there is an error, fix the error and *type check it again*. Repeat this until no errors are found anymore.

Best Practice 11: Always use type hints.

Best Practice 12: Each `Python` file should start with a string describing its purpose [49]. This can either be a single line, like a headline, or a longer text. In the second case, the first line must be a headline, followed by an empty line, followed by the rest of the text. Either way, it must be a string delimited by `"""..."""` [49, 143].

Best Practice 13: Use many static code analysis tools and use them always. They can discover a wide variety of issues, problems, or potential improvements. They can help you to keep your code clean and to enforce a good programming style. Do not just apply them, but also *implement* their suggestions where possible.

Best Practice 14: When you need to use an indexable sequence of objects, use a list if you intent to modify this sequence. If you do not intent to change the sequence, use a tuple.

Best Practice 15: Only put immutable objects into tuples. Mutable objects inside tuples makes the tuples modifiable as well, while other programmers may assume that they are immutable. This can lead to strange errors down the line.

Best Practice 16: Only put immutable objects into sets.

Best Practice 17: Sets are unordered. Never expect anything about how the objects you put into a set are actually stored there.

Best Practice 18: Careful when sorting or comparing strings: The default order is *uppercase characters before lowercase characters*, i.e., `"A" < "a"`. If you want that upper- and lowercase characters are treated the same (e.g., that `"A"` is considered as equal to `"a"`), as is the case in dictionary ordering, i.e., if you want to sort a collection `my_text` of strings in a case-insensitive manner, use `sorted(my_text, key=str.casefold)`.

Best Practice 19: Dictionary keys must be immutable.

Best Practice 20: Blocks are indented by four spaces.

Best Practice 21: Prefer `elif` over nested `else ... if` constructs.

Best Practice 22: When your `if...then...else` statement only assigns values to variables, use the inline variant discussed in Section 6.4, as it is more compact.

Best Practice 23: If we do not care about the value of a variable (or parameter), we should name it `_` [74]. This information is useful for other programmers as well as static code analysis tools.

Best Practice 24: Do not use (un)equality comparisons of `floats` as loop termination criteria, as they may lead to endless loops. There can always be inputs that cause endless oscillations between values or the appearance of `nan` values (see Section 3.3.5).

Best Practice 25: Function names should be lower case, with underscores separating multiple words if need be [143].

Best Practice 26: All parameters and the return value of a function should be annotated with type hints. From my perspective: *A function without type hints is wrong.*

Best Practice 27: The body of a function is indented with four spaces.

Best Practice 28: Each function should be documented with a docstring. If you work in a team or intend to place your code in public repositories like on GitHub, then this very very much increases the chance that your code will be used correctly. From my perspective: *A function without docstring is wrong.*

Best Practice 29: After the function and its body are defined, leave *two* blank lines before writing the next code [143].

Best Practice 30: Package and module names should be short and lowercase. Underscores can be used to improve readability. [143]

Best Practice 31: Always attach a timeout to your unit tests. This timeout can be generous, maybe one hour, but it will serve as sentinel against either endless loops, deadlocks, or other congestion situations which all would be practical test failures. Timeouts protect automated builds or continuous integration systems from clogging.

Best Practice 32: A function which is not unit tested is *wrong*.

Best Practice 33: A good unit test for a given function should cover both expected as well as extreme cases. For a parameter, we should test both the smallest and largest possible argument values, as well as values from its normally expected range.

Best Practice 34: A good unit test for a function should cover all branches of the control flow inside the function. If a function does one thing in one situation and another thing in another situation, then both of these scenarios should have associated unit tests.

Best Practice 35: Default parameter values must always be immutable.

Best Practice 36: Errors should not be ignored and input should be sanitized. Instead, the input of our functions should be checked for validity (where reasonable). `Exceptions` should be raised as early as possible if any unexpected situation occurs.

Best Practice 37: Any function that may raise an exception should explain any exception that it explicitly raises in the docstring. This is done by writing something like `:raises ExceptionType: why` where `ExceptionType` is to be replaced with the type of the exception raised and `why` with a brief explanation why it will be raised.

Best Practice 38: The stack trace and error information printed on the `Python` console in case of an uncaught `Exception` are essential information to identify the problem. They should *always* be read and understood before trying to improve the code.

Best Practice 39: Only `Exceptions` should be caught by `except` blocks that we can meaningfully handle. The `except` block is not to be used to just catch any `Exception`, to implement GIGO, or to try to sanitize erroneous input.

Best Practice 40: If an `Exception` is raised, be aware that the control flow will immediately leave the current block. The statement in which the `Exception` was raised will not be completed but aborted right away. Therefore, no variable assignments or other side-effects can take place anymore and it is possible that variables remain undefined.

Best Practice 41: It is important to cover both the reasonable expected use of our functions as well as unexpected use with incorrect arguments with test cases. The latter case should raise `Exceptions`, which we should verify with unit tests.

Useful Tools

When developing software with Python, we can rely on a wide variety of tools that can help us to achieve high code quality and maintainability. We introduce such tools and show how they can be used throughout this book at different locations.

Useful Tool 1: Mypy [83] is a static type checking tool for Python. This tool can warn you if you, e.g., assign values to a variable that have a different type than the values previously stored in the variable, which often indicates a potential programming error. It can be installed via `pip install mypy`, as illustrated in Figure 4.5 on page 62. You can then apply Mypy using the command `mypy fileToScan.py`. We use the Bash script given in Listing 10.1 on page 175 to apply Mypy to the example programs in this book.

Useful Tool 2: Ruff is a very fast Python linter that checks the code for all kinds of problems, ranging from formatting and style issues over missing documentation to performance problems and potential errors [88]. It can be installed via `pip install ruff` as shown in Figure 5.1 on page 74. You can then apply Ruff using the command `ruff check fileToScan.py`. We provide a script for using Ruff with a reasonable default configuration in Listing 10.2 on page 176.

Useful Tool 3: Pylint is a Python linter that analyzes code for style, potential errors, and possible improvements [110]. It can be installed via `pip install pylint` as shown in Figure 7.1 on page 106. You can then apply Pylint using the command `pylint fileToScan.py`. We provide a script for using Pylint with a reasonable default configuration in Listing 10.3 on page 177.

Useful Tool 4: pytest is a Python is a Python framework for writing and executing software tests. It can be installed via `pip install pytest pytest-timeout` as shown in Figure 8.1 on page 119. You can then apply pytest using the command `pytest --timeout=toInS file(s)`, where `toInS` should be replaced with a reasonable timeout in seconds and `file(s)` is one or multiple files with test cases. We provide a script for using pytest with a reasonable default configuration in Listing 10.4 on page 178.

Glossary

Bash is a the shell used under `Ubuntu Linux`, i.e., the program that "runs" in the terminal and interprets your commands, allowing you to start and interact with other programs [16, 93, 149]. Learn more at `https://www.gnu.org/software/bash/`.

bias The bias is subtracted from the value stored in the exponent field of a floating point number. This allows for representing both positive and negative exponents. In the 64 bit double precision IEEE Standard 754 floating point number layout [56, 60], the bias is 1023. See Section 3.3.1.

C is another programming language, which is very successful in system programming situations [37, 108].

docstring Docstrings are special string constants in `Python` that contain documentation for modules or functions [49]. They must be delimited by `"""..."""` [49, 143].

exit code When a process terminates, it can return a single integer value (the exit status code) to indicate success or failure [68]. Per convention, an exit code of 0 means success. Any non-zero exit code indicates an error. Under `Python`, you can terminate the current process at any time by calling `exit` and optionally passing in the exit code that should be returned. If `exit` is not explicitly called, then the interpreter will return an exit code of 0 once the process normally terminates.If the process was terminated by an uncaught `Exception`, a non-zero exit code, usually 1, is returned.

exponent The exponent is the part of a floating point number that stores a power of 2 with which the significand is multiplied. This allows for covering a wide range of different precisions and representing both very large and very small numbers. In the 64 bit double precision IEEE Standard 754 floating point number layout [56, 60], the exponent is 11 bits with a bias of 1023. See Section 3.3.1.

f-string is a special string in `Python`, which delimited by `f"..."` which can contain expressions in curly braces like `f"a{6-1}b"` that are then turned to text via (string) interpolation, which turns the string to `"a5b"`. f-strings are discussed in Section 3.5.2.

Git Git is a distributed Version Control Systems (VCS) which allows multiple users to work on the same code while preserving the history of the code changes [127, 136].

GitHub GitHub is a website where software projects can be hosted and managed via the Git Version Control System [104, 136]. Learn more at `https://github.com`.

Integrated Development Environment An Integrated Development Environment (IDE) is a program that allows the user do multiple different activities required for software development in one single system. It often offers functionality such as editing source code, debugging, testing, or interaction with a distributed version control system. For this course, we recommend using `PyCharm`.

Java is another very success programming language, with roots in the `C` family of languages [86].

linter A linter is a tool for analyzing static program code to identify bugs, problems, vulnerabilities, and inconsistent code styles [65, 117]. `Ruff` is an example for a linter used in the `Python` world.

Linux is the leading open source operating system, i.e., a free alternative for Windows [7, 55, 135]. Its variant Ubuntu is particularly easy to use and install. We recommend using it for this course, for software development, and for research. Learn more at https://www.linux.org/.

mantissa See significand.

Matplotlib is a Python package for plotting diagrams and charts [59, 64, 100]. Learn more at at https://matplotlib.org.

modulo division is, in Python, done by the operator `%` that computes the remainder of a division. `15 % 6` gives us `3`. Modulo division is mentioned in Sections 3.2 and 3.3.2.

moptipy is the *Metaheuristic Optimization in Python* library [145]. Learn more at https://thomasweise.github.io/moptipy.

Mypy is a static type checking tool for Python [83] that makes use of type hints. Learn more at https://github.com/python/mypy or in Section 4.3 and Useful Tool 1.

NumPy is a fundamental package for scientific computing with Python, which offers efficient array datastructures [34, 54, 64]. Learn more at https://numpy.org/.

Pandas is a Python data analysis and manipulation library [9, 84]. Learn more at https://pandas.pydata.org.

pip is the standard tool to install Python software packages from the PyPI repository [63]. To install a package `thepackage` hosted on PyPI, type `pip install thepackage` into the terminal. Learn more at https://packaging.python.org/installing.

PyCharm is the convenient Python Integrated Development Environment that we recommend for this course [140, 148]. It comes in a free community edition, so it can be downloaded and used at no cost. Learn more at https://www.jetbrains.com/pycharm.

Pylint is a linter for Python that checks for errors, enforces coding standards, and that can make suggestions for improvements [110]. Learn more at https://www.pylint.org, Section 7.5, and in Useful Tool 3.

PyPI The Python Package Index (PyPI) is an online repository that provides the software packages that you can install with pip [13, 132, 139]. Learn more at https://pypi.org.

pytest is a framework for writing and executing unit tests in Python [35, 99, 101]. Learn more at https://pytest.org and in Section 8.3.

Python The Python programming language [58, 82], i.e., what you will learn about in this book. Learn more at https://python.org.

PyTorch is a Python library for deep learning and AI [102, 111]. Learn more at https://pytorch.org.

regular expression Regular expressions, often called "regex" for short, are sequences of characters that define a search pattern for text strings [76, 92, 94, 112]. In Python, the `re` module offers functionality work with regular expressions [76, 112].

Ruff is a linter and code formatting tool for Python [88]. Learn more at https://docs.astral.sh/ruff or in Section 5.1 and Useful Tool 2.

Scikit-learn is a Python library offering various machine learning tools [103, 111]. Learn more at https://scikit-learn.org.

SciPy is a Python library for scientific computing [64, 144]. Learn more at https://scipy.org.

sign bit The sign bit indicates whether a floating point number is positive or negative in the 64 bit double precision IEEE Standard 754 floating point number layout [56, 60]. See Section 3.3.1.

significand The significand is the part of a floating point number that stores the digits of the number (in binary representation). In the 64 bit double precision IEEE Standard 754 floating point number layout [56, 60], the exponent is 52 bits. See Section 3.3.1.

SimPy is a Python library for discrete event simulation [150]. Learn more at https://simpy.readthedocs.io.

stack trace A stack trace gives information the way in which one function invoked another. The term comes from the fact that the data needed to implement function calls is stored in a stack data structure [72]. The data for the most recently invoked function is on top, the data of the function that called is right below, the data of the function that called that one comes next, and so on. Printing a stack trace can be very helpful when trying to find out where an `Exception` occurred, see, for instance, Chapter 9 and Listing 9.2.

standard error stream The standard error stream (stderr) is one of the three pre-defined streams of a console process (together with the standard input stream and the standard output stream) [67]. It is the text stream to which the process writes information about errors and exceptions. If an uncaught `Exception` is raised in Python and the program terminates, then this information is written to stderr. If you run a program in a terminal, then the text that a process writes to its stderr appears in the console.

standard input stream The standard input stream (stdin) is one of the three pre-defined streams of a console process (together with the standard output stream and the standard error stream) [67]. It is the text stream from which the process reads its input text, if any. The Python instruction `input` reads from this stream. If you run a program in a terminal, then the text that you type into the terminal while the process is running appears in this stream.

standard output stream The standard output stream (stdout) is one of the three pre-defined streams of a console process (together with the standard input stream and the standard error stream) [67]. It is the text stream to which the process writes its normal output. The `print` instruction of Python writes text to this stream. If you run a program in a terminal, then the text that a process writes to its stdout appears in the console.

(string) interpolation In Python, string interpolation is the process where all the expressions in an f-string are evaluated and the final string is constructed. An example for string interpolation is turning `f"Rounded {1.234:.2f}"` to `"Rounded 1.23"`. This is discussed in Section 3.5.2.

sudo In order to perform administrative tasks such as installing new software under Linux, root (or "super") user privileges as needed [25]. A normal user can execute a program in the terminal as super user by pre-pending `sudo`, often referred to as "super user do." This requires the root password.

TensorFlow is a Python library for implementing machine learning, especially suitable for training of neural networks [2, 78]. Learn more at https://www.tensorflow.org.

terminal A terminal is a text-based window where you can enter commands and execute them [7, 25]. Knowing what a terminal is and how to use it is very essential in any programming- or system administration-related task. If you want to open a terminal under Windows, you can press [⊞]+[R], type in `cmd`, and hit [↵], as shown, e.g., in page 7. Under Ubuntu Linux, [Ctrl]+[Alt]+[T] opens a terminal, which then runs a Bash shell inside.

type hint are annotations that help programmers and static code analysis tools such as Mypy to better understand what type a variable or function parameter is supposed to be [80, 142]. Python is a dynamically typed programming language where you do not need to specify the type of, e.g., a variable. This creates problems for code analysis, both automated as well as manual: For example, it may not always be clear whether a variable or function parameter should be an integer or floating point number. The annotations allow us to explicitly state which type is expected. They are *ignored* during the program execution. They are a basically a piece of documentation. See Best Practice 11 and Section 4.3.4.

Ubuntu is a variant of the open source operating system `Linux` [25]. We recommend that you use this operating system to follow this class, for software development, and for research. Learn more at `https://ubuntu.com`. If you are in China, you can download it from `https://mirrors.ustc.edu.cn/ubuntu-releases`.

Unicode A standard for assigning characters to numbers [62, 133, 137]. The Unicode standard supports basically all characters from all languages that are currently in use, as well as many special symbols. It is the predominantly used way to represent characters in computers and is regularly updated and improved.

Version Control System A Version Control Systems (VCS) is a software which allows you to manage and preserve the historical development of your program code [136]. A distributed VCS allows multiple users to work on the same code and upload their changes to the server, which then preserves the change history. The most popular distributed VCS is Git.

Windows Microsoft `Windows` is a commerical proprietary operating system [14]. It is widely spread, but we recommend using a `Linux` variant such as `Ubuntu` for software development and for our course. Learn more at `https://www.microsoft.com/windows`.

x-axis The x-axis is the horizontal axis of a two-dimensional coordinate system, often referred to abscissa.

Acronyms

AI Artificial Intelligence, see, e.g., [116]

DS Data Science, see, e.g., [51]

GIGO Garbage In–Garbage Out, see, e.g., [105]

IDE Integrated Development Environment, see Integrated Development Environment

ML Machine Learning, see, e.g., [125]

OS Operating System, the system that runs your computer, see, e.g., `Linux` and `Windows`

regex see regular expression

`stderr` see standard error stream and [67]

`stdin` see standard input stream and [67]

`stdout` see standard output stream and [67]

VCS Version Control Systems, see Version Control System

Symbols

$i!$ The factorial $a!$ of a natural number $a \in \mathbb{N}_1$ is the product of all positive natural numbers less than or equal to a, i.e., $a! = 1 * 2 * 3 * 4 * \cdots * (a-1) * a$ [22, 38, 87]. See also Equation 8.1 and Listing 8.1

π is the ratio of the circumference U of a circle and its diameter d, i.e., $\pi = U/d$. $\pi \in \mathbb{R}$ is an irrational and transcendental number [44, 66, 96], which is approximately $\pi \approx$ 3.141 592 653 589 793 238 462 643. In Python, it is provided by the `math` module as constant `pi` with value `3.141592653589793`.

$i..j$ with $i, j \in \mathbb{Z}$ and $i \leq j$ is the set that contains all integer numbers in the inclusive range from i to j. For example, 5..9 is equivalent to $\{5, 6, 7, 8, 9\}$

e is Euler's number [43], the base of the natural logarithm. $e \in \mathbb{R}$ is an irrational and transcendental number [44, 66], which is approximately $e \approx$ 2.718 281 828 459 045 235 360. In Python, it is provided by the `math` module as constant `e` with value `2.718281828459045`.

$\mathbb{N}_1$ the set of the natural numbers *excluding* 0, i.e., 1, 2, 3, 4, and so on. It holds that $\mathbb{N}_1 \subset \mathbb{Z}$.

$\mathcal{O}(g(x))$ If $f(x) = \mathcal{O}(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $f(x) \leq c * g(x) \forall x \geq x_0$ [6, 79]. In other words, $\mathcal{O}(g(x))$ describes an upper bound for function growth.

$\mathbb{R}$ the set of the real numbers.

$\mathbb{R}^+$ the set of the positive real numbers, i.e., $\mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}$.

$\mathbb{Z}$ the set of the integers numbers including positive and negative numbers and 0, i.e., . . . , -3, -2, -1, 0, 1, 2, 3, . . . , and so on. It holds that $\mathbb{Z} \subset \mathbb{R}$.

Python Commands

!=, 35, 36
!r, 103, 110
(, 24, 29, 38
(...), 77, 112
), 24, 29, 38
*
    function parameter, 127
    list, 73
    multiplication, 24, 28
**
    function parameter, 127
    power, 24, 28
*=, 56, 114
+, 24, 28, 40, 73, 105
+=, 105
,, 59, 69
,.1f, 44
-, 24, 28
->, 113
-inf, 33, 35, 123, 134
., 117
..., 77
.2f, 44, 156
.3f, 44
/, 24, 28, 34, 60, 61, 63, 150
//, 24, 60, 61, 63, 89, 150
:, 40, 43, 63, 89, 90, 98, 113, 130
:.2%, 44
:.3g, 44
:.4%, 44
:.4g, 44
:.5f, 44
:#b, 43
:#x, 43
:b, 43
:e, 44
:param:, 113
:returns:, 113
:returns: description, 113
:x, 43
<, 36
<=, 36
<<, 26
=, 51
    multiple, 59
    swap, 59, 150
==, 33, 35, 36, 49, 65, 121
>, 36, 91
>=, 36
>>, 26
[...], 40, 69, 73, 84
[], 70
[i:j:k], 73
[i:j], 40, 73
#, 51, 150
%, 24, 28, 89, 102, 114
&, 25
^, 25
_, 31, 99, 130, 150, 151
\, 46
\\, 46
\", 46
\', 46
\n, 46
\r\n, 46
\t, 46
\u, 48, 49, 56
|
    bit-wise or, 25
    type hints, 64, 128
{, 44
{...}
    dict, 69, 84
    f-string, 42
    set, 69, 81
{{, 44, 46
{}, 85
}, 44
}}, 44, 46
"...", 38, 40, 45, 46, 150
"", 40, 49
"""...""", 46, 47, 75, 150, 154
', 38, 45, 150
'...', 38, 40, 46
''', 47, 150
'''...''', 47
0.0, 32–34
0b, 25, 43
0o, 26
0x, 26, 43
1.7976931348623157e+308, 27, 33, 34
2.718281828459045, 159
3.141592653589793, 159
5e-324, 33, 121

0, 36

abs, 95
acos, 29
add, 81
and, 37, 38, 89
append, 70, 102, 103
ArithmeticError, 134–138, 145–147
as, 116, 136
asin, 29
assert, 119
AssertionError, 119
asterisk, 127
atan, 29
Axes, 127

bin, 25
bool, 22, 33, 36–38, 45, 48
    conjunction, 37
    disjunction, 37
    function, 45
    negation, 37
break, 100–102, 104, 109–111, 113

Callable, 128, 131
casefold, 83, 151
ceil, 29, 30
collections.abc, 128
continue, 100, 101, 111
cos, 29

def, 112
del, 72, 85
dict, 102, 103, 127
    del, 85
    empty, 85
    items, 85, 104
    iterate over, 103
    keys, 85, 104
    len, 85
    pop, 85

type hint, 84
update, 85
values, 85, 104
difference, 83
difference_update, 83

e, 29, 34, 159
elif, 92–94, 151
else, 91–94, 109–111, 140–144
empty, 70
endswith, 42
enumerate, 106, 107
escaping, 45, 46, 103
except, 136–144, 152
Exception, 34, 119, 132–142, 145, 147, 148, 152, 154, 156
Exceptions, 137
exit, 154
exp, 125
extend, 70

f-string, 42, 51, 52
interpolation, 42
multi-line, 47
f"...", 42, 154
f"""...""", 47
False, 22, 33, 35–38, 40, 42, 45, 89, 91–93, 108
finally, 141–144
find, 40, 41, 109, 136
float, 22, 24, 27–36, 42, 44, 45, 48, 52, 55, 60, 121, 143, 150
function, 45
largest, 27, 33, 34
smallest, 33, 121
floor, 29
for, 98–103, 105, 109, 111, 113, 114, 134, 135
from, 29, 34, 35, 43, 44, 116
function
argument, 113
*, 127
**, 127
asterisk, 127
by name, 125
default, 125, 152
keyword, 127
star, 127
wildcard, 127
body, 113
call, 113, 125
def, 112
docstring, 113
import, 116
module, 116
parameter, 112
*, 127
**, 127
asterisk, 127
by name, 125
default value, 125, 152
star, 127
wildcard, 127
return, 113
return value, 113
testing, 118
type hint, 113, 151
unit test, 118

gcd, 116

hex, 26

if, 89–93
if...elif...else, 93, 95
nested, 94
if...else, 90, 91
inline, 96
nested, 92
import, 29, 34, 35, 43, 44, 56, 101, 116, 119, 130
in, 40, 72, 77, 83, 98
ind, 136
index, 72, 77
IndexError, 40, 132
inf, 33–36, 45, 121, 123, 124, 134–136
input, 156
insert, 72
int, 22–24, 26–31, 34–36, 42, 43, 45, 48, 51, 60, 143, 150
function, 45
intersection, 83
is, 49, 65, 73
isfinite, 35, 36, 123, 134
isinf, 35, 36
isnan, 35, 36, 121
isqrt, 101, 102, 104
items, 85
iteration, 98

join, 104

KeyError, 85
keys, 85

lambda, 130, 131
len, 40, 49, 70, 85, 102, 105
list, 69, 73, 75, 82, 85, 101–103, 105, 127
*, 73
+, 73
append, 70, 102, 103
del, 72
extend, 70
in, 72
index, 72
insert, 72
iterate over, 103
len, 70, 102, 105
not in, 72
remove, 73
reverse, 73
slicing, 73, 104
sort, 73, 78
type hint, 69
unpacking, 70, 73
log, 29, 34, 49
loop, 98
for, 98, 102, 103
lower, 42
lstrip, 42

math, 29, 34–36, 43, 44, 49, 56, 57, 66, 101, 109, 112, 113, 116, 121, 123, 130, 134, 159
max, 91
min, 91
module, 116
import, 116
Mypy, 62, 155

NameError, 138, 140, 141
nan, 34–36, 45, 49, 108, 121, 123, 124, 134–136, 151
None, 22, 49, 50
NoneType, 49, 50
not, 37, 38, 90
Not a Number, 34–36
not in, 72, 77, 83
not is, 73

oct, 26
or, 37, 38, 89
OverflowError, 34, 36, 132, 143, 147

package, 116
pi, 29, 43, 55, 57, 125, 130, 159
pip, 62, 155
plot, 127
pop, 85
print, 21, 46, 47, 49–51, 78, 89, 99, 101, 104, 112, 113, 116, 134, 143, 156

flush, 134
pytest, 145, 147

raise, 121, 134, 137, 138, 142, 145, 147, 148, 152
raises, 145, 148
range, 98, 99, 101, 102, 104, 105, 111, 113
re, 155
remove, 73, 82
replace, 41
return, 113–115
rfind, 41
round, 29, 30
rstrip, 42
ruff, 155

Sequence, 40
set, 80, 82, 102, 103
add, 81
difference, 83
difference_update, 83
empty, 82
in, 83
intersection, 83
iterate over, 103
not in, 83
remove, 82
symmetric_difference, 83
type hint, 81
union, 83
update, 81
sin, 29, 44, 130
slice, 40, 73, 98, 104
slicing, 40, 73, 104
sort, 73
sorted, 82, 83, 151
sqrt, 56, 66, 109, 112, 113, 125
star, 127
startswith, 42
str, 22, 38, 40, 42, 48, 104
+, 40
[...], 40
[], 40
case-sensitive, 40
casefold, 83, 151
concatenation, 40
doc, 75, 150
function, 113
empty, 40
escaping, 45, 46, 103
f, 42, 47, 51, 52
find, 109, 136
function, 42
index, 136, 137
interpolation, 42
join, 104
len, 40
length, 40
literal, 38
lowercase, 42
slicing, 40
unicode, 48
uppercase, 42
strip, 42
swap, 59, 150
symmetric_difference, 83

tan, 29
True, 22, 33–38, 40, 42, 45, 89, 91, 93, 108
trunc, 29, 30
try, 136–144
tuple, 77, 102, 103, 115, 127
in, 77
index, 77
iterate over, 103
not in, 77
type hint, 77
..., 77
unpacking, 78, 104, 106, 115
type, 36, 50, 60
TypeError, 78, 132, 142, 143
typing, 128

union, 83
unpacking, 70, 73, 78, 104, 106, 115
update, 81, 85
upper, 42

ValueError, 136, 137
values, 85

while, 107–111, 115
wildcard, 127
with, 145

ZeroDivisionError, 121, 137–140, 142

Bibliography

[1] *Python 3 Documentation*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3 (visited on 2024-07-05).

[2] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Gordon Murray, Benoit Steiner, Paul A. Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI'16), Nov. 2-4, 2016, Savannah, GA, USA*. Ed. by Kimberly Keeton and Timothy Roscoe. Berkeley, CA, USA: USENIX Association, 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi (visited on 2024-06-26) (cit. on pp. 3, 156).

[3] *American Standard Code for Information Interchange (ASCII 1963)*. Tech. rep. ASA X3.4-1963. New York, NY, USA: American Standards Association Incorporated, June 17, 1963. URL: https://www.sensitiveresearch.com/Archive/CharCodeHist/Files/CODES%20standards%20documents%20ASCII%20Sean%20Leonard%20Oct%202015/ASCII%2063,%20X3.4-1963.pdf (visited on 2024-07-26) (cit. on p. 48).

[4] AndrewBadr. *TimeComplexity*. Beaverton, OR, USA: Python Software Foundation (PSF), Jan. 19, 2023. URL: https://wiki.python.org/moin/TimeComplexity (visited on 2024-08-27) (cit. on pp. 80, 84).

[5] "Annotating `Callable` Objects". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/typing.html#annotating-callables (visited on 2024-10-09) (cit. on p. 128).

[6] Paul Gustav Heinrich Bachmann. *Die Analytische Zahlentheorie / Dargestellt von Paul Bachmann*. Vol. Zweiter Theil of Zahlentheorie: Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Hauptttheilen. Leipzig, Sachsen, Germany: B. G. Teubner, 1894. ISBN: 1-4181-6963-3. URL: http://gallica.bnf.fr/ark:/12148/bpt6k994750 (visited on 2023-12-13) (cit. on p. 159).

[7] Daniel J. Barrett. *Efficient Linux at the Command Line*. Sebastopol, CA, USA: O'Reilly Media, Inc., Feb. 2022. ISBN: 978-1-0981-1340-7 (cit. on pp. 6, 8, 155, 156).

[8] Gary Baumgartner, Danny Heap, and Richard Krueger. "Numerical Systems". In: *Course Notes for CSC165H: Mathematical Expression and Reasoning for Computer Science*. Toronto, ON, Canada: Department of Computer Science, University of Toronto, Aut. 2006. Chap. 7. URL: https://www.cs.toronto.edu/~krueger/csc165h/f06/lectures/ch7.pdf (visited on 2024-07-27) (cit. on pp. 29, 150).

[9] David M. Beazley. "Data Processing with Pandas". *;login: Usenix Magazin* 37(6), Dec. 2012. ISSN: 1044-6397. URL: https://www.usenix.org/publications/login/december-2012-volume-37-number-6/data-processing-pandas (visited on 2024-06-25) (cit. on pp. 3, 155).

[10] Richard Beigel. "Irrationality without Number Theory". *The American Mathematical Monthly* 98(4):332–335, Apr. 1991. ISSN: 1930-0972. doi:10.1080/00029890.1991.12000762. URL: https://cis.temple.edu/~beigel/papers/irrationality-amm.PS.gz (visited on 2024-07-06) (cit. on p. 28).

[11] Jon Bentley. *Programming Pearls*. 2nd ed. Reading, MA, USA: Addison-Wesley Professional, Oct. 7, 1999. ISBN: 978-0-201-65788-3 (cit. on pp. 109, 110).

[12] Fabian Beuke. *GitHut 2.0: GitHub Language Statistics*. San Francisco, CA, USA: GitHub Inc, 2023. URL: https://madnight.github.io/githut/ (visited on 2024-06-24) (cit. on p. 2).

[13] Ethan Bommarito and Michael Bommarito. *An Empirical Analysis of the Python Package Index (PyPI)*. arXiv.org: Computing Research Repository (CoRR) abs/1907.11073. Ithaca, NY, USA: Cornell Universiy Library, July 26, 2019. doi:10.48550/arXiv.1907.11073. URL: https://arxiv.org/abs/1907.11073 (visited on 2024-08-17). arXiv:1907.11073v2 [cs.SE] 26 Jul 2019 (cit. on p. 155).

[14] Ed Bott. *Windows 11 Inside Out*. Hoboken, NJ, USA: Microsoft Press, Pearson Education, Inc., Feb. 2023. ISBN: 978-0-13-769132-6 (cit. on pp. 6, 157).

[15] Georg Brandl and Serhiy Storchaka. *Underscores in Numeric Literals*. Python Enhancement Proposal (PEP) 515. Beaverton, OR, USA: Python Software Foundation (PSF), Feb. 10, 2016. URL: https://peps.python.org/pep-0515 (visited on 2024-09-23) (cit. on pp. 31, 150).

[16] Ron Brash and Ganesh Naik. *Bash Cookbook*. Birmingham, England, UK: Packt Publishing Ltd, July 2018. ISBN: 978-1-78862-936-2 (cit. on pp. 154, 174).

[17] Richard P. Brent. *Further Analysis of the Binary Euclidean Algorithm*. arXiv.org: Computing Research Repository (CoRR) abs/1303.2772. Ithaca, NY, USA: Cornell Universiy Library, Nov. 1999–Mar. 12, 2013. doi:10.48550/arXiv.1303.2772. URL: https://arxiv.org/abs/1303.2772 (visited on 2024-09-28). arXiv:1303.2772v1 [cs.DS] 12 Mar 2013. Report number PRG TR-7-99 of Oxford, Oxfordshire, England, UK: Oxford University Computing Laboratory, Nov. 1999, see https://maths-people.anu.edu.au/~brent/pd/rpb183tr.pdf (cit. on pp. 114, 116).

[18] Florian Bruhin. *Python f-strings*. Winterthur, Switzerland: Bruhin Software, May 31, 2023. URL: https://fstring.help (visited on 2024-07-25) (cit. on p. 42).

[19] "Built-in `Exceptions`". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/exceptions.html (visited on 2024-10-29) (cit. on p. 148).

[20] "Built-in Types". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/stdtypes.html (visited on 2024-08-22) (cit. on p. 69).

[21] Oscar Castro, Pierrick Bruneau, Jean-Sébastien Sottet, and Dario Torregrossa. "Landscape of High-Performance Python to Develop Data Science and Machine Learning Applications". *ACM Computing Surveys (CSUR)* 56(3):65:1–65:30, 2024. ISSN: 0360-0300. doi:10.1145/3617588 (cit. on p. 3).

[22] Noureddine Chabini and Rachid Beguenane. "FPGA-Based Designs of the Factorial Function". In: *IEEE Canadian Conference on Electrical and Computer Engineering (CCECE'22), Sept. 18–20, 2022, Halifax, NS, Canada*. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022, pp. 16–20. ISBN: 978-1-6654-8432-9. doi:10.1109/CCECE49351.2022.9918302 (cit. on pp. 113, 159).

[23] *Chinese Internal Code Specification, National Standard Extended (GBK)*. Technical Supervision Standard Letter 1995 229. China, Beijing: Standardization Department of the State Administration of Technical Supervision, Department of Science, and Quality Supervision of the Ministry of Electronics Industry, Dec. 15, 1995 (cit. on p. 48).

[24] "Class Double". In: *Java® Platform, Standard Edition & Java Development Kit Version 22 API Specification*. Redwood Shores, CA, USA: Oracle Corporation, Apr. 9, 2024. URL: https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/lang/Double.html (visited on 2024-07-07) (cit. on p. 32).

[25] David Clinton and Christopher Negus. *Ubuntu Linux Bible*. 10th ed. Bible Series. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Nov. 10, 2020. ISBN: 978-1-119-72233-5 (cit. on pp. 6, 156, 157).

[26] *Code of Chinese Graphic Character Set for Information Interchange – Primary Set*. National Standard of the People's Republic of China (GB) GB/T 2312-1980. China, Beijing: National Standards Administration, 1980–May 1, 1981. URL: https://openstd.samr.gov.cn/bzgk/gb/newGbInfo?hcno=5664A728BD9D523DE3B99BC37AC7A2CC (visited on 2024-07-26) (cit. on p. 48).

[27] "`collections.abc` – Abstract Base Classes for Containers". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/collections.abc.html (visited on 2024-08-22) (cit. on p. 69).

[28] "`contextlib` – Utilities for `with`-Statement Contexts". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/contextlib.html (visited on 2024-11-01) (cit. on p. 145).

[29] Thomas H. Cormen, Charles E. Leiserson, Ronald Linn Rivest, and Clifford Stein. *Introduction to Algorithms*. 3rd ed. Cambridge, MA, USA: MIT Press, 2009. ISBN: 978-0-262-03384-8 (cit. on pp. 80, 84).

[30] Richard Crandall and Carl Pomerance. *Prime Numbers: A Computational Perspective*. 2nd ed. New York, NY, USA: Springer New York, Aug. 4, 2005. ISBN: 978-0-387-25282-7. doi:10.1007/0-387-28979-8 (cit. on p. 101).

[31] Christopher Cullen. "Learning from Liu Hui? A Different Way to Do Mathematics". *Notices of the American Mathematical Society* 49(7):783–790, Aug. 2002. ISSN: 1088-9477. URL: https://www.ams.org/notices/200207/comm-cullen.pdf (visited on 2024-08-09) (cit. on p. 55).

[32] "Data Model". In: *Python 3 Documentation*. Vol. The Python Language Reference. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. Chap. 3. URL: https://docs.python.org/3/reference/datamodel.html (visited on 2024-08-22) (cit. on p. 69).

[33] Joseph W. Dauben. "Archimedes and Liu Hui on Circles and Spheres". *Ontology Studies (Cuadernos de Ontología)* 10:21–38, 2010. ISSN: 1576-2270. URL: https://ddd.uab.cat/pub/ontstu/15762270n10/15762270n10p21.pdf (visited on 2024-08-10) (cit. on p. 55).

[34] Justin Dennison, Cherokee Boose, and Peter van Rysdam. *Intro to NumPy*. Centennial, CO, USA: ACI Learning. Birmingham, England, UK: Packt Publishing Ltd, June 2024. ISBN: 978-1-83620-863-1 (cit. on pp. 3, 155).

[35] Alfredo Deza and Noah Gift. *Testing In Python*. San Francisco, CA, USA: Pragmatic AI Labs, Feb. 2020. ISBN: 979-8-6169-6064-1 (cit. on pp. 119, 155).

[36] "Dictionaries". In: *Python 3 Documentation*. Vol. The Python Language Reference. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. Chap. 3.2.7.1. URL: https://docs.python.org/3/reference/datamodel.html#dictionaries (visited on 2024-08-27) (cit. on p. 85).

[37] Slobodan Dmitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0224-9 (cit. on p. 154).

[38] Jacques Dutka. "The Early History of the Factorial Function". *Archive for History of Exact Sciences* 43(3):225–249, Sept. 1991. ISSN: 0003-9519. doi:10.1007/BF00389433. Communicated by Umberto Bottazzini (cit. on pp. 113, 159).

[39] The Editors of Encyclopaedia Britannica, Veenu Setia, Emily Rodriguez, Aakanksha Gaur, Meg Matthias, and Gloria Lotha. "Leap Year". In: *Encyclopaedia Britannica*. Aug. 20, 2024. URL: https://www.britannica.com/science/leap-year-calendar (visited on 2024-08-29) (cit. on p. 89).

[40] James F. Epperson. *An Introduction to Numerical Methods and Analysis*. 2nd ed. John Wiley and Sons Ltd., Oct. 2013. ISBN: 978-1-118-36759-9 (cit. on pp. 128, 131).

[41] Euclid of Alexandria (Εὐκλείδης). "Elementary Number Theory". In: *Euclid's Elements of Geometry (Στοιχεῖα). The Greek Text of J.L. Heiberg (1883-1885) from Euclidis Elementa, Edidit et Latine Interpretatus est I.L. Heiberg in Aedibus B. G. Teubneri, 1883-1885. Edited, and provided with a modern English translation, by Richard Fitzpatrick*. Ed. by Richard Fitzpatrick. Trans. by Johan Ludvig Heiberg. revised and corrected. Austin, TX, USA: The University of Texas at Austin, 2008. Chap. Book 7. ISBN: 978-0-615-17984-1. URL: https://farside.ph.utexas.edu/Books/Euclid/Elements.pdf (visited on 2024-09-30) (cit. on p. 114).

[42] Leonhard Euler. "An Essay on Continued Fractions". Trans. by Myra F. Wyman and Bostwick F. Wyman. *Mathematical Systems Theory* 18(1):295–328, Dec. 1985. ISSN: 1432-4350. doi:10.1007/BF01699475. URL: https://www.researchgate.net/publication/301720080 (visited on 2024-09-24). Translation of [43]. (Cit. on p. 166).

[43] Leonhard Euler. "De Fractionibus Continuis Dissertation". *Commentarii Academiae Scientiarum Petropolitanae* 9:98–137, 1737–1744. URL: https://scholarlycommons.pacific.edu/cgi/viewcontent.cgi?article=1070 (visited on 2024-09-24). See [42] for a translation. (Cit. on pp. 159, 165).

[44] Michael Filaseta. "The Transcendence of e and π". In: *Math 785: Transcendental Number Theory*. Columbia, SC, USA: University of South Carolina, Spr. 2011. Chap. 6. URL: https://people.math.sc.edu/filaseta/gradcourses/Math785/Math785Notes6.pdf (visited on 2024-07-05) (cit. on pp. 27, 159).

[45] "Formatted String Literals". In: *Python 3 Documentation*. Vol. The Python Tutorial. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. Chap. 7.1.1. URL: https://docs.python.org/3/tutorial/inputoutput.html#formatted-string-literals (visited on 2024-07-25) (cit. on p. 42).

[46] David Fowler and Eleanor Robson. "Square Root Approximations in Old Babylonian Mathematics: YBC 7289 in Context". *Historia Mathematica* 25(4):366–378, Nov. 1998. ISSN: 0315-0860. doi:10.1006/hmat.1998.2209. URL: https://www.ux1.eiu.edu/~cfcid/Classes/4900/Class%20Notes/Babylonian%20Approximations.pdf (visited on 2024-09-25). Article NO. HM982209 (cit. on p. 107).

[47] Toru Fujita, Koji Nakano, and Yasuaki Ito. "Bulk Execution of Euclidean Algorithms on the CUDA-Enabled GPU". *International Journal of Networking and Computing (IJNC)* 6(1):42–63, Jan. 2016. ISSN: 2185-2839. URL: http://www.ijnc.org (visited on 2024-09-28) (cit. on p. 114).

[48] Alessandro F. Garcia, andCecília M. F. Rubira, Alexander B. Romanovsky, and Jie Xu. "A Comparative Study of Exception Handling Mechanisms for Building Dependable Object-Oriented Software". *Journal of Systems and Software* 59(2):197–222, Nov. 15, 2001. ISSN: 0164-1212. doi:10.1016/S0164-1212(01)00062-0 (cit. on p. 132).

[49] David Goodger and Guido van Rossum. *Docstring Conventions*. Python Enhancement Proposal (PEP) 257. Beaverton, OR, USA: Python Software Foundation (PSF), May 29–June 13, 2001. URL: https://peps.python.org/pep-0257 (visited on 2024-07-27) (cit. on pp. 47, 75, 150, 154).

[50] Linda Grandell, Mia Peltomäki, Ralph-Johan Back, and Tapio Salakoski. "Why complicate things? Introducing Programming in High School using Python". In: *Proceedings of the 8th Australasian Conference on Computing Education (ACE'06), Jan. 16-19, 2006, Hobart, TAS, Australia*. Ed. by Denise Tolhurst and Samuel Mann. Vol. 52. New York, NY, USA: Association for Computing Machinery (ACM), 2006, pp. 71–80. ISBN: 978-1-920682-34-7. doi:10.5555/1151869.1151880 (cit. on p. 3).

[51] Joel Grus. *Data Science from Scratch: First Principles with Python*. 2nd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., May 2019. ISBN: 978-1-4920-4113-9 (cit. on pp. 3, 158).

[52] Sławomir Gryś. *Computer Arithmetic in Practice*. Boca Raton, FL, USA: CRC Press, Sept. 2023. ISBN: 978-1-000-93492-2 (cit. on p. 35).

[53] Mohammad Hammoud. "Binary Search". In: *15-122: Principles of Imperative Computation*. Doha, Qatar: Carnegie Mellon University Qatar, Spr. 2024. Chap. Lecture 06. URL: https://web2.qatar.cmu.edu/~mhhammou/15122-s24/lectures/06-binsearch/slides (visited on 2024-09-26) (cit. on pp. 109, 110).

[54] Charles R. Harris, K. Jarrod Millman, Stéfan van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. "Array programming with NumPy". *Nature* 585:357–362, 2020. ISSN: 0028-0836. doi:10.1038/S41586-020-2649-2 (cit. on pp. 3, 155).

[55] Michael Hausenblas. *Learning Modern Linux*. Sebastopol, CA, USA: O'Reilly Media, Inc., Apr. 2022. ISBN: 978-1-0981-0894-6 (cit. on pp. 6, 155).

[56] Steve Hollasch. "IEEE Standard 754 Floating Point Numbers". In: *CSE401: Introduction to Compiler Construction*. Seattle, WA, USA: University of Washington, Jan. 8, 1997. URL: https://courses.cs.washington.edu/courses/cse401/01au/details/fp.html (visited on 2024-07-05) (cit. on pp. 27, 33, 154–156).

[57] Trey Hunner. *Python Big $\mathcal{O}$: The Time Complexities of Different Data Structures in Python; Python 3.8-3.12*. Reykjavík, Iceland: Python Morsels, Apr. 16, 2024. URL: https://www.pythonmorsels.com/time-complexities (visited on 2024-08-27) (cit. on p. 80).

[58] John Hunt. *A Beginners Guide to Python 3 Programming*. 2nd ed. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (cit. on p. 155).

[59] John D. Hunter. "Matplotlib: A 2D Graphics Environment". *Computing in Science & Engineering* 9(3):90–95, May–June 2007. ISSN: 1521-9615. doi:10.1109/MCSE.2007.55 (cit. on pp. 3, 155).

[60] *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754™-2019 (Revision of IEEE Std 754-2008). New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), June 13, 2019 (cit. on pp. 27, 33, 154–156).

[61] *Information Technology – Chinese Coded Character Set*. National Standard of the People's Republic of China (GB) GB 18030-2022. China, Beijing: State Administration for Market Regulation and Standardization Administration of the People's Republic of China, July 19, 2022–Aug. 1, 2023. URL: https://openstd.samr.gov.cn/bzgk/gb/newGbInfo?hcno=A1931A578FE14957104988029B0833D3 (visited on 2024-07-26) (cit. on p. 48).

[62] *Information Technology – Universal Coded Character Set (UCS)*. International Standard ISO/IEC 10646:2020. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Dec. 2020 (cit. on pp. 48, 157).

[63] "Installing Python Modules". In: *Python 3 Documentation*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/installing (visited on 2024-08-17) (cit. on pp. 116, 155).

[64] Robert Johansson. *Numerical Python: Scientific Computing and Data Science Applications with NumPy, SciPy and Matplotlib*. New York, NY, USA: Apress Media, LLC, Dec. 2018. ISBN: 978-1-4842-4246-9 (cit. on pp. 3, 155).

[65] Stephen Curtis Johnson. *Lint, a C Program Checker*. Computing Science Technical Report 78–1273. New York, NY, USA: Bell Telephone Laboratories, Incorporated, Oct. 25, 1978. URL: https://wolfram.schneider.org/bsd/7thEdManVol2/lint/lint.pdf (visited on 2024-08-23) (cit. on p. 154).

[66] Arthur Jones, Kenneth R. Pearson, and Sidney A. Morris. "Transcendence of e and π". In: *Abstract Algebra and Famous Impossibilities*. Universitext (UTX). New York, NY, USA: Springer New York, 1991. Chap. 9, pp. 115–161. ISSN: 0172-5939. ISBN: 978-1-4419-8552-1. doi:10.1007/978-1-4419-8552-1_8 (cit. on pp. 27, 159).

[67] "stderr, stdin, stdout – Standard I/O Streams". In: *POSIX.1-2024: The Open Group Base Specifications Issue 8, IEEE Std 1003.1™-2024 Edition*. Ed. by Andrew Josey. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and San Francisco, CA, USA: The Open Group, Aug. 8, 2024. URL: https://pubs.opengroup.org/onlinepubs/9799919799/functions/stdin.html (visited on 2024-10-30) (cit. on pp. 156, 158).

[68] "exit – Terminate a Process". In: *POSIX.1-2024: The Open Group Base Specifications Issue 8, IEEE Std 1003.1™-2024 Edition*. Ed. by Andrew Josey. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and San Francisco, CA, USA: The Open Group, Aug. 8, 2024. URL: https://pubs.opengroup.org/onlinepubs/9799919799/functions/exit.html (visited on 2024-10-30) (cit. on p. 154).

[69] Andrew Josey, ed. *POSIX.1-2024: The Open Group Base Specifications Issue 8, IEEE Std 1003.1™-2024 Edition*. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and San Francisco, CA, USA: The Open Group, Aug. 8, 2024. URL: https://pubs.opengroup.org/onlinepubs/9799919799 (visited on 2024-10-30).

[70] Shen Kangshen, John Newsome Crossley, and Anthony W.-C. Lun. *The Nine Chapters on the Mathematical Art: Companion and Commentary*. Oxford, Oxfordshire, England, UK: Oxford University Press, Oct. 7, 1999. ISBN: 978-0-19-853936-0. doi:10.1093/oso/9780198539360.001.0001 (cit. on p. 55).

[71] Faizan Khan, Boqi Chen, Dániel Varró, and Shane McIntosh. "An Empirical Study of Type-Related Defects in Python Projects". *IEEE Transactions on Software Engineering* 48(8):3145–3158, Aug. 1, 2022. ISSN: 0098-5589. doi:10.1109/TSE.2021.3082068. URL: https://www.researchgate.net/publication/351729684 (visited on 2024-08-16) (cit. on p. 61).

[72] Donald Ervin Knuth. *Fundamental Algorithms*. 3rd ed. Vol. 1. The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1997. ISBN: 0-201-89683-4 (cit. on p. 156).

[73] Donald Ervin Knuth. *Sorting and Searching*. Vol. 3. The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1998. ISBN: 978-0-201-89685-5 (cit. on pp. 80, 84, 109, 110).

[74] Tobias Kohn and Guido van Rossum. *Structural Pattern Matching: Motivation and Rationale*. Python Enhancement Proposal (PEP) 635. Beaverton, OR, USA: Python Software Foundation (PSF), Sept. 12, 2020–Feb. 8, 2021. URL: https://peps.python.org/pep-0635 (visited on 2024-09-23) (cit. on pp. 99, 151).

[75] Olga Kosheleva. "Babylonian Method of Computing the Square Root: Justifications based on Fuzzy Techniques and on Computational Complexity". In: *2009 Annual Meeting of the North American Fuzzy Information Processing Society (NAFIPS'09), June 14-19, 2009, Cincinnati, OH, USA*. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2009. ISBN: 978-1-4244-4575-2. doi:10.1109/NAFIPS.2009.5156463. URL: https://www.cs.utep.edu/vladik/2009/olg09-05a.pdf (visited on 2024-09-25) (cit. on pp. 107, 108).

[76] Andrew M. Kuchling. *Regular Expression HOWTO*. Vol. Python HOWTOs. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/howto/regex.html (visited on 2024-11-01) (cit. on pp. 145, 155).

[77] "Lambda". In: *Python 3 Documentation*. Vol. The Python Language Reference. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. Chap. 6.14. URL: https://docs.python.org/3/reference/expressions.html#lambda (visited on 2024-10-09) (cit. on p. 130).

[78] Charles Landau. *TensorFlow Deep Dive: Build, Train, and Deploy Machine Learning Models with TensorFlow*. Sebastopol, CA, USA: O'Reilly Media, Inc., Dec. 2023 (cit. on pp. 3, 156).

[79] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. Leipzig, Sachsen, Germany: B. G. Teubner, 1909. ISBN: 0-8218-2650-6 (cit. on p. 159).

[80] Łukasz Langa. *Literature Overview for Type Hints*. Python Enhancement Proposal (PEP) 482. Beaverton, OR, USA: Python Software Foundation (PSF), Jan. 8, 2015. URL: https://peps.python.org/pep-0482 (visited on 2024-10-09) (cit. on p. 156).

[81] Łukasz Langa. *Type Hinting Generics In Standard Collections*. Python Enhancement Proposal (PEP) 585. Beaverton, OR, USA: Python Software Foundation (PSF), Mar. 3, 2019. URL: https://peps.python.org/pep-0585 (visited on 2024-10-09) (cit. on pp. 69, 77, 81, 84).

[82] Kent D. Lee and Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (cit. on p. 155).

[83] Jukka Lehtosalo, Ivan Levkivskyi, Jared Hance, Ethan Smith, Guido van Rossum, Jelle Zijlstra, Michael J. Sullivan, Shantanu Jain, Xuanda Yang, Jingchen Ye, Nikita Sobolev, and Mypy Contributors. *Mypy – Static Typing for Python*. San Francisco, CA, USA: GitHub Inc, 2024. URL: https://github.com/python/mypy (visited on 2024-08-17) (cit. on pp. 62, 153, 155).

[84] Reuven M. Lerner. *Pandas Workout*. Shelter Island, NY, USA: Manning Publications, June 2024. ISBN: 978-1-61729-972-8 (cit. on pp. 3, 155).

[85] Luan P. Lima, Lincoln S. Rocha, Carla I. M. Bezerra, and Matheus Paixão. "Assessing Exception Handling Testing Practices in Open-Source Libraries". *Empirical Software Engineering: An International Journal* 26(5:85), June–Sept. 2021. ISSN: 1382-3256. doi:10.1007/s10664-021-09983-3. URL: https://arxiv.org/abs/2105.00500 (visited on 2024-10-29). See also arXiv:2105.00500v1 [cs.SE] 2 May 2021 (cit. on p. 145).

[86] Marc Loy, Patrick Niemeyer, and Daniel Leuck. *Learning Java*. 5th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2020. ISBN: 978-1-4920-5627-0 (cit. on p. 154).

[87] Peter Luschny. *A New Kind of Factorial Function*. Highland Park, NJ, USA: The OEIS Foundation Inc., Oct. 4, 2015. URL: https://oeis.org/A000142/a000142.pdf (visited on 2024-09-29) (cit. on pp. 114, 159).

[88] Charlie Marsh. *Ruff: An Extremely Fast Python Linter and Code Formatter, Written in Rust*. New York, NY, USA: Astral Software Inc., Aug. 28, 2022. URL: https://docs.astral.sh/ruff (visited on 2024-08-23) (cit. on pp. 74, 153, 155).

[89] Aaron Maxwell. *What are f-strings in Python and how can I use them?* Oakville, ON, Canada: Infinite Skills Inc, June 2017. ISBN: 978-1-4919-9486-3 (cit. on p. 42).

[90] Pedro Mejia Alvarez, Raul E. Gonzalez Torres, and Susana Ortega Cisneros. *Exception Handling – Fundamentals and Programming*. SpringerBriefs in Computer Science. Cham, Switzerland: Springer, Feb. 2024. ISSN: 2191-5768. ISBN: 978-3-031-50680-2. doi:10.1007/978-3-031-50681-9 (cit. on p. 132).

[91] Mark Mendoza. *Parameter Specification Variables*. Python Enhancement Proposal (PEP) 612. Beaverton, OR, USA: Python Software Foundation (PSF), Dec. 18, 2019–July 13, 2020. URL: https://peps.python.org/pep-0612 (visited on 2024-10-09) (cit. on p. 128).

[92] Zsolt Nagy. *Regex Quick Syntax Reference: Understanding and Using Regular Expressions*. New York, NY, USA: Apress Media, LLC, Aug. 2018. ISBN: 978-1-4842-3876-9 (cit. on pp. 145, 155).

[93] Cameron Newham and Bill Rosenblatt. *Learning the Bash Shell – Unix Shell Programming: Covers Bash 3.0*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., 2005. ISBN: 978-0-596-00965-6 (cit. on pp. 154, 174).

[94] Thomas Nield. *An Introduction to Regular Expressions*. Sebastopol, CA, USA: O'Reilly Media, Inc., June 2019. ISBN: 978-1-4920-8255-2 (cit. on pp. 145, 155).

[95] nishkarsh146. *Complexity Cheat Sheet for Python Operations*. Noida, Uttar Pradesh, India: GeeksforGeeks – Sanchhaya Education Private Limited, Aug. 17, 2022. URL: https://www.geeksforgeeks.org/complexity-cheat-sheet-for-python-operations (visited on 2024-08-27) (cit. on p. 80).

[96] Ivan Niven. "The Transcendence of π". *The American Mathematical Monthly* 46(8):469–471, Oct. 1939. ISSN: 1930-0972. doi:10.2307/2302515 (cit. on pp. 27, 159).

[97] "Numeric Types – `int`, `float`, `complex`". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/stdtypes.html#typesnumeric (visited on 2024-07-05) (cit. on p. 27).

[98] John J. O'Connor and Edmund F. Robertson. *Liu Hui*. St Andrews, Scotland, UK: University of St Andrews, School of Mathematics and Statistics, Dec. 2003. URL: https://mathshistory.st-andrews.ac.uk/Biographies/Liu_Hui (visited on 2024-08-10) (cit. on p. 55).

[99] Brian Okken. *Python Testing with pytest*. Flower Mound, TX, USA: Pragmatic Bookshelf by The Pragmatic Programmers, L.L.C., Feb. 2022. ISBN: 978-1-68050-860-4 (cit. on pp. 119, 155).

[100] Ashwin Pajankar. *Hands-on Matplotlib: Learn Plotting and Visualizations with Python 3*. New York, NY, USA: Apress Media, LLC, Nov. 2021. ISBN: 978-1-4842-7410-1 (cit. on pp. 3, 155).

[101] Ashwin Pajankar. *Python Unit Test Automation: Automate, Organize, and Execute Unit Tests in Python*. New York, NY, USA: Apress Media, LLC, Dec. 2021. ISBN: 978-1-4842-7854-3 (cit. on pp. 119, 155).

[102] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019 (NeurIPS'19), Dec. 8-14, 2019, Vancouver, BC, Canada*. Ed. by Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett. San Diego, CA, USA: The Neural Information Processing Systems Foundation (NeurIPS), 2019, pp. 8024–8035. ISBN: 978-1-7138-0793-3. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (visited on 2024-07-18) (cit. on pp. 3, 155).

[103] Fabian Pedregos, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vander-Plas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. "Scikit-learn: Machine Learning in Python". *Journal of Machine Learning Research (JMLR)* 12:2825–2830, Oct. 2011. ISSN: 1532-4435. doi:10.5555/1953048.2078195 (cit. on pp. 3, 155).

[104] Yasset Pérez-Riverol, Laurent Gatto, Rui Wang, Timo Sachsenberg, Julian Uszkoreit, Felipe da Veiga Leprevost, Christian Fufezan, Tobias Ternent, Stephen J. Eglen, Daniel S. Katz, Tom J. Pollard, Alexander Konovalov, Robert M. Flight, Kai Blin, and Juan Antonio Vizcaíno. "Ten Simple Rules for Taking Advantage of Git and GitHub". *PLOS Computational Biology* 12(7), July 14, 2016. ISSN: 1553-7358. doi:10.1371/JOURNAL.PCBI.1004947 (cit. on p. 154).

[105] Amit Phalgun, Cory Kissinger, Margaret M. Burnett, Curtis R. Cook, Laura Beckwith, and Joseph R. Ruthruff. "Garbage in, Garbage out? An Empirical Look at Oracle Mistakes by End-User Programmers". In: *Proceedings of the 2005 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC '05), Sept. 21-24, 2005, Dallas, TX, USA*. Ed. by Martin Erwig and Andy Schürr. Los Alamitos, CA, USA: IEEE Computer Society, 2005, pp. 45–52. ISSN: 1943-6092. ISBN: 0-7695-2443-5. doi:10.1109/VLHCC.2005.40 (cit. on pp. 132, 158).

[106] Philippe PRADOS and Maggie Moss. *Allow Writing Union Types as* `X | Y`. Python Enhancement Proposal (PEP) 604. Beaverton, OR, USA: Python Software Foundation (PSF), Aug. 28, 2019. URL: https://peps.python.org/pep-0604 (visited on 2024-10-09) (cit. on pp. 64, 93, 128).

[107] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. "1.1 Error, Accuracy, and Stability". In: *Numerical Recipes: The Art of Scientific Computing*. 3rd ed. Cambridge, England, UK: Cambridge University Press & Assessment, 2007–2011. Chap. 1 Preliminaries, pp. 8–12. ISBN: 978-0-521-88068-8. URL: https://numerical.recipes/book.html (visited on 2024-07-27). Version 3.04 (cit. on pp. 29, 150).

[108] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/3IEC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Nov. 2017. URL: https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf (visited on 2024-06-29) (cit. on pp. 23, 154).

[109] "programming: Meaning of *programming* in English". In: *Cambridge Dictionary English (UK)*. Cambridge, England, UK: Cambridge University Press & Assessment, June 2024. URL: https://dictionary.cambridge.org/dictionary/english/programming (visited on 2024-06-17) (cit. on p. 1).

[110] Pylint Contributors. *Pylint*. Toulouse, France: Logilab, 2003–2024. URL: https://pylint.readthedocs.io/en/stable (visited on 2024-09-24) (cit. on pp. 106, 153, 155).

[111] Sebastian Raschka, Yuxi Liu, and Vahid Mirjalili. *Machine Learning with PyTorch and Scikit-learn*. Birmingham, England, UK: Packt Publishing Ltd, Feb. 2022. ISBN: 978-1-80181-931-2 (cit. on pp. 3, 155).

[112] "`re` – Regular Expression Operations". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/re.html#module-re (visited on 2024-11-01) (cit. on pp. 145, 155).

[113] Hans Riesel. *Prime Numbers and Computer Methods for Factorization*. 2nd ed. Progress in Mathematics (PM). New York, NY, USA: Springer Science+Business Media, Oct. 1, 1994–Sept. 30, 2012. ISSN: 0743-1643. ISBN: 978-0-8176-3743-9. doi:10.1007/978-1-4612-0251-6. Boston, MA, USA: Birkhäuser (cit. on p. 101).

[114] Ori Roth. "Python Type Hints Are Turing Complete (Pearl/Brave New Idea)". In: *37th European Conference on Object-Oriented Programming (ECOOP'23), July 17-21, 2023, Seattle, WA, USA*. Ed. by Karim Ali and Guido Salvaneschi. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). Wadern, Saarland, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 44:1–44:15. ISSN: 1868-8969. ISBN: 978-3-95977-281-5. doi:10.4230/LIPICS.ECOOP.2023.44 (cit. on p. 63).

[115] Per Runeson. "IEEE Software". 23(4):22–29, July–Aug. 2006. ISSN: 0740-7459. doi:10.1109/MS.2006.91 (cit. on p. 119).

[116] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach (AIMA)*. 4th ed. Hoboken, NJ, USA: Pearson Education, Inc. ISBN: 978-1-292-40113-3. URL: https://aima.cs.berkeley.edu (visited on 2024-06-27) (cit. on pp. 3, 97, 158).

[117] Yeonhee Ryou, Sangwoo Joh, Joonmo Yang, Sujin Kim, and Youil Kim. "Code Understanding Linter to Detect Variable Misuse". In: *37th IEEE/ACM International Conference on Automated Software Engineering (ASE'22), Oct. 10-14, 2022, Rochester, MI, USA*. New York, NY, USA: Association for Computing Machinery (ACM), 2022, 133:1–133:5. ISBN: 978-1-4503-9475-8. doi:10.1145/3551349.3559497 (cit. on p. 154).

[118] Yoram Sagher. "What Pythagoras Could Have Done". *The American Mathematical Monthly* 95(2):117, Feb. 1988. ISSN: 1930-0972. doi:10.1080/00029890.1988.11971978 (cit. on p. 28).

[119] Larkin Ridgway Scott. "Numerical Algorithms". In: *Numerical Analysis*. Princeton, NJ, USA: Princeton University Press, Apr. 18, 2011. Chap. 1. ISBN: 978-1-4008-3896-7. doi:10.1515/9781400838967-002. URL: https://assets.press.princeton.edu/chapters/s9487.pdf (visited on 2024-09-25) (cit. on pp. 107, 108).

[120] "Sequences". In: *Python 3 Documentation*. Vol. The Python Language Reference. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. Chap. 3.2.5. URL: https://docs.python.org/3/reference/datamodel.html#sequences (visited on 2024-08-24) (cit. on pp. 69, 73).

[121] "Set Types – set, frozenset". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/stdtypes.html#set (visited on 2024-08-27) (cit. on pp. 80, 82).

[122] "Set Types". In: *Python 3 Documentation*. Vol. The Python Language Reference. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. Chap. 3.2.6. URL: https://docs.python.org/3/reference/datamodel.html#set-types (visited on 2024-08-27) (cit. on p. 82).

[123] "Python Setup and Usage". In: *Python 3 Documentation*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/using (visited on 2024-07-05) (cit. on p. 6).

[124] Ali Shahrokni and Robert Feldt. "A Systematic Review of Software Robustness". *Information and Software Technology* 55(1):1–17, Jan. 2013. ISSN: 0950-5849. doi:10.1016/J.INFSOF.2012.06.002. URL: http://robertfeldt.net/publications/shahrokni_2013_sysrev_robustness.pdf (visited on 2024-10-29) (cit. on p. 132).

[125] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge, England, UK: Cambridge University Press & Assessment, July 2014. ISBN: 1-107-05713-2. URL: http://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning (visited on 2024-06-27) (cit. on pp. 3, 97, 158).

[126] Avi Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. 7th ed. New York, NY, USA: McGraw-Hill, Mar. 2019. ISBN: 978-0-07-802215-9 (cit. on pp. 80, 84).

[127] Anna Skoulikari. *Learning Git*. Sebastopol, CA, USA: O'Reilly Media, Inc., May 2023. ISBN: 978-1-0981-3391-7 (cit. on p. 154).

[128] Eric V. Smith. *Literal String Interpolation*. Python Enhancement Proposal (PEP) 498. Beaverton, OR, USA: Python Software Foundation (PSF), Nov. 6, 2016–Sept. 9, 2023. URL: https://peps.python.org/pep-0498 (visited on 2024-07-25) (cit. on p. 42).

[129] Philip D. Straffin Jr. "Liu Hui and the First Golden Age of Chinese Mathematics". *Mathematics Magazine* 71(3):163–181, June 1998. ISSN: 0025-570X. doi:10.2307/2691200. URL: https://www.researchgate.net/publication/237334342 (visited on 2024-08-10) (cit. on p. 55).

[130] "Text Sequence Type – `str`". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/stdtypes.html#textseq (visited on 2024-07-25) (cit. on p. 42).

[131] "The Import System". In: *Python 3 Documentation*. Vol. The Python Language Reference. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. Chap. 5. URL: https://docs.python.org/3/reference/import.html (visited on 2024-10-01) (cit. on p. 116).

[132] *The Python Package Index (PyPI)*. Beaverton, OR, USA: Python Software Foundation (PSF), 2024. URL: https://pypi.org/ (visited on 2024-08-17) (cit. on p. 155).

[133] *The Unicode Standard, Version 15.1: Archived Code Charts*. South San Francisco, CA, USA: The Unicode Consortium, Aug. 25, 2023. URL: https://www.unicode.org/Public/15.1.0/charts/CodeCharts.pdf (visited on 2024-07-26) (cit. on pp. 48, 157).

[134] George K. Thiruvathukal, Konstantin Läufer, and Benjamin Gonzalez. "Unit Testing Considered Useful". *Computing in Science & Engineering* 8(6):76–87, Nov.–Dec. 2006. ISSN: 1521-9615. doi:10.1109/MCSE.2006.124. URL: https://www.researchgate.net/publication/220094077 (visited on 2024-10-01) (cit. on p. 119).

[135] Linus Torvalds. "The Linux Edge". *Communications of the ACM (CACM)* 42(4):38–39, Apr. 1999. ISSN: 0001-0782. doi:10.1145/299157.299165 (cit. on pp. 6, 155).

[136] Mariot Tsitoara. *Beginning Git and GitHub: Version Control, Project Management and Teamwork for the New Developer*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0215-7 (cit. on pp. 154, 157).

[137] *Unicode® 15.1.0*. South San Francisco, CA, USA: The Unicode Consortium, Sept. 12, 2023. ISBN: 978-1-936213-33-7. URL: https://www.unicode.org/versions/Unicode15.1.0 (visited on 2024-07-26) (cit. on pp. 48, 157).

[138] *USA Standard Code for Information Interchange (ASCII 1967)*. Tech. rep. USAS X3.4-1967. New York, NY, USA: United States of America Standards Institute (USAS), July 7, 1967. URL: https://www.sensitiveresearch.com/Archive/CharCodeHist/Files/CODES%20standards%20documents%20ASCII%20Sean%20Leonard%20Oct%202015/ASCII%2068,%20X3.4-1967.pdf (visited on 2024-07-26) (cit. on p. 48).

[139] Marat Valiev, Bogdan Vasilescu, and James D. Herbsleb. "Ecosystem-Level Determinants of Sustained Activity in Open-Source Projects: A Case Study of the PyPI Ecosystem". In: *Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/SIGSOFT FSE'18), Nov. 4-9, 2018, Lake Buena Vista, FL, USA*. Ed. by Gary T. Leavens, Alessandro F. Garcia, and Corina S. Păsăreanu. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 644–655. ISBN: 978-1-4503-5573-5. doi:10.1145/3236024.3236062 (cit. on p. 155).

[140] Bruce M. Van Horn II and Quan Nguyen. *Hands-On Application Development with PyCharm*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-83763-235-0 (cit. on pp. 6, 155).

[141] Guido van Rossum. *Computer Programming for Everybody (Revised Proposal). A Scouting Expedition for the Programmers of Tomorrow*. CNRI Proposal 90120-1a. Reston, VA, USA: Corporation for National Research Initiatives (CNRI), July 1999. URL: https://www.python.org/doc/essays/cp4e (visited on 2024-06-27) (cit. on p. 3).

[142] Guido van Rossum and Łukasz Langa. *Type Hints*. Python Enhancement Proposal (PEP) 484. Beaverton, OR, USA: Python Software Foundation (PSF), Sept. 29, 2014. URL: https://peps.python.org/pep-0484 (visited on 2024-08-22) (cit. on pp. 63, 64, 156).

[143] Guido van Rossum, Barry Warsaw, and Alyssa Coghlan. *Style Guide for Python Code*. Python Enhancement Proposal (PEP) 8. Beaverton, OR, USA: Python Software Foundation (PSF), July 5, 2001. URL: https://peps.python.org/pep-0008 (visited on 2024-07-27) (cit. on pp. 47, 51, 52, 75, 112, 113, 116, 150, 151, 154).

[144] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, CJ Carey, Ilhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregos, Paul van Mulbregt, and SciPy 1.0 Contributors. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". *Nature Methods* 17:261–272, Mar. 2, 2020. ISSN: 1548-7091. doi:10.1038/s41592-019-0686-2. URL: http://arxiv.org/abs/1907.10121 (visited on 2024-06-26). See also arXiv:1907.10121v1 [cs.MS] 23 Jul 2019. (Cit. on pp. 3, 155).

[145] Thomas Weise and Zhize Wu. "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *Companion Proceedings of the Conference on Genetic and Evolutionary Computation (GECCO'23), Companion Volume, July 15-19, 2023, Lisbon, Portugal*. Ed. by Sara Silva and Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, pp. 1891–1899. ISBN: 979-8-4007-0120-7. doi:10.1145/3583133.3596306 (cit. on pp. 3, 155).

[146] Eric Wolfgang Weisstein. "Prime Number". In: *MathWorld – A Wolfram Web Resource*. Champaign, IL, USA: Wolfram Research, Inc., Aug. 22, 2024. URL: https://mathworld.wolfram.com/PrimeNumber.html (visited on 2024-09-24) (cit. on p. 101).

[147] James A. Whittaker. "IEEE Software". 17(1):70–79, Jan.–Feb. 2000. ISSN: 0740-7459. doi:10.1109/52.819971. Practice Tutorial (cit. on p. 119).

[148] Martin Yanev. *PyCharm Productivity and Debugging Techniques*. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2022. ISBN: 978-1-83763-244-2 (cit. on pp. 6, 155).

[149] Giorgio Zarrelli. *Mastering Bash*. Birmingham, England, UK: Packt Publishing Ltd, June 2017. ISBN: 978-1-78439-687-9 (cit. on pp. 154, 174).

[150] Dmitry Zinoviev. *Discrete Event Simulation: It's Easy with SimPy!* arXiv.org: Computing Research Repository (CoRR) abs/2405.01562. Ithaca, NY, USA: Cornell Universiy Library, Apr. 3, 2024. doi:10.48550/ARXIV.2405.01562. URL: https://arxiv.org/abs/2405.01562 (visited on 2024-06-27). arXiv:2405.01562v1 [cs.MS] 3 Apr 2024 (cit. on pp. 3, 156).

Scripts

Here we provide some scripts that are used within this book. These scripts are written for the Bash shell, which is the default interpreter running in Ubuntu Linux terminals. Therefore, they will not work under Windows or other operating systems. Now, our book focuses on Python programming, so Bash shell scripts are not in the center of our attention. We here cannot explain how Bash scripts work or what their syntax is. There exist plenty of books and resources on this interesting topic, such as [16, 93, 149] or https://www.gnu.org/software/bash/.

If we would include the scripts in the places where we use them in this book, then this would lead to confusion or tangents in the text which would mess up the flow of the chapters. Nevertheless, the book would be incomplete if these scripts were not provided at all. So we put them here, at the end of the book, where they do not hurt anyone and where the interested reader may check them out.

In the scripts, we install tools if they are not yet installed. We then apply the tools to whatever the parameters of the scripts state. Now, in a real environment, a tool can either succeed or fail. A unit test executed with pytest will fail and exit with a non-zero exit code if the test, well, fails. A static code analysis tool like Ruff will fail with a non-zero exit code if it discovers any issue with the code. However, our scripts invoking these tools will *not* fail in such cases. They will collect the exit code of the tool they are invoking and print it. Then they will exit with exit code 0. This is necessary for our book building process, which invokes these scripts to construct the outputs of the examples. If any of them would fail with non-zero exit code, the book building would fail as well. However, to illustrate that the tools are useful, we must apply them to cases where they would fail. So for our book, it is necessary that the scripts just print the exit codes while still returning successfully. For a real productive environment, this is usually not what we want: We apply the tools to source code precisely to get them to fail on error, because if nothing fails, we know that everything seems to be OK. In such a practical environment, you would thus not want to use *our* scripts, but you could use the same comments that we use. Anyway, here is the list of tool scripts.

Listing 10.1 is a Bash script which first checks whether Mypy is installed. If Mypy is not installed, it silently installs it. The script then composes the command for applying Mypy to the target file(s) in the target directory. It then prints the command with a prepended `$`. Then, it executes it. Mypy will print its comments and messages to the standard output. Finally, the script shows the Mypy version and exit code in a brief success or failure message (with a prepended `#`). Listings 4.17 and 4.18 are example outputs of this script.

Listing 10.2 works basically the same way, just for Ruff. However, it sets a lot more parameters to Ruff. See, Ruff offers many more configuration options and many more different things that it can check for, compared to Mypy. In it's present form, it does not check for type errors, as far as I know, though. Either way, it is harder to balance the strictness of the tool and there are even some rules which are sometimes mutually exclusive. Hence, we compose a more complex command. Apart from that, this script works pretty much the same as Listing 10.1. Listings 5.10 and 5.14 are examples for the output of this script.

Listing 10.3 offers exactly the same functionality for Pylint. It checks if this tool is installed and installs it if not. It then applies Pylint to a the selected set of files, using a reasonable default configuration. Listing 7.15 is an example of the output of this linter.

Listing 10.4 is similarly structured, but instead of performing *static* code analysis, it executes unit test cases. The directory and list of Python files with the test cases are provided as command line arguments. This script checks if pytest and its plugin `pytest-timeout` are installed and, if not, installs them. It then executes the tests with a fixed ten second timeout that we use in this book. Of course, in practical scenarios, you would use a larger timeout, but within the context of this book, ten seconds are enough. Either way, the script executes the test cases, prints the results as well as potential errors.

Listing 10.1: A Bash script for executing Mypy, which prints the command line and the exit code; see Useful Tool 1. (src)

```bash
#!/bin/bash -

# This is our internal mypy execution script.
# It prints the mypy command and the results of mypy.
# It always exits with an exit code 0, even if mypy fails.
# Argument $1: the folder to run it in
# Argument $2: the file(s) to scan

# We enforce strict error handling, i.e., fail on any unexpected error.
set -o pipefail  # trace errors through pipes
set -o errtrace  # trace errors through commands and functions
set -o nounset   # exit if encountering an uninitialized variable
set -o errexit   # exit if any statement returns a non-0 return value

# Check if mypy is installed. If yes, get its version. If no, install it.
infos="$(python3 -m pip show mypy 2>/dev/null || true)"
if [ -z "$infos" ]; then
  # mypy is not installed, so we install it now.
  # We do this silently, without printing any information...
  python3 -m pip install mypy 1>/dev/null 2>&1
  infos="$(python3 -m pip show mypy 2>/dev/null)"
fi
# We now extract the version of mypy from the information string.
version="$(grep Version: <<< "$infos")"
version="$(sed -n 's/.*Version:\s*\([.0-9]*\)/\1/p' <<< "$version")"

# Construct the mypy command.
command="mypy $2 --no-strict-optional --check-untyped-defs"
echo "\$ $command"  # We print the command line which will be executed.
cd "$1"  # We enter the folder inside of which we should execute mypy.

# Switch of "exit-on-error", run mypy, and afterwards switch it back on.
set +o errexit  # Turn off exit-on-error.
$command 2>&1
exitCode="$?"  # Store exit code of program in variable exitCode.
set -o errexit  # Turn exit-on-error back on.

# Convert exit code to success or failure string.
[ "$exitCode" -eq 0 ] && exitCodeStr="succeeded" || exitCodeStr="failed"

# Finally, we print the result string.
echo "# mypy $version $exitCodeStr with exit code $exitCode."
```

Listing 10.2: A Bash script for executing `Ruff`, which prints the command line and the exit code; see Useful Tool 2. (src)

```bash
#!/bin/bash -

# This is our internal ruff execution script.
# It prints the ruff command and the results of ruff.
# It always exits with an exit code 0, even if ruff fails.
# Argument $1: the folder to run it in
# Argument $2: the file(s) to scan

# We enforce strict error handling, i.e., fail on any unexpected error.
set -o pipefail  # trace errors through pipes
set -o errtrace  # trace errors through commands and functions
set -o nounset   # exit if encountering an uninitialized variable
set -o errexit   # exit if any statement returns a non-0 return value

# Check if ruff is installed. If yes, get its version. If no, install it.
infos="$(python3 -m pip show ruff 2>/dev/null || true)"
if [ -z "$infos" ]; then
  # ruff is not installed, so we install it now.
  # We do this silently, without printing any information...
  python3 -m pip install ruff 1>/dev/null 2>&1
  infos="$(python3 -m pip show ruff 2>/dev/null)"
fi
# We now extract the version of ruff from the information string.
version="$(grep Version: <<< "$infos")"
version="$(sed -n 's/.*Version:\s*\([.0-9]*\)/\1/p' <<< "$version")"

# Construct the ruff command.
command="ruff check --target-version py312 --select=A,AIR,ANN,ASYNC,B,BLE,C,C4,COM,D,DJ,DTZ,E,ERA,EXE,F,FA,FIX,FLY,FURB,G,I,ICN,INP,ISC,INT,LOG,N,NPY,PERF,PIE,PLC,PLE,PLR,PLW,PT,PYI,Q,RET,RSE,RUF,S,SIM,T,T10,TD,TID,TRY,UP,W,YTT --ignore=A005,ANN001,ANN002,ANN003,ANN101,ANN204,ANN401,B008,B009,B010,C901,D203,D208,D212,D401,D407,D413,INP001,N801,PLC2801,PLR0904,PLR0911,PLR0912,PLR0913,PLR0914,PLR0915,PLR0916,PLR0917,PLR1702,PLR2004,PLR6301,PT011,PT012,PT013,PYI041,RUF100,S,T201,TRY003,UP035,W --line-length 79 $2"
echo "\$ $command"  # We print the command line which will be executed.
cd "$1"  # We enter the folder inside of which we should execute ruff.

# Switch of "exit-on-error", run ruff, and afterwards switch it back on.
set +o errexit  # Turn off exit-on-error.
$command 2>&1  # Run ruff.
exitCode="$?"  # Store exit code of program in variable exitCode.
set -o errexit  # Turn exit-on-error back on.

# Convert exit code to success or failure string.
[ "$exitCode" -eq 0 ] && exitCodeStr="succeeded" || exitCodeStr="failed"

# Finally, we print the result string.
echo "# ruff $version $exitCodeStr with exit code $exitCode."
```

Listing 10.3: A Bash script for executing Pylint, which prints the command line and the exit code; see Useful Tool 3. (src)

```bash
#!/bin/bash -

# This is our internal pylint execution script.
# It prints the pylint command and the results of pylint.
# It always exits with an exit code 0, even if pylint fails.
# Argument $1: the folder to run it in
# Argument $2: the file(s) to scan

# We enforce strict error handling, i.e., fail on any unexpected error.
set -o pipefail  # trace errors through pipes
set -o errtrace  # trace errors through commands and functions
set -o nounset   # exit if encountering an uninitialized variable
set -o errexit   # exit if any statement returns a non-0 return value

# Is pylint is installed? If yes, get its version. If no, install it.
infos="$(python3 -m pip show pylint 2>/dev/null || true)"
if [ -z "$infos" ]; then
  # pylint is not installed, so we install it now.
  # We do this silently, without printing any information...
  python3 -m pip install pylint 1>/dev/null 2>&1
  infos="$(python3 -m pip show pylint 2>/dev/null)"
fi
# We now extract the version of pylint from the information string.
version="$(grep Version: <<< "$infos")"
version="$(sed -n 's/.*Version:\s*\([.0-9]*\)/\1/p' <<< "$version")"

# Construct the pylint command.
command="pylint $2 --disable=C0103,C0302,C0325,R0801,R0901,R0902,R0903,R0911,R0912,R0913,R0914,R0915,R1702,R1728,W0212,W0238,W0703"
echo "\$ $command"  # We print the command line which will be executed.
cd "$1"  # We enter the folder inside of which we should execute pylint.

# Switch of "exit-on-error", run pylint, and afterwards switch it back on.
set +o errexit  # Turn off exit-on-error.
$command 2>&1  # Run pylint.
exitCode="$?"  # Store exit code of program in variable exitCode.
set -o errexit  # Turn exit-on-error back on.

# Convert exit code to success or failure string.
[ "$exitCode" -eq 0 ] && exitCodeStr="succeeded" || exitCodeStr="failed"

# Finally, we print the result string.
echo "# pylint $version $exitCodeStr with exit code $exitCode."
```

Listing 10.4: A Bash script for executing test cases with pytest, which prints the command line and the exit code; see Useful Tool 4. (src)

```bash
#!/bin/bash -

# This is our internal pytest execution script.
# It prints the pytest command and the results of pytest.
# It always exits with an exit code 0, even if pytest fails.
# Argument $1: the folder to run it in
# Argument $2: the file(s) to scan

# We enforce strict error handling, i.e., fail on any unexpected error.
set -o pipefail  # trace errors through pipes
set -o errtrace  # trace errors through commands and functions
set -o nounset   # exit if encountering an uninitialized variable
set -o errexit   # exit if any statement returns a non-0 return value

# Check if pytest and all required plugins are installed.
versions=""  # This variable will receive all tool versions.
for pack in "pytest" "pytest-timeout"; do
  infos="$(python3 -m pip show "$pack" 2>/dev/null || true)"
  if [ -z "$infos" ]; then
    # pytest or the plugin is not installed, so we install it now.
    # We do this silently, without printing any information...
    python3 -m pip install "$pack" 1>/dev/null 2>&1
    infos="$(python3 -m pip show "$pack" 2>/dev/null)"
  fi

  # For each tool or plugin, we get the version separately.
  infos="$(grep Version: <<< "$infos")"
  infos="$(sed -n 's/.*Version:\s*\([.0-9]*\)/\1/p' <<< "$infos")"
  if [ -z "$versions" ]; then  # ... and we concatenate them
    versions="$pack $infos with"
  elif [[ "$versions" == *with ]]; then
    versions="$versions $pack $infos"
  else
    versions="$versions, $pack $infos"
  fi
done

# Construct the pytest command.
command="pytest --timeout=10 --no-header --tb=short $2"
echo "\$ $command"  # We print the command line which will be executed.
cd "$1"  # We enter the folder inside of which we should execute pytest.
export COLUMNS=73
# Switch of "exit-on-error", run pytest, and afterwards switch it back on.
set +o errexit  # Turn off exit-on-error.
$command 2>&1
exitCode="$?"  # Store exit code of program in variable exitCode.
set -o errexit  # Turn exit-on-error back on.

# Convert exit code to success or failure string.
[ "$exitCode" -eq 0 ] && exitCodeStr="succeeded" || exitCodeStr="failed"

# Finally, we print the result string.
echo "# $versions $exitCodeStr with exit code $exitCode."
```

Notice that we select some options to make the output less verbose, because for this book, we do want listings that are not too long. In practical scenarios, you may use different options. Either way, the script also prints the command that was executed at the beginning and the `pytest` version used and the exit code of the process at the end of the output. Listings 8.9, 8.12 and 8.15 are examples for the output of `pytest`.